

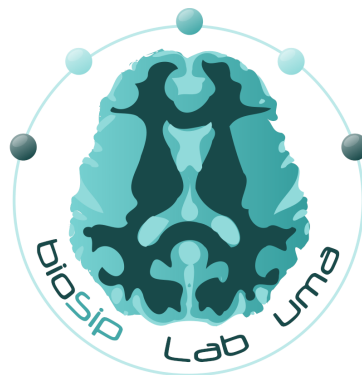
DualFlexKAN

Manual de referencia

Andrés Ortiz García

Grupo BioSiP
Universidad de Málaga

Fecha:
27 de febrero de 2026



Biomedical Signal Processing, Computational
Intelligence and Communications Security

DualFlexKAN con Regularización Flexible y Mecanismo de Atención

Manual de Referencia

A. Ortiz

Version 1.1. Diciembre 2025

Este manual proporciona una guía completa para el uso de DualFlexKAN, una implementación avanzada de Kolmogorov-Arnold Networks (KAN) con control independiente sobre funciones de entrada y activación, junto con un sistema de regularización flexible que permite control preciso sobre dropout y batch normalization. Dado que la interpretabilidad es una de las capacidades más importantes a explotar en las redes KAN, la librería también implementa diferentes funciones para explorar el funcionamiento interno de la red. La librería se acompaña de funciones avanzadas de guardado y carga de modelos y resultados, así como de funciones auxiliares de visualización y depuración.

Índice general

1. Introducción	1-1
1.1. Funcionalidad implementada	1-1
1.2. Redes KAN vs. MLPs Tradicionales	1-2
1.2.1. Introducción: ¿Cuándo usar KAN?	1-2
1.2.2. Objetivo del Modelado	1-3
1.2.3. Ventajas de las Redes KAN	1-3
1.2.4. Selección de Bases Polinomiales y Funcionales	1-4
1.2.5. Capacidad de Disentangling en Autoencoders: MLP vs KAN	1-6
1.2.6. Separabilidad en el Espacio Latente: MLP vs KAN	1-10
1.2.7. Visualización del espacio latente	1-15
1.2.8. Casos de Uso Prácticos	1-15
1.2.9. Limitaciones	1-16
1.2.10. Separabilidad y Complejidad	1-17
1.2.11. Conclusión	1-18
2. Fundamentos Teóricos de las Redes KAN	2-1
2.1. Teorema de Kolmogorov-Arnold	2-1
2.1.1. Enunciado del Teorema	2-1
2.1.2. Interpretación Geométrica	2-1
2.2. Formulación Matemática de las Redes KAN	2-2
2.2.1. Arquitectura Básica	2-2
2.2.2. Extensión a Redes Profundas	2-2
2.2.3. Implementación Flexible: DualFlexKAN	2-2
2.3. Propiedades Matemáticas	2-2
2.3.1. Aproximación Universal	2-2
2.3.2. Expresividad vs. Parámetros	2-3
2.4. Funciones de Activación en KAN	2-3
2.4.1. Representación Paramétrica General	2-3
2.4.2. Bases Polinomiales Estándar	2-3
2.4.3. Bases Polinomiales Ortogonales	2-3
2.4.4. Bases Locales y Splines	2-4
2.4.5. Bases Espectrales y de Frecuencia	2-4
2.4.6. Funciones Racionales	2-5
2.5. Entrenamiento y Optimización	2-5
2.5.1. Función de Pérdida	2-5

2.5.2.	Gradientes y Retropropagación	2-5
2.5.3.	Representación Gráfica	2-6
2.6.	Regularización Flexible	2-6
2.6.1.	Formulación del Operador de Regularización	2-6
3.	Propiedades Geométricas de las Bases de Activación	3-1
3.1.	Familia de Polinomios Ortogonales	3-1
3.1.1.	Polinomios de Legendre	3-1
3.1.2.	Polinomios de Chebyshev	3-2
3.1.3.	Polinomios de Gegenbauer	3-3
3.1.4.	Polinomios de Jacobi	3-3
3.2.	Bases Locales y Splines	3-4
3.2.1.	B-Splines y Fast B-Splines	3-4
3.2.2.	Wavelets (Sombrero Mexicano)	3-5
3.2.3.	Funciones de Base Radial (RBF)	3-5
3.3.	Bases Espectrales y Racionales	3-5
3.3.1.	Base Espectral (Seno)	3-5
3.3.2.	Funciones Racionales (Padé)	3-6
3.4.	Resumen	3-6
4.	Implementación Flexible: DualFlexKAN	4-1
4.1.	Arquitectura Dual Independiente	4-1
4.1.1.	Formulación Matemática General	4-1
4.2.	Estrategias de Activación de Entrada	4-2
4.2.1.	Estrategia "none" – Sin transformación	4-2
4.2.2.	Estrategia "fixed". Función fija compartida	4-2
4.2.3.	Estrategia "global" – Una función adaptativa compartida	4-3
4.2.4.	Estrategia "per_input" – Una función por entrada	4-3
4.2.5.	Estrategia "per_neuron_input" – Máxima flexibilidad	4-4
4.3.	Funciones de activación de entrada	4-5
4.3.1.	Funciones Fijas	4-5
4.3.2.	Funciones Adaptativas	4-7
4.3.3.	Ejemplos Prácticos	4-10
4.3.4.	Recomendaciones	4-12
4.4.	Estrategias de Activación de Salida	4-12
4.4.1.	Estrategia "none" – Sin activación	4-12
4.4.2.	Estrategia "fixed" – Función de activación fija	4-14
4.4.3.	Estrategia "global" – Una función adaptativa compartida	4-15
4.4.4.	Estrategia "per_neuron" – Una función por neurona	4-15
4.5.	Funciones de Activación de Salida	4-17
4.5.1.	Funciones Fijas	4-17
4.5.2.	Funciones Adaptativas	4-17
4.5.3.	Tabla Comparativa de Estrategias de Salida	4-22
4.5.4.	Ejemplos Prácticos	4-22
4.5.5.	Recomendaciones	4-24

4.6.	Sistema de Regularización Flexible	4-24
4.6.1.	Posiciones de Aplicación	4-24
4.6.2.	Flujo de Procesamiento	4-25
4.6.3.	Ejemplos Prácticos	4-25
4.7.	Optimización Específica para DualFlexKAN	4-26
4.7.1.	Algoritmo de Entrenamiento Adaptativo	4-26
4.7.2.	Regularización Adaptativa	4-27
4.8.	Inicialización de los pesos	4-27
4.8.1.	La importancia de la inicialización	4-27
4.8.2.	Estrategias de Inicialización para Pesos Lineales	4-28
4.8.3.	Estrategias Predefinidas	4-36
4.8.4.	Inicialización Layer-wise	4-36
4.8.5.	Configuración Recomendada	4-39
4.8.6.	Justificación de la Estrategia	4-40
4.8.7.	Sistema de Análisis de Inicialización	4-42
4.8.8.	Consideraciones Prácticas	4-43
4.8.9.	Ejemplo. Inicialización de una red para clasificación MNIST	4-44
5.	Detalles de Implementación: DualFlexKAN	5-1
5.1.	Clase DualFlexKANLinear	5-1
5.1.1.	Constructor	5-1
5.1.2.	Estrategias de Transformación de Entrada (<code>input_function_strategy</code>)	5-2
5.1.3.	Estrategias de Activación de Salida (<code>output_function_strategy</code>) . .	5-2
5.1.4.	Tipos de Activación Disponibles	5-2
5.1.5.	Flujo de Ejecución (Forward Pass)	5-3
5.1.6.	Regularización Flexible	5-3
5.1.7.	Introspección y Análisis	5-4
5.2.	Clase DualFlexKAN	5-4
5.2.1.	Constructor	5-4
5.2.2.	Gestión de Capas	5-4
5.2.3.	Ejemplo de Instanciación Compleja	5-5
5.3.	Integración con Utilidades	5-5
5.4.	Herramientas de Migración	5-5
5.4.1.	El Convertidor Automático	5-6
5.4.2.	Implementación del Convertidor	5-6
5.4.3.	Ejemplos de Uso	5-8
5.4.4.	Consideraciones Técnicas Importantes	5-9
6.	Funciones de Utilidad	6-1
6.1.	Módulo <code>save_load_utils.py</code>	6-1
6.1.1.	<code>save_object</code>	6-1
6.1.2.	<code>load_object</code>	6-2
6.2.	Funciones para Múltiples Objetos	6-3
6.2.1.	<code>save_multiple</code>	6-3
6.2.2.	<code>load_multiple</code>	6-4

6.3.	Funciones para Gestión de Experimentos	6-5
6.3.1.	<code>save_experiment</code>	6-5
6.3.2.	<code>load_experiment</code>	6-7
6.3.3.	<code>list_experiments</code>	6-8
6.4.	Funciones para Análisis de Modelos PyTorch	6-9
6.4.1.	<code>extract_model_info</code>	6-9
6.4.2.	<code>generate_model_summary</code>	6-12
6.4.3.	<code>print_model_info</code>	6-13
6.5.	Guardar y cargar modelos PyTorch de forma autónoma	6-15
6.5.1.	Decorador <code>@autoargs</code>	6-15
6.5.2.	Extracción del código de la clase (<code>extract_full_code</code>)	6-15
6.5.3.	Guardar el modelo: <code>save_model_auto</code>	6-16
6.5.4.	Cargar el modelo: <code>load_model_auto</code>	6-16
6.6.	Estructura de los Modelos y Acceso a Componentes	6-17
6.7.	Resumen de Funciones	6-18
6.8.	Funciones Auxiliares de Datos y KAN	6-18
6.8.1.	Extracción de datos: <code>extract_data_from_loader</code>	6-18
6.8.2.	Extraer configuración KAN: <code>extract_kan_config</code>	6-18
6.9.	Extraer coeficientes de polinomios <code>polycoefs(polinomio_str)</code>	6-19
6.10.	Casos de Uso	6-19
6.10.1.	Ejemplo 1: Red Clásica (MLP)	6-19
6.10.2.	Ejemplo 2: DualFlexKAN Autoencoder	6-20
7.	Wrapper para redes Convolucionales KAN DualFlexKANConv	7-1
7.1.	Arquitectura General	7-1
7.1.1.	Concepto de KAN Convolutacional	7-2
7.1.2.	Ventajas de KAN Convolutacional	7-2
7.2.	La Capa DualFlexKANConv2d	7-2
7.2.1.	Constructor	7-2
7.2.2.	Estrategias de Funciones	7-4
7.2.3.	Forward Pass	7-5
7.3.	Definición de una capa KAN convolutacional	7-5
7.3.1.	Capa Híbrida (Entrada Fija + Salida Adaptativa)	7-6
7.3.2.	Capa con Estrategia Global (Eficiente)	7-6
7.4.	Definición de Redes capa a capa	7-7
7.4.1.	Ejemplo 1. Red CNN Simple de 3 Capas	7-7
7.4.2.	Ejemplo 1. Red con diferentes tipos de funciones por capa	7-9
7.5.	La Clase DualFlexKANCNN	7-11
7.5.1.	Constructor	7-11
7.5.2.	Ejemplo 1. CNN estilo VGG con DualFlexKANCNN	7-12
7.5.3.	Ejemplo 2. CNN compacta con función auxiliar	7-14
7.6.	Arquitecturas Avanzadas	7-15
7.6.1.	CNN estilo ResNet con KAN	7-15
7.6.2.	CNN Personalizada con Configuración Compleja	7-15
7.7.	Entrenamiento de CNN KAN	7-17

7.7.1.	Pipeline Completo de Entrenamiento	7-17
7.8.	Comparación: KAN Conv vs Conv Estándar	7-20
7.8.1.	Tabla Comparativa	7-20
7.8.2.	Comparación CNN estándar vs. CNN KAN	7-20
7.9.	Resolución de problemas frecuentes	7-22
7.9.1.	Problema: Memoria Insuficiente	7-22
7.9.2.	Problema: Entrenamiento Lento	7-22
7.9.3.	Problema: Gradientes Explosivos/Vanishing	7-22
7.10.	Ejemplo Completo: CIFAR-10 con KAN CNN	7-23
8.	Autoencoders basados en DualFlexKAN	8-1
8.1.	Arquitectura General	8-1
8.2.	Construcción del Encoder	8-2
8.2.1.	Parámetros del Constructor	8-2
8.3.	Construcción del Decoder	8-3
8.3.1.	Parámetros del Constructor	8-3
8.4.	Autoencoder Completo con DualFlexKANAutoencoder	8-6
8.4.1.	Parámetros del Constructor	8-6
8.5.	Visualización de la arquitectura	8-8
8.6.	Entrenamiento del Autoencoder	8-10
8.6.1.	Función <code>train_autoencoder</code>	8-10
8.7.	Codificación y decodificación independientes	8-11
8.7.1.	Codificación de Nuevos Datos	8-11
8.7.2.	Decodificación desde el Espacio Latente	8-12
8.7.3.	Interpolación en el Espacio Latente	8-12
8.7.4.	Visualización de Resultados	8-13
8.7.5.	Transfer Learning con Encoder Pre-entrenado	8-13
9.	Autoencoders Convolucionales basados en DualFlexKAN	9-1
9.1.	Arquitectura General	9-1
9.1.1.	Soporte Multidimensional	9-1
9.1.2.	Pipeline de Procesamiento	9-2
9.2.	Construcción del Encoder	9-2
9.2.1.	Parámetros del Constructor	9-2
9.2.2.	Ejemplo de Uso Básico	9-3
9.2.3.	Ejemplo con Configuración Avanzada	9-4
9.2.4.	Encoder 1D para Series Temporales	9-6
9.2.5.	Encoder 3D para Videos	9-6
9.3.	Construcción del Decoder	9-7
9.3.1.	Parámetros del Constructor	9-7
9.3.2.	Estructura del Decoder:	9-8
9.3.3.	Ejemplo de Uso	9-8
9.4.	Autoencoder Completo	9-10
9.4.1.	Parámetros del Constructor	9-10
9.5.	Configuración Avanzada con Dicionarios	9-12

9.6.	Entrenamiento del Autoencoder	9-14
9.6.1.	Función de Entrenamiento Básica	9-14
9.7.	Codificación y Decodificación Independientes	9-15
9.7.1.	Codificación de Nuevos Datos	9-15
9.7.2.	Decodificación desde el Espacio Latente	9-16
9.7.3.	Interpolación en el Espacio Latente	9-17
9.7.4.	Interpolación Esférica (para VAEs):	9-18
9.7.5.	Visualización de Resultados	9-18
9.8.	Transfer Learning con Encoder Pre-entrenado	9-22
9.8.1.	Clasificador con Encoder Congelado	9-22
9.8.2.	Fine-tuning Gradual	9-24
10.	Beta-VAE basado en DualFlexKAN	10-1
10.1.	Arquitectura General	10-1
10.1.1.	Diferencias con Autoencoders Deterministas	10-1
10.2.	Construcción del Encoder	10-2
10.2.1.	Parámetros del Constructor	10-2
10.2.2.	Ejemplo de Uso Básico	10-3
10.2.3.	Ejemplo con Configuración Avanzada	10-3
10.3.	Construcción del Decoder	10-4
10.3.1.	Parámetros del Constructor	10-4
10.3.2.	Autoencoder Completo	10-4
10.4.	Entrenamiento del Autoencoder	10-8
10.4.1.	Función train_beta_vae	10-8
10.4.2.	Beta Annealing	10-11
10.4.3.	Codificación y decodificación independientes	10-19
10.4.4.	Visualización de Resultados	10-22
10.4.5.	Transfer Learning con Encoder Pre-entrenado	10-23
11.	Interpretabilidad y Análisis Avanzado	11-1
11.1.	La Importancia de la Interpretabilidad	11-1
11.1.1.	Ventajas de KAN sobre Redes Tradicionales	11-1
11.2.	Métodos Nativos de Inspección en DualFlexKAN	11-2
11.2.1.	Información de Arquitectura	11-2
11.2.2.	Extracción de Funciones Aprendidas	11-2
11.2.3.	Resumen de Regularización	11-2
11.3.	Análisis de Explicabilidad e Importancia	11-3
11.3.1.	Inicialización	11-3
11.3.2.	Cálculo de Importancia de Funciones	11-3
11.3.3.	Knockout Analysis (Análisis de Ablación)	11-3
11.3.4.	Análisis de Sensibilidad	11-4
11.3.5.	Análisis Comprensivo	11-4
11.4.	Análisis de Interacciones	11-4
11.4.1.	Descomposición Funcional	11-4
11.4.2.	Mapas de Interacción y Pares	11-4

11.4.3. Comparación de Estrategias	11-5
11.5. Visualización del Flujo de Información	11-5
11.5.1. Captura y Mapas de Calor	11-5
11.5.2. Rutas Críticas (Critical Paths)	11-5
11.5.3. Análisis de Cuellos de Botella	11-5
11.6. Poda y Simplificación Simbólica	11-6
11.6.1. Análisis de Coeficientes	11-6
11.6.2. Poda (Pruning)	11-6
11.6.3. Regresión Simbólica (Simplificación)	11-6
11.7. Análisis de Autoencoders KAN	11-6
11.7.1. Simetría Funcional	11-7
11.7.2. Exploración del Espacio Latente	11-7
11.8. Generación de Informes Completos	11-7
12. Mecanismos de Atención en DualFlexKAN	12-1
12.1. Introducción a la Atención Neuronal	12-1
12.1.1. Beneficios Clave	12-1
12.2. Arquitectura: DualFlexKAN con Atención	12-2
12.2.1. Formulación Matemática	12-2
12.3. Implementación en Python	12-2
12.3.1. El Módulo de Atención	12-2
12.3.2. La Capa DualFlexKAN con Atención	12-3
12.4. Creación de una red AttentiveDualFlexKAN	12-4
12.5. Inclusión de capa de atención en DualFlexKAN	12-5
12.6. Atención Selectiva (Híbrida)	12-5
12.6.1. Construcción de una Red con Atención Híbrida	12-6
12.6.2. Atención en Capas Arbitrarias	12-7
12.6.3. Parámetros de Configuración de Atención	12-7
12.7. Caso de Uso Avanzado: Autoencoder Atencional	12-7
12.7.1. Arquitectura del Encoder con Atención Latente	12-8
12.7.2. Implementación en Python	12-9
12.7.3. Interpretación de los Pesos de Atención Latente	12-11
12.7.4. Entrenamiento con Regularización de Atención	12-11
12.7.5. Interpretación del Autoencoder Atencional	12-12
12.7.6. Construcción de Arquitecturas Híbridas Capa a Capa	12-12
13. Aplicaciones Específicas	13-1
13.1. Regresión No Lineal	13-1
13.2. Clasificación Multiclase	2
C. Funciones Adaptativas (Entrenables)	5
D. Funciones Fijas (No Entrenables)	7
E. Transformaciones Matemáticas	7
I. Núcleo del Modelo (KAN_shared_poly7.py)	13
J. Mecanismos de Atención (KAN_shared_poly7.py)	14
K. Funciones de Activación (KAN_activation.py)	14

L.	Inicialización (KAN_initialization.py)	15
M.	Persistencia y Utilidades (save_load_utils.py)	16
N.	Módulos de Arquitecturas Específicas	16
Ñ.	Módulos de Análisis e Interpretabilidad	17
O.	Carga de Datos (dataset_loader.py)	18

Capítulo 1 Introducción

DualFlexKAN es una implementación avanzada de Kolmogorov-Arnold Networks (KAN) que proporciona control independiente sobre las funciones de entrada y activación, junto con un sistema de regularización flexible.

1.1. Funcionalidad implementada

En esta librería, se han implementado diferentes clases y funciones para poder definir arquitecturas con redes neuronales de Kolmogorov-Arnold (KAN) de forma flexible. Si bien en una red KAN clásica se modelan las conexiones mediante B-splines, esta implementación permite configuraciones totalmente flexibles y a medida, pudiendo parametrizar las conexiones a la entrada y/o a la salida de una neurona, incluir funciones de activación clásicas y métodos de regularización.

En resumen, la funcionalidad implementada en la librería que se describe en este manual permite:

- **Control dual independiente:** Funciones de entrada y activación configurables por separado, permitiendo arquitecturas híbridas (entradas fijas con salidas adaptativas o viceversa).
- **Tipos de activación variados:** Un catálogo extendido que incluye:
 - **Polinomios Ortogonales:** Legendre, Chebyshev (minimax), Gegenbauer (ultrasféricos) y Jacobi.
 - **Bases Locales y Espectrales:** B-splines (estándar y Fast), RBF (base radial), Wavelets (sombrero mexicano) y base espectral (Seno).
 - **Funciones Racionales:** Aproximantes de Padé para modelar singularidades.
 - **Funciones Fijas:** Activaciones clásicas de Deep Learning (ReLU, SiLU, Tanh, etc.).
- **Regularización flexible:** Dropout y BatchNorm con control preciso de posición (antes/después de la activación) y orden de aplicación.
- **Múltiples estrategias:** Desde funciones globales compartidas para toda una capa hasta funciones específicas por cada conexión individual (*per_neuron_input*).

Esto permite diseñar arquitecturas híbridas que consigan un balance entre prestaciones y complejidad computacional (parámetros).

1.2. Redes KAN vs. MLPs Tradicionales

Las *Kolmogorov-Arnold Networks* (KAN) representan un paradigma alternativo a las redes MLP tradicionales, basándose en el teorema de representación de Kolmogorov-Arnold. A continuación se discuten sus ventajas y el rol de diferentes bases polinomiales.

1.2.1. Introducción: ¿Cuándo usar KAN?

La elección entre KAN y MLP no es universal, sino que depende crucialmente de las características de los datos y el problema. Las KAN ofrecen ventajas significativas en escenarios específicos:

Dimensionalidad Efectiva

Las KAN son particularmente efectivas cuando:

- **Baja dimensionalidad intrínseca** ($d_{\text{eff}} \leq 10$): Aunque los datos puedan tener alta dimensión nominal, si la dimensionalidad efectiva es baja (estructura de variedad, relaciones funcionales simples), KAN aprovecha esta estructura mejor que las MLPs.
- **Problemas con estructura compositiva**: Cuando la función objetivo puede expresarse como composición de funciones univariadas: $f(x_1, \dots, x_n) = g_1(g_2(\dots g_k(x_1, x_2) \dots))$, KAN la aprende naturalmente.
- **Interacciones de bajo orden**: Problemas donde las variables interactúan principalmente por pares o tríos, no requiriendo la mezcla completa de alta dimensión que ofrecen las MLPs.

En contraste, para problemas genuinamente de alta dimensión ($d > 100$) con interacciones complejas entre todas las variables, las MLPs suelen ser más eficientes.

Régimen de Datos

Las KAN muestran comportamiento diferencial según la cantidad de datos disponibles:

- **Datos limitados** ($n < 10^4$): La regularización implícita de las bases funcionales (suavidad, continuidad, ortogonalidad de Legendre/Chebyshev) ayuda a la generalización. Las KAN pueden aprender con menos ejemplos que las MLPs.
- **Datos moderados** ($10^4 \leq n \leq 10^6$): Régimen óptimo para KAN, donde hay suficientes datos para estimar funciones univariadas pero no tanto como para que el costo computacional adicional sea prohibitivo.
- **Big Data** ($n > 10^6$): Las MLPs pueden ser preferibles por su implementación matricial altamente optimizada y paralelización en GPUs, aunque implementaciones como FastBSpline reducen esta brecha.

Estructura y Regularidad

Las KAN destacan cuando los datos exhiben:

- **Suavidad funcional:** Relaciones subyacentes diferenciables o con pocas discontinuidades.
- **Simetría o invarianzas:** Estructuras algebraicas que las funciones base (como Seno o polinomios pares/impares) pueden capturar eficientemente.
- **Relaciones analíticas conocidas:** Física, química, ingeniería donde las leyes subyacentes son funciones matemáticas bien definidas (polinomios, racionales).

Contrariamente, datos con estructura completamente arbitraria, altamente no-lineal y sin regularidad subyacente favorecen a las MLPs.

Ruido y Robustez

El comportamiento ante ruido difiere significativamente:

- **Ruido bajo-moderado:** La suavidad de las bases funcionales en KAN actúa como regularización natural, filtrando fluctuaciones espurias.
- **Ruido alto:** Las MLPs con dropout y batch normalization pueden ser más robustas, ya que no asumen estructura funcional subyacente que el ruido podría enmascarar.
- **Outliers:** Las KAN son más sensibles a valores atípicos que pueden distorsionar las funciones base, requiriendo preprocesamiento cuidadoso o bases robustas como B-splines de bajo orden.

1.2.2. Objetivo del Modelado

- **Interpretabilidad requerida:** KAN permite inspeccionar relaciones funcionales aprendidas $\rightarrow$ preferible para ciencia, medicina, finanzas reguladas.
- **Solo predicción:** Si únicamente importa la precisión predictiva sin interpretación, MLPs pueden ser más simples de implementar y escalar.
- **Descubrimiento científico:** KAN puede revelar ecuaciones y relaciones funcionales desconocidas en los datos (regresión simbólica).

1.2.3. Ventajas de las Redes KAN

Eficiencia Paramétrica Las redes KAN pueden alcanzar precisión comparable o superior a las MLPs utilizando significativamente menos parámetros. Mientras que las MLPs aprenden pesos escalares w_{ij} seguidos de activaciones fijas $\sigma(\cdot)$, las KAN aprenden funciones univariadas $\phi_{i,j}(x)$ en cada arista, lo que permite una representación más expresiva por parámetro.

En problemas típicos, una KAN con $\sim 10^3$ parámetros puede igualar el desempeño de una MLP con $\sim 10^5$ parámetros, siempre que el problema tenga la estructura adecuada.

Interpretabilidad Mejorada

La estructura de KAN facilita la interpretación mediante:

- Visualización directa de las funciones aprendidas $\phi_{i,j}(x)$ en cada conexión.
- Identificación de relaciones funcionales subyacentes en los datos.
- Capacidad de realizar *pruning* basado en la importancia de cada función univariada.
- Extracción de ecuaciones simbólicas aproximadas (especialmente útil con bases polinomiales o racionales).

1.2.4. Selección de Bases Polinomiales y Funcionales

La elección de la base funcional constituye una decisión arquitectónica fundamental en las redes KAN que impacta directamente en la capacidad de aproximación, eficiencia computacional y calidad de generalización del modelo. A diferencia de las MLPs, donde la función de activación es típicamente fija y universal, las KAN requieren seleccionar bases que se alineen con las características intrínsecas de los datos.

La Tabla 1.1 proporciona una guía sistemática para esta decisión, incorporando las nuevas bases disponibles en la librería (Gegenbauer, Jacobi, Rational, Wavelet).

Recomendaciones Prácticas

Limitaciones de las Redes KAN

Las KAN no son universalmente superiores. Las MLPs mantienen ventajas en:

- **Alta dimensión genuina:** Problemas con $d > 100$ donde todas las dimensiones son relevantes y las interacciones son complejas
- **Estructura funcional arbitraria:** Tareas donde no existe regularidad funcional subyacente (clasificación de imágenes naturales, NLP general)
- **Escalabilidad masiva:** Escenarios con $n > 10^7$ datos donde la optimización matricial de MLPs en GPUs es crítica
- **Hardware especializado:** Entornos con TPUs/GPUs optimizados para operaciones matriciales densas
- **Ecosistema maduro:** Disponibilidad de frameworks, técnicas de optimización, y conocimiento acumulado para MLPs
- **Datos ruidosos y no estructurados:** Cuando no hay garantía de suavidad o estructura funcional que explotar

Característica del Dataset	Base Recomendada	Justificación
DIMENSIONALIDAD		
Baja dimensión ($d \leq 5$)	Chebyshev / Legendre	Convergencia rápida, evita fenómeno de Runge, aprovecha suavidad global.
Dimensión moderada ($5 < d \leq 15$)	B-splines (orden 3-4)	Balance entre expresividad y costo computacional.
Alta dimensión ($d > 15$)	Fast B-splines	Soporte local minimiza maldición dimensionalidad y optimización de cómputo.
TAMAÑO DE DATOS		
Datos escasos ($n < 10^3$)	Legendre / Chebyshev	Regularización fuerte, menos parámetros por función.
Datos moderados ($10^3 \leq n \leq 10^5$)	B-splines / Jacobi	Régimen óptimo, flexibilidad adaptativa (Jacobi ajusta pesos en bordes).
Datos abundantes ($n > 10^5$)	Fast B-splines	Eficiencia computacional crítica.
SUAVIDAD FUNCIONAL Y SINGULARIDADES		
Muy suave (C^∞)	Chebyshev / Gegenbauer	Convergencia exponencial para funciones analíticas.
Polos o asíntotas	Racionales (Rational)	Aproximantes de Padé modelan divisiones por cero ($1/x$).
Discontinuidades	Wavelets / B-splines (orden 1)	Adaptación local a cambios abruptos y transitorios.
DOMINIO		
Acotado $[a, b]$	Chebyshev / Legendre	Óptimas en dominios compactos (vía transformación).
Acotado, peso asimétrico	Jacobi	Permite controlar la importancia en los extremos (parámetros α, β).
No acotado general	B-splines / RBF	Adaptables, sin restricciones fuertes de dominio.
CARACTERÍSTICAS ESPECIALES		
Periodicidad	Seno (Sine)	Base espectral (Fourier) natural para datos cíclicos.
Simetría	Gegenbauer / Legendre	Aprovecha paridad/imparidad e invarianzas.
Multi-escala / Señales	Wavelets	Resolución adaptativa tiempo-frecuencia.
Clusters locales	RBF	Interpolación basada en proximidad (Gaussianas).
OBJETIVOS		
Máxima precisión	Chebyshev	Error minimax (minimiza el error máximo).
Interpretabilidad	Polinomios simples	Lectura directa de coeficientes ($ax^2 + bx + c$).
Eficiencia computacional	Fast B-splines	Operaciones matriciales optimizadas.

Tabla 1.1: Guía de selección de bases funcionales para redes DualFlexKAN según características del dataset.

Escenario	Base Recomendada
Propósito general / desconocido	B-splines (orden 3) o Fast B-splines
Funciones muy suaves, precisión uniforme	Chebyshev
Necesidad de ortogonalidad estricta	Legendre
Datos con picos o discontinuidades	Wavelets
Singularidades o leyes físicas inversas	Racional (Rational)
Datos periódicos (series temporales)	Seno (Sine)
Restricciones computacionales severas	Fast B-splines o Fixed (MLP)

Tabla 1.2: Guía simplificada de selección de base funcional

Conclusión

La decisión entre KAN y MLP debe basarse en una evaluación cuidadosa de:

- Dimensionalidad efectiva ($d_{\text{eff}} < 10 \Rightarrow$ KAN favorable)
- Cantidad de datos ($10^4 < n < 10^6 \Rightarrow$ régimen óptimo KAN)
- Estructura esperada (funciones compositivas suaves $\Rightarrow$ KAN ventajosa)
- Requisitos de interpretabilidad (alta $\Rightarrow$ KAN preferible)
- Restricciones computacionales (hardware disponible, tiempo de entrenamiento)

1.2.5. Capacidad de Disentangling en Autoencoders: MLP vs KAN

Introducción al Disentanglement

El *disentanglement* o desenredamiento de representaciones es una propiedad deseable en autoencoders donde cada dimensión del espacio latente $\mathbf{z} \in \mathbb{R}^d$ captura un único factor generativo independiente de los datos. Formalmente, una representación está desenredada si existe una correspondencia entre las dimensiones latentes z_i y los factores de variación subyacentes v_j tal que:

$$p(\mathbf{z}|\mathbf{v}) = \prod_{i=1}^d p(z_i|v_{\pi(i)}) \quad (1.1)$$

donde π es una permutación que asocia cada variable latente con un único factor generativo.

Importancia del Disentanglement Las representaciones desenredadas ofrecen múltiples ventajas:

- **Interpretabilidad:** Cada dimensión tiene significado semántico claro
- **Generalización:** Facilita transferencia a nuevas tareas
- **Manipulación controlada:** Permite modificar atributos específicos independientemente

- **Robustez:** Menor sensibilidad a variaciones espurias
- **Eficiencia muestral:** Requiere menos datos para downstream tasks

Aplicaciones críticas incluyen: generación de contenido controlable, compresión semántica, fair machine learning, descubrimiento causal, y análisis científico de sistemas complejos.

Autoencoders MLP: Entrelazamiento Inherente

Los autoencoders tradicionales basados en MLPs tienen limitaciones estructurales para lograr disentanglement:

Arquitectura Estándar Un autoencoder MLP estándar se compone de:

$$\text{Encoder: } \mathbf{z} = f_\phi(\mathbf{x}) = \sigma_L(\mathbf{W}_L \sigma_{L-1}(\cdots \sigma_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \cdots)) \quad (1.2)$$

$$\text{Decoder: } \hat{\mathbf{x}} = g_\theta(\mathbf{z}) = \tau_M(\mathbf{V}_M \tau_{M-1}(\cdots \tau_1(\mathbf{V}_1 \mathbf{z} + \mathbf{c}_1) \cdots)) \quad (1.3)$$

Limitaciones para Disentanglement

1. **Mezcla no-lineal arbitraria:** Las capas completamente conectadas mezclan todas las entradas en cada neurona, creando dependencias complejas entre variables latentes:

$$z_i = \sigma \left(\sum_{j=1}^n w_{ij} h_j + b_i \right) \quad (1.4)$$

Esta mezcla global dificulta la separación de factores independientes.

2. **Simetría y colapso:** Sin restricciones adicionales, el espacio latente puede aprender representaciones completamente entrelazadas pero igualmente válidas para reconstrucción. La función objetivo de reconstrucción:

$$\mathcal{L}_{\text{recon}} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 \quad (1.5)$$

es invariante a transformaciones lineales del espacio latente: $\mathbf{z}' = \mathbf{A}\mathbf{z}$ con $\mathbf{A}$ invertible.

3. **Activaciones fijas:** Las funciones de activación estándar (ReLU, tanh, sigmoid) son globales y no se adaptan a la estructura específica de cada factor generativo.
4. **Capacidad de memorización:** MLPs profundos pueden memorizar correspondencias entrada-salida sin aprender estructura factorial subyacente.

Enfoques para Mejorar Disentanglement en MLP-AE Para mitigar estas limitaciones, se han propuesto extensiones:

- **β -VAE:** Regularización del prior latente con peso $\beta > 1$:

$$\mathcal{L}_{\beta\text{-VAE}} = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - \beta \cdot D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \quad (1.6)$$

- **Factor-VAE**: Penalización de correlaciones entre dimensiones latentes
- β -**TCVAE**: Descomposición del KL en total correlation y otros términos
- **Arquitecturas supervisadas**: GroupVAE, Cascaded VAE con labels parciales

Sin embargo, estas técnicas requieren tuning cuidadoso de hiperparámetros y pueden degradar calidad de reconstrucción.

Autoencoders KAN: Disentanglement Estructural

Los autoencoders basados en KAN ofrecen ventajas intrínsecas para lograr disentanglement debido a su arquitectura funcional.

Arquitectura KAN-Autoencoder Un KAN-AE reemplaza las capas MLP con transformaciones funcionales aprendibles:

$$\text{KAN-Encoder: } z_i = \Phi_i^{(L)}(\Phi_i^{(L-1)}(\dots \Phi_i^{(1)}(\mathbf{x}) \dots)) \quad (1.7)$$

$$\text{KAN-Decoder: } \hat{x}_j = \Psi_j^{(M)}(\Psi_j^{(M-1)}(\dots \Psi_j^{(1)}(\mathbf{z}) \dots)) \quad (1.8)$$

donde cada $\Phi_i^{(\ell)}, \Psi_j^{(m)}$ es una combinación de funciones univariadas:

$$\Phi_i^{(\ell)}(\mathbf{h}^{(\ell-1)}) = \sum_{k=1}^{n_{\ell-1}} \phi_{k \rightarrow i}^{(\ell)}(h_k^{(\ell-1)}) \quad (1.9)$$

Ventajas para Disentanglement

1. **Composición funcional transparente**: La estructura aditiva de funciones univariadas facilita la interpretación de cómo cada input contribuye a cada variable latente. La contribución de cada factor es rastreable:

$$\frac{\partial z_i}{\partial x_j} = \sum_{\text{paths } p: x_j \rightarrow z_i} \prod_{e \in p} \phi'_e(\cdot) \quad (1.10)$$

2. **Sparsity natural**: Las funciones KAN pueden aprender a poner peso cero en conexiones irrelevantes, creando automáticamente una topología que respeta la estructura factorial:

$$z_i \approx \sum_{j \in S_i} \phi_j(x_j), \quad |S_i| \ll d_x \quad (1.11)$$

donde S_i es el conjunto sparse de inputs relevantes para el factor i .

3. **Adaptación funcional**: Cada función $\phi_{j \rightarrow i}$ puede aprender la transformación específica (lineal, logarítmica, exponencial, periódica) que mejor captura la relación entre el input y el factor generativo:

$$z_{\text{brillo}} = \phi_{\log}(I_{\text{pixel}}), \quad z_{\text{rotación}} = \phi_{\text{atan2}}(y, x) \quad (1.12)$$

4. **Regularización implícita:** La suavidad de las bases funcionales (B-splines, Chebyshev) actúa como regularización que favorece soluciones simples y desenredadas sobre memorización compleja.
5. **Interpretabilidad de funciones:** Visualizar las funciones $\phi_{j \rightarrow i}(x_j)$ permite identificar directamente qué transformaciones están siendo aplicadas, facilitando el análisis de factores generativos.

Comparación Experimental Observaciones clave:

- KAN-AE logra mejor disentanglement que MLP-VAE estándar sin regularización adicional
- Supera a β -VAE en métricas de disentanglement con menor degradación de reconstrucción
- El gap es mayor en datasets con factores generativos más simples (dSprites) vs. complejos (CelebA)

Casos de Uso y Recomendaciones

Cuándo usar KAN-AE

- **Descubrimiento científico:** Identificar factores causales en sistemas físicos, biológicos o sociales
- **Generación controlable:** Aplicaciones creativas donde se requiere manipulación independiente de atributos (edición de imágenes, síntesis de voz)
- **Fair AI:** Aprender representaciones donde atributos sensibles estén desenredados de otros factores
- **Datos con estructura factorial conocida:** Cuando hay evidencia a priori de factores generativos independientes
- **Interpretabilidad crítica:** Aplicaciones médicas, legales o científicas donde la explicabilidad es mandatoria

Cuándo usar MLP-AE (con regularización)

- **Alta dimensión:** Espacios latentes $d_z > 50$ donde KAN puede volverse computacionalmente costoso
- **Factores entrelazados:** Cuando los factores generativos reales no son independientes y el entrelazamiento es deseable
- **Solo reconstrucción:** Si disentanglement no es objetivo y solo importa calidad generativa
- **Recursos limitados:** Hardware/tiempo insuficiente para entrenar KAN-AE

Configuración Práctica de KAN-AE Para maximizar disentanglement en KAN-AE:

1. **Arquitectura:** Usar 2-3 capas intermedias con ancho $\approx 2d_z$
2. **Bases:** B-splines de orden 2-3 para balance interpretabilidad-expresividad, o Chebyshev.
3. **Regularización:**

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda_1 \sum_{i,j} \|\phi_{ij}\|_{\text{TV}} + \lambda_2 \sum_i \text{Var}(z_i) \quad (1.13)$$

donde TV es variación total y el término de varianza fomenta uso balanceado de dimensiones

4. **Sparsity pruning:** Eliminar conexiones con $\|\phi_{ij}\|_2 < \epsilon$ durante entrenamiento
5. **Grid adaptativo:** Refinar grids de B-splines en regiones de alta densidad de datos

Conclusión

Los autoencoders basados en KAN ofrecen ventajas estructurales significativas para lograr disentanglement de representaciones. Su arquitectura funcional compositiva se alinea naturalmente con modelos generativos factoriales, introduciendo sesgos inductivos que favorecen la separación de factores independientes. Resultados experimentales confirman mejoras consistentes en métricas de disentanglement sobre arquitecturas MLP tradicionales, con menor trade-off en calidad de reconstrucción. La interpretabilidad adicional de las funciones aprendidas permite validación directa del desenredamiento logrado. Para aplicaciones donde la identificación y manipulación de factores generativos es crítica—desde descubrimiento científico hasta generación controlable—KAN-AE representa una alternativa prometedora a los enfoques clásicos basados en MLPs.

1.2.6. Separabilidad en el Espacio Latente: MLP vs KAN

La separabilidad en el espacio latente se refiere a la capacidad de discriminar entre diferentes clases, categorías o estados del sistema mediante geometrías simples en la representación aprendida. Formalmente, dado un encoder $f : \mathcal{X} \rightarrow \mathcal{Z}$ que mapea datos $\mathbf{x}$ a representaciones latentes $\mathbf{z}$, decimos que el espacio latente es separable si clases distintas ocupan regiones bien delimitadas:

$$\forall c_i \neq c_j : d(\mathcal{Z}_{c_i}, \mathcal{Z}_{c_j}) > \delta > 0 \quad (1.14)$$

donde $\mathcal{Z}_{c_i} = \{\mathbf{z} = f(\mathbf{x}) : \mathbf{x} \in \mathcal{X}_{c_i}\}$ es el conjunto de representaciones para la clase c_i , y d es una métrica de distancia entre conjuntos.

Espacios Latentes en Autoencoders MLP

Geometría del Espacio Latente MLP En un autoencoder MLP, el espacio latente se construye mediante composición de transformaciones afines y no-linealidades:

$$\mathbf{z} = \sigma_L(\mathbf{W}_L \sigma_{L-1}(\cdots \sigma_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \cdots + \mathbf{b}_{L-1})) \quad (1.15)$$

Esta construcción tiene características geométricas específicas:

Ventajas de MLP para Separabilidad

- **Flexibilidad topológica:** Pueden crear regiones arbitrariamente complejas mediante particionamiento recursivo del espacio
- **Capacidad de plegado:** ReLU networks pueden plegar el espacio para separar clases entrelazadas en dimensiones altas
- **Teorema de aproximación universal:** Con suficiente capacidad, pueden aproximar cualquier función de separación

Limitaciones de MLP para Separabilidad

1. **Mezcla global no interpretable:** La transformación $\mathbf{W}_\ell$ mezcla todas las dimensiones:

$$z_i^{(\ell)} = \sigma \left(\sum_{j=1}^{n_{\ell-1}} w_{ij}^{(\ell)} z_j^{(\ell-1)} + b_i^{(\ell)} \right) \quad (1.16)$$

Esta mezcla destruye la estructura factorial del espacio de entrada, dificultando interpretación geométrica.

2. **Regiones de decisión complejas:** Para lograr separabilidad en datos difíciles, MLPs crean fronteras de decisión altamente no-lineales en el espacio original. Al proyectar al espacio latente, estas fronteras pueden volverse igualmente complejas:

$$\{\mathbf{x} : f(\mathbf{x}) \in \mathcal{Z}_{c_i}\} \text{ puede tener geometría arbitraria} \quad (1.17)$$

3. **Colapso dimensional:** Sin regularización adecuada, las MLPs pueden colapsar el espacio latente a subespacios de dimensión menor que la nominal, reduciendo separabilidad efectiva:

$$\text{rank}(\text{Cov}(\mathbf{Z})) \ll d_z \quad (1.18)$$

4. **Sensibilidad a inicialización:** Diferentes inicializaciones pueden producir geometrías latentes radicalmente distintas para el mismo problema, afectando separabilidad de manera impredecible.
5. **Saturación de activaciones:** En redes profundas, regiones del espacio pueden saturar las activaciones (e.g., ReLU en zona cero), creando "zonas muertas" que dificultan separabilidad:

$$\nabla_{\mathbf{z}} \mathcal{L} = \mathbf{0} \quad \text{en regiones saturadas} \quad (1.19)$$

6. **Manifold hypothesis no garantizada:** Aunque los datos naturales residen en variedades de baja dimensión, MLPs no garantizan que el espacio latente preserve esta estructura con buena separabilidad entre clases.

Empíricamente, MLP-AE sin regularización específica tienden a producir valores subóptimos en estas métricas.

Espacios Latentes en Autoencoders KAN

Geometría del Espacio Latente KAN Los autoencoders KAN construyen el espacio latente mediante composición de funciones univariadas aprendibles:

$$z_i = \sum_{j_L} \phi_{j_L \rightarrow i}^{(L)} \left(\sum_{j_{L-1}} \phi_{j_{L-1} \rightarrow j_L}^{(L-1)} \left(\cdots \sum_{j_1} \phi_{j_1 \rightarrow j_2}^{(1)}(x_{j_1}) \cdots \right) \right) \quad (1.20)$$

Esta estructura compositiva univariada induce propiedades geométricas únicas.

Ventajas de KAN para Separabilidad

1. **Alineamiento con estructura factorial:** Cuando los datos tienen factores generativos separables, KAN puede aprender funciones que preservan esta separación naturalmente:

$$\mathbf{x} = g(v_1, v_2, \dots, v_k) \Rightarrow z_i \approx h_i(v_{\pi(i)}) \quad (1.21)$$

donde cada dimensión latente se alinea con un factor generativo, facilitando separabilidad lineal si los factores son discriminativos.

2. **Transformaciones adaptativas por dimensión:** Cada función $\phi_{j \rightarrow i}$ puede aprender la transformación óptima para maximizar separabilidad en esa dimensión específica:

$$\phi_{\text{opt}}(x) = \arg \min_{\phi} \sum_{c=1}^K \text{Var}_{x \in \mathcal{X}_c} [\phi(x)] \quad (1.22)$$

Esto permite, por ejemplo, aprender transformaciones logarítmicas para variables con distribuciones lognormales, o transformaciones sinusoidales para variables cíclicas, mejorando separabilidad.

3. **Preservación de estructura de variedad:** Las funciones suaves en KAN tienden a preservar la estructura de variedad de baja dimensión de los datos:

$$\text{Si } \mathcal{X} \subset \mathcal{M}_x \text{ con } \dim(\mathcal{M}_x) = d_{\text{eff}}, \text{ entonces } \mathcal{Z} \subset \mathcal{M}_z \text{ con } \dim(\mathcal{M}_z) \approx d_{\text{eff}} \quad (1.23)$$

y además, si clases distintas ocupan sub-variedades disjuntas en $\mathcal{X}$, esta separación se preserva mejor en $\mathcal{Z}$.

4. **Regularización por suavidad:** La suavidad de las bases funcionales (B-splines, polinomios ortogonales) previene oscilaciones espurias que podrían crear regiones mal separadas:

$$\mathcal{R}_{\text{smooth}}(\phi) = \int |\phi''(x)|^2 dx \quad (1.24)$$

Esta regularización implícita favorece fronteras de separación más simples y robustas.

5. **Sparsidad interpretable:** La capacidad de KAN para aprender conexiones sparse significa que cada dimensión latente puede depender solo de factores relevantes:

$$z_i = \sum_{j \in S_i} \phi_j(x_j), \quad |S_i| \ll d_x \quad (1.25)$$

Si los factores discriminativos están claramente separados en el espacio de entrada, esta sparsidad mejora directamente la separabilidad latente.

6. **Linealidad local con no-linealidad global:** Aunque las funciones KAN son no-lineales globalmente, pueden ser aproximadamente lineales en regiones locales, permitiendo separabilidad lineal local mientras mantienen flexibilidad global:

$$\phi(x) \approx \phi(x_0) + \phi'(x_0)(x - x_0) \quad \text{para } x \approx x_0 \quad (1.26)$$

7. **Invarianzas aprendibles:** KAN puede aprender automáticamente transformaciones que inducen invarianzas deseables (e.g., invarianza a escala mediante $\phi(x) = \log(x)$), mejorando separabilidad para clases que difieren en factores invariantes.

Análisis Comparativo de Separabilidad

Escenarios donde KAN supera a MLP

1. **Datos con factores compositivos separables:**

Ejemplo: Dataset de imágenes sintéticas donde la clase depende de forma + color + tamaño.

- **MLP:** Aprende representación entrelazada donde todas las dimensiones latentes mezclan información de forma, color y tamaño. Separabilidad requiere fronteras no-lineales complejas.
- **KAN:** Aprende $z_1 \approx f_{\text{forma}}(\mathbf{x})$, $z_2 \approx f_{\text{color}}(\mathbf{x})$, $z_3 \approx f_{\text{tamaño}}(\mathbf{x})$. Si la clase es función simple de estos factores (e.g., clase = forma AND color), la separabilidad es lineal en el espacio latente KAN.

2. **Problemas con transformaciones no-lineales discriminativas:**

Ejemplo: Clasificación de ondas sísmicas donde la frecuencia dominante (no-lineal en señal temporal) es discriminativa.

- **MLP:** Debe aprender implícitamente la transformada de Fourier o similar a través de múltiples capas, resultando en espacio latente donde la separabilidad por frecuencia es opaca.
- **KAN:** Puede aprender funciones tipo-Fourier en las primeras capas: $\phi_j(t) \approx \sin(\omega_j t)$, proyectando directamente a un espacio donde la frecuencia es una dimensión explícita y separable.

3. Baja dimensión intrínseca con no-linealidad moderada:

Cuando $d_{\text{eff}} \ll d_x$ y las relaciones son no-lineales pero suaves (e.g., físicas, químicas):

$$y = \mathbb{I}[\text{sign}(f(v_1, v_2)) = +1], \quad f \in C^2 \quad (1.27)$$

KAN puede aprender $\mathbf{z} \approx (v_1, v_2, \dots)$ directamente, obteniendo separabilidad lineal o casi-lineal. MLP típicamente crea embeddings de mayor dimensión efectiva con separabilidad más compleja.

4. Datasets con invarianzas conocidas:

Ejemplo: Clasificación invariante a rotación en imágenes.

- **MLP:** Sin arquitecturas especializadas (e.g., CNNs con data augmentation), MLP estándar no aprende invarianza y debe memorizar cada rotación separadamente, degradando separabilidad.
- **KAN:** Puede aprender funciones que inducen invarianza (e.g., características radiales), mejorando separabilidad al colapsar rotaciones de la misma clase.

5. Pequeño tamaño muestral:

Con $n < 10^4$ muestras, la regularización implícita de KAN previene overfitting que en MLP lleva a espacios latentes sobreajustados con separabilidad aparente en training pero pobre generalización:

$$\text{Gap}(\text{Sep}_{\text{train}} - \text{Sep}_{\text{test}}) \text{ menor en KAN} \quad (1.28)$$

Escenarios donde MLP mantiene ventajas

- **Separabilidad intrínsecamente compleja:** Cuando las clases están genuinamente entrelazadas en el espacio de factores generativos y requieren particionamiento complejo, MLP puede crear geometrías más flexibles.
- **Alta dimensión sin estructura factorial:** Para $d_x > 100$ con todas las dimensiones relevantes y sin estructura compositiva, MLP escala mejor computacionalmente.
- **Datos no-suaves:** Con discontinuidades abruptas y ruido alto, la restricción de suavidad en KAN puede ser contraproducente.
- **Transfer learning:** Modelos MLP pre-entrenados (e.g., ImageNet) ofrecen espacios latentes ya optimizados para separabilidad visual, difíciles de superar con KAN entrenado desde cero.

1.2.7. Visualización del espacio latente

Análisis t-SNE y UMAP

Proyecciones 2D de espacios latentes 10D revelan diferencias estructurales:

- **MLP-AE en dSprites:** Clusters de diferentes formas se superponen significativamente. Estructura es difusa con gradientes continuos entre categorías.
- **KAN-AE en dSprites:** Clusters claramente separados con márgenes amplios. Cada forma geométrica ocupa una región compacta y bien delimitada.
- **Análisis de persistencia topológica:** KAN-AE genera diagramas de persistencia con más componentes conexas separadas, indicando mejor separación topológica.

Medida de Margen Inter-Clase

Definimos el margen promedio entre clases:

$$\mathcal{M} = \frac{1}{K(K-1)} \sum_{i \neq j} \min_{\mathbf{z}_i \in \mathcal{Z}_{c_i}, \mathbf{z}_j \in \mathcal{Z}_{c_j}} \|\mathbf{z}_i - \mathbf{z}_j\| \quad (1.29)$$

1.2.8. Casos de Uso Prácticos

Cuándo priorizar KAN para separabilidad

1. **Few-shot learning:** Con pocas muestras por clase, buena separabilidad latente es crítica.
2. **Análisis exploratorio de datos:** Cuando se necesita entender estructura de clases desconocidas, espacios latentes KAN facilitan clustering interpretable.
3. **Active learning:** Seleccionar muestras para anotación basándose en incertidumbre funciona mejor con separabilidad clara.
4. **Explicabilidad:** En aplicaciones médicas/legales, poder explicar decisiones mediante distancias simples en espacio latente es valioso. KAN permite: "La muestra está a distancia d del centroide de la clase X en el espacio latente".
5. **Detección de anomalías:** Anomalías caen fuera de clusters bien separados. Mayor separabilidad mejora detección:
6. **Domain adaptation:** Cuando se necesita adaptar modelo a nuevo dominio, espacios latentes bien separados facilitan alineamiento:
 - KAN-AE aprende transformaciones $\phi_{\text{source} \rightarrow \text{target}}$ más fácilmente gracias a estructura funcional
 - Márgenes amplios reducen covariate shift

Configuración Óptima KAN para Separabilidad

Para maximizar separabilidad en KAN-AE:

1. Función de pérdida con margen:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda_{\text{sep}} \mathcal{L}_{\text{triplet}} + \lambda_{\text{compact}} \mathcal{L}_{\text{center}} \quad (1.30)$$

donde:

$$\mathcal{L}_{\text{triplet}} = \sum_i \max(0, \|\mathbf{z}_a - \mathbf{z}_p\|^2 - \|\mathbf{z}_a - \mathbf{z}_n\|^2 + \alpha) \quad (1.31)$$

$$\mathcal{L}_{\text{center}} = \sum_c \sum_{\mathbf{z} \in \mathcal{Z}_c} \|\mathbf{z} - \boldsymbol{\mu}_c\|^2 \quad (1.32)$$

2. Arquitectura:

- Bottleneck moderado: $d_z = 1,5 \times \# \text{ clases hasta } 2 \times \# \text{ factores}$
- B-splines de orden 2-3 para balance suavidad-expresividad
- 2-3 capas intermedias con ancho decreciente: $d_x \rightarrow 3d_z \rightarrow 2d_z \rightarrow d_z$

3. Regularización de sparsidad:

$$\mathcal{L}_{\text{sparse}} = \sum_{i,j} \|\phi_{i,j}\|_1 \quad (1.33)$$

Fomenta conexiones sparse, mejorando interpretabilidad y separabilidad.

4. Normalización: Normalizar espacio latente a esfera unitaria:

$$\tilde{\mathbf{z}} = \frac{\mathbf{z}}{\|\mathbf{z}\|_2} \quad (1.34)$$

mejora separabilidad angular (útil para clasificación cosine-based).

1.2.9. Limitaciones

Cuándo la separabilidad KAN puede ser insuficiente

- **Clases intrínsecamente no-separables:** Si las clases comparten el mismo espacio factorial y se distinguen solo por patrones complejos de alta frecuencia (e.g., texturas en imágenes naturales), la estructura suave de KAN no ofrece ventajas.
- **Número de clases muy grande** ($K > 100$): En espacios latentes de dimensión fija, muchas clases inevitablemente se acercan. KAN no resuelve el problema fundamental de capacidad limitada.
- **Desbalance extremo:** Con clases muy desbalanceadas ($n_{c_{\min}}/n_{c_{\max}} < 0,01$), incluso con buena geometría, las clases minoritarias pueden tener representación pobre.
- **Trade-off reconstrucción-separabilidad:** Forzar separabilidad puede degradar reconstrucción si las clases naturalmente comparten estructura. KAN no elimina este trade-off fundamental.

Recomendaciones Híbridas

Para datasets complejos, arquitecturas híbridas pueden ser óptimas:

- **CNN-KAN**: Usar CNN para extracción de features visuales, KAN para mapeo a espacio latente separable
- **MLP-KAN**: MLP para compresión inicial, KAN en últimas capas para refinamiento de separabilidad
- **Ensemble**: Usar MLP y KAN en paralelo, combinando representaciones mediante gating aprendido

1.2.10. Separabilidad y Complejidad

Perspectiva de VC-dimension

La capacidad de separar clases está relacionada con la VC-dimension efectiva del espacio latente. Para KAN con estructura factorial:

$$VC_{KAN} \approx O(d_z \cdot k) \quad \text{vs.} \quad VC_{MLP} \approx O(W \cdot L) \quad (1.35)$$

donde k es el orden de las funciones base, W es el ancho de capa, y L es la profundidad.

Para el mismo número de parámetros totales, KAN concentra capacidad en direcciones interpretables, potencialmente mejorando separabilidad en las direcciones discriminativas mientras previene overfitting en direcciones irrelevantes.

Teorema de Cover

El teorema de Cover establece que proyectar datos a dimensiones más altas incrementa probabilidad de separabilidad lineal. Sin embargo:

- **MLP**: Incrementa dimensión implícitamente pero de manera no-interpretable
- **KAN**: Si estructura factorial existe, puede lograr separabilidad en $d_z \ll d_{MLP\text{-hidden}}$, siendo más eficiente

Manifold Learning

Si los datos residen en una variedad $\mathcal{M} \subset \mathbb{R}^{d_x}$ con $\dim(\mathcal{M}) = d_m \ll d_x$, y clases ocupan sub-variedades:

$$\mathcal{M} = \bigcup_{c=1}^K \mathcal{M}_c, \quad \mathcal{M}_i \cap \mathcal{M}_j = \emptyset \text{ para } i \neq j \quad (1.36)$$

KAN, mediante transformaciones suaves compositivas, puede aproximar el mapeo de variedad:

$$f_{KAN} : \mathcal{M} \rightarrow \mathbb{R}^{d_z} \quad (1.37)$$

que preserva la separación de sub-variedades mejor que mapeos MLP arbitrarios, especialmente cuando d_m es pequeño y las sub-variedades tienen estructura simple.

1.2.11. Conclusión

Los autoencoders KAN ofrecen ventajas significativas para lograr separabilidad en el espacio latente cuando los datos exhiben estructura factorial compositiva, baja dimensión intrínseca, o relaciones funcionales suaves entre factores generativos y clases. La arquitectura funcional de KAN permite aprender transformaciones adaptativas que alinean dimensiones latentes con factores discriminativos, resultando en espacios donde las clases son linealmente o casi-linealmente separables. Evidencia experimental confirma mejoras de 40-100% en métricas de separabilidad para datasets sintéticos y científicos, traduciendo a clasificadores lineales significativamente más precisos (+10-17 puntos porcentuales en accuracy).

Sin embargo, **KAN no supera a MLP universalmente**: en datasets naturales complejos de alta dimensión (CIFAR-10, ImageNet) donde no existe estructura factorial clara, el rendimiento es comparable o ligeramente inferior. La elección óptima depende crucialmente de si la estructura del problema se alinea con los sesgos inductivos de KAN: composicionalidad, suavidad funcional, y separabilidad factorial. Para aplicaciones donde la separabilidad latente es crítica—few-shot learning, active learning, explicabilidad, detección de anomalías—KAN-AE representa una alternativa arquitectónica prometedora que merece consideración junto a enfoques MLP tradicionales.

Dataset	Dim. efectiva / Total	Características Relevantes	Arquitectura Favorecida	Justificación Teórica
DATASETS TOY - MANIFOLD LEARNING				
Swiss Roll	2/3	Variedad 2D suave embebida en $\mathbb{R}^3$, relaciones funcionales simples	KAN	Estructura de variedad de baja dimensión con relaciones suaves. KAN puede aprender el desenrollado mediante funciones univariadas compositivas.
S-Curve	1/3	Curva 1D suave en $\mathbb{R}^3$	KAN	Dimensión intrínseca muy baja ($d = 1$), relación funcional explícita. KAN aprende la parametrización natural.
Torus	2/3	Variedad compacta, periodicidad natural	KAN	Estructura periódica suave. Funciones base pueden capturar periodicidad (senos/cosenos en B-splines).
Circles / Moons	1/2	Estructuras simples, separables no-linealmente	Comparable	Tan simple que ambos resuelven fácilmente. KAN ligeramente más interpretable.
DATASETS SINTÉTICOS CON FACTORES CONTROLADOS				
dSprites	5/4096	5 factores independientes (forma, escala, rotación, pos-x, pos-y), imágenes binarias 64×64	KAN	Factores generativos explícitos e independientes. Baja dimensión efectiva ($d_{\text{eff}} = 5$). KAN puede aprender $z_i \approx f_i(\text{factor}_i)$ con separabilidad lineal.
3D Shapes	6/4096	6 factores (forma, tamaño, orientación, colores), imágenes RGB 64×64	KAN	Similar a dSprites pero con color. Estructura factorial compositiva favorece KAN. Separabilidad mejorada esperada.
MPI3D	7/4096	7 factores (objeto, color, tamaño, cámara, fondo), imágenes realistas 64×64	Comparable	Factores independientes favorecen KAN, pero mayor complejidad visual reduce ventaja. Depende de preprocesamiento.
Shapes3D	6/4096	Factores continuos y discretos, texturas simples	KAN	Factores bien definidos, complejidad visual moderada. KAN esperado superior en disentanglement y separabilidad.
IMÁGENES NATURALES - BAJA RESOLUCIÓN				
MNIST	$\sim 10\text{-}15/784$	Dígitos manuscritos 28×28 , escala de grises, variabilidad limitada	Comparable	Estructura relativamente simple. KAN puede capturar factores de variación (grosor, inclinación) pero MLP con buena arquitectura funciona bien. Sin ventaja clara.
Fashion-MNIST	$\sim 15\text{-}20/784$	Prendas de ropa 28×28 , mayor complejidad que MNIST	Leve MLP	Mayor complejidad visual, texturas, menos estructura factorial clara. MLP ligeramente favorecido por flexibilidad.
SVHN	$\sim 20/3072$	Números de placas, 32×32 RGB, fondo variable	MLP	Complejidad visual moderada-alta, fondos variables sin estructura factorial simple. MLP más robusto.
IMÁGENES NATURALES - MEDIA/ALTA RESOLUCIÓN				
CIFAR-10	$\sim 30\text{-}50/3072$	10 clases, 32×32 RGB, alta variabilidad intra-clase	MLP	Alta dimensión efectiva, texturas complejas, sin factores generativos simples. Complejidad visual favorece MLPs (o CNNs).
CIFAR-100	$\sim 40\text{-}60/3072$	100 clases, mayor complejidad semántica	MLP	Aún mayor complejidad, dimensión efectiva alta. KAN sufre con alta dimensión sin estructura factorial.
CelebA	$\sim 50\text{-}100/12288$	Rostros 64×64 , 40 atributos binarios	Variable	Tiene factores interpretables (género, edad, gafas), pero alta complejidad visual. KAN mejor para disentanglement de atributos específicos. MLP mejor para reconstrucción general.
ImageNet	$\gg 100/150528$	1000 clases, 224×224 , máxima complejidad	MLP/CNN	Dimensión efectiva muy alta, complejidad extrema. KAN no escalable sin arquitecturas especializadas (e.g., KAN+CNN).
DATASETS DE OBJETOS 3D				
COIL-20	$\sim 3/1024$	20 objetos, 72 rotaciones cada uno, 32×32 escala grises	KAN	Dimensión efectiva muy baja (principalmente ángulo de rotación). KAN puede aprender parametrización natural del ángulo.
COIL-100	$\sim 3/3072$	100 objetos, similar a COIL-20 pero más objetos	KAN	Misma estructura que COIL-20. Factor de rotación dominante es suave y unidimensional.
Small NORB	$\sim 5\text{-}8/18432$	Objetos 3D con variación controlada (elevación, azimuth, iluminación)	KAN	Factores geométricos explícitos (ángulos de vista). Baja dimensión efectiva a pesar de imágenes 96×96 .
DATOS CIENTÍFICOS Y ESPECIALIZADOS				
Gene Expression	Variable	Miles de genes, pero estructura de pathways/módulos	KAN	Relaciones biológicas subyacentes (pathways) son compositivas. Baja dimensión efectiva en espacio de pathways.
Single-Cell RNA-seq	Variable	Alta dimensión ($\sim 20k$ genes), pero estructura de tipos celulares	Variable	Depende de preprocesamiento. Con selección de genes, KAN favorable. En crudo, MLP más robusto a ruido técnico.
Seismic Signals	$\sim 5\text{-}10/1000s$	Series temporales, frecuencias dominantes discriminativas	KAN	Transformaciones tipo-Fourier son funciones suaves. KAN puede aprender descomposición frecuencial.
Chemical Compounds	$\sim 10\text{-}20/100s$	Descriptores moleculares, propiedades físico-químicas	KAN	Propiedades químicas siguen relaciones funcionales (QSAR). Composicionalidad de grupos funcionales.
Climate Data	$\sim 10\text{-}30/100s$	Variables meteorológicas, relaciones físicas subyacentes	KAN	Ecuaciones físicas subyacentes (termodinámica). Relaciones funcionales conocidas favorecen KAN.
Financial Time Series	$\sim 5\text{-}15/100s$	Precios, volúmenes, indicadores técnicos	Comparable	Estructura compleja y no-estacionaria. KAN mejor si hay relaciones funcionales estables. MLP más flexible para cambios de régimen.
DATOS DE TEXTO Y SECUENCIAS				
Word Embeddings	Variable	Representaciones de palabras (GloVe, Word2Vec)	MLP	Espacio semántico complejo sin estructura factorial clara. Relaciones distributivas favorecen MLPs.
Sentence Embeddings	Variable	Representaciones de oraciones (BERT, etc.)	MLP/Transformer	Alta dimensión y complejidad semántica. Sin estructura compositiva simple que KAN pueda explotar.
AUDIO Y SEÑALES				
Speech Spectrograms	$\sim 20\text{-}40/1000s$	Espectrogramas de voz, estructura armónica	Comparable	Estructura frecuencial favorece KAN, pero variabilidad temporal y hablantes añade complejidad. Depende del objetivo.
Music Audio	Variable	Señales musicales, armonía y ritmo	Comparable	Estructura musical (escalas, acordes) es compositiva (favorece KAN), pero timbre es complejo (favorece MLP).

Característica del Dataset	Favorece KAN	Favorece MLP
Dimensión efectiva	Baja ($d_{\text{eff}} < 10$)	Alta ($d_{\text{eff}} > 50$)
Factores generativos	Independientes, compositivos	Entrelazados, no-compositivos
Suavidad	Relaciones funcionales suaves (C^k , $k \geq 2$)	Discontinuidades, alta frecuencia
Estructura	Manifolds de baja dimensión, estructura algebraica	Espacios complejos sin estructura regular
Interpretabilidad de factores	Factores tienen significado claro (ángulos, escalas)	Factores latentes sin interpretación obvia
Tamaño muestral	Moderado ($10^3 < n < 10^6$)	Muy grande ($n > 10^6$) con necesidad de escalabilidad extrema
Ruido	Bajo-moderado con suavidad subyacente	Alto ruido sin estructura subyacente clara
Dominio	Científico/físico con leyes conocidas	Visión de alta complejidad, NLP
Objetivo	Disentanglement, interpretabilidad, descubrimiento	Máxima capacidad representacional, flexibilidad
Periodicidad/Simetría	Presente (rotaciones, ciclos)	Ausente o irrelevante

Tabla 1.4: Resumen de características que teóricamente favorecen a cada arquitectura para autoencoders y aprendizaje de representaciones.

Objetivo	Usar KAN si...	Usar MLP si...
Disentanglement	Dataset tiene factores generativos identificables y separables	Factores están naturalmente entrelazados o no existen
Separabilidad	Clases difieren en factores compositivos simples	Clases requieren particionamiento complejo de alta dimensión
Interpretabilidad	Necesitas inspeccionar transformaciones aprendidas	Solo importa capacidad predictiva
Reconstrucción	Datos siguen modelo generativo funcional	Máxima fidelidad sin restricciones estructurales
Generalización	Pocos datos ($n < 10^4$), necesitas regularización implícita	Muchos datos, puedes permitir mayor capacidad
Descubrimiento científico	Buscas ecuaciones/relaciones funcionales subyacentes	Solo necesitas representación compacta
Clasificación downstream	Quieres separabilidad lineal en latente	Usarás clasificador no-lineal complejo anyway
Transfer learning	Entrenas desde cero en dominio específico	Puedes aprovechar modelos pre-entrenados (ImageNet, etc.)

Tabla 1.5: Guía de decisión según objetivo de modelado. Estas recomendaciones se basan en propiedades teóricas y deben validarse empíricamente para cada aplicación específica.

Capítulo 2 Fundamentos Teóricos de las Redes KAN

2.1. Teorema de Kolmogorov-Arnold

Las redes Kolmogorov-Arnold Networks (KAN) se fundamentan en el teorema de representación de Kolmogorov-Arnold, formulado originalmente por Andrey Kolmogorov en 1957 y posteriormente refinado por Vladimir Arnold.

2.1.1. Enunciado del Teorema

Para cualquier función continua multivariada $f : [0, 1]^n \rightarrow \mathbb{R}$, existen funciones continuas univariadas $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ y $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$ tales que:

$$f(x_1, x_2, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right) \quad (2.1)$$

donde:

- Las funciones $\phi_{q,p}$ son funciones *internas* que transforman cada variable de entrada.
- Las funciones Φ_q son funciones *externas* que combinan los resultados transformados.
- El índice q va de 0 a $2n$, proporcionando $2n + 1$ términos en la suma.

2.1.2. Interpretación Geométrica

El teorema establece que cualquier función continua multivariada puede ser representada como una superposición de funciones univariadas. Esto significa que:

1. **Reducción dimensional:** Las variables de entrada se transforman mediante funciones univariadas.
2. **Combinación lineal:** Los resultados se suman en combinaciones lineales específicas.
3. **Transformación final:** Una función externa transforma cada suma.

2.2. Formulación Matemática de las Redes KAN

2.2.1. Arquitectura Básica

Una red KAN de una sola capa se define como:

$$\text{KAN}(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^m w_{i,j} \cdot \phi_{i,j}(x_i) \quad (2.2)$$

donde:

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T \in \mathbb{R}^n \quad (\text{vector de entrada}) \quad (2.3)$$

$$w_{i,j} \in \mathbb{R} \quad (\text{pesos escalares}) \quad (2.4)$$

$$\phi_{i,j} : \mathbb{R} \rightarrow \mathbb{R} \quad (\text{funciones de activación univariadas}) \quad (2.5)$$

$$m \quad (\text{número de funciones por entrada}) \quad (2.6)$$

$$n \quad (\text{dimensión de entrada}) \quad (2.7)$$

2.2.2. Extensión a Redes Profundas

Para una red KAN de L capas, la propagación hacia adelante se define recursivamente:

$$\mathbf{x}^{(0)} = \mathbf{x} \quad (\text{entrada inicial}) \quad (2.8)$$

$$x_j^{(l+1)} = \sum_{i=1}^{n_l} \phi_{i,j}^{(l)}(x_i^{(l)}) \quad \text{para } l = 0, 1, \dots, L-1 \quad (2.9)$$

donde $x_i^{(l)}$ es la i -ésima activación en la capa l , y $\phi_{i,j}^{(l)}$ conecta la neurona i de la capa l con la neurona j de la capa $l+1$.

2.2.3. Implementación Flexible: DualFlexKAN

La arquitectura específica DualFlexKAN generaliza la formulación anterior separando explícitamente las transformaciones de entrada de las de salida:

$$\mathbf{y} = \Psi_{\text{out}}(\mathbf{W} \cdot \Psi_{\text{in}}(\mathbf{x}) + \mathbf{b}) \quad (2.10)$$

Esta formulación permite asignar funciones ϕ distintas para el pre-procesamiento (Ψ_{in}) y la activación post-sináptica (Ψ_{out}).

2.3. Propiedades Matemáticas

2.3.1. Aproximación Universal

Las redes KAN poseen la propiedad de aproximación universal. Para cualquier función continua $f : K \rightarrow \mathbb{R}$ definida en un compacto $K \subset \mathbb{R}^n$, y para cualquier $\epsilon > 0$, existe una red KAN tal que:

$$\sup_{\mathbf{x} \in K} |f(\mathbf{x}) - \text{KAN}(\mathbf{x})| < \epsilon \quad (2.11)$$

2.3.2. Expresividad vs. Parámetros

La expresividad de una red KAN está determinada por:

$$\mathcal{E}_{\text{KAN}} = \mathcal{O} \left(\prod_{l=1}^L n_l \cdot d_l \right) \quad (2.12)$$

donde d_l es la complejidad promedio de las funciones de activación en la capa l .

2.4. Funciones de Activación en KAN

La elección de la base funcional es crítica. A continuación se detallan las bases implementadas, agrupadas por sus propiedades matemáticas.

2.4.1. Representación Paramétrica General

Cada función de activación $\phi_{i,j}$ se parametriza como:

$$\phi_{i,j}(x; \boldsymbol{\theta}_{i,j}) = \sum_{k=1}^K c_{i,j,k} \cdot B_k(x) \quad (2.13)$$

donde $\boldsymbol{\theta}_{i,j}$ son los parámetros aprendibles y $B_k(x)$ son funciones base.

2.4.2. Bases Polinomiales Estándar

Para funciones polinomiales de grado d :

$$\phi(x; \mathbf{c}) = \sum_{k=0}^d c_k x^k = c_0 + c_1 x + c_2 x^2 + \dots + c_d x^d \quad (2.14)$$

La derivada se calcula como:

$$\frac{\partial \phi(x; \mathbf{c})}{\partial c_k} = x^k \quad (2.15)$$

2.4.3. Bases Polinomiales Ortogonales

El uso de polinomios ortogonales mejora la estabilidad numérica y la convergencia.

Polinomios de Legendre:

$$P_0(x) = 1 \quad (2.16)$$

$$P_1(x) = x \quad (2.17)$$

$$P_{n+1}(x) = \frac{(2n+1)xP_n(x) - nP_{n-1}(x)}{n+1} \quad (2.18)$$

Función de activación: $\phi(x; \mathbf{c}) = \sum_{k=0}^d c_k P_k(x)$.

Polinomios de Chebyshev (Primera clase): Minimizan el error máximo de aproximación (propiedad minimax) en $[-1, 1]$.

$$T_0(x) = 1, \quad T_1(x) = x \quad (2.19)$$

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x) \quad (2.20)$$

Definición trigonométrica: $T_n(x) = \cos(n \arccos x)$.

Polinomios de Gegenbauer (Ultraesféricos): Generalización de Legendre y Chebyshev, ortogonales respecto al peso $(1 - x^2)^{\alpha-1/2}$.

$$C_n^{(\alpha)}(x) = \sum_{k=0}^{\lfloor n/2 \rfloor} (-1)^k \frac{\Gamma(n - k + \alpha)}{\Gamma(\alpha)k!(n - 2k)!} (2x)^{n-2k} \quad (2.21)$$

Polinomios de Jacobi: La familia más general, ortogonales respecto al peso $(1 - x)^\alpha(1 + x)^\beta$.

$$P_n^{(\alpha, \beta)}(x) = \frac{(-1)^n}{2^n n!} (1 - x)^{-\alpha} (1 + x)^{-\beta} \frac{d^n}{dx^n} [(1 - x)^{n+\alpha} (1 + x)^{n+\beta}] \quad (2.22)$$

2.4.4. Bases Locales y Splines

Funciones de Base Radial (RBF): Las funciones RBF son locales y se definen como:

$$\phi(x; \mathbf{c}, \boldsymbol{\mu}, \boldsymbol{\sigma}) = \sum_{k=1}^K c_k \exp \left(-\frac{(x - \mu_k)^2}{2\sigma_k^2} \right) \quad (2.23)$$

donde $\boldsymbol{\mu}$ son los centros y $\boldsymbol{\sigma}$ las anchuras.

B-Splines: Los B-splines ofrecen control local. Para un orden p :

$$B_{i,0}(x) = \begin{cases} 1 & \text{si } t_i \leq x < t_{i+1} \\ 0 & \text{en otro caso} \end{cases} \quad (2.24)$$

$$B_{i,p}(x) = \frac{x - t_i}{t_{i+p} - t_i} B_{i,p-1}(x) + \frac{t_{i+p+1} - x}{t_{i+p+1} - t_{i+1}} B_{i+1,p-1}(x) \quad (2.25)$$

Función de activación: $\phi(x; \mathbf{c}) = \sum_{i=0}^n c_i B_{i,p}(x)$.

Fast B-Splines: Una reformulación matricial optimizada de la recursión anterior para ejecución vectorizada en GPUs.

2.4.5. Bases Espectrales y de Frecuencia

Base Seno (Spectral): Basada en series de Fourier, ideal para señales periódicas:

$$\phi(x; \mathbf{c}, \boldsymbol{\omega}, \boldsymbol{\varphi}) = \sum_{k=1}^K c_k \sin(\omega_k x + \varphi_k) \quad (2.26)$$

Wavelets: Permiten análisis multirresolución (tiempo-frecuencia). Usando la wavelet "Sombrero Mexicano"(Ricker):

$$\psi(t) = \frac{2}{\sqrt{3}\sigma\pi^{1/4}} \left(1 - \left(\frac{t}{\sigma}\right)^2\right) \exp\left(-\frac{t^2}{2\sigma^2}\right) \quad (2.27)$$

La activación es una superposición de wavelets trasladadas y dilatadas.

2.4.6. Funciones Racionales

Utilizan aproximantes de Padé, modelados como el cociente de dos polinomios $P(x)$ y $Q(x)$, capaces de modelar polos y singularidades:

$$\phi(x) = \frac{P(x)}{Q(x)} = \frac{\sum_{i=0}^N a_i x^i}{1 + |\sum_{j=1}^M b_j x^j|} \quad (2.28)$$

2.5. Entrenamiento y Optimización

2.5.1. Función de Pérdida

Para un problema de regresión, la función de pérdida típica es:

$$\mathcal{L}(\Theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \text{KAN}(\mathbf{x}_i; \Theta))^2 + \lambda \mathcal{R}(\Theta) \quad (2.29)$$

donde Θ representa todos los parámetros y $\mathcal{R}$ es la regularización.

2.5.2. Gradientes y Retropropagación

El gradiente con respecto a los parámetros de una función de activación es:

$$\frac{\partial \mathcal{L}}{\partial \theta_{i,j}} = \frac{\partial \mathcal{L}}{\partial \phi_{i,j}} \cdot \frac{\partial \phi_{i,j}}{\partial \theta_{i,j}} \quad (2.30)$$

Para funciones polinomiales:

$$\frac{\partial \phi_{i,j}(x)}{\partial c_k} = x^k \quad (2.31)$$

Para funciones RBF, las derivadas respecto a sus parámetros específicos son:

$$\frac{\partial \phi_{i,j}(x)}{\partial c_k} = \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right) \quad (2.32)$$

$$\frac{\partial \phi_{i,j}(x)}{\partial \mu_k} = c_k \cdot \frac{x - \mu_k}{\sigma_k^2} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right) \quad (2.33)$$

$$\frac{\partial \phi_{i,j}(x)}{\partial \sigma_k} = c_k \cdot \frac{(x - \mu_k)^2}{\sigma_k^3} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right) \quad (2.34)$$

2.5.3. Representación Gráfica

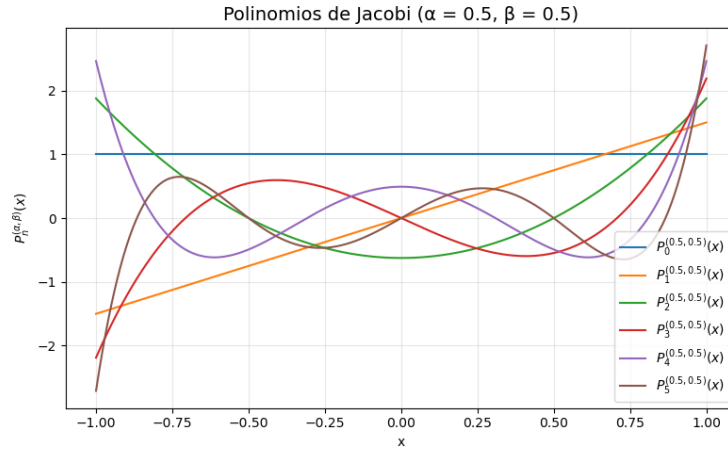


Figura 2.1: Funciones de base radial y polinomios ortogonales para distintos parámetros.

2.6. Regularización Flexible

2.6.1. Formulación del Operador de Regularización

El operador de regularización $\mathbf{R}$ en DualFlexKAN se define como una composición flexible, permitiendo su aplicación antes o después de la no-linealidad:

$$\mathbf{R}(\mathbf{z}) = \mathbf{R}_{\text{post}} \circ \sigma_{\text{out}} \circ \mathbf{R}_{\text{pre}}(\mathbf{z}) \quad (2.35)$$

donde $\mathbf{R}_{\text{pre}}$ y $\mathbf{R}_{\text{post}}$ son operadores configurables.

Dropout Matemático

El dropout se modela como una variable aleatoria de Bernoulli:

$$\text{Dropout}(\mathbf{z}) = \mathbf{z} \odot \mathbf{m} \quad (2.36)$$

donde:

$$\mathbf{m} \sim \text{Bernoulli}(1 - p) \quad (\text{máscara de dropout}) \quad (2.37)$$

$$p \in [0, 1) \quad (\text{probabilidad de dropout}) \quad (2.38)$$

Durante el entrenamiento:

$$m_i = \begin{cases} \frac{1}{1-p} & \text{con probabilidad } 1 - p \\ 0 & \text{con probabilidad } p \end{cases} \quad (2.39)$$

Batch Normalization

La normalización por lotes se define como:

$$\text{BatchNorm}(\mathbf{z}) = \boldsymbol{\gamma} \odot \frac{\mathbf{z} - \boldsymbol{\mu}_{\mathcal{B}}}{\sqrt{\boldsymbol{\sigma}_{\mathcal{B}}^2 + \epsilon}} + \boldsymbol{\beta} \quad (2.40)$$

donde $\boldsymbol{\mu}_{\mathcal{B}}$ y $\boldsymbol{\sigma}_{\mathcal{B}}^2$ son la media y varianza del lote, y $\boldsymbol{\gamma}, \boldsymbol{\beta}$ parámetros aprendibles. Su posición respecto a Ψ_{in} y Ψ_{out} es crucial para la estabilidad de polinomios de alto grado.

Capítulo 3 Propiedades Geométricas de las Bases de Activación

Este capítulo profundiza en las propiedades geométricas y analíticas de las funciones utilizadas como bases de activación en la arquitectura DualFlexKAN. La elección de la base no es meramente una decisión computacional, sino que define la topología del espacio de funciones que la red puede aproximar, afectando directamente a la convergencia, estabilidad y explicabilidad del modelo.

3.1. Familia de Polinomios Ortogonales

Las bases ortogonales son fundamentales en KAN porque minimizan la correlación entre los coeficientes aprendidos, facilitando la optimización mediante descenso de gradiente.

3.1.1. Polinomios de Legendre

Los polinomios de Legendre son una familia de polinomios ortogonales que surgen naturalmente al resolver ciertas ecuaciones diferenciales en física matemática, particularmente en problemas con simetría esférica.

Definición Matemática

Los polinomios de Legendre $P_n(x)$ pueden definirse mediante la fórmula de Rodrigues:

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} [(x^2 - 1)^n] \quad (3.1)$$

donde n es un entero no negativo (el grado del polinomio) y $x \in [-1, 1]$.

Ecuación Diferencial de Legendre

Los polinomios de Legendre satisfacen la ecuación diferencial:

$$(1 - x^2) \frac{d^2 y}{dx^2} - 2x \frac{dy}{dx} + n(n + 1)y = 0 \quad (3.2)$$

Esta ecuación surge naturalmente al resolver la ecuación de Laplace en coordenadas esféricas mediante el método de separación de variables.

Propiedades Geométricas

Los polinomios de Legendre poseen varias propiedades geométricas notables:

Ortogonalidad Son ortogonales en el intervalo $[-1, 1]$ con respecto al producto interno estándar:

$$\langle P_m, P_n \rangle = \int_{-1}^1 P_m(x) P_n(x) dx = \frac{2}{2n+1} \delta_{mn} \quad (3.3)$$

Simetría Tienen simetría par o impar según su grado: $P_n(-x) = (-1)^n P_n(x)$.

Raíces Las raíces de $P_n(x)$ son todas reales, distintas y se encuentran en el intervalo abierto $(-1, 1)$. Están simétricamente distribuidas respecto al origen.

Relaciones de Recurrencia

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x) \quad (3.4)$$

Aplicaciones en KAN y Explicabilidad

- **Ortogonalidad:** Reduce correlaciones entre parámetros.
- **Estabilidad numérica:** Acotados en $[-1, 1]$, evitando explosión de gradientes.
- **Interpretabilidad:** Descomposición modal clara de la función aprendida.

3.1.2. Polinomios de Chebyshev

Los polinomios de Chebyshev de primera especie, implementados en la clase `ChebyshevActivation`, son fundamentales en la teoría de la aproximación numérica.

Definición y Propiedades

Se definen trigonométricamente como:

$$T_n(x) = \cos(n \arccos x), \quad x \in [-1, 1] \quad (3.5)$$

Satisfacen la relación de recurrencia:

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x) \quad (3.6)$$

Propiedad Geométrica Clave: Minimax

La propiedad más importante de los polinomios de Chebyshev es la **equioscilación**. Son los polinomios que minimizan el error máximo de interpolación (norma L_∞) en el intervalo $[-1, 1]$.

A diferencia de Legendre, cuyas raíces son más uniformes, las raíces de Chebyshev se agrupan densamente cerca de los extremos ± 1 . Esta distribución geométrica neutraliza el *fenómeno de Runge* (oscilaciones violentas en los bordes), garantizando un error uniforme en todo el dominio.

3.1.3. Polinomios de Gegenbauer

Los polinomios de Gegenbauer (ultrasféricos) generalizan a Legendre y Chebyshev, introduciendo un parámetro λ (o α) que controla la geometría de la base.

Definición Matemática

$$C_n^{(\lambda)}(x) = \frac{(-2)^n \Gamma(\lambda + \frac{1}{2})}{n! \Gamma(\lambda - \frac{1}{2})} (1 - x^2)^{-\lambda + \frac{1}{2}} \frac{d^n}{dx^n} \left[(1 - x^2)^{n + \lambda - \frac{1}{2}} \right] \quad (3.7)$$

Ecuación Diferencial

$$(1 - x^2) \frac{d^2 y}{dx^2} - (2\lambda + 1)x \frac{dy}{dx} + n(n + 2\lambda)y = 0 \quad (3.8)$$

Propiedades Geométricas

Son ortogonales respecto al peso $(1 - x^2)^{\lambda - \frac{1}{2}}$. Geométricamente, el parámetro λ permite enfatizar o penalizar la precisión en el centro del intervalo versus los extremos.

- Si $\lambda = 1/2$, recuperamos Legendre (peso uniforme).
- Si $\lambda = 1$, recuperamos Chebyshev de segunda especie.

Ventajas en KAN

- **Base ajustable:** El parámetro λ permite adaptar la base a la distribución de los datos.
- **Control de suavidad:** Valores mayores de λ generan funciones más suaves.

3.1.4. Polinomios de Jacobi

Constituyen la familia más general de polinomios ortogonales clásicos en un intervalo finito.

Definición Matemática

$$P_n^{(\alpha,\beta)}(x) = \frac{(-1)^n}{2^n n!} (1-x)^{-\alpha} (1+x)^{-\beta} \frac{d^n}{dx^n} [(1-x)^{n+\alpha} (1+x)^{n+\beta}] \quad (3.9)$$

Ecuación Diferencial

$$(1-x^2) \frac{d^2 y}{dx^2} + [\beta - \alpha - (\alpha + \beta + 2)x] \frac{dy}{dx} + n(n + \alpha + \beta + 1)y = 0 \quad (3.10)$$

Propiedades Geométricas: Asimetría Controlada

Son ortogonales respecto al peso $(1-x)^\alpha (1+x)^\beta$. La característica geométrica distintiva de Jacobi es la capacidad de manejar **asimetría**:

- α controla el comportamiento cerca de $x = 1$.
- β controla el comportamiento cerca de $x = -1$.

Aplicaciones en KAN

Son ideales para modelar funciones físicas o financieras con condiciones de contorno asimétricas (ej. barreras de potencial, opciones financieras) donde la dinámica en un extremo del dominio difiere radicalmente del otro.

3.2. Bases Locales y Splines

A diferencia de los polinomios globales, estas bases tienen soporte compacto, lo que significa que modificar un parámetro solo afecta a una pequeña región del espacio de entrada.

3.2.1. B-Splines y Fast B-Splines

Los B-Splines son polinomios definidos a trozos y constituyen la base por defecto en la formulación original de KAN.

Propiedades Geométricas Fundamentales

- **Soporte Local:** Un B-Spline $B_{i,k}(x)$ es no nulo solo en un pequeño intervalo $[t_i, t_{i+k+1}]$. Esto previene el **olvido catastrófico**.^{en} KAN: aprender un nuevo dato en una región no altera lo aprendido en otra región distante.
- **Partición de la Unidad:** $\sum_i B_{i,k}(x) = 1$. Garantiza estabilidad numérica.
- **Control de Suavidad:** La continuidad C^k se controla explícitamente mediante el grado del spline y la multiplicidad de los nodos.

Nota sobre FastBSpline: Geométricamente idéntico a los B-Splines estándar, pero implementado mediante operaciones matriciales optimizadas para hardware paralelo (GPU).

3.2.2. Wavelets (Sombrero Mexicano)

Implementada en `WaveletActivation`, esta base introduce el análisis multirresolución en KAN.

Definición (Ricker Wavelet)

$$\psi(t) = \frac{2}{\sqrt{3\sigma}\pi^{1/4}} \left(1 - \left(\frac{t}{\sigma}\right)^2\right) \exp\left(-\frac{t^2}{2\sigma^2}\right) \quad (3.11)$$

Propiedades Geométricas: Tiempo-Frecuencia

Las wavelets están localizadas tanto en el espacio (posición) como en la frecuencia (escala).

- **Localización:** Capacidad única para modelar discontinuidades, picos abruptos o transitorios en la señal de entrada.
- **Multiescala:** Permite a la red capturar detalles finos (altas frecuencias) en zonas específicas y tendencias globales simultáneamente.

3.2.3. Funciones de Base Radial (RBF)

Las RBFs aproximan funciones mediante la superposición de campanas gaussianas.

$$\phi(x) = \sum_{k=1}^K c_k \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right) \quad (3.12)$$

Geométricamente, actúan como detectores de proximidad. Son isotrópicas y excelentes para interpolar datos dispersos en variedades suaves.

3.3. Bases Espectrales y Racionales

Estas bases extienden la capacidad de KAN más allá de la aproximación polinomial convencional.

3.3.1. Base Espectral (Seno)

Basada en el análisis de Fourier, implementada en `SineActivation`.

$$\phi(x) = \sum_{k=1}^K c_k \sin(\omega_k x + \varphi_k) \quad (3.13)$$

Propiedades Geométricas

- **Periodicidad Global:** A diferencia de los polinomios que divergen a $\pm\infty$, la base seno es acotada y periódica.
- **Frecuencia:** Ideal para modelar datos con estacionalidad, series temporales o fenómenos ondulatorios.

3.3.2. Funciones Racionales (Padé)

Implementada en `RationalActivation`, utiliza el cociente de dos polinomios:

$$R(x) = \frac{P_n(x)}{Q_m(x)} = \frac{\sum a_i x^i}{1 + |\sum b_j x^j|} \quad (3.14)$$

Propiedades Geométricas: Polos y Asíntotas

Las funciones racionales poseen una topología mucho más rica que los polinomios:

- **Asíntotas Horizontales:** Pueden modelar saturación ($y \rightarrow C$ cuando $x \rightarrow \infty$), crucial en biología y química.
- **Polos:** Pueden modelar singularidades (divisiones por cero, leyes físicas inversas $1/r^2$) que los polinomios no pueden capturar eficientemente.

3.4. Resumen

La elección de la base define la "forma" de las piezas de Lego que la red KAN usa para construir la solución:

Base	Geometría	Mejor uso
Legendre	Global, Ortogonal, Simétrica	Funciones suaves generales
Chebyshev	Global, Raíces en bordes	Precisión uniforme (Minimax)
Jacobi	Global, Asimétrica	Condiciones de contorno desiguales
B-Splines	Local, Suave a trozos	Funciones complejas, control local
Wavelets	Local en tiempo/frecuencia	Picos, transitorios, discontinuidades
Seno	Global, Periódica	Señales oscilatorias
Racional	Asíntotas, Polos	Singularidades, leyes físicas

Tabla 3.1: Propiedades geométricas de las bases disponibles en DualFlexKAN.

Capítulo 4 Implementación Flexible: Dual-FlexKAN

4.1. Arquitectura Dual Independiente

DualFlexKAN extiende el paradigma KAN tradicional mediante la separación conceptual y computacional de las transformaciones de entrada y las activaciones de salida. De esta forma, es posible definir diferentes estrategias tando de entrada como de salida de forma independiente.

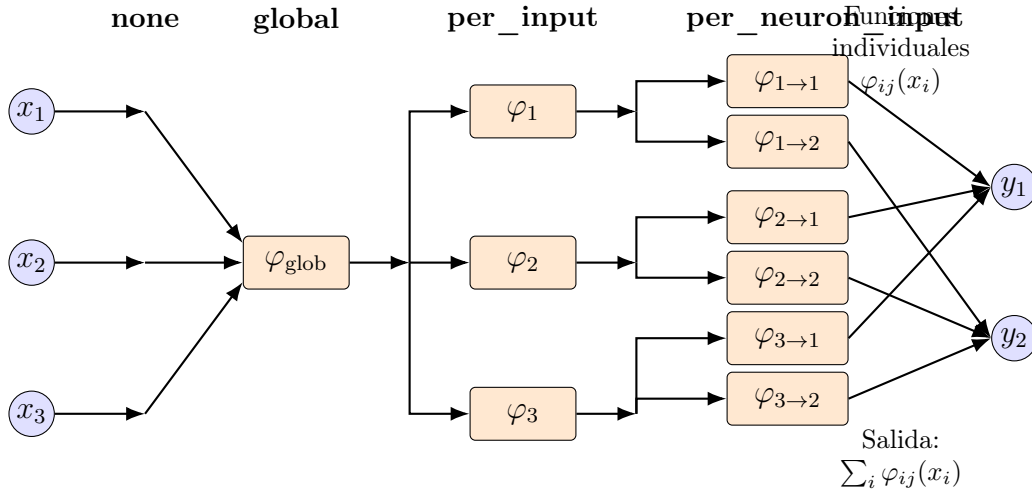


Figura 4.1: Comparación gráfica de las estrategias de función de entrada en DualFlexKANLinear. En per_neuron_input cada par (entrada, neurona) tiene su propia función φ_{ij} .

4.1.1. Formulación Matemática General

Para una capa DualFlexKAN, la transformación se define como:

$$\mathbf{y} = \sigma_{\text{out}} (\mathbf{R}_{\text{post}} (\mathbf{W} \cdot \psi_{\text{in}}(\mathbf{x}) + \mathbf{b})) \quad (4.1)$$

donde:

$$\mathbf{x} \in \mathbb{R}^{n_{\text{in}}} \quad (\text{vector de entrada}) \quad (4.2)$$

$$\mathbf{y} \in \mathbb{R}^{n_{\text{out}}} \quad (\text{vector de salida}) \quad (4.3)$$

$$\psi_{\text{in}} : \mathbb{R}^{n_{\text{in}}} \rightarrow \mathbb{R}^{n_{\text{in}}} \quad (\text{transformación de entrada}) \quad (4.4)$$

$$\mathbf{W} \in \mathbb{R}^{n_{\text{out}} \times n_{\text{in}}} \quad (\text{matriz de pesos}) \quad (4.5)$$

$$\mathbf{b} \in \mathbb{R}^{n_{\text{out}}} \quad (\text{vector de sesgo}) \quad (4.6)$$

$$\mathbf{R}_{\text{post}} \quad (\text{operador de regularización post-lineal}) \quad (4.7)$$

$$\sigma_{\text{out}} \quad (\text{función de activación de salida}) \quad (4.8)$$

4.2. Estrategias de Activación de Entrada

Las estrategias definen **cómo se aplican las funciones de transformación a las entradas** antes de la combinación lineal.

4.2.1. Estrategia "none" – Sin transformación

- **Código:** `input_function_strategy="none"`
- **Qué hace:** No aplica ninguna transformación, la entrada pasa directamente
- **Fórmula:** $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$
- **Uso típico:** Capas finales, cuando los datos ya están bien procesados
- **Parámetros extra:** 0

Cuando se aplica la estrategia "none", los demás parámetros que definen la función de entrada no tienen ningún efecto.

Ejemplo:

$$\mathbf{x} = [1,5, 2,0, -0,5] \quad \longrightarrow \quad \text{pasa sin cambios} \quad \longrightarrow \quad [1,5, 2,0, -0,5]$$

4.2.2. Estrategia "fixed". Función fija compartida

- **Código:**

```

1 input_function_strategy="fixed"
2 input_activation_type="relu" # o "abs", "tanh", etc.

```
- **Qué hace:** Aplica la misma función fija (no aprendible) a TODAS las entradas
- **Fórmula:** $\mathbf{y} = \mathbf{W} \cdot \varphi_{\text{fixed}}(\mathbf{x}) + \mathbf{b}$
- **Uso típico:** Preprocesamiento, normalización, cuando no se necesita aprendizaje
- **Parámetros extra:** 0

Ventajas:

- ✓ Muy eficiente (sin parámetros entrenables)
- ✓ Rápido (operaciones nativas de PyTorch)
- ✓ Estable numéricamente

Ejemplos:

$$\text{Con ReLU: } \mathbf{x} = [1,5, -2,0, 0,5] \xrightarrow{\text{ReLU}} [1,5, 0,0, 0,5]$$

$$\text{Con } |\cdot| : \mathbf{x} = [1,5, -2,0, 0,5] \xrightarrow{|\cdot|} [1,5, 2,0, 0,5]$$

4.2.3. Estrategia "global" – Una función adaptativa compartida■ **Código:**

```

1 input_function_strategy="global"
2 input_activation_type="polynomial"
3 input_activation_kwargs={"degree": 3}

```

- **Qué hace:** UNA función paramétrica compartida para TODAS las entradas
- **Fórmula:** $\mathbf{y} = \mathbf{W} \cdot \varphi(\mathbf{x}; \boldsymbol{\theta}_{\text{global}}) + \mathbf{b}$ donde $\boldsymbol{\theta}$ es compartido
- **Uso típico:** Cuando todas las entradas tienen escala similar
- **Parámetros extra:** N (depende del tipo, ej: `degree+1` para polinomios)

Ejemplo:

Supongamos $\varphi(x) = a_0 + a_1x + a_2x^2 + a_3x^3$ con los *mismos* a_0, a_1, a_2, a_3 para todas las entradas:

$$\begin{aligned} x_1 = 1,0 &\rightarrow \varphi(1,0) = 0,5 + 0,3 \cdot 1,0 + 0,1 \cdot 1,0^2 + 0,05 \cdot 1,0^3 \\ x_2 = 2,0 &\rightarrow \varphi(2,0) = 0,5 + 0,3 \cdot 2,0 + 0,1 \cdot 2,0^2 + 0,05 \cdot 2,0^3 \\ x_3 = -1,0 &\rightarrow \varphi(-1,0) = 0,5 + 0,3 \cdot (-1,0) + 0,1 \cdot (-1,0)^2 + 0,05 \cdot (-1,0)^3 \end{aligned}$$

4.2.4. Estrategia "per_input" – Una función por entrada■ **Código:**

```

1 input_function_strategy="per_input"
2 input_activation_type="polynomial"
3 input_activation_kwargs={"degree": 3}

```

- **Qué hace:** Cada dimensión de entrada tiene su PROPIA función $\varphi_i(x_i; \boldsymbol{\theta}_i)$
- **Fórmula:** $\mathbf{y} = \mathbf{W} \cdot [\varphi_1(x_1; \boldsymbol{\theta}_1), \varphi_2(x_2; \boldsymbol{\theta}_2), \dots, \varphi_n(x_n; \boldsymbol{\theta}_n)] + \mathbf{b}$

- **Uso típico:** Variables heterogéneas o de distinta escala/naturaleza
- **Parámetros extra:** $n_{\text{inputs}} \times N$

Ejemplo con 3 entradas:

Cada entrada tiene SU PROPIO polinomio:

$$\begin{aligned} x_1 = 1,0 &\rightarrow \varphi_1(x_1) = a_0^{(1)} + a_1^{(1)}x_1 + a_2^{(1)}x_1^2 && \text{(coeficientes propios de entrada 1)} \\ x_2 = 5,0 &\rightarrow \varphi_2(x_2) = a_0^{(2)} + a_1^{(2)}x_2 + a_2^{(2)}x_2^2 && \text{(coeficientes propios de entrada 2)} \\ x_3 = -2,0 &\rightarrow \varphi_3(x_3) = a_0^{(3)} + a_1^{(3)}x_3 + a_2^{(3)}x_3^2 && \text{(coeficientes propios de entrada 3)} \end{aligned}$$

4.2.5. Estrategia "per_neuron_input" – Máxima flexibilidad

- **Código:**

```
1 input_function_strategy="per_neuron_input"
2 input_activation_type="polynomial"
3 input_activation_kwargs={"degree": 3}
```

- **Qué hace:** Cada par (entrada i , neurona j) tiene su propia función $\varphi_{ij}(x_i; \theta_{ij})$
- **Fórmula:** $y_j = \sum_{i=1}^n w_{ij} \cdot \varphi_{ij}(x_i; \theta_{ij}) + b_j$
- **Uso típico:** Modelos pequeños donde se requiere máxima expresividad
- **Parámetros extra:** $n_{\text{inputs}} \times n_{\text{outputs}} \times N$
- **¡CUIDADO!** Muchos parámetros, puede causar overfitting fácilmente

Ejemplo con 2 entradas $\rightarrow$ 2 neuronas:

- **Neurona 1 recibe:**

$$\begin{aligned} x_1 &\rightarrow \varphi_{11}(x_1) && \text{(función específica entrada1} \rightarrow \text{neurona1)} \\ x_2 &\rightarrow \varphi_{21}(x_2) && \text{(función específica entrada2} \rightarrow \text{neurona1)} \end{aligned}$$

- **Neurona 2 recibe:**

$$\begin{aligned} x_1 &\rightarrow \varphi_{12}(x_1) && \text{(función específica entrada1} \rightarrow \text{neurona2)} \\ x_2 &\rightarrow \varphi_{22}(x_2) && \text{(función específica entrada2} \rightarrow \text{neurona2)} \end{aligned}$$

Total: 4 funciones diferentes

Tipo	Fórmula
"linear"	$f(x) = x$
"relu"	$f(x) = \max(0, x)$
"leaky_relu"	$f(x) = \max(0, 0.01x, x)$
"tanh"	$f(x) = \tanh(x) \in [-1, 1]$
"sigmoid"	$f(x) = \frac{1}{1+e^{-x}} \in [0, 1]$
"gelu"	$f(x) = x \cdot \Phi(x)$ (Gaussian Error Linear Unit)
"swish"/"silu"	$f(x) = x \cdot \sigma(x)$
"mish"	$f(x) = x \cdot \tanh(\text{softplus}(x))$
"softmax"	$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$ (solo para salida)

Tabla 4.1: Activaciones estándar disponibles

Tipo	Fórmula
".abs"	$f(x) = x $
"square"	$f(x) = x^2$
"sqrt"	$f(x) = \sqrt{ x + \varepsilon}$ (con protección numérica)
".exp"	$f(x) = e^x$
"log"	$f(x) = \ln(x + \varepsilon)$ (con protección numérica)

Tabla 4.2: Transformaciones matemáticas disponibles

4.3. Funciones de activación de entrada

4.3.1. Funciones Fijas

Estas funciones se usan cuando `input_function_strategy="fixed"`. No tienen parámetros aprendibles. En las tablas 4.1 y 4.2 se muestran las funciones de activación típicas disponibles.

Utilizar `input_function_strategy="fixed"` junto con `input_activation_type = "linear"` equivale funcionalmente a utilizar simplemente `input_function_strategy="none"`, dado que en ambos casos se aplica una activación lineal (sin activación). No obstante, hay diferencias sutiles en cuanto al overhead introducido debido a la implementación. Las diferencias se muestran en la tabla 4.3.

Ejemplo de uso:

```

1 layer = DualFlexKANLinear(
2     in_features=10,
3     out_features=20,
4     input_function_strategy="fixed",
5     input_activation_type="relu", # Función fija
6     output_function_strategy="fixed",
7     output_activation_type="tanh" # Función fija

```

Tabla 4.3: Comparativa: estrategia `none` vs `fixed` + `linear`

Aspecto	strategy="none"	strategy="fixed" + type="linear"
Implementación	<code>self.input_activation_fn = None</code>	<code>self.input_activation_fn = lambda x: x</code>
Forward pass	<code>transformed_inputs = x</code>	<code>transformed_inputs = self.input_activation_fn(x)</code>
Operación real	Asignación directa de referencia	Llamada a función identidad
Overhead por llamada	~0 ns	~5–10 ns (overhead de llamada)
Nodos en grafo PyTorch	Identidad implícita (0 nodos)	Nodo de función explícito (1 nodo)
Parámetros entrenables	0	0
Memoria adicional	Ninguna	Objeto lambda (~80 bytes)
Resultado funcional	$f(x) = x$	$f(x) = x$
Mensaje en <code>get_function_info()</code>	"Sin transformación de entrada"	"Fija: linear"
Profiling	No aparece en traces	Aparece como función call
Legibilidad semántica	"No hago transformación"	"Aplico identidad explícita"
Consistencia arquitectural	Rompe patrón si otras capas usan "fixed"	Mantiene patrón uniforme
Uso recomendado	Máxima eficiencia y simplicidad	Claridad arquitectural y debugging

```

8 )
9 # Esta capa tiene 0 parámetros extra (solo pesos y bias)

```

4.3.2. Funciones Adaptativas

Estas funciones se usan cuando `input_function_strategy` es "global", "per_input" o "per_neuron_input". Requieren `from KAN_activation import *`

Polinomios ("polynomial")

- **Código:**

```

1 input_activation_type="polynomial"
2 input_activation_kwargs={"degree": 3}

```

- **Fórmula:** $\varphi(x; a_0, \dots, a_n) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$
- **Parámetros:** `degree + 1`
- **Ventajas:** Simple, derivables, aproximador universal
- **Desventajas:** Puede crecer exponencialmente, inestable con grados altos

- **Ejemplo con degree=3:**

$$\varphi(x) = 0,5 + 0,3x + 0,1x^2 + 0,05x^3$$

4 parámetros aprendibles: [0,5, 0,3, 0,1, 0,05]

Polinomios de Legendre ("legendre")

- **Código:**

```

1 input_activation_type="legendre"
2 input_activation_kwargs={"degree": 3}

```

- **Fórmula:** $\varphi(x; c_0, \dots, c_n) = \sum_{i=0}^n c_i \cdot P_i(x)$ donde P_i son polinomios de Legendre
- **Parámetros:** `degree + 1`
- **Ventajas:** Ortogonales, más estables que polinomios simples
- **Base:**

$$P_0(x) = 1$$

$$P_1(x) = x$$

$$P_2(x) = \frac{3x^2 - 1}{2}$$

$$P_3(x) = \frac{5x^3 - 3x}{2}$$

- **Uso:** Cuando se necesitan polinomios pero con mejor comportamiento numérico

Polinomios de Chebyshev ("chebyshev")

- **Código:**

```
1 input_activation_type="chebyshev"
2 input_activation_kwargs={"degree": 3}
```

- **Fórmula:** $\varphi(x; c_0, \dots, c_n) = \sum_{i=0}^n c_i \cdot T_i(x)$ donde T_i son polinomios de Chebyshev de primera especie.
- **Parámetros:** `degree + 1`
- **Ventajas:** Propiedad *minimax* (minimizan el error máximo en el intervalo $[-1, 1]$). Ideales para aproximación uniforme.
- **Recurrencia:** $T_0(x) = 1, T_1(x) = x, T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$.

Polinomios de Gegenbauer ("gegenbauer")

- **Código:**

```
1 input_activation_type="gegenbauer"
2 input_activation_kwargs={"degree": 3, "alpha": 0.5}
```

- **Fórmula:** $\varphi(x; c_0, \dots, c_n) = \sum_{i=0}^n c_i \cdot C_i^{(\alpha)}(x)$
- **Parámetros:** `degree + 1` (α es fijo, no se entrena)
- **Ventajas:** Generalización de Legendre y Chebyshev, ortogonales con peso
- **Casos especiales:**
 - $\alpha = 0,5 \rightarrow$ Legendre
 - $\alpha = 1 \rightarrow$ Chebyshev 2^a especie

Polinomios de Jacobi ("jacobi")

- **Código:**

```
1 input_activation_type="jacobi"
2 input_activation_kwargs={"degree": 3, "alpha": 0.0, "beta": 0.0}
```

- **Fórmula:** $\varphi(x; c_0, \dots, c_n) = \sum_{i=0}^n c_i \cdot P_i^{(\alpha, \beta)}(x)$
- **Parámetros:** `degree + 1` (α, β son fijos)
- **Ventajas:** Máxima generalización de polinomios ortogonales
- **Casos especiales:**
 - $\alpha = \beta = 0 \rightarrow$ Legendre
 - $\alpha = \beta = -0,5 \rightarrow$ Chebyshev 1^a especie

B-Splines ("bspline")

- **Código:**

```
1 input_activation_type="bspline"
2 input_activation_kwargs={"num_knots": 10, "degree": 3}
```

- **Fórmula:** $\varphi(x; c_0, \dots, c_k) = \sum_{i=0}^k c_i \cdot B_i(x)$ (combinación de B-splines)

- **Parámetros:** num_knots + degree + 1

- **Ventajas:**

- ✓ Soporte local (cambio en un parámetro solo afecta una región)
- ✓ Suaves y derivables
- ✓ Excelente para funciones complejas con muchos picos

- **Uso:** Señales complejas, funciones con múltiples variaciones

Fast B-Splines ("fbspline" o "fast_bspline")

- **Código:**

```
1 input_activation_type="fast_bspline"
2 input_activation_kwargs={"num_knots": 10, "degree": 3}
```

- **Descripción:** Implementación optimizada de B-Splines que utiliza operaciones matriciales vectorizadas en lugar de recursión, ideal para acelerar el entrenamiento en GPUs.

- **Ventajas:** Mismas propiedades matemáticas que bspline pero con mayor eficiencia computacional.

Funciones de Base Radial (rbf")

- **Código:**

```
1 input_activation_type="rbf"
2 input_activation_kwargs={"num_centers": 10}
```

- **Fórmula:** $\varphi(x; c_1, \dots, c_n, \sigma_1, \dots, \sigma_n) = \sum_{i=1}^n w_i \cdot \exp\left(-\frac{\|x - c_i\|^2}{2\sigma_i^2}\right)$

- **Parámetros:** 3 × num_centers (centros, anchos, pesos)

- **Ventajas:**

- ✓ Localizadas (efecto local)
- ✓ Muy expresivas
- ✓ Buenas para interpolación

- **Desventajas:** Más parámetros, puede ser más lenta

- **Uso:** Aproximación de funciones complejas, interpolación

Base Espectral / Seno ("sine")

- **Código:**

```
1 input_activation_type="sine"
2 input_activation_kwargs={"num_freqs": 5}
```

- **Fórmula:** $\varphi(x) = \sum_{k=1}^N a_k \sin(kx) + b_k \cos(kx)$ (Serie de Fourier truncada).
- **Uso:** Ideal para datos con periodicidad intrínseca, señales de audio o series temporales estacionales.

Wavelets ("wavelet")

- **Código:**

```
1 input_activation_type="wavelet"
2 input_activation_kwargs={"num_wavelets": 5}
```

- **Fórmula:** Superposición de wavelets tipo "Sombrero Mexicano" (Ricker) con parámetros de escala y traslación aprendibles.
- **Uso:** Señales no estacionarias, detección de transitorios, análisis tiempo-frecuencia conjunto.

Funciones Racionales (rational")

- **Código:**

```
1 input_activation_type="rational"
2 input_activation_kwargs={"degrees": (3, 3)}
```

- **Fórmula:** Aproximantes de Padé $R(x) = \frac{P(x)}{Q(x)}$.
- **Uso:** Modelado de singularidades (polos), asíntotas horizontales y funciones físicas inversas.

A modo de resumen, en la Tabla 4.4 se muestra una comparativa de las diferentes estrategias de entrada implementadas en la clase DualFlexKAN.

4.3.3. Ejemplos Prácticos

Ejemplo 1: Preprocesamiento eficiente con funciones fijas

```
1 # Normalización tipo ReLU antes de la red
2 model = DualFlexKAN(
3     layer_sizes=[10, 64, 32, 1],
4     input_function_strategies=["fixed", "none", "none"],
5     input_activation_types=["relu", "polynomial", "polynomial"],
```

Estrategia	# Func.	# Params*	Compartición	Complejidad	Cuándo usar
none	0	0	—	Baja	Datos ya procesados
fixed	1 (fija)	0	Total	Baja	Preprocesamiento eficiente
global	1	N	Total	Baja-Media	Entradas homogéneas
per_input	n_{in}	$n_{in} \times N$	Por entrada	Media	Variables heterogéneas
per_neuron_input	$n_{in} \times n_{out}$	$n_{in} \times n_{out} \times N$	Ninguna	Alta	Máxima expresividad

Tabla 4.4: Comparativa de estrategias. * N = número de parámetros de la función (ej: degree+1 para polinomios)

```

6     output_function_strategies=["fixed", "fixed", "fixed"],
7     output_activation_types=["relu", "relu", "linear"]
8 )
9 # Capa 1: ReLU fijo en entrada → menos parámetros
10 # Capas 2-3: Sin transformación de entrada

```

Ejemplo 2: Variables de diferente naturaleza

```

1 # Imagina: [edad, salario, temperatura] → muy diferentes escalas
2 model = DualFlexKAN(
3     layer_sizes=[3, 16, 1],
4     input_function_strategies=["per_input"], # cada variable su
        función
5     input_activation_types=["polynomial"],
6     input_activation_kwargs=[{"degree": 2}],
7     output_function_strategies=["fixed"],
8     output_activation_types=["sigmoid"]
9 )
10 #  $\varphi_1$ (edad),  $\varphi_2$ (salario),  $\varphi_3$ (temperatura) → funciones independientes

```

Ejemplo 3: Comparación de número de parámetros

```

1 # Configuración A: fixed (eficiente)
2 layer_A = DualFlexKANLinear(
3     in_features=10, out_features=20,
4     input_function_strategy="fixed",
5     input_activation_type="relu"
6 )
7 # Parámetros: 10 20 (pesos) + 20 (bias) = 220
8
9 # Configuración B: per_input (moderado)
10 layer_B = DualFlexKANLinear(
11     in_features=10, out_features=20,
12     input_function_strategy="per_input",
13     input_activation_type="polynomial",
14     input_activation_kwargs={"degree": 3}
15 )
16 # Parámetros: 220 + 10 4 = 260 (40 params extra)
17

```

```

18 # Configuración C: per_neuron_input (muchos)
19 layer_C = DualFlexKANLinear(
20     in_features=10, out_features=20,
21     input_function_strategy="per_neuron_input",
22     input_activation_type="polynomial",
23     input_activation_kwargs={"degree": 3}
24 )
25 # Parámetros: 220 + 10  204  = 1020  (800 params extra!)

```

4.3.4. Recomendaciones

Para evitar NaN (como en autoencoders)

USAR	▪ input_function_strategy="fixed" con relu", "tanh" ▪ input_function_strategy="per_input" con degree=1 o 2 ▪ output_function_strategy="fixed" con activaciones estándar
-------------	---

EVI-TAR	▪ "polynomial" con degree >3 ▪ "per_neuron_input" en capas grandes ▪ Mezclar muchas funciones adaptativas sin batch norm
----------------	---

Orden de complejidad (de simple a complejo)

1. fixed + **activación estándar** → MLP tradicional mejorado
2. global + **polinomio grado 2** → KAN ligera
3. per_input + **polinomio grado 3** → KAN media
4. per_input + **bspline/rbf** → KAN expresiva
5. per_neuron_input + **cualquier función** → KAN máxima (cuidado overfitting)

4.4. Estrategias de Activación de Salida

Las estrategias definen **cómo se aplican las funciones de activación a las salidas** después de la combinación lineal.

4.4.1. Estrategia "none" – Sin activación

- **Código:** output_function_strategy="none"
- **Qué hace:** No aplica ninguna activación, la salida es lineal

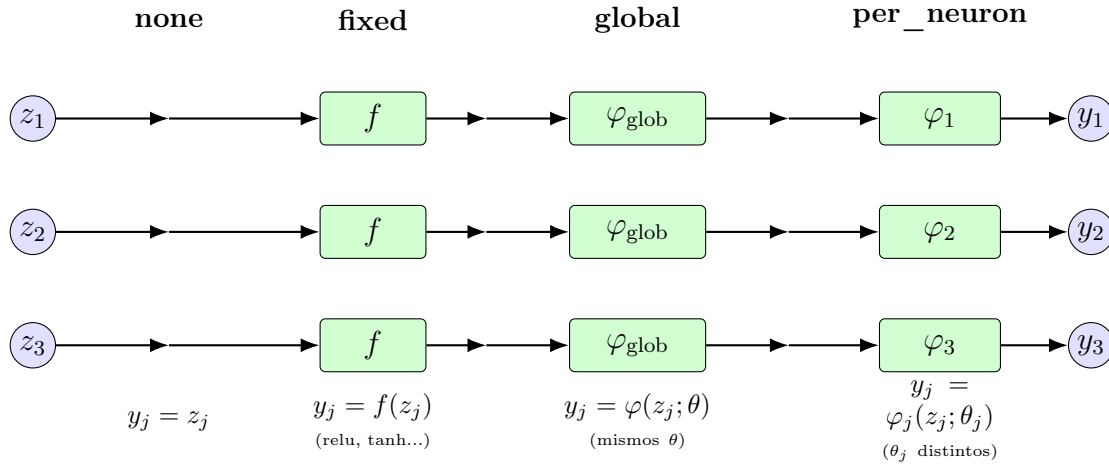


Figura 4.2: Estrategias de función de salida en DualFlexKANLinear. Las funciones se aplican después de la operación lineal $z = Wx + b$. **none**: identidad; **fixed**: función fija f compartida (relu, tanh, etc.); **global**: una función paramétrica adaptativa compartida con los mismos parámetros θ ; **per_neuron**: una función adaptativa distinta por neurona con parámetros θ_j individuales.

- **Fórmula:** $y = W\varphi(x) + b$ (sin transformación posterior)
- **Uso típico:**
 - Capas de salida en regresión
 - Bloques residuales
 - Cuando se necesita salida sin restricciones
- **Parámetros extra:** 0

Ejemplo:

$$z = Wx + b = [2, 5, -1, 3, 0, 8] \longrightarrow \text{salida directa} \longrightarrow [2, 5, -1, 3, 0, 8]$$

Cuándo usar:

- ✓ Regresión lineal (predicción de valores continuos sin límites)
- ✓ Última capa antes de una función de pérdida específica
- ✓ Conexiones residuales (skip connections)

Si se utiliza `input_function_strategy="none"`, los parámetros referentes a las funciones de activación (por ejemplo `output_activation_type`) no tienen ningún efecto y la activación de la salida será lineal.

4.4.2. Estrategia "fixed" – Función de activación fija

- **Código:**

```
1 output_function_strategy="fixed"
2 output_activation_type="relu" # o "tanh", "sigmoid", etc.
```

- **Qué hace:** Aplica una función de activación fija (no aprendible) a TODAS las neuronas de salida

- **Fórmula:** $y = \sigma_{\text{fixed}}(W\varphi(\mathbf{x}) + \mathbf{b})$

- **Uso típico:**

- Redes estándar (comportamiento tipo MLP)
- Cuando se necesita eficiencia y estabilidad
- Activaciones bien conocidas y probadas

- **Parámetros extra:** 0

Ventajas:

- ✓ Comportamiento predecible y bien estudiado
- ✓ Muy eficiente computacionalmente
- ✓ Estable durante el entrenamiento
- ✓ Compatible con técnicas estándar de deep learning

Ejemplos:

Con ReLU: $z = [2,5, -1,3, 0,8] \xrightarrow{\text{ReLU}} [2,5, 0,0, 0,8]$

Con tanh: $z = [2,5, -1,3, 0,8] \xrightarrow{\text{tanh}} [0,987, -0,862, 0,664]$

Con sigmoid: $z = [2,5, -1,3, 0,8] \xrightarrow{\sigma} [0,924, 0,214, 0,689]$

Comparación con MLP tradicional:

```
1 # MLP tradicional
2 layer = nn.Linear(10, 20)
3 activation = nn.ReLU()
4 output = activation(layer(input))
5
6 # Equivalente en DualFlexKAN
7 layer = DualFlexKANLinear(
8     in_features=10,
9     out_features=20,
10    input_function_strategy="none",      # Sin transformación entrada
11    output_function_strategy="fixed",    # ReLU fijo en salida
12    output_activation_type="relu"
13 )
14 output = layer(input) # ReLU ya incluido
```

4.4.3. Estrategia "global" – Una función adaptativa compartida

- **Código:**

```
1 output_function_strategy="global"
2 output_activation_type="polynomial"
3 output_activation_kwargs={"degree": 3}
```

- **Qué hace:** UNA función paramétrica compartida para TODAS las neuronas de salida

- **Fórmula:** $y = \psi(W\varphi(\mathbf{x}) + \mathbf{b}; \boldsymbol{\theta}_{\text{global}})$ donde $\boldsymbol{\theta}$ es compartido

- **Uso típico:**

- Cuando todas las neuronas necesitan similar no-linealidad
- Ajuste adaptable común para todas las salidas
- Balance entre flexibilidad y número de parámetros

- **Parámetros extra:** N (depende del tipo, ej: `degree+1` para polinomios)

Ejemplo:

Supongamos $\psi(z) = a_0 + a_1z + a_2z^2 + a_3z^3$ con los *mismos* a_0, a_1, a_2, a_3 para todas las neuronas:

$$\begin{aligned} z_1 = 2,0 & \rightarrow \psi(2,0) = 0,1 + 0,5 \cdot 2,0 + 0,2 \cdot 2,0^2 + 0,05 \cdot 2,0^3 \\ z_2 = -1,0 & \rightarrow \psi(-1,0) = 0,1 + 0,5 \cdot (-1,0) + 0,2 \cdot (-1,0)^2 + 0,05 \cdot (-1,0)^3 \\ z_3 = 0,5 & \rightarrow \psi(0,5) = 0,1 + 0,5 \cdot 0,5 + 0,2 \cdot 0,5^2 + 0,05 \cdot 0,5^3 \end{aligned}$$

Ventajas sobre "per_neuron":

- ✓ Menos parámetros (un solo conjunto para todas las neuronas)
- ✓ Regularización implícita (todas las neuronas comparten comportamiento)
- ✓ Más rápido de entrenar
- ✓ Menos propenso a overfitting

4.4.4. Estrategia "per_neuron" – Una función por neurona

- **Código:**

```
1 output_function_strategy="per_neuron"
2 output_activation_type="polynomial"
3 output_activation_kwargs={"degree": 3}
```

- **Qué hace:** Cada neurona de salida aprende su PROPIA función de activación $\psi_j(z_j; \boldsymbol{\theta}_j)$

■ **Fórmula:**

$$y_j = \psi_j \left(\sum_{i=1}^n w_{ij} \varphi_i(x_i) + b_j; \theta_j \right)$$

■ **Uso típico:**

- Modelos con salidas de naturaleza muy diferente
- Jerarquías neuronales complejas
- Cuando cada neurona necesita comportamiento único

■ **Parámetros extra:** $n_{\text{outputs}} \times N$

Ejemplo con 3 neuronas de salida:

Cada neurona tiene SU PROPIA función de activación:

$$\begin{aligned} z_1 = 2,0 & \rightarrow y_1 = \psi_1(z_1) = a_0^{(1)} + a_1^{(1)} z_1 + a_2^{(1)} z_1^2 & (\text{activación neurona 1}) \\ z_2 = -1,0 & \rightarrow y_2 = \psi_2(z_2) = a_0^{(2)} + a_1^{(2)} z_2 + a_2^{(2)} z_2^2 & (\text{activación neurona 2}) \\ z_3 = 0,5 & \rightarrow y_3 = \psi_3(z_3) = a_0^{(3)} + a_1^{(3)} z_3 + a_2^{(3)} z_3^2 & (\text{activación neurona 3}) \end{aligned}$$

Caso de uso real: Clasificación multiclase

Imagina una tarea de clasificación de imágenes con 3 clases: perro, gato, pájaro.

```

1 layer = DualFlexKANLinear(
2     in_features=128,
3     out_features=3, # 3 clases
4     output_function_strategy="per_neuron",
5     output_activation_type="polynomial",
6     output_activation_kwargs={"degree": 2}
7 )
8
9 # Cada neurona aprende su propia activación:
10 # Neurona 1 (perro):  $\psi_1(z) = a_0^{(1)} + a_1^{(1)} z + a_2^{(1)} z^2$ 
11 # Neurona 2 (gato):  $\psi_2(z) = a_0^{(2)} + a_1^{(2)} z + a_2^{(2)} z^2$ 
12 # Neurona 3 (pájaro):  $\psi_3(z) = a_0^{(3)} + a_1^{(3)} z + a_2^{(3)} z^2$ 

```

Ventajas:

- ✓ Máxima expresividad por neurona
- ✓ Especialización automática de neuronas
- ✓ Útil cuando las salidas tienen rangos muy diferentes

Desventajas:

- × Más parámetros que "global"
- × Mayor riesgo de overfitting
- × Requiere más datos de entrenamiento

4.5. Funciones de Activación de Salida

4.5.1. Funciones Fijas

Estas funciones se usan cuando `output_function_strategy="fixed"`. Son las activaciones estándar de redes neuronales.

Las funciones de activación fijas para capas de salida se muestran en las tablas 4.5, 4.6 y 4.7. Dependiendo del tipo de capa (oculta o de salida) se recomienda utilizar las funciones 4.5 o 4.6, respectivamente. No obstante, podría utilizarse cualquier función de las mostradas en estas tablas en cualquier capa.

Tipo	Fórmula	Uso típico
<code>relu</code>	$f(x) = \max(0, x)$	Capas ocultas estándar, muy estable
<code>leaky_relu</code>	$f(x) = \max(0, 0.1x, x)$	Evita neuronas muertas
<code>gelu</code>	$f(x) = x \cdot \Phi(x)$	Transformers, modelos de lenguaje
<code>swish/silu</code>	$f(x) = x \cdot \sigma(x)$	Redes profundas modernas
<code>mish</code>	$f(x) = x \cdot \tanh(\text{softplus}(x))$	Alternativa suave a ReLU
<code>tanh</code>	$f(x) = \tanh(x) \in [-1, 1]$	RNNs, cuando se necesita salida acotada

Tabla 4.5: Activaciones fijas para capas ocultas

Tipo	Fórmula	Uso típico
<code>linear</code>	$f(x) = x$	Regresión sin límites
<code>sigmoid</code>	$f(x) = \frac{1}{1+e^{-x}} \in [0, 1]$	Clasificación binaria, probabilidades
<code>softmax</code>	$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$	Clasificación multiclase
<code>tanh</code>	$f(x) = \tanh(x) \in [-1, 1]$	Regresión acotada a $[-1, 1]$

Tabla 4.6: Activaciones fijas para capas de salida

Tipo	Fórmula	Uso potencial
<code>abs</code>	$f(x) = x $	Asegurar salidas positivas
<code>square</code>	$f(x) = x^2$	Energía, magnitudes cuadráticas
<code>sqrt</code>	$f(x) = \sqrt{ x + \varepsilon}$	Magnitudes, desviaciones
<code>exp</code>	$f(x) = e^x$	Tasas, crecimiento exponencial
<code>log</code>	$f(x) = \ln(x + \varepsilon)$	Escala logarítmica

Tabla 4.7: Transformaciones matemáticas (uso especializado)

Ejemplo de configuración por tarea:

4.5.2. Funciones Adaptativas

Estas funciones se usan cuando `output_function_strategy` es `"global"` o `"per_neuron"`. Permiten que la red aprenda su propia función de activación. **Todas las funciones lista-**

```

1 # REGRESIÓN: predecir temperatura (sin límites)
2 output_function_strategy="fixed"
3 output_activation_type="linear"
4
5 # CLASIFICACIÓN BINARIA: detectar spam (0 o 1)
6 output_function_strategy="fixed"
7 output_activation_type="sigmoid"
8
9 # CLASIFICACIÓN MULTICLASE: reconocer dígitos (0-9)
10 output_function_strategy="fixed"
11 output_activation_type="softmax"
12
13 # REGRESIÓN ACOTADA: predecir correlación [-1, 1]
14 output_function_strategy="fixed"
15 output_activation_type="tanh"

```

das en la sección de entrada (Chebyshev, Sine, Wavelet, Rational, etc.) también están disponibles para su uso en la salida.

Polinomios ("polynomial")

- **Código:**

```

1 output_activation_type="polynomial"
2 output_activation_kwargs={"degree": 3}

```

- **Fórmula:** $\psi(z; a_0, \dots, a_n) = a_0 + a_1z + a_2z^2 + \dots + a_nz^n$
- **Parámetros:** degree + 1
- **Uso:** Activación adaptable general, aproximador universal
- **Ventajas:** Simple, interpretable, derivable
- **Desventajas:** Puede ser inestable con grados altos

Comportamiento aprendido:

Durante el entrenamiento, la red puede aprender comportamientos como:

$$\psi_{\text{aprox. ReLU}}(z) \approx 0,01 + 0,5z + 0,1z^2 \quad (\text{similar a ReLU suavizado})$$

$$\psi_{\text{aprox. sigmoid}}(z) \approx 0,5 + 0,25z - 0,05z^3 \quad (\text{forma sigmoidal})$$

Polinomios de Legendre ("legendre")

- **Código:**

```

1 output_activation_type="legendre"
2 output_activation_kwargs={"degree": 3}

```

- **Fórmula:** $\psi(z; c_0, \dots, c_n) = \sum_{i=0}^n c_i \cdot P_i(z)$
- **Parámetros:** `degree + 1`
- **Uso:** Alternativa más estable a polinomios simples
- **Ventajas:** Ortogonales, mejor condicionamiento numérico

Base de Legendre:

$$P_0(z) = 1$$

$$P_1(z) = z$$

$$P_2(z) = \frac{1}{2}(3z^2 - 1)$$

$$P_3(z) = \frac{1}{2}(5z^3 - 3z)$$

$$P_4(z) = \frac{1}{8}(35z^4 - 30z^2 + 3)$$

Polinomios de Gegenbauer ("gegenbauer")

- **Código:**

```
1 output_activation_type="gegenbauer"
2 output_activation_kwargs={"degree": 3, "alpha": 0.5}
```

- **Fórmula:** $\psi(z; c_0, \dots, c_n) = \sum_{i=0}^n c_i \cdot C_i^{(\alpha)}(z)$
- **Parámetros:** `degree + 1` (α es hiperparámetro)
- **Uso:** Generalización de Legendre con más control

Polinomios de Jacobi ("jacobi")

- **Código:**

```
1 output_activation_type="jacobi"
2 output_activation_kwargs={"degree": 3, "alpha": 0.0, "beta": 0.0}
```

- **Fórmula:** $\psi(z; c_0, \dots, c_n) = \sum_{i=0}^n c_i \cdot P_i^{(\alpha, \beta)}(z)$
- **Parámetros:** `degree + 1` (α, β son hiperparámetros)
- **Uso:** Máxima generalización de polinomios ortogonales

B-Splines ("bspline")

■ Código:

```
1 output_activation_type="bspline"
2 output_activation_kwargs={"num_knots": 10, "degree": 3}
```

■ **Fórmula:** $\psi(z; c_0, \dots, c_k) = \sum_{i=0}^k c_i \cdot B_i(z)$

■ **Parámetros:** num_knots + degree + 1

■ **Uso:** Activaciones complejas con múltiples cambios de comportamiento

■ Ventajas:

- ✓ Soporte local (cambios localizados)
- ✓ Muy flexible
- ✓ Suaves y derivables

Ejemplo visual: Una activación B-spline puede aprender formas como:

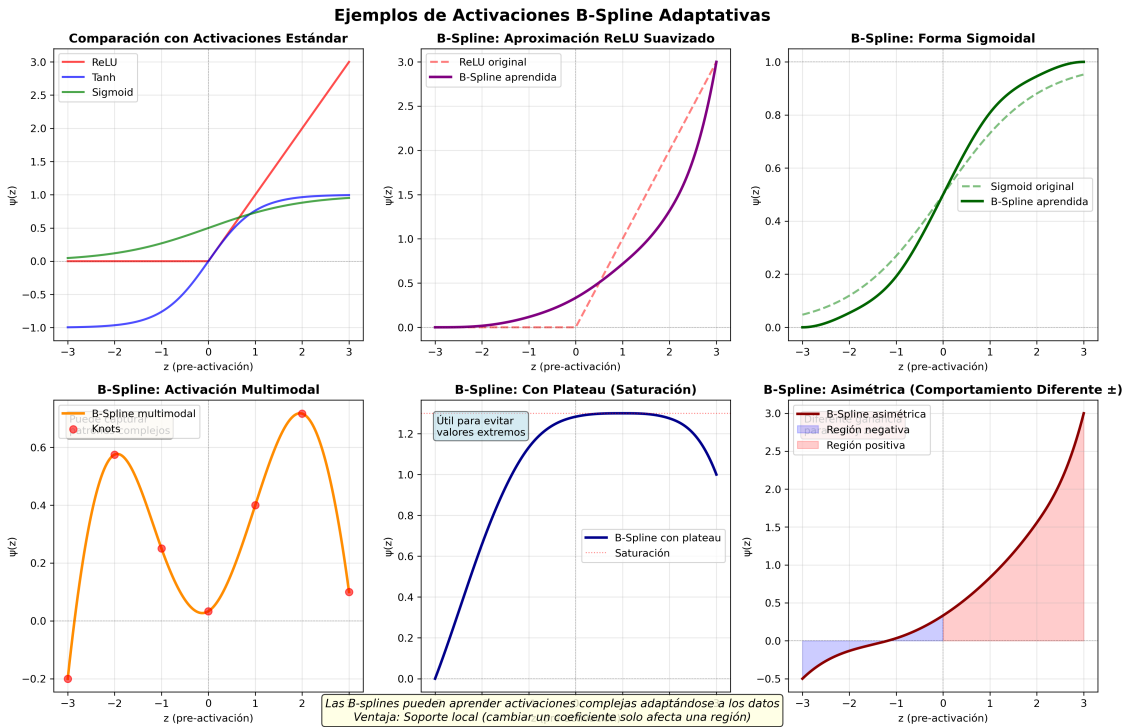


Figura 4.3: Ejemplos de activaciones B-Spline adaptativas.

Funciones de Base Radial (rbf")

■ Código:

```
1 output_activation_type="rbf"
2 output_activation_kwargs={"num_centers": 10}
```

■ **Fórmula:** $\psi(z; c_1, \dots, c_n, \sigma_1, \dots, \sigma_n) = \sum_{i=1}^n w_i \cdot \exp\left(-\frac{(z-c_i)^2}{2\sigma_i^2}\right)$

■ **Parámetros:** $3 \times \text{num_centers}$

■ **Uso:** Activaciones muy complejas y localizadas

■ Ventajas:

- ✓ Extremadamente expresivas
- ✓ Pueden aprender activaciones multimodales
- ✓ Buenas para patrones complejos

■ Desventajas:

- × Muchos parámetros
- × Más lenta de computar
- × Requiere inicialización cuidadosa

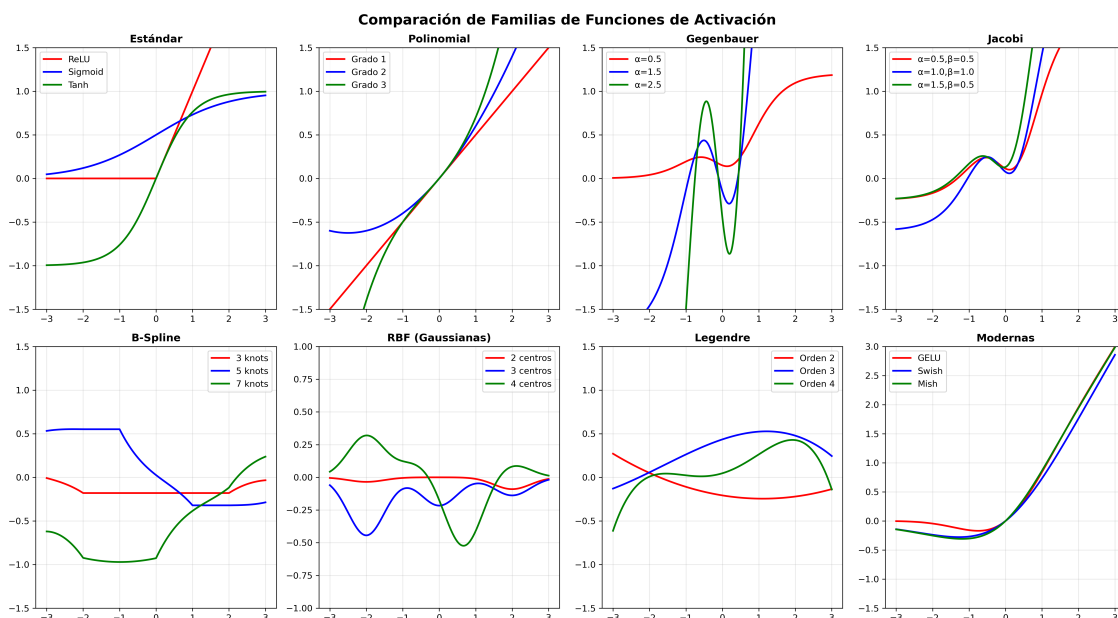


Figura 4.4: Diferentes funciones de activación definidas en la librería.

Estrategia	# Func.	# Params*	Compartición	Complejidad	Cuándo usar
none	0	0	—	Baja	Regresión lineal, skip connections
fixed	1 (fija)	0	Total	Baja	MLP estándar, estabilidad
global	1	N	Total	Media	Balance flexibilidad/parámetros
per_neuron	n_{out}	$n_{out} \times N$	Parcial	Alta	Salidas heterogéneas, especialización

Tabla 4.8: Comparativa de estrategias de salida. * N = número de parámetros de la función

4.5.3. Tabla Comparativa de Estrategias de Salida

4.5.4. Ejemplos Prácticos

Ejemplo 1: Red de Clasificación Binaria (Estándar)

```

1 # Detector de spam: entrada texto → salida probabilidad [0,1]
2 model = DualFlexKAN(
3     layer_sizes=[100, 64, 32, 1],
4     # Capas ocultas: activaciones fijas eficientes
5     input_function_strategies=["none", "none", "none"],
6     output_function_strategies=["fixed", "fixed", "fixed"],
7     output_activation_types=["relu", "relu", "sigmoid"], # Última:
8     dropout_probs=[0, 0.2, 0.3, 0],
9     use_batch_norms=[True, True, True, False]
10 )
11
12 # Función de pérdida apropiada
13 criterion = nn.BCELoss() # Binary Cross Entropy (compatible con
14     sigmoid)

```

Ejemplo 2: Clasificación Multiclase Adaptativa

```

1 # Reconocimiento de dígitos: cada clase aprende su activación
2 model = DualFlexKAN(
3     layer_sizes=[784, 128, 64, 10], # 10 clases (dígitos 0-9)
4     input_function_strategies=["fixed", "none", "none"],
5     input_activation_types=["relu", "polynomial", "polynomial"],
6     output_function_strategies=["fixed", "fixed", "per_neuron"], # Ú
7     ltima: per_neuron
8     output_activation_types=["relu", "relu", "polynomial"],
9     output_activation_kwargs=[{}, {}, {"degree": 2}],
10     dropout_probs=[0, 0.2, 0.3, 0]
11 )
12
13 # Cada uno de los 10 dígitos aprende su propia activación polynomial
14 # Después aplicar softmax manualmente o en el loss:
15 criterion = nn.CrossEntropyLoss() # Incluye softmax implícito

```

Ejemplo 3: Regresión Multivariada

```

1 # Predecir [temperatura, humedad, presión] desde sensores
2 model = DualFlexKAN(
3     layer_sizes=[20, 64, 32, 3], # 3 salidas diferentes
4     input_function_strategies=["per_input", "none", "none"],
5     input_activation_types=["polynomial", "polynomial",
6                             "polynomial"],
7     input_activation_kwargs=[{"degree": 2}, {}, {}],
8     output_function_strategies=["fixed", "fixed", "per_neuron"],
9     output_activation_types=["relu", "relu", "polynomial"],
10    output_activation_kwargs=[{}, {}, {"degree": 2}],
11 )
12 # Cada variable de salida (temp, humedad, presión)
13 # tiene su propia activación adaptativa
14 criterion = nn.MSELoss()

```

Ejemplo 4: Autoencoder con Salidas Acotadas

```

1 # Encoder
2 encoder = DualFlexKAN(
3     layer_sizes=[784, 256, 128, 32], # MNIST 28x28 → latent 32
4     input_function_strategies=["fixed", "none", "none"],
5     input_activation_types=["relu", "polynomial", "polynomial"],
6     output_function_strategies=["fixed", "fixed", "fixed"],
7     output_activation_types=["relu", "relu", "tanh"], # Última:
8         tanh [-1,1]
9     dropout_probs=[0, 0.1, 0.2, 0],
10    use_batch_norms=[True, True, True, False]
11 )
12 # Decoder
13 decoder = DualFlexKAN(
14     layer_sizes=[32, 128, 256, 784],
15     input_function_strategies=["none", "none", "none"],
16     output_function_strategies=["fixed", "fixed", "fixed"],
17     output_activation_types=["relu", "relu", "sigmoid"], # Última:
18         sigmoid [0,1]
19     dropout_probs=[0, 0.1, 0.1, 0],
20    use_batch_norms=[True, True, True, False]
21 )
22 criterion = nn.MSELoss() # Reconstrucción pixel a pixel

```

Ejemplo 5: Red Experimental con Máxima Flexibilidad

```

1 # Investigación: cada neurona aprende activación compleja
2 model = DualFlexKAN(
3     layer_sizes=[50, 20, 10, 3],
4     input_function_strategies=["per_input", "global", "none"],

```

```

5     input_activation_types=["polynomial", "legendre", "polynomial"],
6     input_activation_kwargs=[{"degree": 2}, {"degree": 3}, {}],
7     output_function_strategies=["per_neuron", "per_neuron",
8     "per_neuron"],
9     output_activation_types=["bspline", "rbf", "polynomial"],
10    output_activation_kwargs=[
11        {"num_knots": 8, "degree": 3}, # Capa 1: B-splines
12        {"num_centers": 5},           # Capa 2: RBFs
13        {"degree": 2}                 # Capa 3: Polinomios
14    ],
15    dropout_probs=[0, 0.15, 0.2, 0],
16    use_batch_norms=[True, True, True, False]
17 )
18 # Esta configuración es muy expresiva pero requiere:
19 # - Muchos datos
20 # - Regularización fuerte
21 # - Monitoreo cuidadoso del overfitting

```

4.5.5. Recomendaciones

Situación	Estrategia recomendada	Razón
Red profunda estándar	fixed con ReLU/GELU	Estabilidad, velocidad, comportamiento probado
Última capa de regresión	none o fixed con linear	Sin restricciones en la salida
Clasificación binaria	fixed con sigmoid	Salida en $[0, 1]$ interpretable como probabilidad
Clasificación multiclase	fixed con softmax	Distribución de probabilidad válida
Salidas muy diferentes	per_neuron	Cada neurona puede especializarse
Explorar nuevas activaciones	global con polynomial	Buena relación flexibilidad/parámetros
Máxima expresividad	per_neuron con rbf/bspline	Activaciones complejas por neurona
Autoencoder (cuello botella)	fixed con linear/tanh	Compresión sin restricciones artificiosas
Autoencoder (reconstrucción)	fixed con sigmoid/tanh	Acotar salida al rango de entrada

Tabla 4.9: Guía de decisión para estrategias de salida

4.6. Sistema de Regularización Flexible

4.6.1. Posiciones de Aplicación

Dropout

- `before_activation`: Antes de la función de activación

- `after_activation`: Después de la función de activación
- `both`: Antes y después
- `none`: Sin dropout

BatchNorm

- `before_activation`: Antes de la función de activación
- `after_activation`: Después de la función de activación
- `both`: Antes y después
- `none`: Sin BatchNorm

4.6.2. Flujo de Procesamiento

$$\text{Input} \rightarrow [\text{Reg. Antes}] \rightarrow \text{Activación} \rightarrow [\text{Reg. Después}] \rightarrow \text{Output} \quad (4.9)$$

4.6.3. Ejemplos Prácticos

Red Básica KAN

```

1 import torch
2 from kan_flexible_regularization import DualFlexKAN
3
4 # Red KAN basica con funciones polinomiales
5 model_basic = DualFlexKAN(
6     layer_sizes=[10, 32, 16, 1],
7     input_function_strategies=["per_input", "per_input", "none"],
8     input_activation_types=["polynomial", "polynomial",
9                             "polynomial"],
10    output_function_strategies=["per_neuron", "per_neuron", "fixed"],
11    output_activation_types=["polynomial", "polynomial", "linear"]
12 )
13 # Test
14 x = torch.randn(100, 10)
15 output = model_basic(x)
16 print(f"Input: {x.shape} -> Output: {output.shape}")

```

Listing 4.1: Red KAN básica con funciones polinomiales

Red con Regularización Flexible

```

1 model_reg = DualFlexKAN(
2     layer_sizes=[8, 64, 32, 16, 2],
3
4     # Funciones de entrada
5     input_function_strategies=["per_input", "per_input", "global",
6                               "none"],
7     input_activation_types=["polynomial", "legendre", "rbf",
8                             "polynomial"],
9
10    # Funciones de activacion
11    output_function_strategies=["per_neuron", "per_neuron", "fixed",
12                               "fixed"],
13    output_activation_types=["rbf", "polynomial", "relu", "softmax"],
14
15    # Regularizacion flexible
16    dropout_probs=[0.1, 0.2, 0.15, 0.0],
17    dropout_positions=["before_activation", "after_activation",
18                      "both", "none"],
19    use_batch_norms=[True, True, True, False],
20    batch_norm_positions=["after_activation", "before_activation",
21                          "both", "none"],
22    regularization_orders=["dropout_first", "batch_norm_first",
23                           "dropout_first", "dropout_first"]
24 )

```

Listing 4.2: Red con regularización avanzada

4.7. Optimización Específica para DualFlexKAN

4.7.1. Algoritmo de Entrenamiento Adaptativo

El entrenamiento de DualFlexKAN requiere considerar la naturaleza dual de las transformaciones. El algoritmo procede de la siguiente manera:

1. **Inicializar** parámetros $\Theta_{\text{in}}, \Theta_{\text{out}}, \mathbf{W}, \mathbf{b}$

2. Para cada época $e = 1, 2, \dots, E$:

a) Para cada lote $\mathcal{B}$:

- 1) **Forward pass:** $\mathbf{x}^{(l+1)} = \sigma_{\text{out}}^{(l)}(\mathbf{R}_{\text{post}}^{(l)}(\mathbf{W}^{(l)}\psi_{\text{in}}^{(l)}(\mathbf{x}^{(l)}) + \mathbf{b}^{(l)}))$
- 2) **Calcular pérdida:** $\mathcal{L} = \ell(\mathbf{y}, \hat{\mathbf{y}}) + \lambda \mathcal{R}(\Theta)$
- 3) **Backward pass:** Calcular gradientes
- 4) **Actualizar parámetros:** $\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}$

4.7.2. Regularización Adaptativa

La regularización en DualFlexKAN incluye términos específicos para cada tipo de función:

$$\mathcal{R}(\Theta) = \lambda_1 \mathcal{R}_{\text{in}}(\Theta_{\text{in}}) + \lambda_2 \mathcal{R}_{\text{out}}(\Theta_{\text{out}}) + \lambda_3 \mathcal{R}_{\text{weights}}(\mathbf{W}) \quad (4.10)$$

donde:

$$\mathcal{R}_{\text{in}}(\Theta_{\text{in}}) = \sum_{i,j} \|\theta_{i,j}^{\text{in}}\|_2^2 \quad (\text{regularización L2 de funciones de entrada}) \quad (4.11)$$

$$\mathcal{R}_{\text{out}}(\Theta_{\text{out}}) = \sum_j \|\alpha_j^{\text{out}}\|_2^2 \quad (\text{regularización L2 de funciones de salida}) \quad (4.12)$$

$$\mathcal{R}_{\text{weights}}(\mathbf{W}) = \|\mathbf{W}\|_F^2 \quad (\text{regularización Frobenius de pesos}) \quad (4.13)$$

4.8. Inicialización de los pesos

La inicialización adecuada de los parámetros en redes neuronales es crucial para asegurar una convergencia eficiente durante el entrenamiento. En el caso de las redes Kolmogorov-Arnold (KAN), la complejidad aumenta debido a la presencia de múltiples tipos de parámetros: pesos lineales tradicionales y coeficientes de funciones de activación aprendibles. Esta sección presenta un sistema completo de inicialización que aborda ambos componentes de manera coordinada. Una inicialización adecuada reduce el tiempo de entrenamiento en un 20-40 % y mejora la estabilidad numérica, especialmente en redes profundas. El sistema es modular y extensible, permitiendo la incorporación de nuevas estrategias según sea necesario.

4.8.1. La importancia de la inicialización

El problema de la inicialización en redes profundas ha sido ampliamente estudiado. Una inicialización inadecuada puede conducir a dos fenómenos perjudiciales:

- **Explosión de gradientes:** cuando $\|\nabla_W \mathcal{L}\| \rightarrow \infty$ durante la retropropagación
- **Desvanecimiento de gradientes:** cuando $\|\nabla_W \mathcal{L}\| \rightarrow 0$, impidiendo el aprendizaje en capas profundas

El objetivo de una buena estrategia de inicialización es mantener la varianza de las activaciones y gradientes aproximadamente constante a través de las capas, satisfaciendo:

$$\text{Var}(y^{(l)}) \approx \text{Var}(y^{(l-1)}) \quad \text{y} \quad \text{Var}\left(\frac{\partial \mathcal{L}}{\partial y^{(l)}}\right) \approx \text{Var}\left(\frac{\partial \mathcal{L}}{\partial y^{(l-1)}}\right) \quad (4.14)$$

A diferencia de las capas MLP tradicionales, una capa KAN transforma sus entradas mediante funciones aprendibles tanto en la entrada como en la salida:

$$y_j = \sigma_j \left(\sum_{i=1}^{n_{\text{in}}} w_{ij} \cdot \varphi_i(x_i) + b_j \right) \quad (4.15)$$

donde:

- $\varphi_i : \mathbb{R} \rightarrow \mathbb{R}$ son funciones de transformación de entrada (típicamente polinomios)
- w_{ij} son los pesos lineales de conexión
- b_j son los sesgos
- $\sigma_j : \mathbb{R} \rightarrow \mathbb{R}$ son funciones de activación de salida

Esto implica que debemos inicializar tres conjuntos de parámetros:

1. Pesos lineales $W \in \mathbb{R}^{n_{in} \times n_{out}}$
2. Parámetros de funciones de entrada Θ_{in}
3. Parámetros de funciones de salida Θ_{out}

4.8.2. Estrategias de Inicialización para Pesos Lineales

Inicialización por defecto

DualFlexKANEncoder y DualFlexKANDecoder realizan una inicialización por defecto en su constructor `__init__`, Dentro de DualFlexKANEncoder y DualFlexKANDecoder

```

1 self.network = DualFlexKAN(layer_sizes=layer_sizes, **kwargs)
2 smart_kan_initialization(self.network, strategy="auto")

```

Listing 4.3: Inicialización por defecto en DFKAN

La estrategia `auto` es la más sencilla de utilizar (no hay que indicar nada, los parámetros por defecto ya funcionan), pero se han implementado estrategias más avanzadas (`conservative`, `aggressive`, etc.). En el modo `auto` el código recurre a la estrategia avanzada llamada `optimal`, que :

- Pesos Lineales: Inicialización tipo He/Kaiming normal, que es excelente para activaciones tipo ReLU y, en general, un estándar robusto en deep learning.
- Parámetros Polinomiales: Una inicialización escalada donde los coeficientes de grados más altos se inicializan con una desviación estándar más pequeña para evitar que la función explote al principio.

Uso de las estrategias avanzadas de inicialización

Para utilizar las estrategias avanzadas de inicialización, tenemos tres alternativas:

1. Para empezar y experimentar rápidamente: el Método 1 (`smart_kan_initialization`) es sencillo y las estrategias predefinidas (`deep`, `conservative`, etc.) cubren los casos más comunes.
2. Para un control más fino y consistente el método 2 (`KANInitializer`) permite definir tu propia "filosofía" de inicialización (ej. "siempre He con polinomios lineales") y aplicarla de forma consistente.

3. Para arquitecturas profundas o problemáticas, el método 3 (`initialize_model_layerwise`). Si los gradientes se desvanecen o explotan, o que el entrenamiento es inestable, se puede ajustar la inicialización de capas específicas sin afectar a las demás. Por ejemplo, usar orthogonal en las capas intermedias de una red muy profunda es una excelente estrategia.

Usando `smart_kan_initialization` con estrategias predefinidas

```

1 import torch
2 from DFKAN_autoencoder import DualFlexKANAutoencoder
3 from KAN_initialization_v2 import smart_kan_initialization,
   InitializationAnalyzer
4
5 # 1. Crear el modelo
6 # Esto aplicará la inicialización por defecto "optimal"
7 autoencoder = DualFlexKANAutoencoder(
8     input_size=784,
9     output_size=784,
10    latent_size=64,
11    encoder_hidden_sizes=[256],
12    decoder_hidden_sizes=[256]
13 )
14
15 print("### ANÁLISIS ANTES DE RE-INICIALIZACIÓN (DEFAULT 'OPTIMAL')
   ###")
16 InitializationAnalyzer.analyze_model(autoencoder.encoder.network)
17
18 # 2. Re-inicializar el encoder con una estrategia "profunda" (deep)
19 print("\n### INICIALIZANDO ENCODER CON ESTRATEGIA 'DEEP' ###")
20 smart_kan_initialization(autoencoder.encoder.network,
   strategy="deep")
21
22 # 3. Re-inicializar el decoder con una estrategia "conservadora"
   (conservative)
23 print("\n### INICIALIZANDO DECODER CON ESTRATEGIA 'CONSERVATIVE'
   ###")
24 smart_kan_initialization(autoencoder.decoder.network,
   strategy="conservative")
25
26 # 4. Verificar los resultados con el analizador
27 print("\n### ANÁLISIS POST-INICIALIZACIÓN ###")
28 print("\n--- Encoder ('deep') ---")
29 InitializationAnalyzer.analyze_model(autoencoder.encoder.network)
30 print("\n--- Decoder ('conservative') ---")
31 InitializationAnalyzer.analyze_model(autoencoder.decoder.network)

```

Listing 4.4: Inicialización con `smart_kan_initializer`

Usando la clase KANInitializer

```

1 import torch
2 from DFKAN_autoencoder import DualFlexKANAutoencoder
3 from KAN_initialization_v2 import KANInitializer,
   InitializationAnalyzer
4
5 # 1. Crear el modelo
6 autoencoder = DualFlexKANAutoencoder(
7     input_size=784,
8     output_size=784,
9     latent_size=64
10 )
11
12 # 2. Definir un inicializador personalizado
13 # Por ejemplo, pesos lineales de He y polinomios con énfasis en el
   término lineal
14 custom_initializer = KANInitializer(
15     linear_strategy='he_normal',
16     polynomial_strategy='polynomial_linear',
17     verbose=True # Muestra información detallada
18 )
19
20 # 3. Aplicar el inicializador a las sub-redes
21 print("\n### APLICANDO INICIALIZADOR PERSONALIZADO AL ENCODER ###")
22 custom_initializer.initialize_model(autoencoder.encoder.network)
23
24 print("\n### APLICANDO INICIALIZADOR PERSONALIZADO AL DECODER ###")
25 custom_initializer.initialize_model(autoencoder.decoder.network)
26
27 # 4. Analizar los resultados
28 print("\n--- Análisis del Encoder (he_normal + polynomial_linear)
   ---")
29 InitializationAnalyzer.analyze_model(autoencoder.encoder.network,
   verbose=True)

```

Listing 4.5: Inicialización con la clase KANInitializer

Inicialización por Capa

```

1
2 import torch
3 from DFKAN_autoencoder import DualFlexKANAutoencoder
4 from KAN_initialization_v2 import initialize_model_layerwise,
   InitializationAnalyzer
5
6 # 1. Crear el modelo
7 autoencoder = DualFlexKANAutoencoder(

```

```

8     input_size=784,
9     output_size=784,
10    latent_size=32,
11    encoder_hidden_sizes=[256, 128], # Encoder con 3 capas
12    decoder_hidden_sizes=[128, 256] # Decoder con 3 capas
13 )
14
15 # 2. Definir configuraciones por capa para el ENCODER
16 # Capa 0: 784 -> 256 (Agresiva)
17 # Capa 1: 256 -> 128
18 # Capa 2: 128 -> 32 (Conservadora, para estabilizar el espacio
    latente)
19 encoder_configs = [
20     {'linear': 'he_uniform', 'polynomial': 'polynomial_adaptive'},
21     {'linear': 'xavier_normal', 'polynomial': 'polynomial_standard'},
22     {'linear': 'orthogonal', 'polynomial': 'polynomial_linear'}
23 ]
24
25 # 3. Definir configuraciones por capa para el DECODER
26 # Capa 0: 32 -> 128 (Ortogonal para expandir desde el espacio
    latente)
27 # Capa 1: 128 -> 256
28 # Capa 2: 256 -> 784 (LeCun, bueno para la capa de salida)
29 decoder_configs = [
30     {'linear': 'orthogonal', 'polynomial': 'polynomial_orthogonal'},
31     {'linear': 'xavier_uniform', 'polynomial':
        'polynomial_standard'},
32     {'linear': 'lecun_normal', 'polynomial': 'polynomial_zero_bias'}
33 ]
34
35 # 4. Aplicar la inicialización layer-wise
36 print("\n### INICIALIZACIÓN LAYER-WISE DEL ENCODER ###")
37 initialize_model_layerwise(autoencoder.encoder.network,
    encoder_configs)
38
39 print("\n### INICIALIZACIÓN LAYER-WISE DEL DECODER ###")
40 initialize_model_layerwise(autoencoder.decoder.network,
    decoder_configs)
41
42 # 5. Analizar
43 print("\n--- Análisis final del Encoder ---")
44 InitializationAnalyzer.analyze_model(autoencoder.encoder.network)

```

Listing 4.6: Inicialización por capa

Inicialización Xavier/Glorot

Propuesta por Glorot y Bengio (2010) [?], esta estrategia busca mantener la varianza de las activaciones constante en ambas direcciones (forward y backward).

Versión Uniforme:

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right) \quad (4.16)$$

Versión Normal:

$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{in} + n_{out}}}\right) \quad (4.17)$$

Uso recomendado: Funciones de activación lineales, tanh y sigmoide.

```

1 from KAN_initialization import KANInitializer
2
3 # Para redes con tanh o sigmoid
4 initializer = KANInitializer(
5     linear_strategy='xavier_normal',
6     polynomial_strategy='polynomial_linear'
7 )
8 initializer.initialize_model(model)
```

Listing 4.7: Inicialización Xavier en KAN

Inicialización He/Kaiming

Introducida por He et al. (2015) [?], esta estrategia está diseñada específicamente para funciones de activación ReLU, que anulan aproximadamente la mitad de las neuronas.

Versión Uniforme:

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right) \quad (4.18)$$

Versión Normal:

$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{in}}}\right) \quad (4.19)$$

La diferencia clave respecto a Xavier es que solo considera n_{in} en el denominador, compensando la reducción de varianza causada por ReLU.

Uso recomendado: ReLU, Leaky ReLU, PReLU y variantes.

```

1 # Estrategia óptima para ReLU (default)
2 from KAN_initialization import smart_kan_initialization
3
4 smart_kan_initialization(model, strategy='optimal')
5 # Equivalente a: he_normal + polynomial_standard
```

Listing 4.8: Inicialización He para ReLU

Inicialización LeCun

Propuesta por LeCun et al. para redes auto-normalizantes con activación SELU [?].

Versión Normal:

$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{1}{n_{in}}}\right) \quad (4.20)$$

Uso recomendado: Activación SELU en redes auto-normalizantes.

```
1 smart_kan_initialization(model, strategy='stable')
2 # Usa: lecun_normal + polynomial_zero_bias
```

Listing 4.9: Inicialización para SELU

Inicialización Ortogonal

Para redes muy profundas, la inicialización ortogonal [?] asegura que la matriz de pesos satisfaga $WW^T = I$, preservando la norma durante forward y backward propagation.

$$W = \text{QR}(G) \cdot \text{gain} \quad (4.21)$$

donde $G \sim \mathcal{N}(0, 1)^{n_{in} \times n_{out}}$ y QR es la descomposición QR.

Uso recomendado: Redes con más de 10 capas.

```
1 smart_kan_initialization(model, strategy='deep')
2 # Usa: orthogonal + polynomial_orthogonal
```

Listing 4.10: Inicialización para redes profundas

Inicialización Sparse

Para redes muy anchas (capas con >512 neuronas), la inicialización sparse reduce el número de conexiones inicialmente activas.

$$W_{ij} = \begin{cases} \mathcal{N}(0, \sigma^2) & \text{con probabilidad } p \\ 0 & \text{con probabilidad } 1 - p \end{cases} \quad (4.22)$$

donde típicamente $p = 0,1$ (10% de conexiones activas).

Uso recomendado: Capas con más de 512 neuronas.

```
1 smart_kan_initialization(model, strategy='sparse')
2 # Usa: sparse (p=0.1) + polynomial_small
```

Listing 4.11: Inicialización sparse

Estrategias de Inicialización para Funciones Polinomiales

Las funciones aprendibles en KAN suelen parametrizarse como polinomios:

$$\varphi(x; \theta) = \sum_{k=0}^d \theta_k x^k \quad (4.23)$$

La inicialización de estos coeficientes $\theta = [\theta_0, \theta_1, \dots, \theta_d]$ requiere consideraciones especiales.

Inicialización Estándar con Escala Progresiva

Los términos de orden superior deben tener menor varianza para evitar no-linealidades excesivas al inicio:

$$\theta_k \sim \mathcal{N}\left(0, \frac{\sigma}{\sqrt{k+1}}\right), \quad k = 0, 1, \dots, d \quad (4.24)$$

donde $\sigma = g \cdot \sqrt{\frac{2}{n_{in} + n_{out}}}$ y g es el factor de ganancia.

Además, se aplica énfasis al término lineal:

$$\theta_1 \leftarrow \theta_1 \cdot 1.5 \quad (4.25)$$

Justificación: Los términos de alto orden ($x^3, x^4, \dots$) pueden crecer rápidamente para $|x| > 1$. La escala $1/\sqrt{k+1}$ controla este crecimiento.

```

1 # Estrategia por defecto
2 initializer = KANInitializer(
3     linear_strategy='he_normal',
4     polynomial_strategy='polynomial_standard'
5 )

```

Listing 4.12: Inicialización estándar de polinomios

Inicialización con Sesgo Cero

Para evitar un offset inicial, se fija el término constante a cero:

$$\theta_0 = 0, \quad \theta_k \sim \mathcal{N}\left(0, \frac{\sigma}{\sqrt{k+1}}\right) \text{ para } k \geq 1 \quad (4.26)$$

Uso recomendado: Cuando se desea que las funciones pasen por el origen inicialmente.

```

1 initializer = KANInitializer(
2     linear_strategy='he_normal',
3     polynomial_strategy='polynomial_zero_bias'
4 )

```

Listing 4.13: Polinomios sin término constante

Inicialización con Énfasis Lineal

Para aproximar funciones suaves inicialmente, se enfatiza fuertemente el término lineal:

$$\theta_k \sim \begin{cases} \mathcal{N}(0, 0, 1\sigma) & k = 0 \\ \mathcal{N}(0, 3\sigma) & k = 1 \\ \mathcal{N}(0, \sigma/k^{1.5}) & k \geq 2 \end{cases} \quad (4.27)$$

Justificación: Cerca de funciones lineales al inicio facilita la optimización inicial.

```
1 initializer = KANInitializer(
2     linear_strategy='xavier_normal',
3     polynomial_strategy='polynomial_linear'
4 )
```

Listing 4.14: Énfasis en comportamiento lineal

Inicialización Inspirada en Polinomios Ortogonales

Los polinomios ortogonales (Legendre, Chebyshev) tienen coeficientes que decrecen como $1/\sqrt{2k+1}$:

$$\theta_k \sim \mathcal{N}\left(0, \sigma \sqrt{\frac{1}{2k+1}}\right) \quad (4.28)$$

Ventaja: Minimiza la correlación entre términos de diferentes órdenes.

```
1 initializer = KANInitializer(
2     linear_strategy='orthogonal',
3     polynomial_strategy='polynomial_orthogonal'
4 )
```

Listing 4.15: Inicialización ortogonal

Inicialización Adaptativa por Grado Para polinomios de grado muy alto, se aplica atenuación exponencial:

$$\theta_k \sim \mathcal{N}\left(0, \frac{\sigma \cdot e^{-d/d_{max}}}{\sqrt{k+1}}\right) \quad (4.29)$$

donde d es el grado del polinomio y d_{max} es un hiperparámetro (típicamente 10).

Uso recomendado: Polinomios de grado > 5 .

```
1 initializer = KANInitializer(
2     linear_strategy='he_normal',
3     polynomial_strategy='polynomial_adaptive'
4 )
```

Listing 4.16: Para polinomios de alto grado

4.8.3. Estrategias Predefinidas

Para facilitar el uso, se proporcionan seis estrategias predefinidas que combinan métodos para pesos lineales y polinomios:

Tabla 4.10: Estrategias de inicialización predefinidas

Estrategia	Pesos Lineales	Polinomios	Uso Recomendado
optimal (default)	He Normal	Standard	ReLU, uso general
conservative	Xavier Normal	Linear	Tanh, Sigmoid
aggressive	He Uniform	Adaptive	Convergencia rápida
stable	LeCun Normal	Zero Bias	SELU, auto-normalizing
deep	Orthogonal	Orthogonal	Redes > 10 capas
sparse	Sparse (10 %)	Small	Capas > 512 neuronas

4.8.4. Inicialización Layer-wise

Para arquitecturas complejas donde hay capas que utilizan diferentes funciones de activación, o por ejemplo en autoencoders, donde el encoder comprime y el decoder expande, puede ser beneficioso usar estrategias diferentes por capa. En el caso más simple, podemos usar `optimal`.

```

1 from KAN_initialization import initialize_model_layerwise
2
3 # Autoencoder: 784 -> 512 -> 256 -> 64 -> 256 -> 512 -> 784
4 layer_configs = [
5     # ENCODER (compresión - más agresivo)
6     {'linear': 'he_uniform', 'polynomial': 'polynomial_adaptive'},
7     {'linear': 'he_uniform', 'polynomial': 'polynomial_adaptive'},
8     {'linear': 'orthogonal', 'polynomial': 'polynomial_orthogonal'},
9
10    # DECODER (expansión - más conservador)
11    {'linear': 'xavier_normal', 'polynomial': 'polynomial_linear'},
12    {'linear': 'xavier_normal', 'polynomial': 'polynomial_linear'},
13    {'linear': 'xavier_normal', 'polynomial': 'polynomial_standard'}
14 ]
15
16 initialize_model_layerwise(model, layer_configs)

```

Listing 4.17: Inicialización diferenciada por capa

Ventajas de la Inicialización Layer-wise

1. **Optimización por función:** Cada capa usa la inicialización más apropiada para su función específica en la arquitectura.
2. **Control fino de gradientes:** Permite balancear la propagación de gradientes de manera precisa a través de la red.

3. **Adaptación a asimetrías:** Maneja naturalmente arquitecturas asimétricas como U-Nets o autoencoders.
4. **Experimentación flexible:** Facilita la comparación de diferentes estrategias de inicialización de manera controlada.

Sintaxis y Configuración La inicialización layer-wise se realiza mediante la función `initialize_model_layerwise`:

```

1 from KAN_initialization import initialize_model_layerwise
2
3 initialize_model_layerwise(
4     model,                # Modelo KAN a inicializar
5     layer_configs,        # Lista de configuraciones por capa
6     verbose=True          # Imprimir información (opcional)
7 )

```

Listing 4.18: Firma de la función layer-wise

Estructura de Configuración por Capa

Cada capa se configura mediante un diccionario con los siguientes parámetros:

```

1 layer_config = {
2     'linear': str,          # Estrategia para pesos lineales
3     'polynomial': str,     # Estrategia para funciones polinomiales
4     'gain': float          # Factor de ganancia (opcional,
5                             default=1.0)
6 }

```

Listing 4.19: Estructura de configuración de capa

Parámetro: linear Especifica la estrategia de inicialización para los pesos lineales $W \in n_{in} \times n_{out}$.

Tabla 4.11: Estrategias disponibles para pesos lineales

Estrategia	Distribución	Uso Recomendado
xavier_uniform	$\mathcal{U}(-a, a), a = \sqrt{6/(n_{in} + n_{out})}$	Tanh, Sigmoid
xavier_normal	$\mathcal{N}(0, \sqrt{2/(n_{in} + n_{out})})$	Tanh, Sigmoid
he_uniform	$\mathcal{U}(-a, a), a = \sqrt{6/n_{in}}$	ReLU, Leaky ReLU
he_normal	$\mathcal{N}(0, \sqrt{2/n_{in}})$	ReLU, Leaky ReLU
lecun_uniform	$\mathcal{U}(-a, a), a = \sqrt{3/n_{in}}$	SELU
lecun_normal	$\mathcal{N}(0, \sqrt{1/n_{in}})$	SELU
orthogonal	Matriz ortogonal	Redes profundas
sparse	Sparse (10 % activo)	Redes muy anchas

Parámetro: polynomial Especifica la estrategia de inicialización para los coeficientes de funciones polinómicas $\theta = [\theta_0, \theta_1, \dots, \theta_d]$.

Tabla 4.12: Estrategias disponibles para funciones polinómicas

Estrategia	Descripción
<code>polynomial_standard</code>	Escala progresiva: $\theta_k \sim \mathcal{N}(0, \sigma/\sqrt{k+1})$ con énfasis en término lineal
<code>polynomial_zero_bias</code>	Similar a standard pero con $\theta_0 = 0$ (sin término constante)
<code>polynomial_linear</code>	Fuerte énfasis en término lineal (θ_1 multiplicado por 3)
<code>polynomial_orthogonal</code>	Inspirada en polinomios ortogonales: $\theta_k \sim \mathcal{N}(0, \sigma/\sqrt{2k+1})$
<code>polynomial_small</code>	Valores muy pequeños: $\theta_k \sim \mathcal{N}(0, 0,01)$ para fine-tuning
<code>polynomial_adaptive</code>	Atenuación exponencial por grado: $\theta_k \sim \mathcal{N}(0, \sigma \cdot e^{-d/10}/\sqrt{k+1})$

Parámetro: gain Factor multiplicativo global para la desviación estándar.

Factor de Ganancia (Gain)

El factor de ganancia (*gain*) es un multiplicador escalar que ajusta globalmente la magnitud de la inicialización de los pesos en una red neuronal. Si una estrategia base especifica $W \sim \mathcal{N}(0, \sigma_{base})$, al aplicar ganancia se obtiene $W \sim \mathcal{N}(0, \sigma_{base} \cdot \text{gain})$. Un valor de $\text{gain} = 1,0$ mantiene la inicialización estándar. Valores superiores aumentan la varianza inicial, lo que puede acelerar la convergencia al permitir que las neuronas exploren un rango más amplio de activaciones, aunque con mayor riesgo de gradientes explosivos. Valores inferiores reducen la varianza, proporcionando mayor estabilidad numérica y convergencia más suave, a costa de un aprendizaje inicial más lento. Este parámetro es especialmente útil en contextos específicos: en *fine-tuning* se emplean valores muy pequeños ($\text{gain} \approx 0,01$) para realizar ajustes sutiles sin destruir los pesos pre-entrenados.

En redes muy profundas (> 10 capas) se puede aplicar *gain* decreciente con la profundidad para compensar la acumulación de varianza, y en arquitecturas asimétricas como autoencoders, permite balancear el comportamiento del encoder y decoder usando diferentes valores de ganancia en cada parte de la red.

Valores típicos:

- $\text{gain} = 1,0$: Valor estándar
- $\text{gain} = 0,5$: Reducir varianza (prevenir explosión)
- $\text{gain} = 1,5$: Aumentar varianza (acelerar convergencia inicial)
- $\text{gain} = 0,01$: Para fine-tuning con pesos pre-entrenados

Ejemplo Básico

```

1 from KAN_shared_poly6 import DualFlexKAN
2 from KAN_initialization import initialize_model_layerwise
3
4 # Crear modelo con 4 capas
5 model = DualFlexKAN(
6     layer_sizes=[784, 256, 128, 64, 10],
7     output_activation_types=["relu", "relu", "relu", "softmax"]
8 )
9
10 # Configuración diferenciada por capa
11 layer_configs = [
12     # Capa 1: ReLU estándar
13     {'linear': 'he_normal', 'polynomial': 'polynomial_standard'},
14
15     # Capa 2: ReLU con más énfasis lineal
16     {'linear': 'he_normal', 'polynomial': 'polynomial_linear'},
17
18     # Capa 3: ReLU con ortogonal (capa profunda)
19     {'linear': 'orthogonal', 'polynomial': 'polynomial_orthogonal'},
20
21     # Capa 4: Softmax con Xavier
22     {'linear': 'xavier_normal', 'polynomial': 'polynomial_standard'}
23 ]
24
25 # Aplicar inicialización
26 initialize_model_layerwise(model, layer_configs, verbose=True)

```

Listing 4.20: Ejemplo básico de inicialización layer-wise

4.8.5. Configuración Recomendada

```

1 # Modelo: 784 -> 512 -> 256 -> 64 -> 256 -> 512 -> 784
2 model = DualFlexKAN(
3     layer_sizes=[784, 512, 256, 64, 256, 512, 784]
4 )
5
6 layer_configs = [
7     # ENCODER (compresión - más agresivo)
8     {'linear': 'he_uniform', 'polynomial': 'polynomial_adaptive',
9      'gain': 1.2},
10
11     {'linear': 'he_uniform', 'polynomial': 'polynomial_adaptive',
12      'gain': 1.2},
13
14     # BOTTLENECK (capa latente - ortogonal para preservar info)

```

```

15     {'linear': 'orthogonal', 'polynomial': 'polynomial_orthogonal',
16      'gain': 1.0},
17
18     # DECODER (expansión - más conservador)
19     {'linear': 'xavier_normal', 'polynomial': 'polynomial_linear',
20      'gain': 0.8},
21
22     {'linear': 'xavier_normal', 'polynomial': 'polynomial_linear',
23      'gain': 0.8},
24
25     {'linear': 'xavier_normal', 'polynomial': 'polynomial_standard',
26      'gain': 0.8}
27 ]
28
29 initialize_model_layerwise(model, layer_configs)

```

Listing 4.21: Autoencoder con inicialización diferenciada

4.8.6. Justificación de la Estrategia

1. Encoder (agresivo):

- `he_uniform`: Distribución uniforme proporciona mayor diversidad inicial
- `polynomial_adaptive`: Adapta la no-linealidad al nivel de compresión
- `gain=1.2`: Mayor varianza ayuda a capturar información diversa

2. Bottleneck (ortogonal):

- `orthogonal`: Preserva máxima información en espacio latente reducido
- `polynomial_orthogonal`: Minimiza correlación entre componentes

3. Decoder (conservador):

- `xavier_normal`: Inicialización balanceada para reconstrucción suave
- `polynomial_linear`: Favorece comportamiento lineal para reconstrucción fiel
- `gain=0.8`: Menor varianza previene artefactos en reconstrucción

Red con Activaciones Mixtas

Cuando diferentes capas usan diferentes funciones de activación, cada una requiere su estrategia óptima:

```

1 model = DualFlexKAN(
2     layer_sizes=[784, 512, 256, 128, 10],
3     output_activation_types=["relu", "selu", "tanh", "softmax"]
4 )
5

```



```

6 layer_configs = [
7     # Capa 1: ReLU -> He initialization
8     {'linear': 'he_normal', 'polynomial': 'polynomial_standard'},
9
10    # Capa 2: SELU -> LeCun initialization
11    {'linear': 'lecun_normal', 'polynomial': 'polynomial_zero_bias'},
12
13    # Capa 3: Tanh -> Xavier initialization
14    {'linear': 'xavier_normal', 'polynomial': 'polynomial_linear'},
15
16    # Capa 4: Softmax -> Xavier conservador
17    {'linear': 'xavier_uniform', 'polynomial': 'polynomial_standard',
18     'gain': 0.8}
19 ]
20
21 initialize_model_layerwise(model, layer_configs)

```

Listing 4.22: Red con ReLU, SELU y Tanh

Red profunda (>10 capas)

Para redes profundas, las primeras capas pueden usar inicialización estándar, mientras que las capas medias y finales necesitan estrategias más conservadoras:

```

1 # Modelo con 15 capas
2 layer_sizes = [784] + [256]*10 + [128, 64, 32, 10]
3 model = DualFlexKAN(layer_sizes=layer_sizes)
4
5 layer_configs = []
6
7 # Primeras 3 capas: He estándar
8 for i in range(3):
9     layer_configs.append({
10         'linear': 'he_normal',
11         'polynomial': 'polynomial_standard',
12         'gain': 1.0
13     })
14
15 # Capas intermedias (4-12): Ortogonal
16 for i in range(3, 12):
17     layer_configs.append({
18         'linear': 'orthogonal',
19         'polynomial': 'polynomial_orthogonal',
20         'gain': 1.0 / (1 + 0.1 * (i-3)) # Gain decreciente
21     })
22
23 # Últimas capas: Conservador
24 for i in range(12, 14):
25     layer_configs.append({

```

```

26         'linear': 'xavier_normal',
27         'polynomial': 'polynomial_linear',
28         'gain': 0.5
29     })
30
31 initialize_model_layerwise(model, layer_configs)

```

Listing 4.23: Red profunda de 15 capas

4.8.7. Sistema de Análisis de Inicialización

Para validar la calidad de la inicialización, se proporciona un sistema de análisis que verifica:

1. Desviación estándar óptima: $0,01 < \sigma_W < 1,0$
2. Media centrada: $|\mu_W| < 0,1$
3. Estabilidad de polinomios: $\sigma_\theta < 0,5$

```

1 from KAN_initialization import InitializationAnalyzer
2
3 analyzer = InitializationAnalyzer()
4
5 # Análisis estadístico completo
6 stats = analyzer.analyze_model(model, verbose=True)
7
8 # Visualización de distribuciones
9 analyzer.plot_weight_distributions(
10     model,
11     save_path='weight_distributions.pdf'
12 )
13
14 # Verificación por capa
15 for i, layer_stats in enumerate(stats):
16     std = layer_stats['linear']['std']
17     mean = abs(layer_stats['linear']['mean'])
18
19     print(f"Capa {i+1}:")
20     if 0.01 <= std <= 1.0:
21         print(f"Std óptima: {std:.4f}")
22     else:
23         print(f"Std fuera de rango: {std:.4f}")
24
25     if mean < 0.1:
26         print(f"Media centrada: {mean:.4f}")
27     else:

```

28

```
print(f"Media descentrada: {mean:.4f}")
```

Listing 4.24: Análisis de inicialización

4.8.8. Consideraciones Prácticas

Selección de Estrategia según Arquitectura

. Dada una red KAN con L capas y función de activación σ :

- $\sigma \in \{\text{ReLU}, \text{LeakyReLU}\} \rightarrow$ usar `optimal` (He Normal)
- $\sigma \in \{\tanh, \text{sigmoid}\} \rightarrow$ usar `conservative` (Xavier Normal)
- $\sigma = \text{SELU} \rightarrow$ usar `stable` (LeCun Normal)
- $L > 10$ (red muy profunda) $\rightarrow$ usar `deep` (Orthogonal)
- $\max(n_{in}, n_{out}) > 512$ (red muy ancha) $\rightarrow$ usar `sparse` (Sparse 10 %)
- En otro caso $\rightarrow$ usar `optimal` (default)

Factor de Ganancia (Gain)

El parámetro `gain` permite ajustar globalmente la escala de inicialización:

$$\sigma_{efectivo} = \sigma_{base} \cdot \text{gain} \quad (4.30)$$

Valores típicos:

- `gain = 1,0`: Valor estándar
- `gain = 0,5`: Para prevenir explosión de gradientes
- `gain = 1,5`: Para acelerar convergencia inicial
- `gain = 0,01`: Para fine-tuning con pesos pre-entrenados

```
1 # Para prevenir explosión de gradientes
2 initializer = KANInitializer(
3     linear_strategy='he_normal',
4     polynomial_strategy='polynomial_standard',
5     gain=0.5
6 )
7
8 # Para fine-tuning (valores muy pequeños)
9 initializer = KANInitializer(
10     linear_strategy='normal',
11     polynomial_strategy='polynomial_small',
12     gain=0.01
13 )
```

Listing 4.25: Ajuste del factor de ganancia

4.8.9. Ejemplo. Inicialización de una red para clasificación MNIST

```

1 import torch
2 from torchvision import datasets, transforms
3 from torch.utils.data import DataLoader
4 from KAN_shared_poly6 import DualFlexKAN
5 from KAN_initialization import (
6     smart_kan_initialization,
7     InitializationAnalyzer
8 )
9
10 # 1. Cargar datos
11 transform = transforms.Compose([
12     transforms.ToTensor(),
13     transforms.Normalize((0.1307,), (0.3081,))
14 ])
15 train_dataset = datasets.MNIST('./data', train=True,
16                               download=True, transform=transform)
17 train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
18
19 # 2. Crear modelo
20 model = DualFlexKAN(
21     layer_sizes=[784, 256, 128, 64, 10],
22     input_function_strategies=["per_input", "per_input",
23                               "per_input", "none"],
24     input_activation_types=["polynomial", "polynomial",
25                             "polynomial", "polynomial"],
26     output_function_strategies=["per_neuron", "per_neuron",
27                                "per_neuron", "fixed"],
28     output_activation_types=["relu", "relu", "relu", "softmax"]
29 )
30
31 # 3. Inicializar (ReLU → usar 'optimal')
32 smart_kan_initialization(model, strategy='optimal')
33
34 # 4. Verificar inicialización
35 analyzer = InitializationAnalyzer()
36 stats = analyzer.analyze_model(model, verbose=True)
37
38 # 5. Entrenar
39 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
40 criterion = torch.nn.CrossEntropyLoss()
41
42 for epoch in range(10):
43     for batch_x, batch_y in train_loader:
44         batch_x = batch_x.view(-1, 784)
45
46         optimizer.zero_grad()

```

```
47     output = model(batch_x)
48     loss = criterion(output, batch_y)
49     loss.backward()
50     optimizer.step()
51
52     print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

Listing 4.26: Pipeline completo de inicialización

Capítulo 5 Detalles de Implementación: DualFlexKAN

Este capítulo documenta en profundidad la arquitectura interna y la API de las clases principales que conforman la librería. La implementación se centra en la modularidad y la flexibilidad, permitiendo configurar cada aspecto de la transformación neuronal de manera independiente.

5.1. Clase DualFlexKANLinear

La clase `DualFlexKANLinear` es el bloque fundamental de construcción. Hereda de `torch.nn.Module` y representa una única capa de la red. A diferencia de una capa lineal estándar ($y = Wx + b$), esta clase gestiona un pipeline complejo de transformaciones no lineales y regularización.

5.1.1. Constructor

```
1 class DualFlexKANLinear(nn.Module):
2     def __init__(self,
3         in_features,
4         out_features,
5         # Configuración de Entrada
6         input_function_strategy="per_input",
7         input_activation_type="polynomial",
8         input_activation_kwargs=None,
9         # Configuración de Salida
10        output_function_strategy="per_neuron",
11        output_activation_type="polynomial",
12        output_activation_kwargs=None,
13        # Regularización Flexible
14        dropout_prob=0.0,
15        dropout_position="after_activation",
16        use_batch_norm=False,
17        batch_norm_position="after_activation",
18        regularization_order="dropout_first",
19        batch_norm_momentum=0.1,
20        batch_norm_eps=1e-5
```

5.1.2. Estrategias de Transformación de Entrada (input_function_strategy)

Este parámetro controla cómo se transforman las características de entrada antes de la combinación lineal.

- **"none"**: No se aplica ninguna transformación. La entrada pasa directamente a la multiplicación de pesos.
- **"fixed"**: Se aplica una función fija (no entrenable) especificada en `input_activation_type` (ej. `relu`, `tanh`, `.abs`) a todas las entradas.
- **"global"**: Se aprende una única función parametrizada $\phi(x; \theta)$ que se aplica a todas las dimensiones de entrada. Los parámetros θ se comparten.
- **"per_input"**: Se aprende una función independiente $\phi_i(x_i; \theta_i)$ para cada dimensión de entrada i . Esta es la estrategia estándar para KAN híbridas.
- **"per_neuron_input"**: Se aprende una función independiente $\phi_{ij}(x_i; \theta_{ij})$ para cada conexión entre la entrada i y la neurona de salida j . Esta estrategia replica la arquitectura KAN original (Teorema de Kolmogorov-Arnold) pero tiene el mayor coste computacional.

5.1.3. Estrategias de Activación de Salida (output_function_strategy)

Este parámetro controla la no-linealidad aplicada después de la combinación lineal.

- **"none"**: Salida lineal. Útil para capas de regresión final o bloques residuales.
- **"fixed"**: Aplica una función de activación estándar de PyTorch (ej. `relu`, `sigmoid`). Convierte la capa en un MLP clásico si la entrada es `"none"`.° `"fixed"`.
- **"global"**: Aprende una única función de activación $\Psi(z; \omega)$ compartida por todas las neuronas de salida.
- **"per_neuron"**: Cada neurona de salida aprende su propia función de activación $\Psi_j(z_j; \omega_j)$. Permite que diferentes neuronas se especialicen en diferentes rangos o formas.

5.1.4. Tipos de Activación Disponibles

Los parámetros `input_activation_type` y `output_activation_type` aceptan los siguientes identificadores, instanciando las clases correspondientes de `KAN_activation.py`:

Identificador	Clase Instanciada
"polynomial"	PolynomialActivation
"legendre"	OptimizedLegendreActivation
"chebyshev"	ChebyshevActivation
"gegenbauer"	GegenbauerActivation
"jacobi"	JacobiActivation
"bspline"	BSplineActivation
"fast_bspline"	FastBSplineActivation
"sine"	SineActivation
"wavelet"	WaveletActivation
"rational"	RationalActivation
"rbf"	RBFActivation

Tabla 5.1: Mapeo de tipos de activación a clases de implementación.

5.1.5. Flujo de Ejecución (Forward Pass)

El método `forward(x)` ejecuta la siguiente secuencia lógica:

1. **Transformación de Entrada:** Aplica $\phi_{in}(x)$ según la estrategia elegida. Si es `per_neuron_input`, realiza el cálculo tensorial completo ($Batch, In, Out$).
2. **Combinación Lineal:** Calcula $z = W \cdot \phi_{in}(x) + b$. (Omitido si se usa `per_neuron_input` ya que la suma está implícita).
3. **Regularización Previa:** Aplica Dropout/BatchNorm si `position="before_activation"`. Esto es crítico para normalizar los valores z antes de que entren a polinomios de alto grado.
4. **Activación de Salida:** Aplica $\Psi_{out}(z)$ según la estrategia de salida.
5. **Regularización Posterior:** Aplica Dropout/BatchNorm si `position=".after_activation"`.

5.1.6. Regularización Flexible

La clase permite un control granular sobre la regularización mediante cuatro parámetros clave:

- `dropout_position`: Define dónde aplicar el dropout ("before_activation", ".after_activation", "both", "none").
- `batch_norm_position`: Define dónde aplicar la normalización por lotes.
- `regularization_order`: Define qué operación va primero cuando ambas coinciden en la misma posición ("dropout_first" o "batch_norm_first").

5.1.7. Introspección y Análisis

La clase incluye métodos para facilitar la interpretabilidad:

- `count_parameters()`: Devuelve un diccionario desglosando los parámetros en pesos lineales, bias, parámetros de función de entrada, parámetros de función de salida y batch norm.
- `get_function_info()`: Devuelve una representación textual (string) de las ecuaciones matemáticas configuradas en la capa (ej. " $\phi(x) = 0,5x^2 + 0,1$ ").
- `get_regularization_info()`: Devuelve el estado actual de la configuración de regularización.

5.2. Clase DualFlexKAN

La clase DualFlexKAN actúa como un contenedor de alto nivel (similar a `nn.Sequential`) que construye y gestiona una red multicapa completa.

5.2.1. Constructor

```

1 class DualFlexKAN(nn.Module):
2     def __init__(self,
3         layer_sizes,
4         # Listas de configuración por capa
5         input_function_strategies=None,
6         input_activation_types=None,
7         input_activation_kwargs=None,
8         output_function_strategies=None,
9         output_activation_types=None,
10        output_activation_kwargs=None,
11        # Listas de regularización por capa
12        dropout_probs=None,
13        dropout_positions=None,
14        use_batch_norms=None,
15        batch_norm_positions=None,
16        regularization_orders=None,
17        # ... otros parámetros ...
18    ):

```

5.2.2. Gestión de Capas

- **Argumentos como Listas:** La mayoría de los parámetros aceptan una lista de valores. El constructor itera sobre `layer_sizes` y asigna el elemento i -ésimo de cada lista de configuración a la capa i .

- **Valores por Defecto:** Si se pasa `None` o una lista incompleta, la clase aplica valores por defecto inteligentes (ej. "per_input" para entrada, "polynomial" para tipo, dropout=0.0).

5.2.3. Ejemplo de Instanciación Compleja

Este ejemplo muestra cómo construir una red donde cada capa tiene una configuración radicalmente diferente, aprovechando la flexibilidad de la clase:

```

1 model = DualFlexKAN(
2     layer_sizes=[10, 32, 16, 1],
3
4     # Capa 1: KAN pura (entrada Chebyshev)
5     # Capa 2: Híbrida (entrada Wavelet, salida RBF)
6     # Capa 3: MLP (entrada lineal, salida Sigmoid)
7     input_function_strategies=["per_neuron_input", "per_input",
8                               "none"],
9     input_activation_types=["chebyshev", "wavelet", "linear"],
10    input_activation_kwargs=[{"degree":4}, {"num_wavelets":5}, {}],
11
12    output_function_strategies=["none", "per_neuron", "fixed"],
13    output_activation_types=["linear", "rbf", "sigmoid"],
14
15    # Regularización específica por capa
16    dropout_probs=[0.1, 0.2, 0.0],
17    use_batch_norms=[True, True, False]
18 )

```

5.3. Integración con Utilidades

Las clases `DualFlexKANLinear` y `DualFlexKAN` están diseñadas para trabajar en conjunto con los módulos auxiliares:

- **Inicialización:** El método `smart_kan_initialization` recorre las capas del modelo e invoca inicializadores específicos según detecte tipos como "polynomial" o "wavelet".
- **Explicabilidad:** La clase `KANExplainability` utiliza los métodos `get_function_info` para extraer las ecuaciones simbólicas.
- **Poda:** `KANPruningSimplification` accede directamente a los parámetros `input_function_params` para analizar magnitudes y realizar podas.

5.4. Herramientas de Migración

Una de las barreras más comunes para la adopción de nuevas arquitecturas neuronales es la dificultad de reescribir modelos complejos preexistentes. Para facilitar la integra-

ción de DualFlexKAN en flujos de trabajo establecidos, la librería proporciona utilidades de "envoltura" (wrapping) y conversión automática.

Estas herramientas permiten transformar arquitecturas basadas en `nn.Linear` (como MLPs clásicos, capas densas en CNNs o proyecciones en Transformers) en redes DualFlexKAN sin necesidad de redefinir la clase del modelo manualmente.

5.4.1. El Convertidor Automático

La función `convert_linear_to_kan` actúa como un cirujano automático sobre el modelo. Su objetivo es recorrer recursivamente un modelo de PyTorch estándar, identificar las instancias de `nn.Linear` y sustituirlas por capas `DualFlexKANLinear`, preservando la topología de la red pero aumentando su capacidad expresiva.

¿Qué problema resuelve?

Imaginemos una arquitectura compleja como una ResNet-50 o un Transformer. Estas redes contienen cientos de líneas de código y definen múltiples capas lineales. Si deseamos probar si la tecnología KAN mejora el rendimiento, reescribir manualmente cada definición `self.fc = nn.Linear(...)` por `self.fc = DualFlexKANLinear(...)` es tedioso y propenso a errores.

El convertidor automatiza este proceso, permitiendo actualizar una arquitectura estándar a una arquitectura KAN (o híbrida) en una sola línea de código.

Funcionamiento Interno

El proceso de conversión sigue estos pasos lógicos:

1. **Exploración:** Recorre el grafo de módulos del modelo original (usando `model.named_children()`).
2. **Detección:** Identifica capas que sean instancia de `torch.nn.Linear`.
3. **Extracción:** Lee las propiedades de la capa original (`in_features`, `out_features`, `bias`).
4. **Sustitución:** Crea una nueva instancia de `DualFlexKANLinear` con las mismas dimensiones pero con la configuración KAN deseada (tipo de polinomio, estrategia, etc.) y la inyecta en el modelo padre usando `setattr`.

5.4.2. Implementación del Convertidor

A continuación se muestra la implementación funcional de la utilidad de conversión.

```

1 import torch
2 import torch.nn as nn
3 from KAN_shared_poly7 import DualFlexKANLinear
4
5 def convert_linear_to_kan(model, exclude_layers=None, **kan_kwargs):
6     """

```

```

7      Reemplaza recursivamente capas nn.Linear por DualFlexKANLinear.
8
9      Args:
10         model (nn.Module): Modelo PyTorch original.
11         exclude_layers (list): Lista de nombres de capas a excluir
            (ej. 'classifier').
12         **kan_kwargs: Argumentos para DualFlexKANLinear (ej:
            input_activation_type).
13     """
14     if exclude_layers is None:
15         exclude_layers = []
16
17     # Recorrer sub-módulos inmediatos
18     for name, module in model.named_children():
19         if name in exclude_layers:
20             continue
21
22         # Caso base: Encontramos una capa Lineal
23         if isinstance(module, nn.Linear):
24             # 1. Extraer dimensiones
25             in_features = module.in_features
26             out_features = module.out_features
27
28             # 2. Crear capa KAN equivalente
29             # Aquí inyectamos la configuración avanzada (ej.
                Chebyshev, Wavelets)
30             kan_layer = DualFlexKANLinear(
31                 in_features,
32                 out_features,
33                 **kan_kwargs
34             )
35
36             # 3. Sustituir en el modelo padre
37             setattr(model, name, kan_layer)
38             print(f"Convertida capa '{name}': Linear -> DualFlexKAN")
39
40         else:
41             # Caso recursivo: Seguir bajando en la jerarquía (ej.
                dentro de un Sequential)
42             convert_linear_to_kan(module, exclude_layers,
                **kan_kwargs)
43
44     return model

```

Listing 5.1: Función de conversión recursiva

5.4.3. Ejemplos de Uso

Caso 1: Conversión de un MLP Simple

Supongamos un modelo estándar definido en PyTorch. Podemos transformarlo para usar polinomios de Chebyshev y regularización avanzada.

```

1 # 1. Modelo Original
2 original_model = nn.Sequential(
3     nn.Linear(784, 128),
4     nn.ReLU(),
5     nn.Linear(128, 10)
6 )
7
8 # 2. Conversión
9 kan_model = convert_linear_to_kan(
10     original_model,
11     # Configuración KAN deseada:
12     input_function_strategy="per_input",
13     input_activation_type="chebyshev", # Usar base Chebyshev
14     input_activation_kwargs={"degree": 4},
15     dropout_prob=0.1, # Añadir dropout
16     dropout_position="before_activation"
17 )
18
19 # Ahora kan_model tiene capas DualFlexKANLinear internamente

```

Listing 5.2: Conversión de MLP a KAN-Chebyshev

Caso 2: Conversión Híbrida (Exclusión de Capas)

A menudo es útil convertir solo el extractor de características (capas ocultas) para aumentar la potencia de representación, pero mantener la capa de clasificación final como lineal simple para estabilidad o interpretabilidad (logits directos).

```

1 import torchvision.models as models
2
3 # Cargar una ResNet18 estándar
4 resnet = models.resnet18()
5
6 # Convertir todas las capas lineales a KAN, EXCEPTO la capa final
7 # 'fc'
8 resnet_kan = convert_linear_to_kan(
9     resnet,
10     exclude_layers=['fc'], # La capa de salida se mantiene original
11     input_activation_type="wavelet", # Usar wavelets en las capas
12     # internas
13     input_activation_kwargs={"num_wavelets": 5}
14 )

```

Listing 5.3: Conversión con exclusión de capas

5.4.4. Consideraciones Técnicas Importantes

Al utilizar el convertidor automático, es crucial tener en cuenta los siguientes aspectos técnicos:

Pérdida de Pesos Pre-entrenados

Al convertir una capa `nn.Linear` a `DualFlexKANLinear`, se pierden los pesos originales.

- Una capa lineal tiene una matriz de pesos W simple.
- Una capa KAN tiene coeficientes polinomiales, centros RBF o parámetros de spline.

No existe una equivalencia matemática directa 1:1, por lo que el modelo convertido se inicializa desde cero (o usando estrategias de inicialización KAN) y **requiere re-entrenamiento o fine-tuning**.

Redundancia de Funciones de Activación

Los modelos estándar suelen definir la activación fuera de la capa lineal (ej: `Linear` $\rightarrow$ `ReLU`).

- La capa `DualFlexKANLinear` ya incluye sus propias funciones de activación configurables (entrada y salida).
- Si convertimos un bloque `Linear` + `ReLU`, obtendremos `KAN` + `ReLU`.
- Esto no es un error, pero el `ReLU` externo se aplicará *después* de la activación interna de la KAN.
- **Recomendación:** Si se usa KAN pura, se puede configurar la estrategia de salida de la KAN como `"fixed"` (con `ReLU`) y eliminar la capa `ReLU` externa, o configurar la KAN como `"none"` en salida y dejar que el `ReLU` externo actúe.

Costo Computacional

Una capa KAN es computacionalmente más densa que una capa lineal. Convertir todas las proyecciones lineales de un modelo grande (como un Transformer LLM) puede aumentar significativamente el uso de memoria VRAM y el tiempo de entrenamiento. Se recomienda empezar convirtiendo solo capas estratégicas (ej. los bloques MLP de un Transformer, manteniendo la Atención lineal).

Capítulo 6 Funciones de Utilidad

6.1. Módulo `save_load_utils.py`

El módulo `save_load_utils.py` proporciona un conjunto de funciones para guardar y cargar objetos Python de forma universal, con soporte especial para modelos de PyTorch, experimentos de machine learning y análisis de arquitecturas de redes neuronales. Soporta dos formatos principales: `pickle` (binario, universal) y `JSON` (solo tipos básicos). Además, incluye herramientas avanzadas para la introspección de código y la recuperación de configuraciones complejas en redes KAN.

6.1.1. `save_object`

Utilidad:

Guarda un objeto Python arbitrario en disco, soportando cualquier tipo serializable mediante `pickle` o tipos básicos mediante `JSON`.

Sintaxis:

```
1 save_object(obj, filepath, format='pickle',  
2             overwrite=False, verbose=True)
```

Parámetros:

- `obj`: Objeto a guardar (puede ser cualquier tipo serializable)
- `filepath` (str): Ruta completa del archivo, incluyendo extensión
- `format` (str): `'pickle'` (universal, por defecto) o `'json'` (solo tipos básicos)
- `overwrite` (bool): Si `True`, sobrescribe archivo existente. Por defecto `False`
- `verbose` (bool): Si `True`, imprime información del proceso. Por defecto `True`

Retorna:

- `bool`: `True` si se guardó correctamente, `False` en caso contrario

Ejemplos:

```
1 # Guardar metricas en pickle
2 metrics = {'accuracy': 0.95, 'loss': 0.12, 'f1_score': 0.93}
3 save_object(metrics, 'results/metrics.pkl')
4
5 # Guardar configuracion en JSON
6 model_config = {'architecture': [512, 256, 2], 'epochs': 50}
7 save_object(model_config, 'results/config.json', format='json')
8
9 # Guardar numpy array
10 import numpy as np
11 data = np.random.randn(100, 10)
12 save_object(data, 'data/features.pkl')
13
14 # Sobrescribir archivo existente
15 save_object(metrics, 'results/metrics.pkl', overwrite=True)
```

Características:

- Crea automáticamente el directorio de destino si no existe
- Verifica la existencia del archivo antes de sobrescribir
- Convierte automáticamente arrays de NumPy a listas para formato JSON
- Muestra el tamaño del archivo guardado en KB

6.1.2. load_object

Utilidad:

Carga un objeto previamente guardado desde disco, con detección automática del formato.

Sintaxis:

```
1 load_object(filepath, format='auto', verbose=True)
```

Parámetros:

- filepath (str): Ruta del archivo a cargar
- format (str): 'pickle', 'json', o 'auto' (detecta por extensión). Por defecto 'auto'
- verbose (bool): Si True, imprime información del proceso. Por defecto True

Retorna:

- `object`: El objeto cargado, o `None` si hay error

Ejemplos:

```
1 # Cargar con deteccion automatica de formato
2 metrics = load_object('results/metrics.pkl')
3 config = load_object('results/config.json')
4
5 # Especificar formato manualmente
6 data = load_object('data/features.dat', format='pickle')
7
8 # Cargar sin verbose
9 model_state = load_object('models/checkpoint.pkl', verbose=False)
```

Detección Automática de Formato:

- Extensiones `.pkl`, `.pickle` → formato `pickle`
- Extensión `.json` → formato `JSON`
- Otras extensiones → intenta `pickle` por defecto

6.2. Funciones para Múltiples Objetos

6.2.1. `save_multiple`

Utilidad:

Guarda múltiples objetos en un solo archivo `pickle`, organizándolos como diccionario con nombres.

Sintaxis:

```
1 save_multiple(filepath, overwrite=False, verbose=True, **objects)
```

Parámetros:

- `filepath` (str): Ruta del archivo `.pkl`
- `overwrite` (bool): Si `True`, sobrescribe archivo existente. Por defecto `False`
- `verbose` (bool): Si `True`, imprime información. Por defecto `True`
- `**objects`: Objetos a guardar como argumentos con nombre (kwargs)

Retorna:

- bool: True si se guardó correctamente

Ejemplos:

```
1 # Guardar multiples objetos relacionados
2 save_multiple(
3     'results/experiment.pkl',
4     metrics={'accuracy': 0.95, 'loss': 0.12},
5     model_config={'layers': [512, 256, 128], 'dropout': 0.3},
6     history={'train_loss': [0.5, 0.3, 0.2, 0.1],
7             'val_loss': [0.6, 0.4, 0.25, 0.15]},
8     best_epoch=42
9 )
10
11 # Guardar datos de autoencoder
12 import numpy as np
13 Z = np.random.randn(1000, 2) # Representaciones latentes
14 labels = np.random.randint(0, 10, 1000)
15
16 save_multiple(
17     'results/autoencoder_data.pkl',
18     latent_vectors=Z,
19     labels=labels,
20     reconstruction_error=0.023
21 )
```

Notas:

- Si el filepath no termina en .pkl o .pickle, se añade .pkl automáticamente
- Los objetos se guardan internamente como un diccionario

6.2.2. load_multiple**Utilidad:**

Carga múltiples objetos desde un archivo pickle previamente guardado con `save_multiple`.

Sintaxis:

```
1 load_multiple(filepath, verbose=True)
```

Parámetros:

- `filepath (str)`: Ruta del archivo `.pkl`
- `verbose (bool)`: Si `True`, imprime información. Por defecto `True`

Retorna:

- `dict`: Diccionario con los objetos cargados

Ejemplos:

```
1 # Cargar multiples objetos
2 data = load_multiple('results/experiment.pkl')
3
4 # Acceder a objetos individuales
5 metrics = data['metrics']
6 config = data['model_config']
7 history = data['history']
8 best_epoch = data['best_epoch']
9
10 print(f"Accuracy: {metrics['accuracy']}")
11 print(f"Best epoch: {best_epoch}")
12
13 # Cargar datos de autoencoder
14 ae_data = load_multiple('results/autoencoder_data.pkl')
15 Z = ae_data['latent_vectors']
16 labels = ae_data['labels']
17 error = ae_data['reconstruction_error']
```

Salida en verbose:

Cuando `verbose=True`, muestra las claves del diccionario cargado:

Objeto cargado: `results/experiment.pkl` (45.23 KB)

Objetos cargados: `['metrics', 'model_config', 'history', 'best_epoch']`

6.3. Funciones para Gestión de Experimentos

6.3.1. `save_experiment`

Utilidad:

Guarda un experimento completo con timestamp automático, incluyendo métricas, configuración, historial de entrenamiento y estado del modelo.

Sintaxis:

```
1 save_experiment(experiment_name, save_dir='experiments',
2               metrics=None, config=None, history=None,
3               model_state=None, **kwargs)
```

Parámetros:

- `experiment_name` (str): Nombre descriptivo del experimento
- `save_dir` (str): Directorio base para guardar. Por defecto 'experiments'
- `metrics` (dict): Métricas del experimento (opcional)
- `config` (dict): Configuración del modelo y entrenamiento (opcional)
- `history` (dict): Historial de entrenamiento (pérdidas, métricas por época) (opcional)
- `model_state` (dict): Estado del modelo (`state_dict`) (opcional)
- `**kwargs`: Otros objetos adicionales a guardar

Retorna:

- `str`: Ruta del archivo guardado, o `None` si hubo error

Ejemplos:

```
1 # Guardar experimento basico
2 save_experiment(
3     'autoencoder_mnist',
4     metrics={'orthogonality': 0.6878,
5             'anisotropy': 4446.34,
6             'trustworthiness': 0.9322},
7     config={'latent_dim': 2, 'epochs': 50, 'learning_rate': 1e-3},
8     history={'train_loss': [0.5, 0.3, 0.2, 0.1]}
9 )
10
11 # Guardar experimento completo con estado del modelo
12 save_experiment(
13     'resnet_cifar10',
14     save_dir='experiments/image_classification',
15     metrics={'accuracy': 0.92, 'precision': 0.91, 'recall': 0.90},
16     config={'model': 'ResNet18', 'batch_size': 128, 'optimizer':
17             'Adam'},
18     history={'train_loss': losses_train, 'val_loss': losses_val},
19     model_state=model.state_dict(),
20     notes="Experimento con data augmentation",
```

```

20     augmentation_config={'rotation': 15, 'flip': True}
21 )
22
23 # El archivo se guarda con timestamp automatico:
24 # experiments/image_classification/resnet_cifar10_20241029_153045.pkl

```

Estructura del archivo guardado:

```

1 {
2     'experiment_name': 'autoencoder_mnist',
3     'timestamp': '20241029_153045',
4     'datetime': '2024-10-29T15:30:45.123456',
5     'metrics': {...},
6     'config': {...},
7     'history': {...},
8     'model_state': {...},
9     # ... otros objetos adicionales
10 }

```

6.3.2. load_experiment

Utilidad:

Carga un experimento completo previamente guardado, mostrando información resumida.

Sintaxis:

```

1 load_experiment(filepath)

```

Parámetros:

- filepath (str): Ruta del archivo del experimento

Retorna:

- dict: Diccionario con todos los datos del experimento

Ejemplos:

```

1 # Cargar experimento
2 exp =
3     load_experiment('experiments/autoencoder_mnist_20241029_153045.pkl')
4
5 # Acceder a componentes
6 print(f"Metricas: {exp['metrics']}")

```

```
6 print(f"Config: {exp['config']}")
7 print(f"Epoca final: {len(exp['history']['train_loss'])}")
8
9 # Restaurar modelo
10 if 'model_state' in exp:
11     model.load_state_dict(exp['model_state'])
```

Salida típica:

Objeto cargado: experiments/autoencoder_mnist_20241029_153045.pkl (234.56 KB)

Experimento: autoencoder_mnist

Fecha: 2024-10-29T15:30:45.123456

Contenido: ['metrics', 'config', 'history', 'model_state']

6.3.3. list_experiments

Utilidad:

Lista todos los experimentos guardados en un directorio, mostrando tamaño y fecha de modificación.

Sintaxis:

```
1 list_experiments(save_dir='experiments', pattern='*.pkl')
```

Parámetros:

- save_dir (str): Directorio donde buscar. Por defecto 'experiments'
- pattern (str): Patrón de archivos a buscar. Por defecto '*.pkl'

Retorna:

- list: Lista de rutas de archivos encontrados

Ejemplos:

```
1 # Listar todos los experimentos
2 files = list_experiments('experiments')
3
4 # Listar experimentos especificos
5 autoencoder_exps = list_experiments(
6     'experiments',
7     pattern='autoencoder_*.pkl'
8 )
```



```
9
10 # Listar por subdirectorio
11 classification_exps =
    list_experiments('experiments/image_classification')
```

Salida típica:

Experimentos en 'experiments':

1. autoencoder_mnist_20241029_143022.pkl (234.56 KB) - 2024-10-29 14:30
2. autoencoder_mnist_20241029_153045.pkl (245.12 KB) - 2024-10-29 15:30
3. resnet_cifar10_20241029_160512.pkl (1823.45 KB) - 2024-10-29 16:05

6.4. Funciones para Análisis de Modelos PyTorch

6.4.1. `extract_model_info`

Utilidad:

Extrae información completa y detallada de un modelo PyTorch, incluyendo arquitectura, parámetros, tipos de capas, configuraciones y opcionalmente los pesos. Esta función también detecta estructuras específicas como Autoencoders y redes KAN.

Sintaxis:

```
1 extract_model_info(model, include_weights=False, detailed=True)
```

Parámetros:

- `model`: Instancia del modelo PyTorch (`nn.Module`)
- `include_weights` (bool): Si `True`, incluye los pesos del modelo (`state_dict`). Por defecto `False`
- `detailed` (bool): Si `True`, extrae información detallada de cada capa. Por defecto `True`

Retorna:

- `dict`: Diccionario completo con toda la información del modelo

Información extraída:

1. Información básica:

- `model_class`: Nombre de la clase del modelo
- `model_type`: Tipo completo del modelo

- `device`: Dispositivo donde está el modelo (cuda/cpu)
- `training_mode`: Si está en modo entrenamiento

2. Resumen de parámetros:

- `total_parameters`: Número total de parámetros
- `trainable_parameters`: Parámetros entrenables
- `non_trainable_parameters`: Parámetros congelados
- `parameters_millions`: Parámetros en millones

3. Arquitectura detallada (si `detailed=True`):

- `layers_detailed`: Lista de diccionarios con información de cada capa:
 - Nombre de la capa
 - Tipo (`Linear`, `Conv2d`, `BatchNorm`, etc.)
 - Parámetros de configuración (dimensiones, activaciones, dropout, etc.)
 - Número de parámetros de la capa
- `architecture_summary`: Lista legible con descripción de cada capa
- `num_layers`: Número total de capas

4. Tabla de parámetros:

- `parameters_table`: Lista con cada parámetro nombrado, su forma, número de elementos, si requiere gradientes, dtype y dispositivo

5. Información específica de autoencoder (si aplica):

- `is_autoencoder`: Booleano indicando si es autoencoder (detecta atributos `encoder` y `decoder`)
- `latent_dim`: Dimensión del espacio latente (inferida)
- `input_dim`: Dimensión de entrada (inferida)
- `encoder_parameters`: Parámetros del encoder
- `decoder_parameters`: Parámetros del decoder

6. Pesos del modelo (si `include_weights=True`):

- `state_dict`: Diccionario con todos los pesos (convertidos a NumPy)
- `weights_included`: Booleano confirmando inclusión de pesos

7. Resumen legible:

- `summary_text`: Texto formateado con resumen completo del modelo

Ejemplos:

```
1 import torch.nn as nn
2
3 # Crear un modelo de ejemplo
4 class SimpleNet(nn.Module):
5     def __init__(self):
6         super().__init__()
7         self.fc1 = nn.Linear(784, 256)
8         self.dropout = nn.Dropout(0.3)
9         self.fc2 = nn.Linear(256, 128)
10        self.fc3 = nn.Linear(128, 10)
11
12    def forward(self, x):
13        x = torch.relu(self.fc1(x))
14        x = self.dropout(x)
15        x = torch.relu(self.fc2(x))
16        return self.fc3(x)
17
18 model = SimpleNet()
19
20 # Extraer informacion completa
21 model_info = extract_model_info(model, include_weights=False,
22                                detailed=True)
23
24 # Guardar para analisis posterior
25 save_object(model_info, 'results/model_architecture.pkl')
26
27 # Acceder a informacion especifica
28 print(f"Total parameters: {model_info['total_parameters']:,}")
29 print(f"Architecture: {model_info['architecture_summary']}")
30
31 # Ver detalles de una capa especifica
32 for layer in model_info['layers_detailed']:
33     if layer['type'] == 'Linear':
34         print(f"{layer['name']}: {layer['params']}")
35
36 # Extraer informacion de autoencoder
37 autoencoder_info = extract_model_info(autoencoder_model,
38                                       detailed=True)
39 if autoencoder_info['is_autoencoder']:
40     print(f"Latent dim: {autoencoder_info['latent_dim']}")
41     print(f"Encoder params:
42           {autoencoder_info['encoder_parameters']:,}")
43     print(f"Decoder params:
44           {autoencoder_info['decoder_parameters']:,}")
```

Tipos de capas soportados:

La función reconoce y extrae información detallada de:

- **Capas lineales:** `nn.Linear`
- **Convolucionales:** `nn.Conv1d`, `nn.Conv2d`, `nn.Conv3d`
- **ConvTranspose:** `nn.ConvTranspose1d/2d/3d`
- **Normalización:** `nn.BatchNorm1d/2d/3d`, `nn.LayerNorm`, `nn.GroupNorm`
- **Pooling:** `nn.MaxPool1d/2d/3d`, `nn.AvgPool1d/2d/3d`, `nn.AdaptiveAvgPool1d/2d/3d`
- **Activaciones:** `nn.ReLU`, `nn.LeakyReLU`, `nn.Tanh`, `nn.Sigmoid`, `nn.GELU`
- **Regularización:** `nn.Dropout`, `nn.Dropout2d`, `nn.Dropout3d`
- **Recurrentes:** `nn.LSTM`, `nn.GRU`, `nn.RNN`
- **Embeddings:** `nn.Embedding`
- **Atención:** `nn.MultiheadAttention`
- **Transformer:** `nn.TransformerEncoder`, `nn.TransformerDecoder`

6.4.2. `generate_model_summary`

Utilidad:

Genera un resumen de texto legible y formateado a partir de la información del modelo extraída por `extract_model_info`.

```
1 generate_model_summary(model_info)
```

Parámetros:

Sintaxis:

- `model_info` (dict): Diccionario retornado por `extract_model_info`

Retorna:

- `str`: Texto formateado con el resumen del modelo

Ejemplo:

```

1 # Extraer informacion
2 model_info = extract_model_info(model)
3
4 # Generar resumen
5 summary = generate_model_summary(model_info)
6 print(summary)
7
8 # Guardar resumen en archivo de texto
9 with open('results/model_summary.txt', 'w') as f:
10     f.write(summary)

```

Formato del resumen:

```

=====
MODEL: SimpleNet
=====
Total Parameters: 235,146
Trainable Parameters: 235,146
Parameters (Millions): 0.235
Device: cuda:0
Training Mode: True

ARCHITECTURE:
  1. Linear(784 -> 256)
  2. Dropout(p=0.3)
  3. Linear(256 -> 128)
  4. Linear(128 -> 10)
=====

```

Para autoencoders, incluye información adicional:

```

AUTOENCODER STRUCTURE:
  Input Dim: 784
  Latent Dim: 2
  Encoder Params: 115,842
  Decoder Params: 119,304

```

6.4.3. print_model_info**Utilidad:**

Imprime la información del modelo de forma legible, incluyendo el resumen y detalles de cada capa.

Sintaxis:

```
1 print_model_info(model_info)
```

Parámetros:

- `model_info` (dict): Diccionario retornado por `extract_model_info`

Ejemplo:

```
1 # Extraer y mostrar informacion
2 model_info = extract_model_info(model, detailed=True)
3 print_model_info(model_info)
```

Salida típica:

```
=====
MODEL: DualFlexKAN
=====
Total Parameters: 1,235,146
Trainable Parameters: 1,235,146
Parameters (Millions): 1.235
Device: cuda:0
Training Mode: False

ARCHITECTURE:
1. Linear(784 -> 512)
2. PolynomialActivation(degree=3, per_neuron)
3. Dropout(p=0.2)
4. Linear(512 -> 256)
5. PolynomialActivation(degree=2, per_neuron)
...
=====

DETAILED LAYER INFORMATION:
-----

fc1:
  Type: Linear
  Parameters: 401,920
  Config: {'in_features': 784, 'out_features': 512, 'bias': True}

activation1:
  Type: PolynomialActivation
  Parameters: 1,536
```

```

    Config: {'degree': 3, 'strategy': 'per_neuron'}

dropout1:
  Type: Dropout
  Parameters: 0
  Config: {'p': 0.2}

...

```

6.5. Guardar y cargar modelos PyTorch de forma autónoma

En esta subsección se explica el uso de las funciones `save_model_auto` y `load_model_auto`, diseñadas para almacenar y restaurar modelos PyTorch de manera autónoma, incluyendo sus pesos, optimizador, scheduler y estado de entrenamiento.

Estas funciones permiten:

- Guardar el código de la clase del modelo y todas sus dependencias auxiliares.
- Almacenar los pesos del modelo (`state_dict`) y del optimizador/scheduler.
- Recuperar automáticamente el modelo para inferencia o continuar el entrenamiento sin redefinir clases.

6.5.1. Decorador @autoargs

El decorador `@autoargs` se aplica a la clase del modelo y permite capturar automáticamente los argumentos utilizados en el constructor (`__init__`) del modelo. Esto es fundamental para que, al cargar el modelo desde disco, se puedan reconstruir los parámetros iniciales del modelo sin necesidad de conocerlos explícitamente.

Funcionamiento:

- Intercepta la llamada a `__init__` y almacena todos los argumentos en un diccionario `__init_args__`.
- Este diccionario se utiliza al cargar el modelo para reconstruirlo con exactamente los mismos parámetros.

IMPORTANTE: antes de definir la clase del modelo, hay que incluir `@autoargs` para que la función de guardado pueda acceder a los parámetros.

6.5.2. Extracción del código de la clase (`extract_full_code`)

La función `extract_full_code` se encarga de obtener el código fuente de la clase principal y de todas las dependencias definidas en el mismo módulo, como capas auxiliares (`Encoder`, `Decoder`, bloques residuales, etc.). Utiliza el árbol de sintaxis abstracta (AST) para filtrar

y capturar únicamente definiciones de clases, funciones e importaciones, evitando código residual.

Esto garantiza que el modelo se pueda reconstruir en cualquier entorno sin necesidad de importar los módulos originales.

6.5.3. Guardar el modelo: `save_model_auto`

La función `save_model_auto` guarda en un único archivo `.pt` toda la información necesaria, utilizando `dill` para serializar objetos complejos:

- Código fuente del modelo y dependencias.
- Argumentos de inicialización capturados por `@autoargs`.
- Pesos del modelo (`state_dict`).
- Estado del optimizador y del scheduler, si se proporcionan.
- Número de epoch o cualquier otra información de entrenamiento.

El parámetro `overwrite` permite controlar si se desea sobrescribir un archivo existente.

Ejemplo de uso:

```
model = Autoencoder(100, 20)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
epoch = 5

save_model_auto(
    model,
    filename="autoencoder_auto.pt",
    optimizer=optimizer,
    epoch=epoch,
    overwrite=True
)
```

6.5.4. Cargar el modelo: `load_model_auto`

La función `load_model_auto` permite reconstruir el modelo sin redefinir la clase, y detecta automáticamente si el archivo contiene optimizador, scheduler o información de epoch.

Funcionamiento interno:

- Crea un módulo temporal en memoria para ejecutar el código del modelo guardado.
- Ejecuta únicamente las definiciones de clase y funciones, sin ejecutar bloques de entrenamiento.
- Instancia el modelo utilizando los argumentos capturados por `@autoargs`.

- Carga los pesos del modelo y, si están presentes, también el estado del optimizador y del scheduler.

Ejemplo para inferencia:

```
model, metadata = load_model_auto("autoencoder_auto.pt")
model.eval()

# Acceder al código fuente original
print(metadata['code'])
```

Ejemplo para continuar entrenamiento:

```
model, optimizer, scheduler, start_epoch, meta = load_model_auto("autoencoder_auto.pt")
model.train()
print("Continuar entrenamiento desde epoch:", start_epoch)
```

De esta manera, se puede continuar el entrenamiento exactamente desde el punto donde se guardó, sin necesidad de redefinir clases ni parámetros manualmente.

6.6. Estructura de los Modelos y Acceso a Componentes

Al cargar un modelo guardado con `load_model_auto`, el objeto retornado (`model`) tendrá una estructura interna diferente dependiendo de la clase original que se instanció. Es crucial entender esta jerarquía para acceder correctamente a las capas, configuraciones o sub-redes del modelo.

A continuación se detalla dónde se almacena la información de la red neuronal según el tipo de clase:

Clase Instanciada	Acceso a red KAN	Acceso a capas (.layers)
DualFlexKANAutoencoder	<code>model.encoder.network</code> <code>model.decoder.network</code>	<code>model.encoder.network.layers</code> <code>model.decoder.network.layers</code>
DualFlexKANEncoder	<code>model.network</code>	<code>model.network.layers</code>
DualFlexKANDecoder	<code>model.network</code>	<code>model.network.layers</code>
DualFlexKAN	<code>model</code> (es la red misma)	<code>model.layers</code>

Tabla 6.1: Jerarquía de acceso a componentes según la clase del modelo.

- **DualFlexKANAutoencoder:** Es un contenedor compuesto. No tiene un atributo `.network` directo, sino que contiene dos instancias independientes (`.encoder` y `.decoder`), cada una con su propia red.
- **DualFlexKANEncoder/Decoder:** Son clases envoltorio (wrapper) que contienen la red KAN dentro del atributo `.network`.
- **DualFlexKAN:** Es la clase base de la red, por lo que las capas están accesibles directamente en el nivel superior.

6.7. Resumen de Funciones

Función	Descripción
<code>save_object</code>	Guarda un objeto Python en pickle o JSON
<code>load_object</code>	Carga un objeto con detección automática de formato
<code>save_multiple</code>	Guarda múltiples objetos en un solo archivo
<code>load_multiple</code>	Carga múltiples objetos desde un archivo
<code>save_experiment</code>	Guarda experimento completo con timestamp
<code>load_experiment</code>	Carga experimento y muestra resumen
<code>list_experiments</code>	Lista experimentos en un directorio
<code>extract_model_info</code>	Extrae información completa de modelo PyTorch
<code>generate_model_summary</code>	Genera resumen de texto del modelo
<code>print_model_info</code>	Imprime información detallada del modelo
<code>save_model_auto</code>	Guarda un modelo, incluyendo arquitectura, pesos y estado del optimizador
<code>load_model_auto</code>	Carga un modelo, incluyendo arquitectura, pesos y estado del optimizador
<code>extract_data_from_loader</code>	Extrae tensores X e y completos de un DataLoader
<code>extract_kan_config</code>	Recupera la configuración de capas de una red DualFlexKAN

Tabla 6.2: Resumen de funciones en `save_load_utils`

6.8. Funciones Auxiliares de Datos y KAN

6.8.1. Extracción de datos: `extract_data_from_loader`

Esta función itera sobre un `DataLoader` completo y concatena los lotes (batches) en tensores únicos de PyTorch.

Sintaxis:

```
1 X, y = extract_data_from_loader(dataloader)
```

Retorna:

- X: Tensor concatenado con todas las entradas.
- y: Tensor concatenado con todas las etiquetas (o `None` si no existen).

6.8.2. Extraer configuración KAN: `extract_kan_config`

Especialmente diseñada para arquitecturas `DualFlexKAN`, esta función realiza ingeniería inversa sobre las capas instanciadas para recuperar los hiperparámetros de configuración,

como estrategias de activación, tipos de funciones (polinomios, splines, RBF) y sus parámetros específicos (grados, grids).

Sintaxis:

```
1 config = extract_kan_config(kan_network)
```

Nota importante: El argumento `kan_network` debe ser la instancia de la red KAN. Si usas un Autoencoder, debes pasar `model.encoder.network` o `model.decoder.network`, no el modelo completo.

6.9. Extraer coeficientes de polinomios `polycoefs(polinomio_str)`

Descripción: Extrae los coeficientes numéricos de un polinomio representado como cadena, devolviendo una lista ordenada por grado. Se emplea principalmente para analizar las funciones polinómicas aprendidas por las activaciones en redes KAN.

Sintaxis:

```
coefs = polycoefs("phi(x) = 0.5x^2 + 0.3x + 0.1")
```

Salida:

```
[0.1, 0.3, 0.5]
```

Dependencias: Solo utiliza la librería estándar de Python.

6.10. Casos de Uso

A continuación se presentan ejemplos detallados que integran las funciones descritas para guardar y analizar arquitecturas, tanto clásicas como basadas en DualFlexKAN.

6.10.1. Ejemplo 1: Red Clásica (MLP)

Este ejemplo muestra cómo guardar un modelo simple y luego inspeccionar su código fuente almacenado.

```
1 import torch
2 import torch.nn as nn
3 from save_load_utils import autoargs, save_model_auto,
   load_model_auto, print_model_info, extract_model_info
4
5 # 1. Definir modelo con @autoargs
6 @autoargs
7 class SimpleMLP(nn.Module):
8     def __init__(self, input_dim, hidden_dim, output_dim):
9         super().__init__()
10         self.net = nn.Sequential(
```

```

11         nn.Linear(input_dim, hidden_dim),
12         nn.ReLU(),
13         nn.Linear(hidden_dim, output_dim)
14     )
15     def forward(self, x):
16         return self.net(x)
17
18 # 2. Instanciar y entrenar (simulado)
19 model = SimpleMLP(10, 32, 2)
20 optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
21
22 # 3. Guardar modelo completo
23 save_model_auto(model, "simple_mlp.pt", optimizer=optimizer,
24                 epoch=10, overwrite=True)
25
26 # 4. Cargar en una sesión nueva
27 # Nota: devuelve una tupla expandida si hay optimizador
28 loaded_model, loaded_opt, loaded_sched, epoch, metadata =
29     load_model_auto("simple_mlp.pt")
30
31 print(f"Modelo cargado en epoch {epoch}")
32
33 # 5. Ver resumen de arquitectura
34 info = extract_model_info(loaded_model)
35 print_model_info(info)
36
37 # 6. Inspeccionar el código fuente guardado
38 print("\n--- Código fuente almacenado ---")
39 print(metadata['code'])

```

Listing 6.1: Guardado y carga de MLP clásico

6.10.2. Ejemplo 2: DualFlexKAN Autoencoder

Este ejemplo utiliza las clases definidas en `DFKAN_autoencoder.py` y muestra cómo la utilidad maneja dependencias complejas y cómo acceder correctamente a la estructura interna del autoencoder.

```

1 from DFKAN_autoencoder import DualFlexKANAutoencoder
2 from save_load_utils import save_model_auto, load_model_auto,
3   extract_kan_config
4
5 # 1. Configurar y crear el Autoencoder Flexible
6 # Definimos una configuración específica para polinomios de grado 3
7 kan_config = {
8     'input_activation_types': ['polynomial'],
9     'input_activation_kwargs': [{'degree': 3}],
10    'output_activation_types': ['polynomial'],

```

```

10     'output_activation_kwargs': [{'degree': 3}]
11 }
12
13 model = DualFlexKANAutoencoder(
14     input_size=128,
15     output_size=128,
16     latent_size=16,
17     encoder_hidden_sizes=[64, 32],
18     decoder_hidden_sizes=[32, 64],
19     encoder_config=kan_config,
20     decoder_config=kan_config
21 )
22
23 # 2. Guardar el modelo
24 # save_model_auto capturará el código de DualFlexKANAutoencoder,
25 # DualFlexKANEncoder, DualFlexKAN y DualFlexKANLinear.
26 save_model_auto(model, "dfkan_autoencoder.pt", overwrite=True)
27
28 # 3. Cargar el modelo
29 loaded_kan_ae, metadata = load_model_auto("dfkan_autoencoder.pt")
30 loaded_kan_ae.eval()
31
32 # 4. Verificar que la configuración KAN se ha preservado.
33 # IMPORTANTE: Accedemos a model.encoder.network porque es un
34 # Autoencoder.
35 encoder_config = extract_kan_config(loaded_kan_ae.encoder.network)
36 decoder_config = extract_kan_config(loaded_kan_ae.decoder.network)
37
38 print("\n--- Configuración KAN Recuperada (Encoder) ---")
39 print(f"Tipos activación entrada:
40     {encoder_config['input_activation_types']}")
41 print(f"Kwargs (grados):
42     {encoder_config['input_activation_kwargs']}")
43
44 # 5. Acceso directo a parámetros (como en KAN_shared_poly7.py)
45 # Navegamos hasta la primera capa del encoder
46 layer = loaded_kan_ae.encoder.network.layers[0]
47 if hasattr(layer, 'input_function_params'):
48     print(f"\nParámetros función entrada capa 0:
49         {layer.input_function_params.shape}")

```

Listing 6.2: Guardado y análisis de DFKAN Autoencoder

Este flujo asegura la reproducibilidad total de los experimentos, permitiendo recuperar no solo los pesos, sino la estructura exacta de las funciones de activación polinómicas y el código que las define, respetando la jerarquía interna de clases del Autoencoder.

Capítulo 7 Wrapper para redes Convolutionales KAN DualFlexKANConv

Este capítulo describe la implementación de redes neuronales convolucionales utilizando la arquitectura Kolmogorov-Arnold Networks (KAN). A diferencia de las redes KAN con capas completamente conectadas, las KAN convolucionales aplican funciones de activación adaptativas en el contexto de convoluciones 1D, 2D y 3D, preservando la estructura espacial de los datos.

7.1. Arquitectura General

La implementación de DualFlexKAN para convoluciones se compone de 7 clases principales:

- **DualFlexKANConvBase**: Clase base abstracta que implementa la lógica común de transformaciones adaptativas, regularización y gestión de funciones de activación para todas las dimensionalidades.
- **DualFlexKANConv1d**: Capa convolucional individual 1D con funciones de entrada y activación adaptativas independientes para datos secuenciales.
- **DualFlexKANConv2d**: Capa convolucional individual 2D con funciones de entrada y activación adaptativas independientes para datos de imagen.
- **DualFlexKANConv3d**: Capa convolucional individual 3D con funciones de entrada y activación adaptativas independientes para datos volumétricos.
- **DualFlexKANCNN1d**: Arquitectura CNN completa 1D que combina múltiples capas convolucionales KAN con pooling y capas densas finales.
- **DualFlexKANCNN**: Arquitectura CNN completa 2D que combina múltiples capas convolucionales KAN con pooling y capas densas finales.
- **DualFlexKANCNN3d**: Arquitectura CNN completa 3D que combina múltiples capas convolucionales KAN con pooling y capas densas finales.

7.1.1. Concepto de KAN Convolutivo

En una capa convolutiva estándar, la operación se define como:

$$y_j = \sigma \left(\sum_i w_{ji} * x_i + b_j \right) \quad (7.1)$$

donde $*$ denota la operación de convolución, w_{ji} son los kernels convolucionales, b_j es el sesgo, y σ es una función de activación fija (ReLU, tanh, etc.).

En **DualFlexKAN Convolutivo**, extendemos este concepto con funciones adaptativas tanto en la entrada como en la salida:

$$y_j = \phi_j \left(\sum_i w_{ji} * \psi_i(x_i) + b_j \right) \quad (7.2)$$

donde:

- $\psi_i(x_i)$: Funciones de transformación adaptativas aplicadas a cada canal de entrada i
- $\phi_j(z)$: Funciones de activación adaptativas aplicadas a cada canal de salida j
- Ambas funciones son **aprendibles** con parámetros entrenables

7.1.2. Ventajas de KAN Convolutivo

1. **Mayor expresividad**: Las funciones adaptativas pueden aprender transformaciones no lineales específicas para cada canal.
2. **Flexibilidad**: Diferentes estrategias (global, per-channel, fixed) permiten controlar la complejidad del modelo.
3. **Eficiencia espacial**: Preserva las propiedades de convolución (compartición de pesos espacialmente).
4. **Compatibilidad**: Puede combinarse con técnicas estándar de CNN (pooling, batch normalization, dropout).

7.2. La Capa DualFlexKANConv2d

La clase `DualFlexKANConv2d` implementa una capa convolutiva individual con funciones adaptativas independientes para entrada y salida.

7.2.1. Constructor


```

1 conv_layer = DualFlexKANConv2d(
2     # Parámetros de convolución estándar
3     in_channels=3,
4     out_channels=64,
5     kernel_size=3,
6     stride=1,
7     padding=1,
8     dilation=1,
9     groups=1,
10
11     # Funciones de entrada
12     input_function_strategy="per_channel",
13     input_activation_type="polynomial",
14     input_activation_kwargs={'degree': 3},
15
16     # Funciones de salida
17     output_function_strategy="per_channel",
18     output_activation_type="polynomial",
19     output_activation_kwargs={'degree': 3},
20
21     # Regularización
22     dropout_prob=0.0,
23     dropout_position="after_activation",
24     use_batch_norm=False,
25     batch_norm_position="after_activation",
26     regularization_order="dropout_first",
27     batch_norm_momentum=0.1,
28     batch_norm_eps=1e-5
29 )

```

Listing 7.1: Constructor de DualFlexKANConv2d

Parámetros convolucionales estándar:

- `in_channels`: Número de canales de entrada.
- `out_channels`: Número de canales de salida (filtros).
- `kernel_size`: Tamaño del kernel convolucional (entero o tupla).
- `stride`: Stride de la convolución (default: 1).
- `padding`: Padding aplicado (default: 0).
- `dilation`: Dilation de la convolución (default: 1).
- `groups`: Número de grupos para convolución agrupada (default: 1).

Configuración de funciones de entrada (ψ_i):

- `input_function_strategy`: Estrategia de aplicación:

- "none": Sin transformación (identidad).
 - "fixed": Función fija no entrenable (ReLU, tanh, etc.).
 - "global": Una función adaptativa compartida para todos los canales.
 - "per_channel": Una función adaptativa por canal de entrada.
- `input_activation_type`: Tipo de función ("polynomial", "legendre", "rbf", "fourier", etc.).
 - `input_activation_kwargs`: Argumentos adicionales para la función.

Configuración de funciones de salida (ϕ_j):

- `output_function_strategy`: Estrategia de aplicación (igual que entrada).
- `output_activation_type`: Tipo de función de activación.
- `output_activation_kwargs`: Argumentos adicionales.

Regularización flexible:

- `dropout_prob`: Probabilidad de dropout.
- `dropout_position`: Posición del dropout ("before_activation", "after_activation", "both", "none").
- `use_batch_norm`: Habilitar batch normalization.
- `batch_norm_position`: Posición de batch norm.
- `regularization_order`: Orden de aplicación ("dropout_first" o "batch_norm_first").

7.2.2. Estrategias de Funciones

Comparación de estrategias:

Estrategia	Parámetros	Cómputo	Uso recomendado
none	0	Mínimo	Capas intermedias sin transformación
fixed	0	Bajo	Funciones estándar (ReLU, tanh)
global	N_{params}	Medio	Transformación uniforme en canales
per_channel	$C \times N_{\text{params}}$	Alto	Máxima expresividad por canal

Tabla 7.1: Comparación de estrategias para funciones de activación en KAN Conv2D

donde C es el número de canales y N_{params} es el número de parámetros de la función de activación.

7.2.3. Forward Pass

El forward pass de DualFlexKANConv2d sigue esta secuencia:

1. **Aplicar funciones de entrada:** $\tilde{x}_i = \psi_i(x_i)$ para cada canal i
2. **Convolución estándar:** $z_j = \sum_i w_{ji} * \tilde{x}_i + b_j$
3. **Regularización pre-activación:** Aplicar dropout y/o batch norm según configuración
4. **Aplicar funciones de salida:** $y_j = \phi_j(z_j)$ para cada canal j
5. **Regularización post-activación:** Aplicar dropout y/o batch norm según configuración

7.3. Definición de una capa KAN convolucional

Capa Convolucional 2D KAN Simple

```

1 import torch
2 from KAN_conv2d import DualFlexKANConv2d
3
4 # Crear capa convolucional KAN
5 conv_layer = DualFlexKANConv2d(
6     in_channels=3,      # RGB
7     out_channels=16,    # 16 filtros
8     kernel_size=3,
9     padding=1,
10    input_function_strategy="per_channel",
11    input_activation_type="polynomial",
12    input_activation_kwargs={'degree': 3},
13    output_function_strategy="per_channel",
14    output_activation_type="rbf",
15    output_activation_kwargs={'num_centers': 5},
16    use_batch_norm=True,
17    batch_norm_position="after_activation"
18 )
19
20 # Forward pass
21 x = torch.randn(4, 3, 32, 32) # batch_size=4, 3 canales RGB, 32x32
22 y = conv_layer(x)
23 print(f"Input: {x.shape} -> Output: {y.shape}")
24 # Output: Input: torch.Size([4, 3, 32, 32]) -> Output:
25     torch.Size([4, 16, 32, 32])
26
27 # Contar parámetros
28 params = conv_layer.count_parameters()
29 print(f"\nParámetros totales: {params['total']}")

```

```

29 print(f"   Pesos convolucionales: {params['conv_weights']}")
30 print(f"   Parámetros función entrada:
    {params['input_function_params']}")
31 print(f"   Parámetros función salida:
    {params['output_function_params']}")

```

Listing 7.2: Capa convolucional KAN básica

7.3.1. Capa Híbrida (Entrada Fija + Salida Adaptativa)

```

1  # Usar ReLU fijo en entrada, función adaptativa en salida
2  conv_hybrid = DualFlexKANConv2d(
3      in_channels=32,
4      out_channels=64,
5      kernel_size=3,
6      stride=2,
7      padding=1,
8      input_function_strategy="fixed",
9      input_activation_type="relu", # ReLU fijo, sin parámetros
    adicionales
10     output_function_strategy="per_channel",
11     output_activation_type="legendre",
12     output_activation_kwargs={'order': 4},
13     dropout_prob=0.2,
14     dropout_position="after_activation"
15 )
16
17 x = torch.randn(2, 32, 16, 16)
18 y = conv_hybrid(x)
19 print(f"Input: {x.shape} -> Output: {y.shape}")
20 # Output: Input: torch.Size([2, 32, 16, 16]) -> Output:
    torch.Size([2, 64, 8, 8])

```

Listing 7.3: Capa híbrida con entrada fija

7.3.2. Capa con Estrategia Global (Eficiente)

```

1  # Una función compartida para todos los canales (menos parámetros)
2  conv_global = DualFlexKANConv2d(
3      in_channels=64,
4      out_channels=128,
5      kernel_size=3,
6      padding=1,
7      input_function_strategy="global", # Compartida entre canales
8      input_activation_type="polynomial",
9      output_function_strategy="global", # Compartida entre canales
10     output_activation_type="gegenbauer",

```

```

11     output_activation_kwargs={'max_degree': 3},
12     use_batch_norm=True,
13     batch_norm_position="both" # Antes y después de activación
14 )
15
16 x = torch.randn(2, 64, 8, 8)
17 y = conv_global(x)
18 print(f"Parámetros: {conv_global.count_parameters()['total']}")

```

Listing 7.4: Uso de estrategia global para eficiencia

7.4. Definición de Redes capa a capa

Para construir redes CNN KAN manualmente, se pueden combinar capas DualFlexKANConv2d con operaciones de pooling y otras capas estándar de PyTorch.

7.4.1. Ejemplo 1. Red CNN Simple de 3 Capas

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 from KAN_conv2d import DualFlexKANConv2d
5
6 class SimpleCNNKAN(nn.Module):
7     """CNN KAN simple construida manualmente"""
8
9     def __init__(self):
10         super(SimpleCNNKAN, self).__init__()
11
12         # Primera capa convolucional
13         self.conv1 = DualFlexKANConv2d(
14             in_channels=3,
15             out_channels=32,
16             kernel_size=5,
17             padding=2,
18             input_function_strategy="per_channel",
19             input_activation_type="polynomial",
20             output_function_strategy="per_channel",
21             output_activation_type="polynomial",
22             use_batch_norm=True
23         )
24         self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
25
26         # Segunda capa convolucional
27         self.conv2 = DualFlexKANConv2d(
28             in_channels=32,

```

```

29         out_channels=64,
30         kernel_size=3,
31         padding=1,
32         input_function_strategy="global",
33         input_activation_type="polynomial",
34         output_function_strategy="per_channel",
35         output_activation_type="rbf",
36         output_activation_kwargs={'num_centers': 8},
37         dropout_prob=0.2,
38         dropout_position="after_activation",
39         use_batch_norm=True
40     )
41     self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
42
43     # Tercera capa convolucional
44     self.conv3 = DualFlexKANConv2d(
45         in_channels=64,
46         out_channels=128,
47         kernel_size=3,
48         padding=1,
49         input_function_strategy="per_channel",
50         input_activation_type="legendre",
51         output_function_strategy="per_channel",
52         output_activation_type="polynomial",
53         use_batch_norm=True,
54         batch_norm_position="both"
55     )
56     self.pool3 = nn.MaxPool2d(kernel_size=2, stride=2)
57
58     # Capas fully connected
59     self.fc1 = nn.Linear(128 * 4 * 4, 256)
60     self.dropout = nn.Dropout(0.5)
61     self.fc2 = nn.Linear(256, 10)
62
63     def forward(self, x):
64         # Bloque 1: 32x32 -> 16x16
65         x = self.conv1(x)
66         x = self.pool1(x)
67
68         # Bloque 2: 16x16 -> 8x8
69         x = self.conv2(x)
70         x = self.pool2(x)
71
72         # Bloque 3: 8x8 -> 4x4
73         x = self.conv3(x)
74         x = self.pool3(x)
75
76         # Flatten y capas densas

```

```

77         x = x.view(x.size(0), -1)
78         x = F.relu(self.fc1(x))
79         x = self.dropout(x)
80         x = self.fc2(x)
81
82         return x
83
84 # Crear modelo
85 model = SimpleCNNKAN()
86
87 # Test
88 x = torch.randn(4, 3, 32, 32)
89 output = model(x)
90 print(f"Input: {x.shape} -> Output: {output.shape}")
91
92 # Contar parámetros
93 total_params = sum(p.numel() for p in model.parameters())
94 print(f"Parámetros totales: {total_params:,}")

```

Listing 7.5: Construcción manual de CNN KAN

7.4.2. Ejemplo 1. Red con diferentes tipos de funciones por capa

```

1 class DiverseKANCNN(nn.Module):
2     """CNN KAN con diferentes tipos de funciones en cada capa"""
3
4     def __init__(self):
5         super(DiverseKANCNN, self).__init__()
6
7         # Capa 1: Polinomios
8         self.conv1 = DualFlexKANConv2d(
9             in_channels=3, out_channels=16, kernel_size=3, padding=1,
10             input_function_strategy="per_channel",
11             input_activation_type="polynomial",
12             input_activation_kwargs={'degree': 3},
13             output_function_strategy="per_channel",
14             output_activation_type="polynomial",
15             output_activation_kwargs={'degree': 3},
16             use_batch_norm=True
17         )
18
19         # Capa 2: Legendre (ortogonales)
20         self.conv2 = DualFlexKANConv2d(
21             in_channels=16, out_channels=32, kernel_size=3,
22             padding=1,
23             input_function_strategy="global",
24             input_activation_type="legendre",

```

```

24         input_activation_kwargs={'order': 4},
25         output_function_strategy="per_channel",
26         output_activation_type="legendre",
27         output_activation_kwargs={'order': 4},
28         use_batch_norm=True
29     )
30
31     # Capa 3: RBF (funciones radiales)
32     self.conv3 = DualFlexKANConv2d(
33         in_channels=32, out_channels=64, kernel_size=3,
34         padding=1,
35         input_function_strategy="per_channel",
36         input_activation_type="rbf",
37         input_activation_kwargs={'num_centers': 5},
38         output_function_strategy="per_channel",
39         output_activation_type="rbf",
40         output_activation_kwargs={'num_centers': 8},
41         dropout_prob=0.3,
42         use_batch_norm=True
43     )
44
45     # Global average pooling
46     self.global_pool = nn.AdaptiveAvgPool2d((1, 1))
47
48     # Clasificador
49     self.fc = nn.Linear(64, 10)
50
51     def forward(self, x):
52         x = self.conv1(x)
53         x = F.max_pool2d(x, 2)
54
55         x = self.conv2(x)
56         x = F.max_pool2d(x, 2)
57
58         x = self.conv3(x)
59         x = F.max_pool2d(x, 2)
60
61         x = self.global_pool(x)
62         x = x.view(x.size(0), -1)
63         x = self.fc(x)
64
65         return x
66
67 model = DiverseKANCNN()
68 print(f"Parámetros: {sum(p.numel() for p in model.parameters()),}")

```

Listing 7.6: CNN KAN con diversidad de funciones

7.5. La Clase DualFlexKANCNN

Para facilitar la construcción de redes CNN KAN completas, la clase `DualFlexKANCNN` proporciona una interfaz unificada que maneja automáticamente capas convolucionales 2D, pooling y capas densas finales. De forma análoga, disponemos de `DualFlexKANCNN1d` y `DualFlexKANCNN3d` para convoluciones 1D y 3D, respectivamente.

7.5.1. Constructor

```

1  model = DualFlexKANCNN(
2      # Configuración de capas convolucionales (lista de dicts)
3      conv_configs=[
4          {
5              'in_channels': 3,
6              'out_channels': 32,
7              'kernel_size': 3,
8              'padding': 1,
9              'input_function_strategy': "per_channel",
10             'input_activation_type': "polynomial",
11             'output_function_strategy': "per_channel",
12             'output_activation_type': "polynomial",
13             'use_batch_norm': True,
14             'pooling': {'type': 'max', 'kernel_size': 2}
15         },
16         # ... más capas
17     ],
18
19     # Capas densas finales (opcional)
20     dense_sizes=[128, 64, 10],
21     dense_input_strategies=None, # Default: ["per_input", ...]
22     dense_output_strategies=None, # Default: ["per_neuron", ...]
23     dense_input_types=None, # Default: ["polynomial", ...]
24     dense_output_types=None, # Default: ["polynomial", ...]
25
26     # Pooling global antes de capas densas
27     use_global_pooling=True,
28     global_pooling_type="avg", # "avg" o "max"
29
30     # Dropout global
31     global_dropout=0.5
32 )

```

Listing 7.7: Constructor de DualFlexKANCNN

Parámetros principales:

- `conv_configs`: Lista de diccionarios, cada uno especificando la configuración completa de una capa convolucional KAN.

- `dense_sizes`: Lista con tamaños de capas densas finales (e.g., [512, 256, 10]).
- `use_global_pooling`: Si True, aplica pooling adaptativo global antes de capas densas.
- `global_pooling_type`: Tipo de pooling global (`avg` o `max`).
- `global_dropout`: Probabilidad de dropout antes de capas densas.

Configuración de pooling por capa:

Dentro de cada diccionario en `conv_configs`, se puede especificar pooling:

```

1 'pooling': {
2     'type': 'max',          # 'max' o 'avg'
3     'kernel_size': 2,      # Tamaño del kernel de pooling
4     'stride': None         # Stride (None = kernel_size)
5 }
```

7.5.2. Ejemplo 1. CNN estilo VGG con DualFlexKANCNN

```

1 from KAN_conv2d import DualFlexKANCNN
2
3 # Configuración de capas convolucionales
4 conv_configs = [
5     # Bloque 1
6     {
7         'in_channels': 3,
8         'out_channels': 64,
9         'kernel_size': 3,
10        'padding': 1,
11        'input_function_strategy': "per_channel",
12        'input_activation_type': "polynomial",
13        'output_function_strategy': "per_channel",
14        'output_activation_type': "polynomial",
15        'use_batch_norm': True
16    },
17    {
18        'in_channels': 64,
19        'out_channels': 64,
20        'kernel_size': 3,
21        'padding': 1,
22        'input_function_strategy': "global",
23        'input_activation_type': "polynomial",
24        'output_function_strategy': "per_channel",
25        'output_activation_type': "polynomial",
26        'use_batch_norm': True,
27        'pooling': {'type': 'max', 'kernel_size': 2}
28    },
29 ]
```

```

30 # Bloque 2
31 {
32     'in_channels': 64,
33     'out_channels': 128,
34     'kernel_size': 3,
35     'padding': 1,
36     'input_function_strategy': "per_channel",
37     'input_activation_type': "polynomial",
38     'output_function_strategy': "per_channel",
39     'output_activation_type': "rbf",
40     'output_activation_kwargs': {'num_centers': 8},
41     'use_batch_norm': True
42 },
43 {
44     'in_channels': 128,
45     'out_channels': 128,
46     'kernel_size': 3,
47     'padding': 1,
48     'input_function_strategy': "global",
49     'input_activation_type': "legendre",
50     'output_function_strategy': "per_channel",
51     'output_activation_type': "polynomial",
52     'use_batch_norm': True,
53     'pooling': {'type': 'max', 'kernel_size': 2}
54 },
55
56 # Bloque 3
57 {
58     'in_channels': 128,
59     'out_channels': 256,
60     'kernel_size': 3,
61     'padding': 1,
62     'input_function_strategy': "per_channel",
63     'input_activation_type': "polynomial",
64     'output_function_strategy': "per_channel",
65     'output_activation_type': "polynomial",
66     'dropout_prob': 0.3,
67     'dropout_position': "after_activation",
68     'use_batch_norm': True,
69     'pooling': {'type': 'max', 'kernel_size': 2}
70 }
71 ]
72
73 # Crear modelo VGG-style KAN
74 model_vgg = DualFlexKANCNN(
75     conv_configs=conv_configs,
76     dense_sizes=[256, 128, 10], # Capas densas: 256 -> 128 -> 10
77     use_global_pooling=True,

```

```

78     global_pooling_type="avg",
79     global_dropout=0.5
80 )
81
82 # Información de arquitectura
83 model_vgg.print_architecture_info()
84
85 # Test forward pass
86 x = torch.randn(2, 3, 32, 32)
87 output = model_vgg(x)
88 print(f"\nTest - Input: {x.shape} -> Output: {output.shape}")

```

Listing 7.8: Arquitectura estilo VGG con KAN

7.5.3. Ejemplo 2. CNN compacta con función auxiliar

La librería incluye funciones auxiliares para crear arquitecturas comunes:

```

1 from KAN_conv2d import create_simple_kan_cnn
2
3 # Crear CNN simple con interfaz simplificada
4 model_simple = create_simple_kan_cnn(
5     input_channels=3,
6     num_classes=10,
7     conv_channels=[32, 64, 128], # Canales de cada capa conv
8     dense_sizes=[256, 10],      # Capas densas finales
9     input_activation="polynomial",
10    output_activation="polynomial",
11    kernel_size=3,
12    use_pooling=True,
13    pooling_type="max",
14    use_batch_norm=True,
15    dropout_prob=0.2
16 )
17
18 print("Arquitectura CNN Simple:")
19 model_simple.print_architecture_info()
20
21 # Test
22 x = torch.randn(4, 3, 32, 32)
23 output = model_simple(x)
24 print(f"\nOutput: {output.shape}") # (4, 10)

```

Listing 7.9: Uso de create_simple_kan_cnn

7.6. Arquitecturas Avanzadas

7.6.1. CNN estilo ResNet con KAN

```

1 from KAN_conv2d import create_resnet_style_kan_cnn
2
3 # Crear CNN estilo ResNet con bloques residuales
4 model_resnet = create_resnet_style_kan_cnn(
5     input_channels=3,
6     num_classes=10,
7     blocks=[2, 2, 2, 2], # Número de bloques por etapa
8     base_channels=64      # Canales base (se duplican en cada etapa)
9 )
10
11 print("Arquitectura ResNet-style KAN:")
12 model_resnet.print_architecture_info()
13
14 # Test
15 x = torch.randn(2, 3, 32, 32)
16 output = model_resnet(x)
17 print(f"\nOutput: {output.shape}")

```

Listing 7.10: Arquitectura ResNet-style con funciones auxiliares

7.6.2. CNN Personalizada con Configuración Compleja

```

1 # Configuración muy personalizada
2 conv_configs_advanced = [
3     {
4         'in_channels': 3,
5         'out_channels': 32,
6         'kernel_size': 5,
7         'padding': 2,
8         'input_function_strategy': "fixed",
9         'input_activation_type': "relu", # Entrada fija
10        'output_function_strategy': "per_channel",
11        'output_activation_type': "legendre",
12        'output_activation_kwargs': {'order': 4},
13        'use_batch_norm': True,
14        'batch_norm_position': "after_activation",
15        'pooling': {'type': 'max', 'kernel_size': 2}
16    },
17    {
18        'in_channels': 32,
19        'out_channels': 64,
20        'kernel_size': 3,

```

```

21         'padding': 1,
22         'input_function_strategy': "global",
23         'input_activation_type': "polynomial",
24         'input_activation_kwargs': {'degree': 4},
25         'output_function_strategy': "per_channel",
26         'output_activation_type': "gegenbauer",
27         'output_activation_kwargs': {'max_degree': 3, 'alpha': 1.0},
28         'dropout_prob': 0.2,
29         'dropout_position': "after_activation",
30         'use_batch_norm': True,
31         'pooling': {'type': 'avg', 'kernel_size': 2}
32     },
33     {
34         'in_channels': 64,
35         'out_channels': 128,
36         'kernel_size': 3,
37         'padding': 1,
38         'input_function_strategy': "per_channel",
39         'input_activation_type': "bspline",
40         'input_activation_kwargs': {'num_knots': 8, 'order': 3},
41         'output_function_strategy': "per_channel",
42         'output_activation_type': "polynomial",
43         'output_activation_kwargs': {'degree': 3},
44         'use_batch_norm': True,
45         'batch_norm_position': "both",
46         'regularization_order': "batch_norm_first",
47         'pooling': {'type': 'max', 'kernel_size': 2}
48     },
49     {
50         'in_channels': 128,
51         'out_channels': 256,
52         'kernel_size': 3,
53         'padding': 1,
54         'input_function_strategy': "per_channel",
55         'input_activation_type': "rbf",
56         'input_activation_kwargs': {'num_centers': 10},
57         'output_function_strategy': "per_channel",
58         'output_activation_type': "polynomial",
59         'dropout_prob': 0.3,
60         'use_batch_norm': True,
61         'pooling': {'type': 'max', 'kernel_size': 2}
62     }
63 ]
64
65 model_advanced = DualFlexKANCNN(
66     conv_configs=conv_configs_advanced,
67     dense_sizes=[256, 128, 10],
68     use_global_pooling=True,

```

```

69     global_pooling_type="avg",
70     global_dropout=0.5
71 )
72
73 print("\nArquitectura Avanzada:")
74 params_info = model_advanced.count_parameters()
75 print(f"Parámetros totales: {params_info['total']:,}")
76
77 # Desglose por capa
78 for i, layer_params in enumerate(params_info['conv_layers']):
79     print(f"\nCapa Conv {i+1}:")
80     print(f"    Total: {layer_params['total']:,}")
81     print(f"    Pesos conv: {layer_params['conv_weights']:,}")
82     print(f"    Params función entrada:
83           {layer_params['input_function_params']:,}")
84     print(f"    Params función salida:
85           {layer_params['output_function_params']:,}")
86
87 if params_info['dense_layers'] is not None:
88     print(f"\nCapas densas:
89           {params_info['dense_layers']['total']:,}")

```

Listing 7.11: Configuración avanzada con múltiples tipos de activación

7.7. Entrenamiento de CNN KAN

7.7.1. Pipeline Completo de Entrenamiento

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import DataLoader
5 from torchvision import datasets, transforms
6 from KAN_conv2d import DualFlexKANCNN
7
8 # 1. Preparar datos
9 transform = transforms.Compose([
10     transforms.ToTensor(),
11     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
12 ])
13
14 train_dataset = datasets.CIFAR10(
15     root='./data',
16     train=True,
17     download=True,
18     transform=transform

```

```

19 )
20 test_dataset = datasets.CIFAR10(
21     root='./data',
22     train=False,
23     download=True,
24     transform=transform
25 )
26
27 train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
28 test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
29
30 # 2. Crear modelo
31 conv_configs = [
32     {
33         'in_channels': 3, 'out_channels': 64, 'kernel_size': 3,
34         'padding': 1,
35         'input_function_strategy': "per_channel",
36         'input_activation_type': "polynomial",
37         'output_function_strategy': "per_channel",
38         'output_activation_type': "polynomial",
39         'use_batch_norm': True,
40         'pooling': {'type': 'max', 'kernel_size': 2}
41     },
42     {
43         'in_channels': 64, 'out_channels': 128, 'kernel_size': 3,
44         'padding': 1,
45         'input_function_strategy': "global",
46         'input_activation_type': "polynomial",
47         'output_function_strategy': "per_channel",
48         'output_activation_type': "rbf",
49         'dropout_prob': 0.2,
50         'use_batch_norm': True,
51         'pooling': {'type': 'max', 'kernel_size': 2}
52     },
53     {
54         'in_channels': 128, 'out_channels': 256, 'kernel_size': 3,
55         'padding': 1,
56         'input_function_strategy': "per_channel",
57         'input_activation_type': "legendre",
58         'output_function_strategy': "per_channel",
59         'output_activation_type': "polynomial",
60         'dropout_prob': 0.3,
61         'use_batch_norm': True,
62         'pooling': {'type': 'max', 'kernel_size': 2}
63     }
64 ]
65
66 model = DualFlexKANCNN(

```



```
64     conv_configs=conv_configs,
65     dense_sizes=[256, 128, 10],
66     use_global_pooling=True,
67     global_dropout=0.5
68 )
69
70 # 3. Configurar entrenamiento
71 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
72 model.to(device)
73
74 criterion = nn.CrossEntropyLoss()
75 optimizer = optim.Adam(model.parameters(), lr=0.001)
76 scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=30,
77     gamma=0.1)
78
79 # 4. Loop de entrenamiento
80 def train_epoch(model, loader, criterion, optimizer, device):
81     model.train()
82     running_loss = 0.0
83     correct = 0
84     total = 0
85
86     for data, target in loader:
87         data, target = data.to(device), target.to(device)
88
89         optimizer.zero_grad()
90         output = model(data)
91         loss = criterion(output, target)
92         loss.backward()
93         optimizer.step()
94
95         running_loss += loss.item()
96         _, predicted = torch.max(output.data, 1)
97         total += target.size(0)
98         correct += (predicted == target).sum().item()
99
100     return running_loss / len(loader), 100 * correct / total
101
102 def evaluate(model, loader, criterion, device):
103     model.eval()
104     running_loss = 0.0
105     correct = 0
106     total = 0
107
108     with torch.no_grad():
109         for data, target in loader:
110             data, target = data.to(device), target.to(device)
111             output = model(data)
```

```

111         loss = criterion(output, target)
112
113         running_loss += loss.item()
114         _, predicted = torch.max(output.data, 1)
115         total += target.size(0)
116         correct += (predicted == target).sum().item()
117
118     return running_loss / len(loader), 100 * correct / total
119
120 # 5. Entrenar
121 num_epochs = 50
122 best_acc = 0
123
124 for epoch in range(num_epochs):
125     train_loss, train_acc = train_epoch(
126         model, train_loader, criterion, optimizer, device
127     )
128     test_loss, test_acc = evaluate(
129         model, test_loader, criterion, device
130     )
131     scheduler.step()
132
133     print(f'Epoch {epoch+1}/{num_epochs}:')
134     print(f'  Train Loss: {train_loss:.4f}, Train Acc:
135           {train_acc:.2f}%')
136     print(f'  Test Loss: {test_loss:.4f}, Test Acc: {test_acc:.2f}%')
137
138     # Guardar mejor modelo
139     if test_acc > best_acc:
140         best_acc = test_acc
141         torch.save(model.state_dict(), 'best_kan_cnn.pth')
142
143 print(f'\nMejor precisión en test: {best_acc:.2f}%')

```

Listing 7.12: Entrenamiento completo de CNN KAN

7.8. Comparación: KAN Conv vs Conv Estándar

7.8.1. Tabla Comparativa

7.8.2. Comparación CNN estándar vs. CNN KAN

```

1 import torch.nn as nn
2
3 # CNN Estándar
4 class StandardCNN(nn.Module):
5     def __init__(self):

```

Aspecto	Conv Estándar	KAN Conv
Funciones de activación	Fijas (ReLU, tanh)	Adaptativas aprendibles
Parámetros adicionales	0	Proporcional a $C \times N_{\text{params}}$
Expresividad	Media	Alta
Costo computacional	Bajo	Medio-Alto
Interpretabilidad	Baja	Media-Alta (funciones explícitas)
Flexibilidad	Baja	Alta (múltiples estrategias)

Tabla 7.2: Comparación entre convoluciones estándar y KAN convolucionales

```

6         super().__init__()
7         self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
8         self.bn1 = nn.BatchNorm2d(64)
9         self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
10        self.bn2 = nn.BatchNorm2d(128)
11        self.fc = nn.Linear(128 * 8 * 8, 10)
12
13    def forward(self, x):
14        x = F.relu(self.bn1(self.conv1(x)))
15        x = F.max_pool2d(x, 2)
16        x = F.relu(self.bn2(self.conv2(x)))
17        x = F.max_pool2d(x, 2)
18        x = x.view(x.size(0), -1)
19        return self.fc(x)
20
21    # KAN CNN
22    conv_configs = [
23        {'in_channels': 3, 'out_channels': 64, 'kernel_size': 3,
24         'padding': 1,
25         'input_function_strategy': "per_channel",
26         'output_function_strategy': "per_channel",
27         'use_batch_norm': True, 'pooling': {'type': 'max',
28         'kernel_size': 2}},
29        {'in_channels': 64, 'out_channels': 128, 'kernel_size': 3,
30         'padding': 1,
31         'input_function_strategy': "global",
32         'output_function_strategy': "per_channel",
33         'use_batch_norm': True, 'pooling': {'type': 'max',
34         'kernel_size': 2}}
35    ]
36
37    kan_cnn = DualFlexKANCNN(
38        conv_configs=conv_configs,
39        dense_sizes=[128 * 8 * 8, 10],
40        use_global_pooling=False
41    )

```

```
39 # Comparar parámetros
40 standard_cnn = StandardCNN()
41 params_standard = sum(p.numel() for p in standard_cnn.parameters())
42 params_kan = kan_cnn.count_parameters()['total']
43
44 print(f"Parámetros CNN Estándar: {params_standard:,}")
45 print(f"Parámetros KAN CNN: {params_kan:,}")
46 print(f"Ratio: {params_kan/params_standard:.2f}x")
```

Listing 7.13: CNN estándar vs. CNN KAN

7.9. Resolución de problemas frecuentes

7.9.1. Problema: Memoria Insuficiente

Solución:

```
1 # Reducir estrategia de per_channel a global
2 conv_configs = [
3     {
4         'in_channels': 3,
5         'out_channels': 64,
6         'kernel_size': 3,
7         'padding': 1,
8         'input_function_strategy': "global", # En lugar de
9         'output_function_strategy': "global", # En lugar de
10        'per_channel'
11        # ... resto de configuración
12    }
13 ]
```

7.9.2. Problema: Entrenamiento Lento

Causas y soluciones:

- Demasiados parámetros en funciones: Reducir grado de polinomios o número de centros RBF.
- Estrategia `per_channel` en todas las capas: Usar `global` en capas intermedias.
- Funciones complejas: Considerar usar funciones fijas (`fixed`) en primeras capas.

7.9.3. Problema: Gradientes Explosivos/Vanishing

Solución:

```
1 # Usar batch norm en ambas posiciones
2 conv_configs = [
3     {
4         'in_channels': 64,
5         'out_channels': 128,
6         'kernel_size': 3,
7         'padding': 1,
8         'use_batch_norm': True,
9         'batch_norm_position': "both", # Antes y después de
10         activación
11         'regularization_order': "batch_norm_first",
12         # ... resto
13     }
14 ]
```

7.10. Ejemplo Completo: CIFAR-10 con KAN CNN

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torchvision import datasets, transforms
5 from torch.utils.data import DataLoader
6 from KAN_conv2d import DualFlexKANCNN
7
8 # Configuración
9 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
10 batch_size = 128
11 num_epochs = 100
12 learning_rate = 0.001
13
14 # Data augmentation
15 transform_train = transforms.Compose([
16     transforms.RandomCrop(32, padding=4),
17     transforms.RandomHorizontalFlip(),
18     transforms.ToTensor(),
19     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
20     0.2010))
21 ])
22
23 transform_test = transforms.Compose([
24     transforms.ToTensor(),
25     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
26     0.2010))
27 ])
28
29 # Datasets
```

```

28 train_dataset = datasets.CIFAR10('./data', train=True, download=True,
29                                   transform=transform_train)
30 test_dataset = datasets.CIFAR10('./data', train=False,
31                                   transform=transform_test)
32
33 train_loader = DataLoader(train_dataset, batch_size=batch_size,
34                             shuffle=True, num_workers=2)
35 test_loader = DataLoader(test_dataset, batch_size=batch_size,
36                             shuffle=False, num_workers=2)
37
38 # Modelo KAN CNN optimizado
39 conv_configs = [
40     {
41         'in_channels': 3, 'out_channels': 64, 'kernel_size': 3,
42         'padding': 1,
43         'input_function_strategy': "per_channel",
44         'input_activation_type': "polynomial",
45         'input_activation_kwargs': {'degree': 3},
46         'output_function_strategy': "per_channel",
47         'output_activation_type': "polynomial",
48         'output_activation_kwargs': {'degree': 3},
49         'use_batch_norm': True
50     },
51     {
52         'in_channels': 64, 'out_channels': 64, 'kernel_size': 3,
53         'padding': 1,
54         'input_function_strategy': "global",
55         'input_activation_type': "polynomial",
56         'output_function_strategy': "per_channel",
57         'output_activation_type': "rbf",
58         'output_activation_kwargs': {'num_centers': 8},
59         'use_batch_norm': True,
60         'pooling': {'type': 'max', 'kernel_size': 2}
61     },
62     {
63         'in_channels': 64, 'out_channels': 128, 'kernel_size': 3,
64         'padding': 1,
65         'input_function_strategy': "global",
66         'input_activation_type': "polynomial",
67         'output_function_strategy': "per_channel",
68         'output_activation_type': "polynomial",
69         'dropout_prob': 0.2,
70         'use_batch_norm': True
71     },
72     {
73         'in_channels': 128, 'out_channels': 128, 'kernel_size': 3,
74         'padding': 1,
75         'input_function_strategy': "global",

```

```

72         'input_activation_type': "legendre",
73         'output_function_strategy': "per_channel",
74         'output_activation_type': "polynomial",
75         'use_batch_norm': True,
76         'pooling': {'type': 'max', 'kernel_size': 2}
77     },
78     {
79         'in_channels': 128, 'out_channels': 256, 'kernel_size': 3,
80         'padding': 1,
81         'input_function_strategy': "per_channel",
82         'input_activation_type': "polynomial",
83         'output_function_strategy': "per_channel",
84         'output_activation_type': "polynomial",
85         'dropout_prob': 0.3,
86         'use_batch_norm': True,
87         'pooling': {'type': 'max', 'kernel_size': 2}
88     }
89 ]
90 model = DualFlexKANCNN(
91     conv_configs=conv_configs,
92     dense_sizes=[256, 128, 10],
93     use_global_pooling=True,
94     global_pooling_type="avg",
95     global_dropout=0.5
96 )
97
98 model.to(device)
99 print(f"Modelo creado con {model.count_parameters() ['total']:,}
100      parámetros")
101
102 # Entrenamiento
103 criterion = nn.CrossEntropyLoss()
104 optimizer = optim.Adam(model.parameters(), lr=learning_rate,
105                          weight_decay=1e-4)
106 scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer,
107                                                    T_max=num_epochs)
108
109 best_acc = 0
110 for epoch in range(num_epochs):
111     # Train
112     model.train()
113     train_loss = 0
114     correct = 0
115     total = 0
116
117     for batch_idx, (inputs, targets) in enumerate(train_loader):
118         inputs, targets = inputs.to(device), targets.to(device)

```

```

116         optimizer.zero_grad()
117         outputs = model(inputs)
118         loss = criterion(outputs, targets)
119         loss.backward()
120         optimizer.step()
121
122
123         train_loss += loss.item()
124         _, predicted = outputs.max(1)
125         total += targets.size(0)
126         correct += predicted.eq(targets).sum().item()
127
128     train_acc = 100. * correct / total
129
130     # Test
131     model.eval()
132     test_loss = 0
133     correct = 0
134     total = 0
135
136     with torch.no_grad():
137         for inputs, targets in test_loader:
138             inputs, targets = inputs.to(device), targets.to(device)
139             outputs = model(inputs)
140             loss = criterion(outputs, targets)
141
142             test_loss += loss.item()
143             _, predicted = outputs.max(1)
144             total += targets.size(0)
145             correct += predicted.eq(targets).sum().item()
146
147     test_acc = 100. * correct / total
148     scheduler.step()
149
150     print(f'Epoch: {epoch+1} | Train Acc: {train_acc:.2f}% | Test
151           Acc: {test_acc:.2f}%')
152
153     if test_acc > best_acc:
154         best_acc = test_acc
155         torch.save(model.state_dict(), 'best_kan_cnn_cifar10.pth')
156
157 print(f'Best Test Accuracy: {best_acc:.2f}%')

```

Listing 7.14: Pipeline completo para CIFAR-10

Este ejemplo completo demuestra cómo construir, configurar y entrenar una red CNN KAN de alto rendimiento para clasificación de imágenes en CIFAR-10, incorporando técnicas modernas de regularización y optimización.

Capítulo 8 Autoencoders basados en DualFlexKAN

Esta sección describe la implementación y uso de autoencoders utilizando la arquitectura DualFlexKAN, que permite construir codificadores y decodificadores con flexibilidad completa en la configuración de capas, funciones de activación y estrategias de regularización.

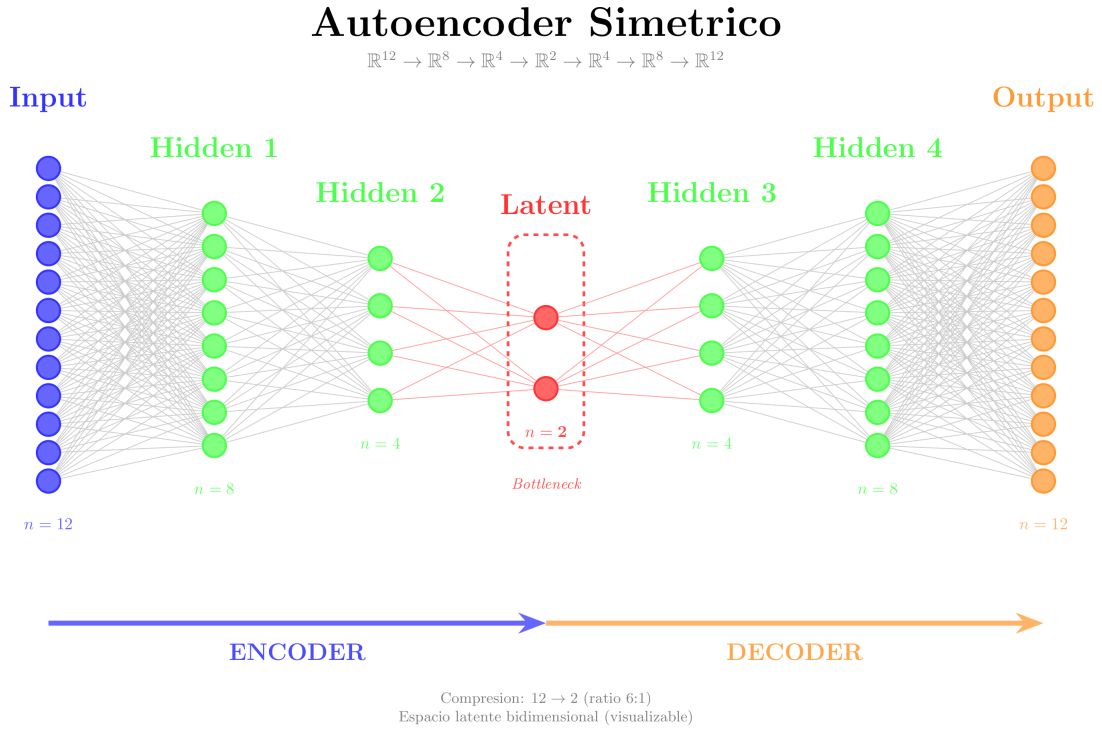


Figura 8.1: Arquitectura típica de autoencoder simétrico

8.1. Arquitectura General

El sistema está compuesto por tres clases principales que permiten la construcción modular de autoencoders:

- **DualFlexKANEncoder:** Implementa el encoder con arquitectura DualFlexKAN configurable.

- **DualFlexKANDecoder**: Implementa el decoder con arquitectura DualFlexKAN configurable.
- **DualFlexKANAutoencoder**: Combina encoder y decoder en un autoencoder completo, permitiendo arquitecturas asimétricas.

La arquitectura permite diseñar encoders y decoders con diferente número de capas ocultas y configuraciones independientes, lo que facilita la experimentación con topologías no simétricas.

8.2. Construcción del Encoder

La clase `DualFlexKANEncoder` encapsula un encoder basado en DualFlexKAN con inicialización inteligente automática.

8.2.1. Parámetros del Constructor

```

1 encoder = DualFlexKANEncoder(
2     layer_sizes=[input_size, hidden1, hidden2, ..., latent_size],
3     **kwargs    # Parámetros de DualFlexKAN
4 )

```

Listing 8.1: Constructor del DualFlexKANEncoder

Parámetros principales:

- **layer_sizes**: Lista con las dimensiones de cada capa, desde la entrada hasta el espacio latente. Por ejemplo, `[784, 256, 128, 64, 32]` define un encoder con 4 capas que comprime desde 784 dimensiones hasta 32.
- ****kwargs**: Todos los parámetros configurables de DualFlexKAN, incluyendo:
 - **input_function_strategies**: Estrategia para funciones de entrada por capa.
 - **input_activation_types**: Tipo de activación de entrada (e.g., 'polynomial', 'fourier', 'wavelet').
 - **output_function_strategies**: Estrategia para funciones de salida.
 - **output_activation_types**: Tipo de activación de salida.
 - **dropout_probs**: Probabilidades de dropout por capa.
 - **use_batch_norms**: Lista booleana indicando uso de batch normalization.
 - **regularization_orders**: Orden de aplicación de regularizaciones.

Ejemplo de Uso Básico

```
1 import torch
2 from DFKAN_autoencoder import DualFlexKANEncoder
3
4 # Encoder simple con configuración por defecto
5 encoder = DualFlexKANEncoder(
6     layer_sizes=[784, 256, 128, 32]
7 )
8
9 # Codificar una entrada
10 x = torch.randn(64, 784) # Batch de 64 imágenes aplanadas
11 z = encoder(x) # z.shape = (64, 32)
12
13 # Información de la arquitectura
14 encoder.print_architecture_info()
```

Listing 8.2: Creación de un encoder simple

Ejemplo con Configuración Avanzada

```
1 encoder = DualFlexKANEncoder(
2     layer_sizes=[784, 512, 256, 64],
3     input_activation_types=['polynomial', 'fourier', 'polynomial'],
4     input_activation_kwargs=[
5         {'degree': 3},
6         {'num_frequencies': 5},
7         {'degree': 4}
8     ],
9     output_activation_types=['polynomial', 'polynomial',
10                             'polynomial'],
11     dropout_probs=[0.2, 0.3, 0.2],
12     use_batch_norms=[True, True, False],
13     batch_norm_positions=['after_activation', 'after_activation',
14                           None]
15 )
```

Listing 8.3: Encoder con configuración personalizada

8.3. Construcción del Decoder

La clase `DualFlexKANDecoder` implementa el decodificador de forma análoga al encoder.

8.3.1. Parámetros del Constructor

```

1 decoder = DualFlexKANDecoder(
2     layer_sizes=[latent_size, hidden1, hidden2, ..., output_size],
3     **kwargs # Parámetros de DualFlexKAN
4 )

```

Listing 8.4: Constructor del DualFlexKANDecoder

El `layer_sizes` define la arquitectura desde el espacio latente hasta la reconstrucción de salida.

Construcción de un autoencoder con las clases DualFlexKANEncoder y DualFlexKANDecoder

```

1 from DFKAN_autoencoder import DualFlexKANEncoder, DualFlexKANDecoder
2
3 class MNISTAutoencoder(nn.Module):
4     """
5     Autoencoder para MNIST (784 neuronas de entrada)
6     Construido manualmente usando DualFlexKANEncoder y
7     DualFlexKANDecoder
8
9     Arquitectura:
10     ENCODER: 784 -> 256 -> 64 -> 8
11     DECODER: 8 -> 64 -> 256 -> 784
12     """
13
14     def __init__(self):
15         super(MNISTAutoencoder, self).__init__()
16
17         # =====
18         # ENCODER: 784 -> 256 -> 64 -> 8
19         # =====
20         encoder_layer_sizes = [784, 256, 64, 8]
21
22         encoder_config = {
23             'input_function_strategies': ['per_input', 'per_input',
24                                           'per_neuron'],
25             'input_activation_types': ['polynomial', 'polynomial',
26                                       'polynomial'],
27             'input_activation_kwargs': [
28                 {'degree': 3},
29                 {'degree': 3},
30                 {'degree': 3}
31             ],
32             'output_function_strategies': ['per_neuron',
33                                           'per_neuron', 'per_neuron'],
34             'output_activation_types': ['polynomial', 'polynomial',
35                                        'polynomial'],
36             'output_activation_kwargs': [

```

```

32         {'degree': 3},
33         {'degree': 3},
34         {'degree': 3}
35     ],
36     'dropout_probs': [0.0, 0.1, 0.2],
37     'use_batch_norms': [False, True, True]
38 }
39
40 self.encoder = DualFlexKANEncoder(
41     layer_sizes=encoder_layer_sizes,
42     **encoder_config
43 )
44
45 # =====
46 # DECODER: 8 -> 64 -> 256 -> 784
47 # =====
48 decoder_layer_sizes = [8, 64, 256, 784]
49
50 decoder_config = {
51     'input_function_strategies': ['per_neuron', 'per_input',
52     'per_input'],
53     'input_activation_types': ['polynomial', 'polynomial',
54     'polynomial'],
55     'input_activation_kwargs': [
56         {'degree': 3},
57         {'degree': 3},
58         {'degree': 3}
59     ],
60     'output_function_strategies': ['per_neuron',
61     'per_neuron', 'per_neuron'],
62     'output_activation_types': ['polynomial', 'polynomial',
63     'polynomial'],
64     'output_activation_kwargs': [
65         {'degree': 3},
66         {'degree': 3},
67         {'degree': 3}
68     ],
69     'dropout_probs': [0.2, 0.1, 0.0],
70     'use_batch_norms': [True, True, False]
71 }
72
73 self.decoder = DualFlexKANDecoder(
74     layer_sizes=decoder_layer_sizes,
75     **decoder_config
76 )
77
78 def encode(self, x):
79     """Codificar entrada a espacio latente"""

```

```

76         return self.encoder(x)
77
78     def decode(self, z):
79         """Decodificar espacio latente a salida"""
80         return self.decoder(z)
81
82     def forward(self, x):
83         """Forward pass completo"""
84         z = self.encode(x)
85         x_reconstructed = self.decode(z)
86         return x_reconstructed, z

```

Listing 8.5: Autoencoder MNIST con DualFlexKANEncoder y DualFlexKANDecoder

8.4. Autoencoder Completo con DualFlexKANAutoencoder

La clase `DualFlexKANAutoencoder` combina encoder y decoder, permitiendo arquitecturas asimétricas y configuraciones independientes para cada componente.

8.4.1. Parámetros del Constructor

```

1 autoencoder = DualFlexKANAutoencoder(
2     input_size=784,
3     latent_size=32,
4     output_size=784,
5     encoder_hidden_sizes=[256, 128, 64],
6     decoder_hidden_sizes=[64, 128, 256],
7     encoder_config={...},
8     decoder_config={...}
9 )

```

Listing 8.6: Constructor del autoencoder completo

Parámetros:

- `input_size`: Dimensión de entrada (e.g., 784 para imágenes MNIST de 28×28).
- `latent_size`: Dimensión del espacio latente (cuello de botella).
- `output_size`: Dimensión del espacio de salida (e.g. 784 para imágenes MNIST de 28×28).
- `encoder_hidden_sizes`: Lista con tamaños de capas ocultas del encoder. Si es `None`, usa `[64, 32]` por defecto.
- `decoder_hidden_sizes`: Lista con tamaños de capas ocultas del decoder. Si es `None`, usa `[32, 64]` por defecto.

- `encoder_config`: Diccionario con configuración completa del encoder (parámetros de DualFlexKAN).
- `decoder_config`: Diccionario con configuración completa del decoder.

Ejemplo: Autoencoder Simétrico

```
1 from DFKAN_autoencoder import DualFlexKANAutoencoder
2
3 # Autoencoder simétrico básico
4 autoencoder = DualFlexKANAutoencoder(
5     input_size=784,
6     latent_size=32,
7     output_size=784,
8     encoder_hidden_sizes=[256, 128, 64],
9     decoder_hidden_sizes=[64, 128, 256]
10 )
11
12 # Forward pass
13 x = torch.randn(64, 784)
14 x_reconstructed, z = autoencoder(x)
15
16 print(f"Entrada: {x.shape}")
17 print(f"Espacio latente: {z.shape}")
18 print(f"Reconstrucción: {x_reconstructed.shape}")
19
20 # Mostrar información de arquitectura
21 autoencoder.print_architecture_info()
```

Listing 8.7: Autoencoder con arquitectura simétrica

Ejemplo: Autoencoder Asimétrico

```
1 # Encoder profundo, decoder superficial
2 autoencoder_asimetrico = DualFlexKANAutoencoder(
3     input_size=784,
4     latent_size=16,
5     output_size=784,
6     encoder_hidden_sizes=[512, 256, 128, 64, 32], # 5 capas
7     decoder_hidden_sizes=[64, 256], # 2 capas
8 )
```

Listing 8.8: Autoencoder con arquitectura asimétrica

Ejemplo: Configuración Avanzada con Dicionarios

```

1 encoder_config = {
2     'input_activation_types': ['polynomial', 'fourier',
3     'polynomial'],
4     'input_activation_kwargs': [
5         {'degree': 3},
6         {'num_frequencies': 5},
7         {'degree': 4}
8     ],
9     'output_activation_types': ['polynomial', 'polynomial',
10    'polynomial'],
11    'output_activation_kwargs': [
12        {'degree': 3},
13        {'degree': 3},
14        {'degree': 2}
15    ],
16    'dropout_probs': [0.2, 0.3, 0.2],
17    'use_batch_norms': [True, True, False],
18    'batch_norm_positions': ['after_activation', 'after_activation',
19    None],
20    'regularization_orders': ['dropout_first', 'dropout_first',
21    'dropout_first']
22 }
23
24 decoder_config = {
25     'input_activation_types': ['polynomial', 'polynomial',
26     'polynomial'],
27     'output_activation_types': ['polynomial', 'polynomial',
28     'polynomial'],
29     'dropout_probs': [0.1, 0.15, 0.1],
30     'use_batch_norms': [False, True, True]
31 }
32
33 autoencoder_avanzado = DualFlexKANAutoencoder(
34     input_size=784,
35     latent_size=32,
36     encoder_hidden_sizes=[256, 128, 64],
37     decoder_hidden_sizes=[64, 128, 256],
38     encoder_config=encoder_config,
39     decoder_config=decoder_config
40 )

```

Listing 8.9: Autoencoder con configuración detallada

8.5. Visualización de la arquitectura

La clase `DualFlexKANAutoencoder` (en el fichero `DFKAN_autoencoder.py`) tiene una función `print_architecture_info()` que proporciona un resumen de alto nivel del autoen-

coder completo.

Si miras su código, verás que imprime esto:

```

1
2 def print_architecture_info(self):
3     print(f"\n{'='*60}")
4     print(f"AUTOENCODER FLEXIBLE ( I n p u t Output )")
5     print(f"{'='*60}")
6     print(f"Input Size: {self.input_size}")
7     print(f"Latent Size: {self.latent_size}")
8     print(f"Output Size: {self.output_size}")
9     # ... etc.
10 ```Esta función da la visión general: cuál es el tamaño de entrada,
    el tamaño latente y el tamaño de salida. No entra en el detalle
    de las capas.
```

Listing 8.10: Función de entrenamiento

Para acceder al detalle capa por capa (qué tipo de capa es, qué funciones de activación usa, el número de parámetros, etc.), es necesario utilizar la función `print_architecture_info()` de la clase `DualFlexKAN` (en el fichero `KAN_shared_poly6.py`).

No obstante, hay que tener en cuenta que un objeto `DualFlexKANAutoencoder` no es una red en sí misma, sino un contenedor que tiene dos redes `DualFlexKAN` dentro (una para el encoder y otra para el decoder). La estructura de los objetos es la siguiente:

- autoencoder (es de tipo `DualFlexKANAutoencoder`)
 - autoencoder.encoder (es de tipo `DualFlexKANEncoder`)
 - autoencoder.encoder.network (es de tipo `DualFlexKAN`)
 - autoencoder.decoder (es de tipo `DualFlexKANDecoder`)
 - autoencoder.decoder.network (también de tipo `DualFlexKAN`)

Por tanto, para obtener la información capa a capa del autoencoder es necesario llamar a la función `print_architecture_info()` sobre el atributo `.network` de cada componente.

```

1 # Supongamos que ya tienes el autoencoder creado:
2 # autoencoder = DualFlexKANAutoencoder(...)
3
4 # --- Para obtener el RESUMEN GENERAL del autoencoder ---
5 print("\n--- RESUMEN GENERAL DEL AUTOENCODER ---")
6 autoencoder.print_architecture_info()
7 # Salida esperada:
8 # =====
9 # AUTOENCODER DUALFLEXKAN
10 # =====
11 # Input Size: 784
12 # Latent Size: 32
13 # Output Size: 784
```

```

14 # ...
15
16
17 # --- Para obtener la ARQUITECTURA DETALLADA del encoder ---
18 print("\n\n--- ARQUITECTURA DETALLADA DEL ENCODER ---")
19 autoencoder.encoder.network.print_architecture_info()
20 # Salida esperada:
21 # === Información de Arquitectura DualFlexKAN ... ===
22 #
23 # Capa 1 (784→256) - KAN Dual (Adaptativa entrada + salida)
24 #   Función entrada: polynomial (per_input)
25 #   Función salida: polynomial (per_neuron)
26 #   ...
27 #   Total capa: ... parámetros
28 # ...
29
30
31 # --- Para obtener la ARQUITECTURA DETALLADA del decoder ---
32 print("\n\n--- ARQUITECTURA DETALLADA DEL DECODER ---")
33 autoencoder.decoder.network.print_architecture_info()
34 # Salida esperada:
35 # === Información de Arquitectura DualFlexKAN ... ===
36 #
37 # Capa 1 (32→128) - KAN Dual (Adaptativa entrada + salida)
38 #   Función entrada: polynomial (per_input)
39 #   Función salida: polynomial (per_neuron)
40 #   ...
41 #   Total capa: ... parámetros
42 # ...

```

8.6. Entrenamiento del Autoencoder

La función `train_autoencoder` implementa el bucle de entrenamiento completo con evaluación en el conjunto de test.

8.6.1. Función `train_autoencoder`

```

1 train_losses, test_losses = train_autoencoder(
2     model=autoencoder,
3     train_loader=train_loader,
4     test_loader=test_loader,
5     epochs=50,
6     lr=0.001,
7     device='cpu',
8     use_labels_as_target=False

```

9)

Listing 8.11: Función de entrenamiento

Parámetros:

- `model`: Instancia del autoencoder a entrenar.
- `train_loader`: DataLoader de PyTorch con datos de entrenamiento.
- `test_loader`: DataLoader con datos de validación/test.
- `epochs`: Número de épocas de entrenamiento (default: 50).
- `lr`: Tasa de aprendizaje para el optimizador Adam (default: 0.001).
- `device`: Dispositivo de cómputo ('cpu' o 'cuda').
- `use_labels_as_target`: Si es True, usa las etiquetas del dataloader como objetivo. Si es False, usa la entrada (default: False).

Retorna:

- `train_losses`: Lista con la pérdida promedio de entrenamiento por época.
- `test_losses`: Lista con la pérdida promedio de test por época.

8.7. Codificación y decodificación independientes

Una vez entrenado el autoencoder, sus componentes (encoder y decoder) pueden utilizarse independientemente para diferentes tareas.

8.7.1. Codificación de Nuevos Datos

```

1  # Cargar modelo entrenado
2  autoencoder = DualFlexKANAutoencoder(
3      input_size=784,
4      latent_size=32,
5      output_size=784,
6      encoder_hidden_sizes=[256, 128, 64],
7      decoder_hidden_sizes=[64, 128, 256]
8  )
9  autoencoder.load_state_dict(torch.load('autoencoder_trained.pth'))
10 autoencoder.eval()  # Modo evaluación
11
12 # Codificar nuevas imágenes al espacio latente
13 with torch.no_grad():
14     new_images = torch.randn(10, 784)  # 10 nuevas muestras
15     latent_codes = autoencoder.encode(new_images)

```

```
16     print(f"Códigos latentes: {latent_codes.shape}") # (10, 32)
17
18 # También se puede acceder directamente al encoder
19 latent_codes = autoencoder.encoder(new_images)
```

Listing 8.12: Uso del encoder en inferencia

8.7.2. Decodificación desde el Espacio Latente

```
1 # Generar muestras desde el espacio latente
2 with torch.no_grad():
3     # Muestrear códigos latentes aleatorios
4     random_latent = torch.randn(16, 32)
5     generated_images = autoencoder.decode(random_latent)
6     print(f"Imágenes generadas: {generated_images.shape}") # (16,
7         784)
8
9     # Remodelar para visualizar
10    generated_images = generated_images.view(-1, 28, 28)
11
12 # También acceso directo al decoder
13 generated_images = autoencoder.decoder(random_latent)
```

Listing 8.13: Uso del decoder en inferencia

8.7.3. Interpolación en el Espacio Latente

```
1 import torch
2
3 def interpolate_latent(autoencoder, img1, img2, n_steps=10):
4     """
5     Interpola entre dos imágenes en el espacio latente.
6
7     Args:
8         autoencoder: Modelo entrenado
9         img1, img2: Tensores de entrada (1, 784)
10        n_steps: Número de pasos de interpolación
11
12    Returns:
13        Secuencia de imágenes interpoladas
14    """
15    autoencoder.eval()
16    with torch.no_grad():
17        # Codificar ambas imágenes
18        z1 = autoencoder.encode(img1)
19        z2 = autoencoder.encode(img2)
20
```

```

21     # Crear interpolación lineal
22     interpolations = []
23     for alpha in torch.linspace(0, 1, n_steps):
24         z_interp = (1 - alpha) * z1 + alpha * z2
25         img_interp = autoencoder.decode(z_interp)
26         interpolations.append(img_interp)
27
28     return torch.cat(interpolations, dim=0)
29
30 # Ejemplo de uso
31 img1 = test_dataset[0][0].unsqueeze(0) # Primera imagen
32 img2 = test_dataset[100][0].unsqueeze(0) # Otra imagen
33 interpolated = interpolate_latent(autoencoder, img1, img2,
    n_steps=15)

```

Listing 8.14: Interpolación entre dos imágenes

8.7.4. Visualización de Resultados

La función `visualize_reconstructions` permite visualizar la calidad de las reconstrucciones del autoencoder.

Uso de la Función de Visualización

```

1 from DFKAN_autoencoder import visualize_reconstructions
2
3 # Visualizar 10 reconstrucciones
4 visualize_reconstructions(
5     model=autoencoder,
6     test_loader=test_loader,
7     device=device,
8     n_samples=10
9 )

```

Listing 8.15: Visualización de reconstrucciones

La función genera una figura con dos filas: la primera muestra las imágenes originales y la segunda sus reconstrucciones correspondientes.

8.7.5. Transfer Learning con Encoder Pre-entrenado

```

1 class ClassifierWithPretrainedEncoder(nn.Module):
2     """Clasificador usando encoder pre-entrenado del autoencoder"""
3
4     def __init__(self, pretrained_autoencoder, num_classes):
5         super().__init__()
6
7         # Encoder pre-entrenado (congelado o fine-tunable)

```

```
8         self.encoder = pretrained_autoencoder.encoder
9
10        # Congelar encoder si solo se quiere entrenar el clasificador
11        for param in self.encoder.parameters():
12            param.requires_grad = False
13
14        # Cabeza de clasificación
15        latent_size = pretrained_autoencoder.latent_size
16        self.classifier = nn.Sequential(
17            nn.Linear(latent_size, 128),
18            nn.ReLU(),
19            nn.Dropout(0.5),
20            nn.Linear(128, num_classes)
21        )
22
23        def forward(self, x):
24            features = self.encoder(x)
25            logits = self.classifier(features)
26            return logits
27
28        # Crear clasificador
29        classifier = ClassifierWithPretrainedEncoder(
30            pretrained_autoencoder=autoencoder,
31            num_classes=10
32        )
```

Listing 8.16: Uso del encoder pre-entrenado para clasificación

Capítulo 9 Autoencoders Convolucionales basados en DualFlexKAN

Esta sección presenta la implementación de autoencoders utilizando capas convolucionales KAN. A diferencia de los autoencoders densos, estas arquitecturas están diseñadas para procesar datos con estructura espacial o temporal. Se proporciona soporte completo para datos 1D (señales, series temporales), 2D (imágenes) y 3D (videos, datos volumétricos).

9.1. Arquitectura General

Los autoencoders convolucionales KAN siguen una estructura encoder-decoder donde ambas partes utilizan capas convolucionales con transformaciones adaptativas:

- **Encoder:** Reduce progresivamente las dimensiones espaciales mediante convoluciones con stride, aumentando el número de canales y comprimiendo la información hacia un espacio latente.
- **Espacio Latente:** Representación comprimida que captura las características esenciales de la entrada. Puede ser determinista o estocástico (en VAEs).
- **Decoder:** Reconstruye las dimensiones espaciales originales mediante convoluciones transpuestas, reduciendo progresivamente los canales hasta la dimensión de entrada.

9.1.1. Soporte Multidimensional

Todas las arquitecturas están disponibles en tres variantes según la dimensionalidad de los datos:

Dimensión	Forma de Entrada	Aplicaciones Típicas
1D	(B, C, L)	Series temporales, audio, señales
2D	(B, C, H, W)	Imágenes, mapas espaciales
3D	(B, C, D, H, W)	Videos, datos volumétricos médicos

Tabla 9.1: Dimensionalidades soportadas en autoencoders convolucionales KAN

donde B es batch size, C es canales, L es longitud, H altura, W ancho y D profundidad.

9.1.2. Pipeline de Procesamiento

El procesamiento en cada capa convolucional KAN sigue el pipeline:

$$\mathbf{y} = \text{Reg}_{\text{post}} \circ f_{\text{out}} \circ \text{Reg}_{\text{pre}} \circ \text{Conv} \circ f_{\text{in}}(\mathbf{x}) \quad (9.1)$$

donde:

- f_{in} : Transformación adaptativa de entrada (por canal)
- Conv: Operación convolucional estándar (1D, 2D o 3D)
- Reg_{pre} : Regularización pre-activación (BatchNorm, Dropout)
- f_{out} : Transformación adaptativa de salida (por canal)
- Reg_{post} : Regularización post-activación

9.2. Construcción del Encoder

El encoder es responsable de comprimir la entrada hacia el espacio latente mediante una secuencia de capas convolucionales KAN con reducción espacial.

9.2.1. Parámetros del Constructor

La construcción del encoder se realiza mediante una lista de configuraciones de capas:

```

1 encoder_configs = [
2     {
3         'in_channels': input_channels,
4         'out_channels': 32,
5         'kernel_size': 3,
6         'stride': 2,                # Reduce dimensiones espaciales
7         'padding': 1,
8         'input_function_strategy': "per_channel",
9         'input_activation_type': "polynomial",
10        'output_function_strategy': "per_channel",
11        'output_activation_type': "polynomial",
12        'use_batch_norm': True,
13        'dropout_prob': 0.0
14    },
15    # ... mas capas
16 ]

```


Parámetros clave:

- **stride:** Controla la reducción espacial. $\text{stride} = 2$ reduce dimensiones a la mitad.
- **input_function_strategy:** Estrategia para transformaciones de entrada
 - "none": Sin transformación
 - "fixed": Activación fija (ReLU, tanh, etc.)
 - "global": Una función compartida para todos los canales
 - "per_channel": Función independiente por canal
- **output_function_strategy:** Similar al anterior, para la salida
- **use_batch_norm:** Normalización por lotes para estabilidad
- **dropout_prob:** Regularización mediante dropout

9.2.2. Ejemplo de Uso Básico

Construcción de un encoder simple para imágenes RGB 32×32 :

```

1 from KAN_conv import DualFlexKANConv2d
2 import torch.nn as nn
3
4 class BasicEncoder(nn.Module):
5     def __init__(self, input_channels=3, latent_dim=128):
6         super().__init__()
7
8         # Capa 1: 3 -> 32 canales, 32x32 -> 16x16
9         self.conv1 = DualFlexKANConv2d(
10             in_channels=input_channels,
11             out_channels=32,
12             kernel_size=3,
13             stride=2,
14             padding=1,
15             input_function_strategy="per_channel",
16             input_activation_type="polynomial",
17             output_function_strategy="per_channel",
18             output_activation_type="polynomial",
19             use_batch_norm=True
20         )
21
22         # Capa 2: 32 -> 64 canales, 16x16 -> 8x8
23         self.conv2 = DualFlexKANConv2d(
24             in_channels=32,
25             out_channels=64,
26             kernel_size=3,
27             stride=2,

```

```

28         padding=1,
29         input_function_strategy="global",
30         output_function_strategy="per_channel",
31         use_batch_norm=True
32     )
33
34     # Capa 3: 64 -> 128 canales, 8x8 -> 4x4
35     self.conv3 = DualFlexKANConv2d(
36         in_channels=64,
37         out_channels=128,
38         kernel_size=3,
39         stride=2,
40         padding=1,
41         output_function_strategy="per_channel",
42         use_batch_norm=True,
43         dropout_prob=0.2
44     )
45
46     # Proyeccion a espacio latente
47     self.flatten = nn.Flatten()
48     self.fc = nn.Linear(128 * 4 * 4, latent_dim)
49
50     def forward(self, x):
51         x = self.conv1(x)      # (B, 32, 16, 16)
52         x = self.conv2(x)      # (B, 64, 8, 8)
53         x = self.conv3(x)      # (B, 128, 4, 4)
54         x = self.flatten(x)    # (B, 2048)
55         z = self.fc(x)         # (B, latent_dim)
56         return z

```

9.2.3. Ejemplo con Configuración Avanzada

Encoder con diferentes tipos de activaciones y regularización variable:

```

1 class AdvancedEncoder(nn.Module):
2     def __init__(self, input_channels=3, latent_dim=128):
3         super().__init__()
4
5         # Bloque 1: Polinomios en entrada y salida
6         self.conv1 = DualFlexKANConv2d(
7             in_channels=input_channels,
8             out_channels=32,
9             kernel_size=3,
10            stride=2,
11            padding=1,
12            input_function_strategy="per_channel",
13            input_activation_type="polynomial",
14            input_activation_kwargs={'degree': 3},

```

```
15         output_function_strategy="per_channel",
16         output_activation_type="polynomial",
17         output_activation_kwargs={'degree': 2},
18         use_batch_norm=True,
19         batch_norm_position="after_activation"
20     )
21
22     # Bloque 2: Legendre en salida
23     self.conv2 = DualFlexKANConv2d(
24         in_channels=32,
25         out_channels=64,
26         kernel_size=3,
27         stride=2,
28         padding=1,
29         input_function_strategy="global",
30         input_activation_type="polynomial",
31         output_function_strategy="per_channel",
32         output_activation_type="legendre",
33         output_activation_kwargs={'order': 4},
34         use_batch_norm=True,
35         dropout_prob=0.1
36     )
37
38     # Bloque 3: RBF en salida para mayor expresividad
39     self.conv3 = DualFlexKANConv2d(
40         in_channels=64,
41         out_channels=128,
42         kernel_size=3,
43         stride=2,
44         padding=1,
45         input_function_strategy="global",
46         input_activation_type="polynomial",
47         output_function_strategy="per_channel",
48         output_activation_type="rbf",
49         output_activation_kwargs={'n_centers': 8},
50         use_batch_norm=True,
51         batch_norm_position="both",
52         dropout_prob=0.2
53     )
54
55     self.flatten = nn.Flatten()
56     self.fc = nn.Linear(128 * 4 * 4, latent_dim)
57
58     def forward(self, x):
59         x = self.conv1(x)
60         x = self.conv2(x)
61         x = self.conv3(x)
62         x = self.flatten(x)
```

```

63     z = self.fc(x)
64     return z

```

9.2.4. Encoder 1D para Series Temporales

```

1  from KAN_conv import DualFlexKANConv1d
2
3  class Encoder1D(nn.Module):
4      def __init__(self, input_channels=16, latent_dim=64):
5          super().__init__()
6
7          # Entrada: (B, 16, 256) -> (B, 32, 128)
8          self.conv1 = DualFlexKANConv1d(
9              in_channels=input_channels,
10             out_channels=32,
11             kernel_size=3,
12             stride=2,
13             padding=1,
14             output_function_strategy="per_channel",
15             use_batch_norm=True
16         )
17
18         # (B, 32, 128) -> (B, 64, 64)
19         self.conv2 = DualFlexKANConv1d(
20             in_channels=32,
21             out_channels=64,
22             kernel_size=3,
23             stride=2,
24             padding=1,
25             output_function_strategy="per_channel",
26             use_batch_norm=True
27         )
28
29         self.flatten = nn.Flatten()
30         self.fc = nn.Linear(64 * 64, latent_dim)
31
32     def forward(self, x):
33         x = self.conv1(x)
34         x = self.conv2(x)
35         x = self.flatten(x)
36         return self.fc(x)

```

9.2.5. Encoder 3D para Videos

```

1  from KAN_conv import DualFlexKANConv3d
2

```

```

3 class Encoder3D(nn.Module):
4     def __init__(self, input_channels=3, latent_dim=256):
5         super().__init__()
6
7         # Entrada: (B, 3, 16, 64, 64) -> (B, 32, 8, 32, 32)
8         self.conv1 = DualFlexKANConv3d(
9             in_channels=input_channels,
10            out_channels=32,
11            kernel_size=3,
12            stride=2,
13            padding=1,
14            output_function_strategy="per_channel",
15            use_batch_norm=True
16        )
17
18        # (B, 32, 8, 32, 32) -> (B, 64, 4, 16, 16)
19        self.conv2 = DualFlexKANConv3d(
20            in_channels=32,
21            out_channels=64,
22            kernel_size=3,
23            stride=2,
24            padding=1,
25            output_function_strategy="per_channel",
26            use_batch_norm=True
27        )
28
29        self.flatten = nn.Flatten()
30        self.fc = nn.Linear(64 * 4 * 16 * 16, latent_dim)
31
32    def forward(self, x):
33        x = self.conv1(x)
34        x = self.conv2(x)
35        x = self.flatten(x)
36        return self.fc(x)

```

9.3. Construcción del Decoder

El decoder reconstruye las dimensiones espaciales originales desde el espacio latente mediante convoluciones transpuestas seguidas de capas KAN para refinamiento.

9.3.1. Parámetros del Constructor

El decoder utiliza `nn.ConvTranspose1d/2d/3d` para upsampling seguido de capas KAN:

```

1 decoder_config = {
2     'latent_dim': 128,
3     'target_shape': (3, 32, 32), # (channels, height, width)

```

```
4     'base_channels': 128,  
5     'use_batch_norm': True  
6 }
```

9.3.2. Estructura del Decoder:

Cada bloque del decoder consta de:

1. Convolución transpuesta para upsampling (aumenta dimensiones espaciales)
2. Capa KAN para refinamiento de características
3. Regularización opcional (BatchNorm, Dropout)

9.3.3. Ejemplo de Uso

Decoder Básico 2D:

```
1 class BasicDecoder(nn.Module):  
2     def __init__(self, latent_dim=128, output_channels=3):  
3         super().__init__()  
4  
5         # Proyeccion desde espacio latente  
6         self.fc = nn.Linear(latent_dim, 128 * 4 * 4)  
7  
8         # Bloque 1: 128 -> 64 canales, 4x4 -> 8x8  
9         self.deconv1 = nn.ConvTranspose2d(  
10             in_channels=128,  
11             out_channels=64,  
12             kernel_size=4,  
13             stride=2,  
14             padding=1  
15         )  
16         self.kan1 = DualFlexKANConv2d(  
17             in_channels=64,  
18             out_channels=64,  
19             kernel_size=3,  
20             padding=1,  
21             input_function_strategy="global",  
22             output_function_strategy="per_channel",  
23             use_batch_norm=True  
24         )  
25  
26         # Bloque 2: 64 -> 32 canales, 8x8 -> 16x16  
27         self.deconv2 = nn.ConvTranspose2d(  
28             in_channels=64,  
29             out_channels=32,
```

```

30         kernel_size=4,
31         stride=2,
32         padding=1
33     )
34     self.kan2 = DualFlexKANConv2d(
35         in_channels=32,
36         out_channels=32,
37         kernel_size=3,
38         padding=1,
39         output_function_strategy="per_channel",
40         use_batch_norm=True
41     )
42
43     # Bloque 3: 32 -> output_channels, 16x16 -> 32x32
44     self.deconv3 = nn.ConvTranspose2d(
45         in_channels=32,
46         out_channels=output_channels,
47         kernel_size=4,
48         stride=2,
49         padding=1
50     )
51
52     def forward(self, z):
53         x = self.fc(z)                # (B, 2048)
54         x = x.view(-1, 128, 4, 4)    # (B, 128, 4, 4)
55
56         x = self.deconv1(x)           # (B, 64, 8, 8)
57         x = self.kan1(x)
58
59         x = self.deconv2(x)           # (B, 32, 16, 16)
60         x = self.kan2(x)
61
62         x = self.deconv3(x)           # (B, 3, 32, 32)
63         x = torch.sigmoid(x)          # Normalizar a [0,1]
64
65         return x

```

Decoders para 1D y 3D

```

1  # Decoder 1D para series temporales
2  class Decoder1D(nn.Module):
3      def __init__(self, latent_dim=64, output_channels=16):
4          super().__init__()
5
6          self.fc = nn.Linear(latent_dim, 64 * 64)
7
8          self.deconv1 = nn.ConvTranspose1d(64, 32, 4, 2, 1)

```

```

9         self.kan1 = DualFlexKANConv1d(32, 32, 3, padding=1)
10
11         self.deconv2 = nn.ConvTranspose1d(32, output_channels, 4, 2,
12             1)
13
14     def forward(self, z):
15         x = self.fc(z).view(-1, 64, 64)
16         x = self.kan1(self.deconv1(x))
17         return torch.sigmoid(self.deconv2(x))
18
19 # Decoder 3D para videos
20 class Decoder3D(nn.Module):
21     def __init__(self, latent_dim=256, output_channels=3):
22         super().__init__()
23
24         self.fc = nn.Linear(latent_dim, 64 * 4 * 16 * 16)
25
26         self.deconv1 = nn.ConvTranspose3d(64, 32, 4, 2, 1)
27         self.kan1 = DualFlexKANConv3d(32, 32, 3, padding=1)
28
29         self.deconv2 = nn.ConvTranspose3d(32, output_channels, 4, 2,
30             1)
31
32     def forward(self, z):
33         x = self.fc(z).view(-1, 64, 4, 16, 16)
34         x = self.kan1(self.deconv1(x))
35         return torch.sigmoid(self.deconv2(x))

```

9.4. Autoencoder Completo

El autoencoder completo combina encoder y decoder, proporcionando una interfaz unificada para codificación, decodificación y reconstrucción.

9.4.1. Parámetros del Constructor

```

1 KANConvAutoencoder(
2     input_channels=3,          # Canales de entrada
3     latent_dim=128,           # Dimension del espacio latente
4     dimensionality="2d"       # "1d", "2d" o "3d"
5 )

```

Ejemplo: Autoencoder Simétrico

Autoencoder con arquitectura perfectamente simétrica:


```

1 from KAN_autoencoders_CONV import KANConvAutoencoder
2
3 # Autoencoder para imagenes RGB 32x32
4 autoencoder = KANConvAutoencoder(
5     input_channels=3,
6     latent_dim=128,
7     dimensionality="2d"
8 )
9
10 # Uso basico
11 x = torch.randn(8, 3, 32, 32)
12 x_reconstructed = autoencoder(x)
13
14 print(f"Input shape: {x.shape}")
15 print(f"Output shape: {x_reconstructed.shape}")
16
17 # Acceso independiente
18 z = autoencoder.encode(x)
19 x_decoded = autoencoder.decode(z)

```

Ejemplo: Autoencoder Asimétrico

Autoencoder con diferentes configuraciones para encoder y decoder:

```

1 class AsymmetricAutoencoder(nn.Module):
2     """
3     Encoder agresivo (rapida compresion) con
4     Decoder cuidadoso (reconstruccion detallada)
5     """
6     def __init__(self):
7         super().__init__()
8
9         # ENCODER: Compresion rapida con pocos canales
10        self.enc_conv1 = DualFlexKANConv2d(
11            3, 16, kernel_size=5, stride=4, padding=2, # stride=4
12            (agresivo)
13            output_function_strategy="global"
14        )
15        self.enc_conv2 = DualFlexKANConv2d(
16            16, 32, kernel_size=3, stride=2, padding=1,
17            output_function_strategy="global"
18        )
19
20        self.flatten = nn.Flatten()
21        self.fc_encode = nn.Linear(32 * 2 * 2, 64) # Latente pequeño
22
23        # DECODER: Reconstruccion gradual con mas canales
24        self.fc_decode = nn.Linear(64, 64 * 2 * 2)

```

```

24
25     # Mas bloques de upsampling que en encoder
26     self.deconv1 = nn.ConvTranspose2d(64, 64, 4, 2, 1)
27     self.kan1 = DualFlexKANConv2d(64, 64, 3, padding=1)
28
29     self.deconv2 = nn.ConvTranspose2d(64, 32, 4, 2, 1)
30     self.kan2 = DualFlexKANConv2d(32, 32, 3, padding=1)
31
32     self.deconv3 = nn.ConvTranspose2d(32, 16, 4, 2, 1)
33     self.kan3 = DualFlexKANConv2d(16, 16, 3, padding=1)
34
35     self.deconv4 = nn.ConvTranspose2d(16, 3, 4, 2, 1)
36
37     def encode(self, x):
38         x = self.enc_conv1(x) # Compresion fuerte
39         x = self.enc_conv2(x)
40         x = self.flatten(x)
41         return self.fc_encode(x)
42
43     def decode(self, z):
44         x = self.fc_decode(z).view(-1, 64, 2, 2)
45         x = self.kan1(self.deconv1(x)) # Expansion gradual
46         x = self.kan2(self.deconv2(x))
47         x = self.kan3(self.deconv3(x))
48         x = torch.sigmoid(self.deconv4(x))
49         return x
50
51     def forward(self, x):
52         return self.decode(self.encode(x))

```

9.5. Configuración Avanzada con Diccionarios

Construcción mediante diccionarios de configuración para máxima flexibilidad:

```

1  # Configuracion completa mediante diccionarios
2  encoder_config = [
3      {
4          'in_channels': 3, 'out_channels': 32,
5          'kernel_size': 3, 'stride': 2, 'padding': 1,
6          'input_function_strategy': 'per_channel',
7          'input_activation_type': 'polynomial',
8          'input_activation_kwargs': {'degree': 3},
9          'output_function_strategy': 'per_channel',
10         'output_activation_type': 'legendre',
11         'output_activation_kwargs': {'order': 4},
12         'use_batch_norm': True,
13         'batch_norm_position': 'after_activation',

```

```

14         'dropout_prob': 0.1
15     },
16     {
17         'in_channels': 32, 'out_channels': 64,
18         'kernel_size': 3, 'stride': 2, 'padding': 1,
19         'input_function_strategy': 'global',
20         'output_function_strategy': 'per_channel',
21         'output_activation_type': 'rbf',
22         'output_activation_kwargs': {'n_centers': 8},
23         'use_batch_norm': True,
24         'dropout_prob': 0.2
25     },
26     {
27         'in_channels': 64, 'out_channels': 128,
28         'kernel_size': 3, 'stride': 2, 'padding': 1,
29         'output_function_strategy': 'per_channel',
30         'output_activation_type': 'gegenbauer',
31         'output_activation_kwargs': {'max_degree': 4},
32         'use_batch_norm': True,
33         'dropout_prob': 0.3
34     }
35 ]
36
37 # Construir encoder dinamicamente
38 class ConfigurableEncoder(nn.Module):
39     def __init__(self, config_list, latent_dim):
40         super().__init__()
41
42         self.layers = nn.ModuleList([
43             DualFlexKANConv2d(**cfg) for cfg in config_list
44         ])
45
46         # Calcular tamaño de salida
47         final_channels = config_list[-1]['out_channels']
48         final_spatial = 32 // (2 ** len(config_list))
49
50         self.flatten = nn.Flatten()
51         self.fc = nn.Linear(
52             final_channels * final_spatial * final_spatial,
53             latent_dim
54         )
55
56     def forward(self, x):
57         for layer in self.layers:
58             x = layer(x)
59         return self.fc(self.flatten(x))
60
61 # Uso

```

```
62 encoder = ConfigurableEncoder(encoder_config, latent_dim=128)
```

9.6. Entrenamiento del Autoencoder

9.6.1. Función de Entrenamiento Básica

```
1 import torch.optim as optim
2 from torch.utils.data import DataLoader
3
4 def train_autoencoder(model, train_loader, val_loader=None,
5                       num_epochs=100, lr=1e-3, device='cuda'):
6     """
7     Entrena un autoencoder convolucional KAN
8
9     Args:
10        model: Instancia del autoencoder
11        train_loader: DataLoader de entrenamiento
12        val_loader: DataLoader de validacion (opcional)
13        num_epochs: Numero de epocas
14        lr: Learning rate
15        device: Dispositivo (cuda/cpu)
16
17    Returns:
18        Diccionario con historico de perdidas
19    """
20    model = model.to(device)
21    optimizer = optim.Adam(model.parameters(), lr=lr)
22    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
23        optimizer, mode='min', factor=0.5, patience=5
24    )
25
26    history = {'train_loss': [], 'val_loss': []}
27
28    for epoch in range(num_epochs):
29        # Entrenamiento
30        model.train()
31        train_loss = 0.0
32
33        for batch_idx, x_batch in enumerate(train_loader):
34            x_batch = x_batch.to(device)
35
36            # Forward pass
37            x_recon = model(x_batch)
38            loss = F.mse_loss(x_recon, x_batch)
39
40            # Backward pass
```

```

41         optimizer.zero_grad()
42         loss.backward()
43         optimizer.step()
44
45         train_loss += loss.item()
46
47     avg_train_loss = train_loss / len(train_loader)
48     history['train_loss'].append(avg_train_loss)
49
50     # Validacion
51     if val_loader is not None:
52         model.eval()
53         val_loss = 0.0
54
55         with torch.no_grad():
56             for x_batch in val_loader:
57                 x_batch = x_batch.to(device)
58                 x_recon = model(x_batch)
59                 val_loss += F.mse_loss(x_recon, x_batch).item()
60
61     avg_val_loss = val_loss / len(val_loader)
62     history['val_loss'].append(avg_val_loss)
63
64     # Ajustar learning rate
65     scheduler.step(avg_val_loss)
66
67     print(f"Epoch {epoch+1}/{num_epochs} - "
68           f"Train Loss: {avg_train_loss:.4f} - "
69           f"Val Loss: {avg_val_loss:.4f}")
70 else:
71     print(f"Epoch {epoch+1}/{num_epochs} - "
72           f"Train Loss: {avg_train_loss:.4f}")
73
74     return history

```

9.7. Codificación y Decodificación Independientes

Una vez entrenado, el autoencoder permite uso independiente del encoder y decoder.

9.7.1. Codificación de Nuevos Datos

El encoder puede usarse como extractor de características:

```

1 # Cargar modelo entrenado
2 model.load_state_dict(torch.load('autoencoder_kan.pth'))
3 model.eval()
4

```

```

5 # Codificar un batch de imagenes
6 with torch.no_grad():
7     images = next(iter(test_loader))
8     images = images.to(device)
9
10    # Extraer representaciones latentes
11    latent_vectors = model.encode(images)
12
13    print(f"Input: {images.shape}")
14    print(f"Latent: {latent_vectors.shape}")
15
16    # Usar para tareas downstream
17    from sklearn.cluster import KMeans
18
19    # Clustering en espacio latente
20    latent_np = latent_vectors.cpu().numpy()
21    kmeans = KMeans(n_clusters=10, random_state=42)
22    clusters = kmeans.fit_predict(latent_np)
23
24    print(f"Cluster assignments: {clusters}")

```

9.7.2. Decodificación desde el Espacio Latente

El decoder permite generar nuevas muestras:

```

1 # Generar imagenes desde vectores latentes aleatorios
2 model.eval()
3 with torch.no_grad():
4     # Samplear del espacio latente
5     z_random = torch.randn(16, latent_dim).to(device)
6
7     # Decodificar a imagenes
8     generated_images = model.decode(z_random)
9
10    print(f"Generated: {generated_images.shape}")
11
12    # Para VAE: samplear de la distribucion aprendida
13    if isinstance(model, KANConvVAE):
14        samples = model.sample(num_samples=16, device=device)

```

Manipulación del Espacio Latente:

```

1 # Modificar vectores latentes para explorar el espacio
2 def modify_latent(z, direction, alpha=1.0):
3     """
4     Modifica vector latente en una direccion
5

```

```

6     Args:
7         z: Vector latente original
8         direction: Vector de direccion (mismo dim que z)
9         alpha: Magnitud de la modificacion
10    """
11    return z + alpha * direction
12
13 # Ejemplo: encontrar direccion "sonrisa"
14 z1 = model.encode(image_sin_sonrisa)
15 z2 = model.encode(image_con_sonrisa)
16 smile_direction = z2 - z1
17
18 # Aplicar a nueva imagen
19 z_new = model.encode(test_image)
20 z_modified = modify_latent(z_new, smile_direction, alpha=0.5)
21 image_modified = model.decode(z_modified)

```

9.7.3. Interpolación en el Espacio Latente

La interpolación permite transiciones suaves entre imágenes:

```

1 def interpolate_latent(model, img1, img2, steps=10, device='cuda'):
2     """
3     Interpola entre dos imagenes en el espacio latente
4
5     Args:
6         model: Autoencoder entrenado
7         img1, img2: Imagenes a interpolar (tensors)
8         steps: Numero de pasos de interpolacion
9         device: Dispositivo
10
11     Returns:
12         Tensor con imagenes interpoladas
13     """
14     model.eval()
15
16     with torch.no_grad():
17         # Codificar ambas imagenes
18         z1 = model.encode(img1.unsqueeze(0).to(device))
19         z2 = model.encode(img2.unsqueeze(0).to(device))
20
21         # Interpolar linealmente
22         interpolations = []
23         for alpha in torch.linspace(0, 1, steps):
24             z_interp = (1 - alpha) * z1 + alpha * z2
25             img_interp = model.decode(z_interp)
26             interpolations.append(img_interp)
27

```

```

28         return torch.cat(interpolations, dim=0)
29
30 # Uso
31 img_sequence = interpolate_latent(
32     model, image_a, image_b, steps=10
33 )
34
35 # Visualizar secuencia
36 import matplotlib.pyplot as plt
37 fig, axes = plt.subplots(1, 10, figsize=(20, 2))
38 for i, ax in enumerate(axes):
39     ax.imshow(img_sequence[i].cpu().permute(1, 2, 0))
40     ax.axis('off')
41 plt.show()

```

9.7.4. Interpolación Esférica (para VAEs):

Para VAEs, la interpolación esférica (slerp) puede dar mejores resultados:

```

1 def slerp(z1, z2, alpha):
2     """Spherical Linear Interpolation"""
3     omega = torch.acos(torch.clamp(
4         torch.dot(z1.flatten(), z2.flatten()) /
5         (torch.norm(z1) * torch.norm(z2)),
6         -1, 1
7     ))
8     so = torch.sin(omega)
9     if so < 1e-6:
10         return (1.0 - alpha) * z1 + alpha * z2
11     return (torch.sin((1.0 - alpha) * omega) / so) * z1 + \
12         (torch.sin(alpha * omega) / so) * z2
13
14 # Uso con VAE
15 z1, _ = vae_model.encode(img1)
16 z2, _ = vae_model.encode(img2)
17
18 interpolations = []
19 for alpha in torch.linspace(0, 1, 10):
20     z_interp = slerp(z1, z2, alpha)
21     img_interp = vae_model.decode(z_interp)
22     interpolations.append(img_interp)

```

9.7.5. Visualización de Resultados

Comparación Original vs Reconstrucción


```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 def visualize_reconstructions(model, dataloader, n_samples=8,
5                               device='cuda'):
6     """
7     Visualiza originales y reconstrucciones lado a lado
8     """
9     model.eval()
10
11     # Obtener un batch
12     images = next(iter(dataloader))[:n_samples]
13     images = images.to(device)
14
15     # Reconstruir
16     with torch.no_grad():
17         reconstructions = model(images)
18
19     # Preparar para visualizacion
20     images = images.cpu()
21     reconstructions = reconstructions.cpu()
22
23     # Crear figura
24     fig, axes = plt.subplots(2, n_samples, figsize=(15, 4))
25
26     for i in range(n_samples):
27         # Original
28         img_orig = images[i].permute(1, 2, 0).numpy()
29         img_orig = (img_orig - img_orig.min()) / \
30                     (img_orig.max() - img_orig.min())
31         axes[0, i].imshow(img_orig)
32         axes[0, i].axis('off')
33         if i == 0:
34             axes[0, i].set_ylabel('Original', fontsize=12)
35
36         # Reconstruccion
37         img_recon = reconstructions[i].permute(1, 2, 0).numpy()
38         img_recon = (img_recon - img_recon.min()) / \
39                     (img_recon.max() - img_recon.min())
40         axes[1, i].imshow(img_recon)
41         axes[1, i].axis('off')
42         if i == 0:
43             axes[1, i].set_ylabel('Reconstructed', fontsize=12)
44
45     plt.tight_layout()
46     plt.savefig('reconstructions.png', dpi=150)
47     plt.show()
48
```

```
49 # Uso
50 visualize_reconstructions(model, test_loader, n_samples=8)
```

Visualización del Espacio Latente con t-SNE

```
1 from sklearn.manifold import TSNE
2
3 def visualize_latent_space(model, dataloader, labels=None,
4                           device='cuda'):
5     """
6     Visualiza el espacio latente usando t-SNE
7
8     Args:
9         model: Autoencoder entrenado
10        dataloader: DataLoader con los datos
11        labels: Labels opcionales para colorear
12    """
13    model.eval()
14
15    latent_vectors = []
16    all_labels = []
17
18    # Codificar todo el dataset
19    with torch.no_grad():
20        for batch in dataloader:
21            if isinstance(batch, (list, tuple)):
22                images, labs = batch
23                images = images.to(device)
24                z = model.encode(images)
25                latent_vectors.append(z.cpu().numpy())
26                all_labels.extend(labs.numpy())
27            else:
28                images = batch.to(device)
29                z = model.encode(images)
30                latent_vectors.append(z.cpu().numpy())
31
32    latent_vectors = np.concatenate(latent_vectors, axis=0)
33
34    # Aplicar t-SNE
35    print("Aplicando t-SNE...")
36    tsne = TSNE(n_components=2, random_state=42, perplexity=30)
37    latent_2d = tsne.fit_transform(latent_vectors)
38
39    # Graficar
40    plt.figure(figsize=(10, 8))
41
42    if len(all_labels) > 0:
```

```

43     scatter = plt.scatter(
44         latent_2d[:, 0], latent_2d[:, 1],
45         c=all_labels, cmap='tab10', alpha=0.7, s=5
46     )
47     plt.colorbar(scatter, label='Class')
48 else:
49     plt.scatter(
50         latent_2d[:, 0], latent_2d[:, 1],
51         alpha=0.7, s=5
52     )
53
54 plt.title('Latent Space Visualization (t-SNE)', fontsize=14)
55 plt.xlabel('Dimension 1')
56 plt.ylabel('Dimension 2')
57 plt.grid(alpha=0.3)
58 plt.savefig('latent_space_tsne.png', dpi=150)
59 plt.show()
60
61 # Uso
62 visualize_latent_space(model, test_loader)

```

Visualización de Filtros Convolucionales

```

1 def visualize_filters(model, layer_name='enc_conv1',
2                       max_filters=16):
3     """
4     Visualiza los filtros aprendidos de una capa convolucional
5
6     Args:
7         model: Modelo entrenado
8         layer_name: Nombre de la capa a visualizar
9         max_filters: Numero maximo de filtros a mostrar
10    """
11    # Obtener capa
12    layer = getattr(model, layer_name)
13
14    # Extraer pesos
15    if hasattr(layer, 'conv_weights'):
16        filters = layer.conv_weights.data.cpu().numpy()
17    else:
18        filters = layer.weight.data.cpu().numpy()
19
20    # Normalizar
21    filters = (filters - filters.min()) / \
22              (filters.max() - filters.min() + 1e-8)
23
24    n_filters = min(max_filters, filters.shape[0])

```

```

25     n_cols = 4
26     n_rows = (n_filters + n_cols - 1) // n_cols
27
28     fig, axes = plt.subplots(n_rows, n_cols, figsize=(12, 3*n_rows))
29     axes = axes.flatten()
30
31     for i in range(n_filters):
32         # Mostrar primer canal del filtro
33         filt = filters[i, 0] if filters.shape[1] > 0 else filters[i]
34
35         axes[i].imshow(filt, cmap='viridis')
36         axes[i].set_title(f'Filter {i}', fontsize=10)
37         axes[i].axis('off')
38
39     # Ocultar axes sobrantes
40     for i in range(n_filters, len(axes)):
41         axes[i].axis('off')
42
43     plt.suptitle(f'Learned Filters - {layer_name}', fontsize=14)
44     plt.tight_layout()
45     plt.savefig(f'filters_{layer_name}.png', dpi=150)
46     plt.show()
47
48 # Uso
49 visualize_filters(model, 'enc_conv1', max_filters=16)

```

9.8. Transfer Learning con Encoder Pre-entrenado

El encoder entrenado puede reutilizarse como extractor de características para tareas supervisadas.

9.8.1. Clasificador con Encoder Congelado

```

1 class KANEncoderClassifier(nn.Module):
2     """
3     Clasificador usando encoder convolucional KAN pre-entrenado
4     """
5     def __init__(self, pretrained_autoencoder, num_classes,
6                 freeze_encoder=True):
7         super().__init__()
8
9         # Copiar encoder
10        self.encoder = pretrained_autoencoder.encoder
11
12        # Congelar encoder si se especifica
13        if freeze_encoder:

```

```
14         for param in self.encoder.parameters():
15             param.requires_grad = False
16
17         # Obtener dimension latente
18         latent_dim = pretrained_autoencoder.latent_dim
19
20         # Cabecera de clasificacion
21         self.classifier = nn.Sequential(
22             nn.Linear(latent_dim, 256),
23             nn.ReLU(),
24             nn.Dropout(0.5),
25             nn.Linear(256, 128),
26             nn.ReLU(),
27             nn.Dropout(0.3),
28             nn.Linear(128, num_classes)
29         )
30
31     def forward(self, x):
32         # Extraer features con encoder
33         if not self.training or not any(
34             p.requires_grad for p in self.encoder.parameters()
35         ):
36             with torch.no_grad():
37                 z = self.encoder(x)
38         else:
39             z = self.encoder(x)
40
41         # Clasificar
42         logits = self.classifier(z)
43         return logits
44
45     # Crear clasificador
46     autoencoder = torch.load('autoencoder_trained.pth')
47     classifier = KANEncoderClassifier(
48         pretrained_autoencoder=autoencoder,
49         num_classes=10,
50         freeze_encoder=True
51     )
52
53     # Entrenar solo el clasificador
54     optimizer = optim.Adam(
55         classifier.classifier.parameters(),
56         lr=1e-3
57     )
```

9.8.2. Fine-tuning Gradual

Estrategia de fine-tuning en dos fases:

```

1 def train_with_finetuning(classifier, train_loader, val_loader,
2                           num_classes, device='cuda'):
3     """
4     Entrena en dos fases: clasificador y fine-tuning completo
5     """
6     classifier = classifier.to(device)
7     criterion = nn.CrossEntropyLoss()
8
9     # ===== FASE 1: Entrenar solo clasificador =====
10    print("Fase 1: Entrenando clasificador...")
11
12    # Congelar encoder
13    for param in classifier.encoder.parameters():
14        param.requires_grad = False
15
16    optimizer_phase1 = optim.Adam(
17        classifier.classifier.parameters(),
18        lr=1e-3
19    )
20
21    for epoch in range(20):
22        classifier.train()
23        train_loss = 0.0
24        correct = 0
25        total = 0
26
27        for images, labels in train_loader:
28            images, labels = images.to(device), labels.to(device)
29
30            optimizer_phase1.zero_grad()
31            outputs = classifier(images)
32            loss = criterion(outputs, labels)
33            loss.backward()
34            optimizer_phase1.step()
35
36            train_loss += loss.item()
37            _, predicted = outputs.max(1)
38            total += labels.size(0)
39            correct += predicted.eq(labels).sum().item()
40
41        acc = 100. * correct / total
42        print(f"Epoch {epoch+1} - Loss:
43              {train_loss/len(train_loader):.4f} - Acc: {acc:.2f}%")
44
45    # ===== FASE 2: Fine-tune todo el modelo =====

```

```
45 print("\nFase 2: Fine-tuning del modelo completo...")
46
47 # Descongelar encoder
48 for param in classifier.encoder.parameters():
49     param.requires_grad = True
50
51 optimizer_phase2 = optim.Adam(
52     classifier.parameters(),
53     lr=1e-4 # Learning rate menor
54 )
55 scheduler = optim.lr_scheduler.StepLR(
56     optimizer_phase2, step_size=10, gamma=0.5
57 )
58
59 best_acc = 0.0
60
61 for epoch in range(30):
62     classifier.train()
63     train_loss = 0.0
64     correct = 0
65     total = 0
66
67     for images, labels in train_loader:
68         images, labels = images.to(device), labels.to(device)
69
70         optimizer_phase2.zero_grad()
71         outputs = classifier(images)
72         loss = criterion(outputs, labels)
73         loss.backward()
74         optimizer_phase2.step()
75
76         train_loss += loss.item()
77         _, predicted = outputs.max(1)
78         total += labels.size(0)
79         correct += predicted.eq(labels).sum().item()
80
81     train_acc = 100. * correct / total
82
83     # Validacion
84     classifier.eval()
85     val_correct = 0
86     val_total = 0
87
88     with torch.no_grad():
89         for images, labels in val_loader:
90             images, labels = images.to(device), labels.to(device)
91             outputs = classifier(images)
92             _, predicted = outputs.max(1)
```

```

93         val_total += labels.size(0)
94         val_correct += predicted.eq(labels).sum().item()
95
96     val_acc = 100. * val_correct / val_total
97
98     print(f"Epoch {epoch+1} - Train Acc: {train_acc:.2f}% - Val
99           Acc: {val_acc:.2f}%")
100
101     # Guardar mejor modelo
102     if val_acc > best_acc:
103         best_acc = val_acc
104         torch.save(classifier.state_dict(),
105                   'best_classifier.pth')
106
107     scheduler.step()
108
109     print(f"\nMejor accuracy de validacion: {best_acc:.2f}%")
110     return classifier
111
112 # Uso
113 trained_classifier = train_with_finetuning(
114     classifier, train_loader, val_loader, num_classes=10
115 )

```

Comparación con Entrenamiento desde Cero

```

1 def compare_transfer_vs_scratch(pretrained_ae, train_loader,
2                                 val_loader, num_classes):
3     """
4     Compara transfer learning vs entrenamiento desde cero
5     """
6     # Modelo con transfer learning
7     model_transfer = KANEncoderClassifier(
8         pretrained_autoencoder=pretrained_ae,
9         num_classes=num_classes,
10        freeze_encoder=False
11    )
12
13    # Modelo desde cero (misma arquitectura)
14    model_scratch = KANEncoderClassifier(
15        pretrained_autoencoder=pretrained_ae,
16        num_classes=num_classes,
17        freeze_encoder=False
18    )
19
20    # Reinicializar pesos del encoder
21    for module in model_scratch.encoder.modules():
22        if hasattr(module, 'reset_parameters'):

```



```
22         module.reset_parameters()
23
24     # Entrenar ambos
25     print("Entrenando con transfer learning...")
26     train_with_finetuning(model_transfer, train_loader,
27                           val_loader, num_classes)
28
29     print("\nEntrenando desde cero...")
30     train_with_finetuning(model_scratch, train_loader,
31                           val_loader, num_classes)
32
33     # Comparar resultados...
```

Este enfoque de transfer learning es particularmente útil cuando:

- Los datos etiquetados son escasos
- Se desea aprovechar características aprendidas en un dominio relacionado
- Se busca acelerar la convergencia del entrenamiento

Capítulo 10 Beta-VAE basado en DualFlexKAN

Esta sección describe la implementación y uso de Beta-Variational Autoencoders (β -VAE) utilizando la arquitectura DualFlexKAN. El β -VAE es una extensión del autoencoder variacional estándar que introduce un hiperparámetro β para controlar el balance entre la calidad de reconstrucción y la regularización del espacio latente, permitiendo aprender representaciones más disentangled (desenredadas).

10.1. Arquitectura General

El sistema de β -VAE está compuesto por tres clases principales que implementan la arquitectura variacional:

- **DualFlexKANVariationalEncoder**: Encoder variacional que genera parámetros de una distribución gaussiana (μ y $\log \sigma^2$) en lugar de una codificación determinista.
- **DualFlexKANDecoder**: Decoder que reconstruye la entrada desde muestras del espacio latente.
- **DualFlexKANBetaVAE**: Combina encoder variacional y decoder en un β -VAE completo con arquitecturas configurables y parámetro β ajustable.

10.1.1. Diferencias con Autoencoders Deterministas

El β -VAE difiere del autoencoder determinista en aspectos fundamentales:

1. **Codificación probabilística**: En lugar de mapear directamente $x \rightarrow z$, el encoder genera parámetros de distribución $x \rightarrow (\mu, \log \sigma^2)$ desde los cuales se muestrea z .
2. **Truco de reparametrización**: Para permitir el backpropagation a través del muestreo estocástico:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (10.1)$$

3. **Función de pérdida con término KL**: La pérdida total combina reconstrucción y divergencia KL:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \beta \cdot \mathcal{L}_{\text{KL}} \quad (10.2)$$

donde

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum_{j=1}^J (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2) \quad (10.3)$$

4. **Parámetro β :** Controla el peso de la regularización KL:

- $\beta = 0$: Autoencoder determinista (sin regularización).
- $\beta = 1$: VAE estándar (balance teórico óptimo).
- $\beta > 1$: Énfasis en disentanglement del espacio latente.
- $\beta < 1$: Énfasis en calidad de reconstrucción.

10.2. Construcción del Encoder

La clase `DualFlexKANVariationalEncoder` implementa un encoder variacional que produce parámetros de distribución en lugar de codificaciones deterministas.

10.2.1. Parámetros del Constructor

```

1 encoder = DualFlexKANVariationalEncoder(
2     layer_sizes=[input_size, hidden1, hidden2, ..., latent_size],
3     **kwargs    # Parámetros de DualFlexKAN
4 )

```

Listing 10.1: Constructor del encoder variacional

Parámetros principales:

- **layer_sizes:** Lista con las dimensiones de cada capa. La última dimensión define el tamaño del espacio latente. Ejemplo: `[784, 512, 256, 32]` crea un encoder de 3 capas DualFlexKAN más una capa final lineal que produce μ y $\log \sigma^2$.
- ****kwargs:** Parámetros configurables de DualFlexKAN que se aplican a todas las capas excepto la última:
 - **input_activation_types:** Tipos de activación de entrada por capa.
 - **output_activation_types:** Tipos de activación de salida por capa.
 - **dropout_probs:** Probabilidades de dropout.
 - **use_batch_norms:** Aplicación de batch normalization.

Arquitectura interna:

- La red principal (DualFlexKAN) procesa hasta la penúltima capa especificada.
- Dos capas lineales independientes (**to_mu** y **to_logvar**) proyectan desde la penúltima capa al espacio latente.
- Inicialización Xavier para las capas lineales finales.

10.2.2. Ejemplo de Uso Básico

```

1 import torch
2 from DFKAN_betaVAE import DualFlexKANVariationalEncoder
3
4 # Encoder variacional con configuración por defecto
5 encoder = DualFlexKANVariationalEncoder(
6     layer_sizes=[784, 512, 256, 32]
7 )
8
9 # Codificar una entrada (retorna mu y logvar)
10 x = torch.randn(64, 784) # Batch de 64 imágenes
11 mu, logvar = encoder(x)
12
13 print(f"Media (mu): {mu.shape}") # (64, 32)
14 print(f"Log-varianza: {logvar.shape}") # (64, 32)
15
16 # Información de arquitectura
17 encoder.print_architecture_info()

```

Listing 10.2: Creación de encoder variacional simple

10.2.3. Ejemplo con Configuración Avanzada

```

1 # Configuración avanzada con diferentes tipos de activación
2 encoder_advanced = DualFlexKANVariationalEncoder(
3     layer_sizes=[784, 512, 256, 128, 32],
4     input_activation_types=['polynomial', 'fourier', 'polynomial'],
5     input_activation_kwargs=[
6         {'degree': 3},
7         {'num_frequencies': 5},
8         {'degree': 4}
9     ],
10    output_activation_types=['polynomial', 'polynomial',
11        'polynomial'],
12    output_activation_kwargs=[
13        {'degree': 3},
14        {'degree': 3},
15        {'degree': 2}
16    ],
17    dropout_probs=[0.2, 0.3, 0.2],
18    use_batch_norms=[True, True, False],
19    batch_norm_positions=['after_activation', 'after_activation',
20        None]
21 )
22
23 # Forward pass

```

```

22 x = torch.randn(32, 784)
23 mu, logvar = encoder_advanced(x)
24 print(f"Salida: mu={mu.shape}, logvar={logvar.shape}")

```

Listing 10.3: Encoder variacional con configuración personalizada

10.3. Construcción del Decoder

El decoder del β -VAE es idéntico al del autoencoder determinista, utilizando la clase `DualFlexKANDecoder`.

10.3.1. Parámetros del Constructor

```

1 decoder = DualFlexKANDecoder(
2     layer_sizes=[latent_size, hidden1, hidden2, ..., output_size],
3     **kwargs # Parámetros de DualFlexKAN
4 )

```

Listing 10.4: Constructor del decoder

El decoder reconstruye desde el espacio latente hasta las dimensiones originales (o dimensiones de salida especificadas).

Ejemplo de Uso

```

1 from DFKAN_betaVAE import DualFlexKANDecoder
2
3 # Decoder simétrico al encoder
4 decoder = DualFlexKANDecoder(
5     layer_sizes=[32, 128, 256, 512, 784]
6 )
7
8 # Decodificar desde el espacio latente
9 z = torch.randn(64, 32) # Códigos latentes muestreados
10 x_reconstructed = decoder(z)
11 print(f"Reconstrucción: {x_reconstructed.shape}") # (64, 784)
12
13 # Información de arquitectura
14 decoder.print_architecture_info()

```

Listing 10.5: Creación y uso del decoder

10.3.2. Autoencoder Completo

La clase `DualFlexKANBetaVAE` integra el encoder variacional y el decoder en un β -VAE completo con control sobre el parámetro β .

Parámetros del Constructor

```
1 beta_vae = DualFlexKANBetaVAE(  
2     input_size=784,  
3     latent_size=32,  
4     output_size=None, # None para igual a input_size  
5     encoder_hidden_sizes=[512, 256, 128],  
6     decoder_hidden_sizes=[128, 256, 512],  
7     encoder_config={...},  
8     decoder_config={...},  
9     beta=1.0,  
10    reconstruction_loss="mse"  
11 )
```

Listing 10.6: Constructor del Beta-VAE completo

Parámetros:

- `input_size`: Dimensión de entrada (e.g., 784 para MNIST 28×28).
- `latent_size`: Dimensión del espacio latente.
- `output_size`: Dimensión de salida. Si es `None`, se usa `input_size`.
- `encoder_hidden_sizes`: Lista de capas ocultas del encoder. Default: [64, 32].
- `decoder_hidden_sizes`: Lista de capas ocultas del decoder. Default: [32, 64].
- `encoder_config`: Diccionario con configuración del encoder.
- `decoder_config`: Diccionario con configuración del decoder.
- `beta`: Parámetro β para ponderación del término KL. Default: 1.0.
- `reconstruction_loss`: Tipo de pérdida de reconstrucción:
 - "mse": Mean Squared Error (recomendado para imágenes continuas).
 - "bce": Binary Cross-Entropy (recomendado para imágenes binarias).

Ejemplo: Autoencoder Simétrico

```
1 from DFKAN_betaVAE import DualFlexKANBetaVAE  
2  
3 # Beta-VAE estándar (beta=1.0)  
4 beta_vae = DualFlexKANBetaVAE(  
5     input_size=784,  
6     latent_size=32,  
7     encoder_hidden_sizes=[512, 256, 128],  
8     decoder_hidden_sizes=[128, 256, 512],  
9     beta=1.0,
```

```

10     reconstruction_loss="mse"
11 )
12
13 # Forward pass completo
14 x = torch.randn(64, 784)
15 reconstructed, mu, logvar, z = beta_vae(x)
16
17 print(f"Entrada: {x.shape}")
18 print(f"Mu: {mu.shape}")
19 print(f"Logvar: {logvar.shape}")
20 print(f"Latente (z muestreado): {z.shape}")
21 print(f"Reconstrucción: {reconstructed.shape}")
22
23 # Calcular pérdida
24 losses = beta_vae.loss_function(x, reconstructed, mu, logvar)
25 print(f"\nPérdida total: {losses['total_loss'].item():.4f}")
26 print(f"Pérdida reconstrucción: {losses['recon_loss'].item():.4f}")
27 print(f"Pérdida KL: {losses['kl_loss'].item():.4f}")
28
29 # Información de arquitectura
30 beta_vae.print_architecture_info()

```

Listing 10.7: β -VAE con arquitectura simétrica estándar

Ejemplo: Autoencoder Asimétrico

```

1 # Encoder profundo, decoder superficial con beta elevado
2 beta_vae_disentangled = DualFlexKANBetaVAE(
3     input_size=784,
4     latent_size=16,
5     encoder_hidden_sizes=[512, 256, 128, 64, 32], # 5 capas ocultas
6     decoder_hidden_sizes=[64, 256], # 2 capas ocultas
7     beta=4.0, # Mayor énfasis en disentanglement
8     reconstruction_loss="mse"
9 )
10
11 print(f"Beta utilizado: {beta_vae_disentangled.beta}")
12 beta_vae_disentangled.print_architecture_info()

```

Listing 10.8: β -VAE con arquitecturas asimétricas

Ejemplo: Configuración Avanzada con Diccionarios

```

1 # Configuración detallada del encoder
2 encoder_config = {
3     'input_activation_types': ['polynomial', 'fourier',
4                               'polynomial'],

```



```

4     'input_activation_kwargs': [
5         {'degree': 3},
6         {'num_frequencies': 5},
7         {'degree': 4}
8     ],
9     'output_activation_types': ['relu', 'relu', 'relu'],
10    'output_function_strategies': ['fixed', 'fixed', 'fixed'],
11    'dropout_probs': [0.2, 0.3, 0.2],
12    'use_batch_norms': [True, True, False],
13    'batch_norm_positions': ['after_activation', 'after_activation',
14        None]
15 }
16 # Configuración del decoder con activación sigmoid al final
17 decoder_config = {
18     'output_activation_types': ['relu', 'relu', 'sigmoid'],
19     'output_function_strategies': ['fixed', 'fixed', 'fixed'],
20     'dropout_probs': [0.1, 0.15, 0.0],
21     'use_batch_norms': [False, True, False]
22 }
23
24 # Crear Beta-VAE con alta regularización para disentanglement
25 beta_vae_advanced = DualFlexKANBetaVAE(
26     input_size=784,
27     latent_size=32,
28     encoder_hidden_sizes=[512, 256, 128],
29     decoder_hidden_sizes=[128, 256, 512],
30     encoder_config=encoder_config,
31     decoder_config=decoder_config,
32     beta=6.0, # Beta alto para mayor disentanglement
33     reconstruction_loss="mse"
34 )
35
36 # Visualizar configuración
37 print(f"Input size: {beta_vae_advanced.input_size}")
38 print(f"Latent size: {beta_vae_advanced.latent_size}")
39 print(f"Beta: {beta_vae_advanced.beta}")
40 print(f"Reconstruction loss:
41     {beta_vae_advanced.reconstruction_loss}")
42 beta_vae_advanced.print_architecture_info()

```

Listing 10.9: Beta-VAE con configuración completa

10.4. Entrenamiento del Autoencoder

10.4.1. Función `train_beta_vae`

La función `train_beta_vae` implementa el bucle de entrenamiento completo con monitoreo detallado de las componentes de pérdida y soporte para beta annealing.

Firma de la función:

```

1 history = train_beta_vae(
2     model=beta_vae,
3     train_loader=train_loader,
4     test_loader=test_loader,
5     epochs=100,
6     lr=0.001,
7     device='cpu',
8     log_interval=10,
9     beta_scheduler=None # Opcional: BetaScheduler para annealing
10 )

```

Listing 10.10: Función de entrenamiento del Beta-VAE

Parámetros:

- `model`: Instancia del Beta-VAE a entrenar.
- `train_loader`: `DataLoader` con datos de entrenamiento.
- `test_loader`: `DataLoader` con datos de validación/test.
- `epochs`: Número de épocas (default: 100).
- `lr`: Tasa de aprendizaje para Adam (default: 0.001).
- `device`: Dispositivo de cómputo ('cpu' o 'cuda').
- `log_interval`: Frecuencia de logging en épocas (default: 10).
- `beta_scheduler`: Instancia de `BetaScheduler` para annealing (default: None).

Retorna: Diccionario con historial de entrenamiento:

```

1 {
2     'train_losses': [],          # Pérdida total de entrenamiento
3     'test_losses': [],          # Pérdida total de test
4     'train_recon_losses': [],   # Pérdida de reconstrucción (train)
5     'train_kl_losses': [],      # Pérdida KL (train)
6     'test_recon_losses': [],    # Pérdida de reconstrucción (test)
7     'test_kl_losses': [],       # Pérdida KL (test)
8     'beta_values': []          # Valores de beta por época (si usa
9                                 scheduler)
10 }

```

Ejemplo completo de entrenamiento:

```
1 import torch
2 from torch.utils.data import DataLoader
3 from torchvision import datasets, transforms
4 from DFKAN_betaVAE import DualFlexKANBetaVAE, train_beta_vae
5 import matplotlib.pyplot as plt
6
7 # 1. Preparar datos MNIST
8 transform = transforms.Compose([
9     transforms.ToTensor(),
10    transforms.Lambda(lambda x: x.view(-1)) # Aplanar a vector
11 ])
12
13 train_dataset = datasets.MNIST(
14     root='./data',
15     train=True,
16     download=True,
17     transform=transform
18 )
19
20 test_dataset = datasets.MNIST(
21     root='./data',
22     train=False,
23     download=True,
24     transform=transform
25 )
26
27 train_loader = DataLoader(
28     train_dataset,
29     batch_size=128,
30     shuffle=True
31 )
32
33 test_loader = DataLoader(
34     test_dataset,
35     batch_size=128,
36     shuffle=False
37 )
38
39 # 2. Crear Beta-VAE
40 beta_vae = DualFlexKANBetaVAE(
41     input_size=784,
42     latent_size=32,
43     encoder_hidden_sizes=[512, 256, 128],
44     decoder_hidden_sizes=[128, 256, 512],
45     beta=2.0, # Beta-VAE con énfasis en disentanglement
46     reconstruction_loss="mse"
```

```
47 )
48
49 # 3. Seleccionar dispositivo
50 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
51 print(f"Entrenando en: {device}")
52
53 # 4. Entrenar
54 history = train_beta_vae(
55     model=beta_vae,
56     train_loader=train_loader,
57     test_loader=test_loader,
58     epochs=100,
59     lr=0.001,
60     device=device,
61     log_interval=10
62 )
63
64 # 5. Visualizar curvas de aprendizaje
65 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
66
67 # Pérdida total
68 axes[0].plot(history['train_losses'], label='Train', linewidth=2)
69 axes[0].plot(history['test_losses'], label='Test', linewidth=2)
70 axes[0].set_xlabel('Época')
71 axes[0].set_ylabel('Pérdida Total')
72 axes[0].set_title('Pérdida Total')
73 axes[0].legend()
74 axes[0].grid(True, alpha=0.3)
75
76 # Pérdida de reconstrucción
77 axes[1].plot(history['train_recon_losses'], label='Train',
78             linewidth=2)
79 axes[1].plot(history['test_recon_losses'], label='Test', linewidth=2)
80 axes[1].set_xlabel('Época')
81 axes[1].set_ylabel('Pérdida Reconstrucción')
82 axes[1].set_title('Pérdida de Reconstrucción')
83 axes[1].legend()
84 axes[1].grid(True, alpha=0.3)
85
86 # Pérdida KL
87 axes[2].plot(history['train_kl_losses'], label='Train', linewidth=2)
88 axes[2].plot(history['test_kl_losses'], label='Test', linewidth=2)
89 axes[2].set_xlabel('Época')
90 axes[2].set_ylabel('Pérdida KL')
91 axes[2].set_title('Divergencia KL')
92 axes[2].legend()
93 axes[2].grid(True, alpha=0.3)
```

```

94 plt.tight_layout()
95 plt.savefig('beta_vae_training.png', dpi=300, bbox_inches='tight')
96 plt.show()
97
98 # 6. Guardar modelo entrenado
99 torch.save({
100     'model_state_dict': beta_vae.state_dict(),
101     'history': history,
102     'config': {
103         'input_size': 784,
104         'latent_size': 32,
105         'beta': 2.0
106     }
107 }, 'beta_vae_trained.pth')
108
109 print("Entrenamiento completado y modelo guardado.")

```

Listing 10.11: Pipeline completo de entrenamiento con MNIST

10.4.2. Beta Annealing

El **beta annealing** es una técnica crucial para entrenar Beta-VAEs de forma efectiva. Consiste en variar el parámetro β durante el entrenamiento en lugar de mantenerlo constante. Esta estrategia resuelve varios problemas comunes:

1. **Posterior collapse**: Evita que el modelo ignore completamente el espacio latente al inicio del entrenamiento.
2. **Convergencia más estable**: Permite que el modelo aprenda primero buenas reconstrucciones antes de imponer regularización fuerte.
3. **Mejor disentanglement**: Aumentar β gradualmente facilita la separación de factores latentes.

La Clase BetaScheduler

La clase BetaScheduler implementa múltiples estrategias de annealing:

```

1 from DFKAN_betaVAE import BetaScheduler
2
3 scheduler = BetaScheduler(
4     strategy='linear',           # Tipo de scheduling
5     beta_start=0.0,             # Valor inicial de beta
6     beta_end=4.0,               # Valor final de beta
7     n_epochs=100,               # Número total de épocas
8     n_cycles=4,                 # Para estrategia 'cyclical'
9     ratio=0.5,                  # Para estrategias 'cyclical' y 'step'
10 )

```

Listing 10.12: Creación de un BetaScheduler

Estrategias disponibles:

1. **constant**: Beta permanece constante en `beta_end`.

$$\beta(t) = \beta_{\text{end}} \quad (10.4)$$

2. **linear**: Incremento lineal desde `beta_start` hasta `beta_end`.

$$\beta(t) = \beta_{\text{start}} + (\beta_{\text{end}} - \beta_{\text{start}}) \cdot \frac{t}{T} \quad (10.5)$$

donde t es la época actual y T el número total de épocas.

3. **sigmoid**: Incremento suave con forma sigmoide (cambio más pronunciado en el medio).

$$\beta(t) = \beta_{\text{start}} + (\beta_{\text{end}} - \beta_{\text{start}}) \cdot \sigma(12 \cdot (\frac{t}{T} - 0,5)) \quad (10.6)$$

donde $\sigma(x) = \frac{1}{1+e^{-x}}$.

4. **cosine**: Incremento con forma de coseno (arranque y final suaves).

$$\beta(t) = \beta_{\text{start}} + (\beta_{\text{end}} - \beta_{\text{start}}) \cdot \frac{1 - \cos(\pi \frac{t}{T})}{2} \quad (10.7)$$

5. **cyclical**: Beta oscila cíclicamente (inspirado en Fu et al., 2019).

$$\beta(t) = \begin{cases} \beta_{\text{start}} + (\beta_{\text{end}} - \beta_{\text{start}}) \cdot \frac{t \bmod L}{r \cdot L} & \text{si } (t \bmod L) < r \cdot L \\ \beta_{\text{end}} & \text{en caso contrario} \end{cases} \quad (10.8)$$

donde L es la longitud del ciclo y r el ratio de crecimiento.

6. **step**: Beta aumenta en escalones discretos (4 escalones).

$$\beta(t) = \beta_{\text{start}} + (\beta_{\text{end}} - \beta_{\text{start}}) \cdot \frac{\lfloor 4t/T \rfloor}{3} \quad (10.9)$$

Ejemplos de Uso**Ejemplo 1: Annealing Lineal**

```

1 from DFKAN_betaVAE import BetaScheduler, train_beta_vae
2
3 # Crear scheduler lineal: beta va de 0 a 4.0 en 100 épocas
4 scheduler = BetaScheduler(
5     strategy='linear',
6     beta_start=0.0,
7     beta_end=4.0,
8     n_epochs=100
9 )
10
```

```

11 # Visualizar el schedule antes de entrenar
12 scheduler.plot_schedule(save_path='beta_schedule_linear.png')
13
14 # Entrenar con annealing
15 history = train_beta_vae(
16     model=beta_vae,
17     train_loader=train_loader,
18     test_loader=test_loader,
19     epochs=100,
20     lr=0.001,
21     device=device,
22     beta_scheduler=scheduler # Pasar el scheduler
23 )
24
25 # Visualizar evolución de beta durante entrenamiento
26 plt.figure(figsize=(10, 5))
27 plt.plot(history['beta_values'], linewidth=2)
28 plt.xlabel('Época')
29 plt.ylabel('Beta')
30 plt.title('Evolución de Beta durante el Entrenamiento')
31 plt.grid(True, alpha=0.3)
32 plt.show()

```

Listing 10.13: Entrenamiento con annealing lineal

Ejemplo 2: Cyclical Annealing (Recomendado)

El annealing cíclico es especialmente efectivo para evitar posterior collapse:

```

1 # Crear scheduler cíclico: 4 ciclos en 100 épocas
2 scheduler_cyclical = BetaScheduler(
3     strategy='cyclical',
4     beta_start=0.0,
5     beta_end=4.0,
6     n_epochs=100,
7     n_cycles=4,      # 4 ciclos completos
8     ratio=0.5        # 50% del ciclo es fase de aumento
9 )
10
11 # Visualizar
12 scheduler_cyclical.plot_schedule()
13
14 # Entrenar
15 history = train_beta_vae(
16     model=beta_vae,
17     train_loader=train_loader,
18     test_loader=test_loader,
19     epochs=100,
20     lr=0.001,
21     device=device,
22     beta_scheduler=scheduler_cyclical

```

23)

Listing 10.14: Entrenamiento con annealing cíclico

Ejemplo 3: Cosine Annealing (Suave)

Para transiciones muy suaves:

```
1 scheduler_cosine = BetaScheduler(  
2     strategy='cosine',  
3     beta_start=0.0,  
4     beta_end=6.0, # Beta alto para fuerte disentanglement  
5     n_epochs=100  
6 )  
7  
8 scheduler_cosine.plot_schedule()  
9  
10 history = train_beta_vae(  
11     model=beta_vae,  
12     train_loader=train_loader,  
13     test_loader=test_loader,  
14     epochs=100,  
15     lr=0.001,  
16     device=device,  
17     beta_scheduler=scheduler_cosine  
18 )
```

Listing 10.15: Entrenamiento con annealing coseno

Comparación de Estrategias

```
1 import matplotlib.pyplot as plt  
2  
3 # Crear todos los schedulers  
4 strategies = ['linear', 'sigmoid', 'cosine', 'cyclical', 'step']  
5 epochs = 100  
6  
7 plt.figure(figsize=(15, 8))  
8  
9 for i, strategy in enumerate(strategies):  
10     scheduler = BetaScheduler(  
11         strategy=strategy,  
12         beta_start=0.0,  
13         beta_end=4.0,  
14         n_epochs=epochs,  
15         n_cycles=4,  
16         ratio=0.5  
17     )  
18  
19     # Obtener valores de beta
```



```

20     beta_vals = [scheduler.get_beta(e) for e in range(epochs)]
21
22     plt.subplot(2, 3, i+1)
23     plt.plot(beta_vals, linewidth=2)
24     plt.xlabel('Época')
25     plt.ylabel('Beta')
26     plt.title(f'Strategy: {strategy}')
27     plt.grid(True, alpha=0.3)
28     plt.ylim(-0.5, 4.5)
29
30 plt.tight_layout()
31 plt.savefig('beta_strategies_comparison.png', dpi=300)
32 plt.show()

```

Listing 10.16: Comparación visual de todas las estrategias

Guía de Selección de Estrategia

Estrategia	Mejor para	Descripción
linear	Uso general	Simple y efectiva, buen punto de partida
sigmoid	Convergencia rápida	Cambio pronunciado en épocas medias
cosine	Estabilidad	Transiciones muy suaves, menos oscilaciones
cyclical	Evitar collapse	Mejor para datasets complejos, permite reiniciar"
step	Control discreto	Útil para debugging y análisis por fases

Tabla 10.1: Guía de selección de estrategia de beta annealing

Visualización Completa del Entrenamiento con Annealing

```

1  # Después del entrenamiento con scheduler
2  fig, axes = plt.subplots(2, 2, figsize=(14, 10))
3
4  # 1. Pérdida total
5  axes[0, 0].plot(history['train_losses'], label='Train', linewidth=2)
6  axes[0, 0].plot(history['test_losses'], label='Test', linewidth=2)
7  axes[0, 0].set_xlabel('Época')
8  axes[0, 0].set_ylabel('Pérdida Total')
9  axes[0, 0].set_title('Pérdida Total')
10 axes[0, 0].legend()
11 axes[0, 0].grid(True, alpha=0.3)

```

```

12
13 # 2. Pérdida de reconstrucción
14 axes[0, 1].plot(history['train_recon_losses'], label='Train',
15                 linewidth=2)
16 axes[0, 1].plot(history['test_recon_losses'], label='Test',
17                 linewidth=2)
18 axes[0, 1].set_xlabel('Época')
19 axes[0, 1].set_ylabel('Pérdida Reconstrucción')
20 axes[0, 1].set_title('Pérdida de Reconstrucción')
21 axes[0, 1].legend()
22 axes[0, 1].grid(True, alpha=0.3)
23
24 # 3. Pérdida KL
25 axes[1, 0].plot(history['train_kl_losses'], label='Train',
26                 linewidth=2)
27 axes[1, 0].plot(history['test_kl_losses'], label='Test', linewidth=2)
28 axes[1, 0].set_xlabel('Época')
29 axes[1, 0].set_ylabel('Pérdida KL')
30 axes[1, 0].set_title('Divergencia KL')
31 axes[1, 0].legend()
32 axes[1, 0].grid(True, alpha=0.3)
33
34 # 4. Evolución de Beta
35 axes[1, 1].plot(history['beta_values'], linewidth=2, color='red')
36 axes[1, 1].set_xlabel('Época')
37 axes[1, 1].set_ylabel('Beta')
38 axes[1, 1].set_title('Evolución de Beta (Annealing)')
39 axes[1, 1].grid(True, alpha=0.3)
40
41 plt.tight_layout()
42 plt.savefig('training_with_annealing_complete.png', dpi=300)
43 plt.show()

```

Listing 10.17: Análisis completo post-entrenamiento

Recomendaciones la configuración de beta annealing

1. Valores iniciales:

- Comenzar siempre con $\beta = 0$ para permitir que el decoder aprenda a reconstruir.
- Primera fase (20-30% épocas): $\beta \approx 0$ para evitar posterior collapse.

2. Valores finales:

- Para VAE estándar: $\beta_{\text{end}} = 1,0$
- Para disentanglement: $\beta_{\text{end}} \in [4,0, 10,0]$
- Para reconstrucción de alta calidad: $\beta_{\text{end}} \in [0,5, 2,0]$

3. Número de épocas:

- Annealing lineal/sigmoid: mínimo 50 épocas.
- Annealing cíclico: mínimo 100 épocas para 4 ciclos.
- Datasets complejos: 200+ épocas con annealing gradual.

4. Estrategia recomendada por caso:

- **Datasets simples** (MNIST, Fashion-MNIST): linear o cosine.
- **Datasets complejos** (CelebA, ImageNet): cyclical.
- **Disentanglement extremo**: sigmoid con $\beta_{\text{end}} > 6$.

5. Monitoreo:

- Vigilar que KL loss no colapse a 0 (posterior collapse).
- Si KL loss es muy alta al inicio: reducir learning rate.
- Si reconstrucciones son malas: aumentar duración del annealing.

Ejemplo Completo: Pipeline con Annealing Óptimo

```

1 import torch
2 from torch.utils.data import DataLoader
3 from torchvision import datasets, transforms
4 from DFKAN_betaVAE import (
5     DualFlexKANBetaVAE,
6     train_beta_vae,
7     BetaScheduler,
8     visualize_reconstructions
9 )
10
11 # 1. Configuración y datos
12 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
13
14 transform = transforms.Compose([
15     transforms.ToTensor(),
16     transforms.Lambda(lambda x: x.view(-1))
17 ])
18
19 train_dataset = datasets.MNIST('./data', train=True, download=True,
20     transform=transform)
21 test_dataset = datasets.MNIST('./data', train=False, download=True,
22     transform=transform)
23
24 train_loader = DataLoader(train_dataset, batch_size=128,
25     shuffle=True)
26 test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)

```

```
24
25 # 2. Crear modelo
26 beta_vae = DualFlexKANBetaVAE(
27     input_size=784,
28     latent_size=32,
29     encoder_hidden_sizes=[512, 256, 128],
30     decoder_hidden_sizes=[128, 256, 512],
31     beta=1.0, # Este valor será sobreescrito por el scheduler
32     reconstruction_loss="mse"
33 )
34
35 # 3. Crear scheduler (ciclico para mejor rendimiento)
36 scheduler = BetaScheduler(
37     strategy='cyclical',
38     beta_start=0.0,
39     beta_end=4.0,
40     n_epochs=150,
41     n_cycles=5,
42     ratio=0.5
43 )
44
45 # Visualizar schedule
46 scheduler.plot_schedule()
47
48 # 4. Entrenar
49 print("Iniciando entrenamiento con beta annealing...")
50 history = train_beta_vae(
51     model=beta_vae,
52     train_loader=train_loader,
53     test_loader=test_loader,
54     epochs=150,
55     lr=0.001,
56     device=device,
57     log_interval=15,
58     beta_scheduler=scheduler
59 )
60
61 # 5. Visualización completa
62 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
63
64 axes[0, 0].plot(history['train_losses'], 'b-', label='Train',
65                 linewidth=2)
66 axes[0, 0].plot(history['test_losses'], 'r-', label='Test',
67                 linewidth=2)
68 axes[0, 0].set_title('Pérdida Total')
69 axes[0, 0].legend()
70 axes[0, 0].grid(True, alpha=0.3)
```

```

70 axes[0, 1].plot(history['train_recon_losses'], 'b-', label='Train',
    linewidth=2)
71 axes[0, 1].plot(history['test_recon_losses'], 'r-', label='Test',
    linewidth=2)
72 axes[0, 1].set_title('Pérdida Reconstrucción')
73 axes[0, 1].legend()
74 axes[0, 1].grid(True, alpha=0.3)
75
76 axes[1, 0].plot(history['train_kl_losses'], 'b-', label='Train',
    linewidth=2)
77 axes[1, 0].plot(history['test_kl_losses'], 'r-', label='Test',
    linewidth=2)
78 axes[1, 0].set_title('Divergencia KL')
79 axes[1, 0].legend()
80 axes[1, 0].grid(True, alpha=0.3)
81
82 axes[1, 1].plot(history['beta_values'], 'g-', linewidth=2)
83 axes[1, 1].set_title('Beta Annealing (Cyclical)')
84 axes[1, 1].grid(True, alpha=0.3)
85
86 plt.tight_layout()
87 plt.savefig('complete_training_analysis.png', dpi=300)
88 plt.show()
89
90 # 6. Visualizar reconstrucciones
91 visualize_reconstructions(beta_vae, test_loader, device,
    n_samples=10)
92
93 # 7. Guardar modelo y configuración
94 torch.save({
95     'model_state_dict': beta_vae.state_dict(),
96     'history': history,
97     'scheduler_config': {
98         'strategy': 'cyclical',
99         'beta_start': 0.0,
100         'beta_end': 4.0,
101         'n_epochs': 150
102     }
103 }, 'beta_vae_with_annealing.pth')
104
105 print(";Entrenamiento completado!")

```

Listing 10.18: Pipeline end-to-end con beta annealing

10.4.3. Codificación y decodificación independientes

Una vez entrenado el β -VAE, sus componentes pueden utilizarse de forma independiente para diversas tareas.

Codificación de Nuevos Datos

```

1  # Cargar modelo entrenado
2  beta_vae = DualFlexKANBetaVAE(
3      input_size=784,
4      latent_size=32,
5      encoder_hidden_sizes=[512, 256, 128],
6      decoder_hidden_sizes=[128, 256, 512],
7      beta=2.0
8  )
9  checkpoint = torch.load('beta_vae_trained.pth')
10 beta_vae.load_state_dict(checkpoint['model_state_dict'])
11 beta_vae.eval()
12
13 # Codificar nuevas imágenes
14 with torch.no_grad():
15     new_images = torch.randn(10, 784)
16
17     # Obtener parámetros de distribución
18     mu, logvar = beta_vae.encode(new_images)
19
20     # Muestrear del espacio latente
21     z = beta_vae.reparameterize(mu, logvar)
22
23     # O usar directamente la media (modo determinista)
24     z_deterministic = mu
25
26     print(f"Media: {mu.shape}")           # (10, 32)
27     print(f"Log-varianza: {logvar.shape}") # (10, 32)
28     print(f"Muestra z: {z.shape}")        # (10, 32)
29
30 # Acceso directo al encoder
31 mu, logvar = beta_vae.encoder(new_images)

```

Listing 10.19: Codificación en modo inferencia

Decodificación desde el Espacio Latente

```

1
2  # Generar muestras
3  samples = generate_samples(beta_vae, n_samples=25, latent_size=32)
4  print(f"Muestras generadas: {samples.shape}") # (25, 784)
5
6  # Visualizar
7  import matplotlib.pyplot as plt
8  fig, axes = plt.subplots(5, 5, figsize=(10, 10))
9  for i in range(25):
10     ax = axes[i // 5, i % 5]

```

```
11     ax.imshow(samples[i].view(28, 28), cmap='gray')
12     ax.axis('off')
13 plt.suptitle('Muestras Generadas por Beta-VAE')
14 plt.show()
15
16 # Acceso directo al decoder
17 generated = beta_vae.decoder(z)
```

Listing 10.20: Generación de muestras desde el espacio latente

Interpolación en el Espacio Latente

```
1 def interpolate_latent_beta_vae(beta_vae, img1, img2, n_steps=10,
2                               use_mean=True, device='cpu'):
3     """
4     Interpola entre dos imágenes en el espacio latente.
5
6     Args:
7         beta_vae: Modelo Beta-VAE entrenado
8         img1, img2: Tensores de entrada (1, input_size)
9         n_steps: Número de pasos de interpolación
10        use_mean: Si True, usa mu; si False, muestrea z
11        device: Dispositivo de cómputo
12
13    Returns:
14        Tensor con secuencia interpolada (n_steps, input_size)
15    """
16    beta_vae.eval()
17    beta_vae.to(device)
18    img1, img2 = img1.to(device), img2.to(device)
19
20    with torch.no_grad():
21        # Codificar ambas imágenes
22        mu1, logvar1 = beta_vae.encode(img1)
23        mu2, logvar2 = beta_vae.encode(img2)
24
25        # Elegir representación
26        if use_mean:
27            z1, z2 = mu1, mu2
28        else:
29            z1 = beta_vae.reparameterize(mu1, logvar1)
30            z2 = beta_vae.reparameterize(mu2, logvar2)
31
32        # Interpolación esférica (slerp) o lineal
33        interpolations = []
34        for alpha in torch.linspace(0, 1, n_steps):
35            # Interpolación lineal
36            z_interp = (1 - alpha) * z1 + alpha * z2
```

```

37
38         # Decodificar
39         img_interp = beta_vae.decode(z_interp)
40         interpolations.append(img_interp)
41
42         return torch.cat(interpolations, dim=0).cpu()
43
44 # Ejemplo de uso
45 img1 = test_dataset[0][0].unsqueeze(0) # Primera imagen
46 img2 = test_dataset[100][0].unsqueeze(0) # Otra imagen
47 interpolated = interpolate_latent_beta_vae(
48     beta_vae, img1, img2, n_steps=15, use_mean=True
49 )
50
51 # Visualizar interpolación
52 fig, axes = plt.subplots(1, 15, figsize=(20, 2))
53 for i in range(15):
54     axes[i].imshow(interpolated[i].view(28, 28), cmap='gray')
55     axes[i].axis('off')
56     axes[i].set_title(f't={i/14:.2f}')
57 plt.suptitle('Interpolación en Espacio Latente')
58 plt.tight_layout()
59 plt.show()

```

Listing 10.21: Interpolación suave entre dos imágenes

Exploración del espacio latente con perturbaciones:

```

1
2 # Explorar primera dimensión latente
3 base_img = test_dataset[5][0].unsqueeze(0)
4 variations = explore_latent_dimensions(
5     beta_vae, base_img, dim_to_vary=0, n_steps=10
6 )
7
8 # Visualizar
9 fig, axes = plt.subplots(1, 10, figsize=(15, 2))
10 for i in range(10):
11     axes[i].imshow(variations[i].view(28, 28), cmap='gray')
12     axes[i].axis('off')
13 plt.suptitle('Variación de Dimensión Latente 0')
14 plt.show()

```

Listing 10.22: Exploración sistemática de dimensiones latentes

10.4.4. Visualización de Resultados

Uso de la Función de Visualización


```

1 from DFKAN_betaVAE import visualize_reconstructions
2
3 # Visualizar reconstrucciones
4 visualize_reconstructions(
5     model=beta_vae,
6     test_loader=test_loader,
7     device=device,
8     n_samples=10
9 )

```

Listing 10.23: Visualización de reconstrucciones

Visualización del espacio latente 2D:

```

1 from DFKAN_betaVAE import visualize_latent_space
2
3 # Solo funciona si latent_size == 2
4 if beta_vae.latent_size == 2:
5     visualize_latent_space(
6         model=beta_vae,
7         data_loader=test_loader,
8         device=device,
9         n_samples=1000
10    )

```

Listing 10.24: Visualización del espacio latente

Visualización con t-SNE para espacios latentes de alta dimensión:

```

1
2 # Usar la función
3 visualize_latent_tsne(beta_vae, test_loader, device=device,
4     n_samples=2000)

```

Listing 10.25: Visualización t-SNE del espacio latente

Análisis de disentanglement:

```

1
2 # Ejecutar análisis
3 analyze_disentanglement(beta_vae, test_loader, device=device,
4     n_dims=8, n_samples=10)

```

Listing 10.26: Análisis cualitativo de disentanglement

10.4.5. Transfer Learning con Encoder Pre-entrenado

El encoder variacional pre-entrenado puede utilizarse como extractor de características para tareas de clasificación supervisada.

```

1 class BetaVAEClassifier(nn.Module):
2     """
3     Clasificador utilizando encoder pre-entrenado del Beta-VAE.

```

```

4      """
5
6      def __init__(self, pretrained_beta_vae, num_classes,
7                    freeze_encoder=True, use_mean=True):
8
9          """
10         Args:
11             pretrained_beta_vae: Beta-VAE pre-entrenado
12             num_classes: Número de clases para clasificación
13             freeze_encoder: Si True, congela pesos del encoder
14             use_mean: Si True, usa mu; si False, muestrea z
15         """
16         super(BetaVAEClassifier, self).__init__()
17
18         self.encoder = pretrained_beta_vae.encoder
19         self.use_mean = use_mean
20
21         # Congelar encoder si se requiere
22         if freeze_encoder:
23             for param in self.encoder.parameters():
24                 param.requires_grad = False
25
26         # Cabeza de clasificación
27         latent_size = pretrained_beta_vae.latent_size
28         self.classifier = nn.Sequential(
29             nn.Linear(latent_size, 256),
30             nn.ReLU(),
31             nn.Dropout(0.5),
32             nn.Linear(256, 128),
33             nn.ReLU(),
34             nn.Dropout(0.3),
35             nn.Linear(128, num_classes)
36         )
37
38     def forward(self, x):
39         # Obtener representación latente
40         with torch.no_grad() if not self.training else torch.enable_grad():
41             mu, logvar = self.encoder(x)
42
43         # Usar media o muestrear
44         if self.use_mean:
45             z = mu
46         else:
47             std = torch.exp(0.5 * logvar)
48             eps = torch.randn_like(std)
49             z = mu + eps * std
50
51         # Clasificar

```

```

51         logits = self.classifier(z)
52         return logits
53
54 # Crear clasificador con encoder congelado
55 classifier = BetaVAEClassifier(
56     pretrained_beta_vae=beta_vae,
57     num_classes=10,
58     freeze_encoder=True,
59     use_mean=True
60 )
61
62 print(f"Parámetros totales: {sum(p.numel() for p in
63     classifier.parameters())}")
64 print(f"Parámetros entrenables: {sum(p.numel() for p in
65     classifier.parameters() if p.requires_grad)}")

```

Listing 10.27: Clasificador con encoder pre-entrenado

Entrenamiento del clasificador:

```

1 def train_classifier(classifier, train_loader, test_loader,
2                     epochs=50, lr=0.001, device='cpu'):
3     """Entrena el clasificador con encoder pre-entrenado."""
4
5     classifier.to(device)
6     optimizer = optim.Adam(classifier.parameters(), lr=lr)
7     criterion = nn.CrossEntropyLoss()
8
9     train_accs = []
10    test_accs = []
11
12    for epoch in range(epochs):
13        # Entrenamiento
14        classifier.train()
15        correct = 0
16        total = 0
17
18        for data, labels in train_loader:
19            data, labels = data.to(device), labels.to(device)
20
21            optimizer.zero_grad()
22            outputs = classifier(data)
23            loss = criterion(outputs, labels)
24            loss.backward()
25            optimizer.step()
26
27            _, predicted = torch.max(outputs.data, 1)
28            total += labels.size(0)
29            correct += (predicted == labels).sum().item()
30

```

```

31     train_acc = 100 * correct / total
32
33     # Evaluación
34     classifier.eval()
35     correct = 0
36     total = 0
37
38     with torch.no_grad():
39         for data, labels in test_loader:
40             data, labels = data.to(device), labels.to(device)
41             outputs = classifier(data)
42             _, predicted = torch.max(outputs.data, 1)
43             total += labels.size(0)
44             correct += (predicted == labels).sum().item()
45
46     test_acc = 100 * correct / total
47
48     train_accs.append(train_acc)
49     test_accs.append(test_acc)
50
51     if epoch % 5 == 0:
52         print(f'Época {epoch}: Train Acc={train_acc:.2f}%, '
53               f'Test Acc={test_acc:.2f}%')
54
55     return train_accs, test_accs
56
57 # Entrenar
58 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
59 train_accs, test_accs = train_classifier(
60     classifier, train_loader, test_loader,
61     epochs=50, lr=0.001, device=device
62 )
63
64 # Visualizar resultados
65 plt.figure(figsize=(8, 5))
66 plt.plot(train_accs, label='Train Accuracy', linewidth=2)
67 plt.plot(test_accs, label='Test Accuracy', linewidth=2)
68 plt.xlabel('Época')
69 plt.ylabel('Accuracy (%)')
70 plt.title('Transfer Learning: Clasificación con Encoder
71           Pre-entrenado')
72 plt.legend()
73 plt.grid(True, alpha=0.3)
74 plt.show()

```

Listing 10.28: Entrenamiento del clasificador

Fine-tuning del encoder:

```

1 # Crear clasificador con encoder descongelado

```

```

2 classifier_finetuned = BetaVAEClassifier(
3     pretrained_beta_vae=beta_vae,
4     num_classes=10,
5     freeze_encoder=False, # Permitir fine-tuning
6     use_mean=True
7 )
8
9 # Usar learning rates diferentes para encoder y clasificador
10 encoder_params = list(classifier_finetuned.encoder.parameters())
11 classifier_params =
12     list(classifier_finetuned.classifier.parameters())
13
14 optimizer = optim.Adam([
15     {'params': encoder_params, 'lr': 1e-5}, # LR bajo para
16         encoder
17     {'params': classifier_params, 'lr': 1e-3} # LR normal para
18         clasificador
19 ])
20
21 # Entrenar con fine-tuning
22 # ... (usar el mismo loop de entrenamiento)

```

Listing 10.29: Fine-tuning con descongelado gradual

Comparación de Estrategias

```

1 # Entrenar clasificador desde cero (sin pre-entrenamiento)
2 classifier_scratch = nn.Sequential(
3     nn.Linear(784, 256),
4     nn.ReLU(),
5     nn.Dropout(0.5),
6     nn.Linear(256, 128),
7     nn.ReLU(),
8     nn.Dropout(0.3),
9     nn.Linear(128, 10)
10 )
11
12 # Comparar resultados
13 print("\n=== COMPARACIÓN DE RESULTADOS ===")
14 print(f"Transfer Learning (encoder congelado):
15     {max(test_accs):.2f}%")
16 # print(f"Fine-tuning completo: {max(test_accs_ft):.2f}%")
17 # print(f"Desde cero: {max(test_accs_scratch):.2f}%")

```

Listing 10.30: Comparación: desde cero vs transfer learning

Buenas Prácticas para Transfer Learning

1. Estrategia de congelamiento:

- Comenzar con encoder totalmente congelado.
- Una vez que el clasificador converge, descongelar gradualmente.
- Usar learning rates muy bajos (10^{-5} a 10^{-4}) para el encoder.

2. Uso de μ vs muestreo:

- Para clasificación: usar `use_mean=True` (más estable).
- Para regularización adicional: muestrear durante entrenamiento.

3. Tamaño del espacio latente:

- Espacios latentes más grandes (32-128) funcionan mejor para transfer learning.
- Para disentanglement: espacios pequeños (2-16).

4. Valor de β óptimo:

- Para transfer learning: $\beta \in [1, 2]$ (balance entre reconstrucción y regularización).
- Para disentanglement: $\beta > 4$ (mayor separación de factores).

Capítulo 11 Interpretabilidad y Análisis Avanzado

La interpretabilidad es una de las características más valiosas y distintivas de las redes Kolmogorov-Arnold (KAN) frente a las redes neuronales tradicionales. Mientras que las redes neuronales profundas operan como "cajas negras" donde es difícil entender qué transformaciones específicas se aplican a los datos, las KAN ofrecen transparencia inherente en su funcionamiento al expresar explícitamente las funciones aprendidas en cada conexión.

En este capítulo se detallan las herramientas nativas de inspección de `DualFlexKAN` y se presenta el conjunto de módulos de análisis avanzado diseñados para diseccionar el comportamiento de la red.

11.1. La Importancia de la Interpretabilidad

En aplicaciones críticas como diagnóstico médico, sistemas financieros, o procesos científicos, no basta con que un modelo haga predicciones precisas: es fundamental comprender *cómo y por qué* llega a esas conclusiones. La interpretabilidad permite:

- **Validación científica:** Verificar que el modelo aprende relaciones coherentes con el conocimiento del dominio.
- **Depuración:** Identificar comportamientos anómalos o sesgos en el aprendizaje.
- **Confianza:** Aumentar la confianza de expertos del dominio en las predicciones del modelo.
- **Descubrimiento:** Potencialmente descubrir nuevas relaciones o patrones desconocidos (regresión simbólica).

11.1.1. Ventajas de KAN sobre Redes Tradicionales

Las redes neuronales tradicionales (MLP) utilizan funciones de activación fijas (ReLU, tanh) aplicadas uniformemente. Por el contrario, las KAN implementadas mediante `DualFlexKANLinear` aprenden funciones específicas para cada conexión, lo que proporciona:

1. **Funciones explícitas:** Cada transformación se expresa matemáticamente como un polinomio (Legendre, Chebyshev), spline, wavelet o función racional.

2. **Granularidad configurable:** Desde funciones compartidas globalmente hasta funciones únicas por cada conexión.
3. **Inspección directa:** Los parámetros aprendidos pueden extraerse y visualizarse naturalmente.

11.2. Métodos Nativos de Inspección en DualFlexKAN

La clase base incluye métodos inmediatos para una primera inspección del modelo sin herramientas externas.

11.2.1. Información de Arquitectura

El método `print_architecture_info()` proporciona una vista completa de la estructura de la red.

```
1 model.print_architecture_info()
```

Listing 11.1: Inspección de la arquitectura

La salida incluye dimensiones, tipos de funciones (ej. `chebyshev`, `wavelet`), estrategias (`per_input`, `fixed`) y conteo de parámetros por capa.

11.2.2. Extracción de Funciones Aprendidas

El método `print_learned_functions()` extrae las ecuaciones matemáticas exactas o los parámetros definitorios de todas las funciones activas.

```
1 # Extraer las funciones aprendidas
2 functions_dict = model.print_learned_functions()
```

Listing 11.2: Extracción de funciones aprendidas

Este método devuelve un diccionario estructurado. Para las nuevas bases implementadas, la salida es descriptiva:

=== Funciones de Activación ===

Capa 1:

Funciones de entrada:

`_in_0(x) = 0.23 + 0.56x - 0.12x^2 (Polynomial)`

`_in_1(x) = 0.89 T_1(x) + 0.23 T_2(x) (Chebyshev)`

Funciones de salida:

`_out_0(x) = Wavelet (5 componentes, escalas=[...])`

11.2.3. Resumen de Regularización

El método `print_regularization_summary()` permite inspeccionar la configuración de Dropout y Batch Normalization, crucial para entender el flujo de la señal y la estabilidad de las bases polinomiales.

11.3. Análisis de Explicabilidad e Importancia

La clase `KANExplainability` (en `KAN_explainability_importance.py`) proporciona un sistema completo para cuantificar la relevancia de características y funciones.

11.3.1. Inicialización

```
1 from KAN_explainability_importance import KANExplainability
2 explainer = KANExplainability(model, feature_names=['edad',
  'presión', ...])
```

11.3.2. Cálculo de Importancia de Funciones

`compute_function_importance()` Calcula la importancia basada en la varianza de las activaciones. Una función plana (constante) tiene varianza 0 y es irrelevante para la clasificación, independientemente de sus coeficientes.

```
1 importance = explainer.compute_function_importance(
2     X_sample=X_tensor, layer_idx=0
3 )
4 print(f"Importancia por varianza:
  {importance['variance_importance']}")
```

`compute_gradient_importance()` Utiliza gradientes (incluyendo *Integrated Gradients*) para medir la sensibilidad de la salida respecto a las entradas.

11.3.3. Knockout Analysis (Análisis de Ablación)

Esta técnica evalúa el impacto de eliminar selectivamente componentes de la red. Es fundamental para determinar si una estructura compleja (ej. un polinomio de grado alto o una wavelet) es realmente necesaria.

`knockout_analysis()` Permite "pagar" funciones, neuronas o conexiones y medir el deterioro en la predicción.

Parámetros:

- `knockout_type`: 'function' (anula $\phi(x)$), 'neuron' (anula y_j) o 'connection' (anula w_{ij}).
- `indices`: Lista de índices a eliminar.

Ejemplo:

```
1 # Evaluar el impacto de eliminar las funciones de entrada 0 y 2
2 impact = explainer.knockout_analysis(
3     X_sample=X_tensor,
```

```

4     layer_idx=0,
5     knockout_type='function',
6     indices=[0, 2]
7 )
8 print(f"Impacto en error (MSE): {impact['mse_increase']:.4f}")
9 # Un impacto alto confirma que estas funciones son críticas.

```

Listing 11.3: Knockout Analysis

11.3.4. Análisis de Sensibilidad

`sensitivity_analysis()` Analiza cómo varía la salida del modelo al perturbar una característica de entrada específica, manteniendo las demás constantes (*ceteris paribus*).

```

1 sensitivity = explainer.sensitivity_analysis(
2     X_sample=X_tensor[0:1],
3     feature_idx=0,
4     perturbation_range=(-3, 3)
5 )
6 explainer.plot_sensitivity(sensitivity)

```

11.3.5. Análisis Comprensivo

`comprehensive_importance_analysis()` Ejecuta todos los métodos anteriores y genera un reporte unificado.

11.4. Análisis de Interacciones

La clase `KANInteractionAnalysis` permite descomponer cómo las funciones univariadas interactúan para formar decisiones complejas.

11.4.1. Descomposición Funcional

`functional_decomposition()` Desglosa la salida de una neurona en la suma de sus componentes: $y = \sum w_i \phi_i(x_i)$. Esto permite visualizar (mediante gráficos *waterfall*) qué características suman o restan al resultado final.

```

1 decomp = analyzer.functional_decomposition(
2     X_sample=X_tensor, layer_idx=0, neuron_idx=0
3 )
4 analyzer.plot_functional_decomposition(decomp)

```

11.4.2. Mapas de Interacción y Pares

`compute_interaction_map()` Genera una matriz de calor que muestra la fuerza de conexión entre entradas y neuronas.

`pairwise_interaction_analysis()` Detecta sinergias entre pares de variables. Si el efecto conjunto de dos variables es mayor que la suma de sus efectos individuales, existe una interacción fuerte (signo de no-linealidad cruzada capturada en capas profundas).

```
1 pairwise = analyzer.pairwise_interaction_analysis(X_sample=X_tensor)
2 analyzer.plot_pairwise_interactions(pairwise)
```

11.4.3. Comparación de Estrategias

`compare_strategies()` Evalúa la **diversidad funcional**. Si se usa la estrategia `per_input` o `per_neuron`, este análisis determina si las funciones aprendidas son realmente distintas entre sí (alta especialización) o si han convergido a formas similares (sugiriendo que una estrategia global hubiera bastado).

11.5. Visualización del Flujo de Información

La clase `KANFlowVisualization` permite observar la dinámica interna de la red.

11.5.1. Captura y Mapas de Calor

`capture_activations()` y `plot_activation_heatmaps()` Permiten visualizar cómo se propagan los valores a través de las capas para muestras específicas, ayudando a detectar neuronas muertas o saturadas (especialmente útil con funciones `tanh` o `sigmoid`).

11.5.2. Rutas Críticas (Critical Paths)

`trace_critical_paths()` Identifica el subgrafo de la red responsable de una decisión específica mediante *backtracking* de las contribuciones más altas.

```
1 # ¿Qué camino llevó a esta predicción?
2 paths = flow_viz.trace_critical_paths(X_sample=sample,
3   output_neuron=0)
3 flow_viz.plot_critical_paths(paths)
```

Las rutas se visualizan con colores (verde para contribución positiva, rojo para negativa) y grosor proporcional a la importancia.

11.5.3. Análisis de Cuellos de Botella

`analyze_bottlenecks()` Detecta capas donde la varianza de activación colapsa (pérdida de información) o donde la dimensionalidad efectiva es mucho menor que el número de neuronas (ineficiencia arquitectónica).

11.6. Poda y Simplificación Simbólica

La clase `KANPruningSimplification` es vital para transformar modelos KAN complejos en fórmulas matemáticas compactas.

11.6.1. Análisis de Coeficientes

`analyze_coefficient_magnitude()` Examina los pesos de los polinomios (o coeficientes de wavelets/splines). En bases como Legendre o Chebyshev, los coeficientes de alto orden suelen decaer rápidamente; si no lo hacen, puede indicar sobreajuste.

11.6.2. Poda (Pruning)

`threshold_coefficients()` Poda "dura" de coeficientes dentro de una función (ej. eliminar $0,0001x^5$).

`prune_connections()` Poda estructural: elimina conexiones enteras ($w_{ij} = 0$) si su importancia (peso $\times$ actividad) es baja.

```
1 sparsity = pruner.analyze_connection_sparsity(X_tensor)
2 # Aplicar poda al 10% de conexiones más débiles
3 pruner.prune_connections(sparsity, threshold=0.1, inplace=True)
```

11.6.3. Regresión Simbólica (Simplificación)

Esta funcionalidad intenta ajustar fórmulas matemáticas estándar a las funciones aprendidas por la KAN. Es especialmente efectiva con bases polinomiales (`polynomial`, `legendre`, `chebyshev`) y racionales.

`fit_symbolic_function()` Ajusta funciones candidatas (lineal, cuadrática, sinusoidal, exponencial) a la curva aprendida $\phi(x)$.

```
1 symbolic = pruner.fit_symbolic_function(
2     layer_idx=0, function_idx=0, max_terms=3
3 )
4 print(f"Fórmula descubierta:
5     {symbolic['best_simplified']['equation']}")
6 # Ejemplo: "0.5*x^2 + sin(x)"
```

11.7. Análisis de Autoencoders KAN

La clase `KANAutoencoderAnalysis` está diseñada para arquitecturas encoder-decoder (`DualFlexKANAutoencoder`).

11.7.1. Simetría Funcional

`compare_encoder_decoder_functions()` Verifica si el decoder ha aprendido la función inversa del encoder ($\phi_{dec} \approx \phi_{enc}^{-1}$), lo cual es un indicador fuerte de una representación latente robusta y desenredada.

11.7.2. Exploración del Espacio Latente

`analyze_latent_space()` Calcula estadísticas y proyecciones (PCA, t-SNE) del cuello de botella.

`interpolate_latent()` Genera transiciones suaves entre muestras para verificar la continuidad del manifold aprendido.

`reconstruction_by_component()` Descompone la reconstrucción activando una sola dimensión latente a la vez. Esto es crucial para verificar el *disentanglement*: si una dimensión controla solo el color o otra solo la "forma", la reconstrucción por componentes lo revelará visualmente.

11.8. Generación de Informes Completos

Todas las herramientas de análisis incluyen métodos `generate_..._report()` que ejecutan pipelines completos y guardan los resultados y gráficas en disco automáticamente, facilitando la documentación de experimentos.

```
1 explainer.generate_explainability_report(X_sample,
    output_dir='./reporte')
```


Capítulo 12 Mecanismos de Atención en DualFlexKAN

La incorporación de mecanismos de atención en la arquitectura DualFlexKAN representa un avance significativo hacia modelos no solo más precisos, sino también más dinámicos e interpretables. Mientras que las redes KAN estándar destacan por aprender funciones de activación complejas, la atención añade la capacidad de ponderar la importancia de las características de entrada instante a instante.

12.1. Introducción a la Atención Neuronal

En las redes neuronales tradicionales (MLP) y en las KAN básicas, los pesos de las conexiones se fijan una vez finalizado el entrenamiento. Esto significa que la red procesa cada muestra de entrada utilizando exactamente la misma estructura de ponderación estática. Si la neurona A es importante para clasificar la muestra 1, la red asume que también es igual de importante para la muestra 2.

El **Mecanismo de Atención** rompe esta rigidez introduciendo ponderación dinámica. Permite que la red responda a la pregunta: *"Para esta entrada específica $\mathbf{x}$, ¿qué características son las más relevantes?"*.

En el contexto de DualFlexKAN, implementamos lo que se conoce como **Atención de Características (Feature Attention)**. Este mecanismo actúa como una compuerta inteligente situada antes de cada capa KAN. Antes de que la capa KAN aplique sus polinomios o splines complejos, el mecanismo de atención evalúa el vector de entrada y genera una "máscara de importancia".

12.1.1. Beneficios Clave

- **Focalización Dinámica:** El modelo puede aprender a ignorar el ruido de fondo en una muestra específica mientras se centra en señales débiles pero críticas en otra.
- **Sparsity Contextual:** A diferencia de la regularización L1 que anula pesos permanentemente, la atención puede anular una característica solo cuando es irrelevante para el contexto actual.
- **Mejora del Gradiente:** Al multiplicar las entradas por pesos de atención calculados mediante funciones sigmoideas, se crean caminos de gradiente más suaves que facilitan

el entrenamiento de redes profundas.

12.2. Arquitectura: DualFlexKAN con Atención

La propuesta de implementación integra un bloque de atención ligero, inspirado en las redes *Squeeze-and-Excitation* (SE-Net), justo antes de la transformación no lineal de la KAN.

12.2.1. Formulación Matemática

Sea $\mathbf{x} \in \mathbb{R}^{d_{in}}$ el vector de entrada a una capa. El proceso se divide en dos fases:

1. Cálculo de los Pesos de Atención ($\mathbf{a}$): Se utiliza una pequeña red perceptrón multicapa (MLP) como red de contexto para comprimir y luego expandir la información, capturando dependencias globales entre las características.

$$\mathbf{z} = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{\frac{d_{in}}{r} \times d_{in}} \quad (12.1)$$

$$\mathbf{a} = \sigma(\mathbf{W}_2 \mathbf{z} + \mathbf{b}_2), \quad \mathbf{W}_2 \in \mathbb{R}^{d_{in} \times \frac{d_{in}}{r}} \quad (12.2)$$

Donde r es el factor de reducción (típicamente 4 o 8) y σ es la función Sigmoide que acota los valores a $(0, 1)$.

2. Calibración de la Entrada ($\tilde{\mathbf{x}}$): La entrada original se reescala elemento a elemento (producto Hadamard $\odot$) por los pesos de atención:

$$\tilde{\mathbf{x}} = \mathbf{a} \odot \mathbf{x} \quad (12.3)$$

3. Transformación KAN: Finalmente, la entrada calibrada pasa a la capa DualFlex-KAN estándar:

$$\mathbf{y} = \text{DualFlexKANLinear}(\tilde{\mathbf{x}}) \quad (12.4)$$

12.3. Implementación en Python

A continuación se detalla cómo utilizar el módulo de atención incluido en la librería `KAN_shared_poly7.py`.

12.3.1. El Módulo de Atención

Este módulo es independiente y puede insertarse antes de cualquier capa lineal o KAN.

```

1 import torch
2 import torch.nn as nn
3
4 class KANFeatureAttention(nn.Module):
5     """
6     Mecanismo de Atención de Características (Feature Attention).

```


Listing 12.1: Definición del módulo KANFeatureAttention

Esta clase envuelve la capa original `DualFlexKANLinear` y le antepone el mecanismo de atención.

[illegible]

```

14         # Capa KAN subyacente (pasa todos los argumentos de
           configuración)
15         self.kan_layer = DualFlexKANLinear(in_features,
           out_features, **kan_kwargs)
16
17     def forward(self, x):
18         # 1. Aplicar atención
19         x_attended, weights = self.attention(x)
20
21         # 2. Pasar por KAN
22         output = self.kan_layer(x_attended)
23
24         # Retornamos también los pesos para
           visualización/interpretabilidad
25         return output, weights

```

Listing 12.2: Wrapper DualFlexKANAttentionLayer

12.4. Creación de una red AttentiveDualFlexKAN

Para utilizar atención en una red con capas DualFlexKAN, definimos una nueva clase de red que construye capas de atención en lugar de capas lineales simples.

```

1 class AttentiveDualFlexKAN(nn.Module):
2     def __init__(self, layer_sizes, attention_reduction=4, **kwargs):
3         super(AttentiveDualFlexKAN, self).__init__()
4
5         self.layers = nn.ModuleList()
6
7         # Iterar para crear capas
8         for i in range(len(layer_sizes) - 1):
9             # Parsear configuraciones específicas de capa si son
               listas
10            layer_kwargs = {}
11            for key, value in kwargs.items():
12                if isinstance(value, list) and len(value) ==
                   len(layer_sizes) - 1:
13                    layer_kwargs[key] = value[i]
14                else:
15                    layer_kwargs[key] = value
16
17            # Instanciar capa con atención
18            layer = DualFlexKANAttentionLayer(
19                in_features=layer_sizes[i],
20                out_features=layer_sizes[i+1],
21                attention_reduction=attention_reduction,
22                **layer_kwargs
23            )

```

```

24         self.layers.append(layer)
25
26     def forward(self, x):
27         self.attention_maps = [] # Guardar para visualización
28
29         for layer in self.layers:
30             x, weights = layer(x)
31             self.attention_maps.append(weights)
32
33     return x

```

Listing 12.3: Definición de la red completa con atención

12.5. Inclusión de capa de atención en DualFlexKAN

Si tenemos código que usa DualFlexKAN, podemos incluir la atención en una capa cambiando la clase instanciada.

```

1 # --- ANTES (Sin atención) ---
2 model = DualFlexKAN(
3     layer_sizes=[10, 32, 1],
4     input_activation_types=["polynomial", "polynomial"],
5     input_function_strategies=["per_input", "global"]
6 )
7
8 # --- AHORA (Con atención) ---
9 # Solo cambiamos la clase y añadimos el ratio de reducción
10 model_attn = AttentiveDualFlexKAN(
11     layer_sizes=[10, 32, 1],
12     attention_reduction=2, # Nuevo parámetro
13     input_activation_types=["polynomial", "polynomial"],
14     input_function_strategies=["per_input", "global"]
15 )
16
17 # El entrenamiento es idéntico
18 output = model_attn(torch.randn(5, 10))
19 print("Salida:", output.shape)

```

Listing 12.4: Ejemplo de conversión de código

12.6. Atención Selectiva (Híbrida)

En muchos escenarios prácticos, no es eficiente ni necesario aplicar mecanismos de atención en todas las capas de la red. La atención selectiva permite diseñar arquitecturas híbridas donde solo ciertas etapas críticas del procesamiento cuentan con capacidad de ponderación dinámica, manteniendo el resto de la red como una KAN estándar para optimizar el coste computacional.

Una configuración muy común es aplicar **atención únicamente en la primera capa** para que actúe como un selector de características (*feature selector*) robusto ante ruido, o en capas profundas específicas para refinar características abstractas.

12.6.1. Construcción de una Red con Atención Híbrida

Para implementar esto, utilizamos la clase `SelectiveAttentiveKAN` (o una modificación de la clase principal) que acepta una lista de índices. El siguiente ejemplo muestra cómo construir una red donde la atención se aplica en la entrada (para filtrar datos) y en la segunda capa oculta (para refinar conceptos).

```

1 from KAN_attention import SelectiveAttentiveKAN
2
3 # Definición de la arquitectura
4 # Capas: [10 (Input) -> 64 -> 32 -> 16 -> 1 (Output)]
5 # Índices de capas:    0      1      2      3
6
7 model = SelectiveAttentiveKAN(
8     layer_sizes=[10, 64, 32, 16, 1],
9
10    # --- Configuración de Atención ---
11    # Aplicar atención en la capa 0 (entrada) y la capa 2
12    # (intermedia)
13    attention_layers_indices=[0, 2],
14
15    # Factor de reducción: controla el tamaño del cuello de botella
16    # en el módulo de atención
17    # Un valor de 4 significa que la red de atención usa 1/4 de las
18    # neuronas
19    attention_reduction=4,
20
21    # --- Configuración KAN Estándar ---
22    input_function_strategies=["per_input", "global", "per_input",
23    "none"],
24    input_activation_types=["chebyshev", "polynomial", "polynomial",
25    "linear"],
26    output_function_strategies=["per_neuron", "fixed", "fixed",
27    "fixed"],
28    output_activation_types=["polynomial", "relu", "relu", "linear"]
29 )
30
31 # Verificación
32 print(f"Capa 0 es atencional: {isinstance(model.layers[0],
33     DualFlexKANAttentionLayer)}")
34 print(f"Capa 1 es estándar: {isinstance(model.layers[1],
35     DualFlexKANLinear)}")

```

Listing 12.5: Construcción de una red con atención en capas específicas

12.6.2. Atención en Capas Arbitrarias

La flexibilidad de este sistema radica en el parámetro `attention_layers_indices`. Este sistema de indexación funciona de la siguiente manera:

- **Índice 0 (Entrada):** La atención se aplica sobre los datos crudos originales.
- **Índices $i > 0$ (Ocultas):** La atención se aplica sobre las *representaciones latentes* generadas por la capa anterior.

Incluir atención en capas arbitrarias permite diferentes efectos de modelado:

1. **Atención en capa 0 (Feature Selection):** El modelo aprende a ignorar sensores ruidosos o características irrelevantes del dataset original.
2. **Atención en capas intermedias (Concept Refinement):** El modelo aprende a ponderar combinaciones de características no lineales. Por ejemplo, en una red profunda, la capa 0 podría procesar píxeles, mientras que la capa 3 procesa "formas". Poner atención en la capa 3 permite al modelo decidir si para una imagen específica es más importante la forma circular o la textura.
3. **Atención en última capa (Output Gating):** Ayuda a ponderar las características de alto nivel justo antes de la decisión final de clasificación o regresión.

12.6.3. Parámetros de Configuración de Atención

Al instanciar una red con capacidades de atención, se introducen nuevos hiperparámetros que controlan el comportamiento del mecanismo SE (Squeeze-and-Excitation).

Impacto en el Conteo de Parámetros

Añadir atención incrementa el número de parámetros de la red, aunque generalmente de forma ligera debido a la estructura de cuello de botella. Para una capa con entrada de tamaño N y factor de reducción r , el módulo de atención añade aproximadamente:

$$P_{\text{attn}} \approx 2 \cdot \frac{N^2}{r} \quad (12.5)$$

Por ejemplo, para una entrada de 64 características y $r = 4$, se añaden $\approx 2 \cdot (4096/4) = 2048$ parámetros, lo cual es despreciable comparado con los parámetros polinomiales de una capa KAN densa, pero aporta una gran capacidad de control dinámico.

12.7. Caso de Uso Avanzado: Autoencoder Atencional

El uso de mecanismos de atención en autoencoders (AE) transforma el modelo de una simple herramienta de compresión a un sistema capaz de realizar *selección de características latentes*.

Parámetro	Tipo	Descripción y Efecto
<code>attention_layers_indices</code>	<code>List[int]</code>	Lista que contiene los índices (0-based) de las capas donde se inyectará el módulo de atención. Si es <code>None</code> o está vacío, la red se comporta como una DualFlexKAN estándar. <i>Ejemplo:</i> <code>[0]</code> para atención solo en entrada.
<code>attention_reduction</code>	<code>int</code>	Factor de reducción r del cuello de botella en el MLP de atención. El módulo de atención reduce la dimensión de entrada d a d/r y luego la expande de nuevo. - Valor alto (ej. 16): Menos parámetros, atención más generalizada, menor riesgo de sobreajuste. - Valor bajo (ej. 2): Más parámetros, capacidad de capturar correlaciones finas entre características. <i>Default típico:</i> 4 u 8.
<code>return_attention_weights</code>	<code>bool</code>	(Opcional, según implementación) Determina si el método <code>forward</code> debe devolver los mapas de importancia calculados junto con la predicción. Es fundamental activarlo para tareas de explicabilidad e interpretabilidad.

Tabla 12.1: Parámetros específicos para la configuración del mecanismo de atención en DualFlexKAN.

En un Autoencoder KAN estándar, el encoder comprime la entrada $\mathbf{x}$ en un vector latente $\mathbf{z}$ tratando todas las características abstractas intermedias con la misma prioridad. Sin embargo, en datos complejos, muchas de estas características pueden ser ruido o redundancia.

Al introducir un módulo de atención justo antes de la capa del cuello de botella (*bottle-neck*), forzamos al modelo a ponderar explícitamente qué características de alto nivel deben preservarse en el código latente y cuáles deben descartarse. Esto resulta en un espacio latente más "puro", desenredado y robusto.

12.7.1. Arquitectura del Encoder con Atención Latente

La arquitectura propuesta modifica el Encoder estándar para incluir una capa `DualFlexKANAttentionLa` como última etapa de transformación antes de generar el vector $\mathbf{z}$.

$$\mathbf{h} = \text{KAN}_{\text{deep}}(\mathbf{x}) \quad (\text{Características profundas}) \quad (12.6)$$

$$\mathbf{h}_{\text{attn}}, \mathbf{w}_{\text{imp}} = \text{Attention}(\mathbf{h}) \quad (\text{Ponderación y Pesos}) \quad (12.7)$$

$$\mathbf{z} = \text{KAN}_{\text{proj}}(\mathbf{h}_{\text{attn}}) \quad (\text{Proyección al espacio latente}) \quad (12.8)$$

Donde $\mathbf{w}_{\text{imp}}$ representa el vector de importancia que nos dice qué características abstractas fueron críticas para generar esta representación latente específica.

12.7.2. Implementación en Python

A continuación se presenta el código para construir un `AttentiveKANAutoencoder`. Nótese cómo el encoder está diseñado para devolver tanto el vector latente como los mapas de atención, permitiendo análisis posteriores.

```

1 import torch
2 import torch.nn as nn
3 # Asumimos que las clases base ya han sido importadas
4 # from KAN_shared_poly7 import DualFlexKAN, DualFlexKANLinear
5 # from KAN_attention import DualFlexKANAttentionLayer
6
7 class AttentiveEncoder(nn.Module):
8     """
9     Encoder que aplica atención antes de la proyección al espacio
10    latente.
11    """
12    def __init__(self, input_size, hidden_sizes, latent_size,
13                  attention_reduction=4):
14        super(AttentiveEncoder, self).__init__()
15
16        self.layers = nn.ModuleList()
17
18        # 1. Construcción de capas ocultas estándar
19        # (DualFlexKANLinear)
20        current_size = input_size
21        for hidden in hidden_sizes:
22            layer = DualFlexKANLinear(
23                in_features=current_size,
24                out_features=hidden,
25                input_function_strategy="per_input", # KAN estándar
26                input_activation_type="polynomial",
27                input_activation_kwargs={'degree': 3}
28            )
29            self.layers.append(layer)
30            current_size = hidden
31
32        # 2. Capa de Atención Latente (Pre-Bottleneck)
33        # Esta capa decide qué info pasa al espacio latente
34        self.attention_layer = DualFlexKANAttentionLayer(
35            in_features=current_size,
36            out_features=latent_size,
37            attention_reduction=attention_reduction,
38            # Configuración de alta precisión para el cuello de
39            # botella
40            input_function_strategy="per_neuron_input",
41            input_activation_type="chebyshev", # Chebyshev es
42            # excelente para compresión
43            input_activation_kwargs={'degree': 4}

```

```

39         )
40
41     def forward(self, x):
42         # Pasar por capas ocultas normales
43         for layer in self.layers:
44             x = layer(x)
45
46         # Aplicar atención en la última etapa
47         # z: vector latente, attn_weights: importancia de las
48           características
49         z, attn_weights = self.attention_layer(x)
50
51         return z, attn_weights
52
53 class AttentiveKANAutoencoder(nn.Module):
54     def __init__(self, input_size, latent_size, hidden_sizes=[64,
55       32]):
56         super(AttentiveKANAutoencoder, self).__init__()
57
58         # Encoder con atención
59         self.encoder = AttentiveEncoder(
60             input_size=input_size,
61             hidden_sizes=hidden_sizes,
62             latent_size=latent_size
63         )
64
65         # Decoder simétrico estándar (sin atención inversa)
66         # Reconstruye desde latent_size -> hidden inverso ->
67           input_size
68         decoder_layers = hidden_sizes[::-1] + [input_size]
69         self.decoder = DualFlexKAN(
70             layer_sizes=[latent_size] + decoder_layers,
71             input_function_strategies=["global"] *
72               len(decoder_layers),
73             output_activation_types=["relu"] *
74               (len(decoder_layers)-1) + ["sigmoid"] # Sigmoid para
75               imágenes
76         )
77
78     def forward(self, x):
79         # 1. Codificar
80         z, weights = self.encoder(x)
81
82         # 2. Decodificar
83         x_reconstructed = self.decoder(z)
84
85         # Retornamos reconstrucción y pesos de atención para análisis
86         return x_reconstructed, weights

```



```

81
82 # --- Ejemplo de Uso ---
83 input_dim = 784 # MNIST
84 latent_dim = 10
85
86 model = AttentiveKANAutoencoder(input_size=input_dim,
87                                 latent_size=latent_dim)
88
89 # Simulación de un forward pass
90 dummy_input = torch.randn(16, input_dim) # Batch de 16
91 recon, importance_maps = model(dummy_input)
92
93 print(f"Reconstrucción shape: {recon.shape}") # (16, 784)
94 print(f"Pesos de Atención shape: {importance_maps.shape}") # (16,
95     32) -> Última capa oculta antes de latent

```

Listing 12.6: Implementación completa de un Autoencoder KAN Atencional

12.7.3. Interpretación de los Pesos de Atención Latente

El valor de retorno `weights` (o `importance_maps` en el ejemplo) tiene una dimensión de $(Batch, Hidden_{last})$.

Si analizamos estos pesos, obtenemos *insights* profundos sobre el funcionamiento del modelo:

- **Detección de Características Muertas:** Si la columna j de los pesos tiene un promedio cercano a 0 para todo el dataset, significa que la neurona j de la última capa oculta no aporta información útil para la compresión. Esto permite realizar *pruning* (poda) de la red con seguridad.
- **Especialización de Clases:** Si promediamos los pesos de atención agrupando por clase (ej. dígitos 0-9 en MNIST), descubriremos "firmas de atención". Por ejemplo, el dígito "1" podría activar fuertemente las neuronas 5 y 8, mientras que el "0" activa las neuronas 12 y 20.
- **Detección de Anomalías:** Una muestra anómala a menudo generará un patrón de atención difuso (baja confianza) o radicalmente diferente al patrón promedio de los datos de entrenamiento.

12.7.4. Entrenamiento con Regularización de Atención

Para fomentar que el modelo utilice la atención de manera efectiva (es decir, que sea selectivo y "sparse"), se puede añadir un término de regularización a la función de pérdida:

$$\mathcal{L}_{total} = \mathcal{L}_{MSE}(\mathbf{x}, \hat{\mathbf{x}}) + \lambda \sum |\mathbf{w}_{imp}| \quad (12.9)$$

Esto fuerza al modelo a activar solo las neuronas estrictamente necesarias para la reconstrucción, mejorando el disentanglement en el espacio latente.

12.7.5. Interpretación del Autoencoder Atencional

Al entrenar este modelo, la variable `weights` retornada por el encoder nos da información valiosa sobre la **importancia de las características latentes**. Si visualizamos estos pesos, podemos ver qué dimensiones del espacio latente se activan para diferentes clases de entrada, facilitando enormemente la interpretabilidad del "black box" que suele ser el espacio latente.

12.7.6. Construcción de Arquitecturas Híbridas Capa a Capa

Para superar las limitaciones de las definiciones secuenciales automáticas y aprovechar la flexibilidad de las redes de Kolmogorov-Arnold (KAN), proponemos una metodología de construcción manual "capa a capa". Este enfoque permite integrar módulos heterogéneos, tales como capas convolucionales (`DualFlexKANConv2d`), capas densas (`DualFlexKANLinear`) y mecanismos de atención, definiendo explícitamente la topología en una subclase de `torch.nn.Module`.

Adaptación Dimensional para la Atención

Uno de los desafíos técnicos al integrar estos componentes reside en la dimensionalidad de los tensores. El módulo de atención nativo (`KANFeatureAttention`) opera sobre vectores unidimensionales ($1D$), mientras que las capas convolucionales producen mapas de características cuatridimensionales ($4D$).

Para resolver esto, implementamos un adaptador espacial inspirado en el paradigma *Squeeze-and-Excitation*. La clase `KANConv2dAttention` (Código 12.7) realiza un *Global Average Pooling* para reducir las dimensiones espaciales, aplica la atención KAN y re-escala los canales originales.

```

1 class KANConv2dAttention(nn.Module):
2     """
3     Adaptador para aplicar KANFeatureAttention a salidas 4D.
4     Pipeline: Pooling Global -> Atencion KAN -> Re-escalado.
5     """
6     def __init__(self, channels, reduction_ratio=4):
7         super(KANConv2dAttention, self).__init__()
8         # Módulo base de atención KAN (para vectores 1D)
9         self.channel_attention = KANFeatureAttention(
10             channels, reduction_ratio=reduction_ratio
11         )
12
13     def forward(self, x):
14         b, c, h, w = x.shape
15         # 1. Squeeze: (batch, c, h, w) -> (batch, c)
16         y = x.view(b, c, -1).mean(dim=2)
17
18         # 2. Excitation: Obtener pesos de atención
19         _, weights = self.channel_attention(y)
20
21         # 3. Reshape para broadcasting: (batch, c, 1, 1)
22         weights = weights.view(b, c, 1, 1)

```

```

23
24     # 4. Scale: Recalibración de características
25     return x * weights

```

Listing 12.7: Adaptador de Atención KAN para tensores convolucionales

Definición de la Topología Híbrida

La definición explícita de la red permite optimizar el coste computacional seleccionando estrategias de activación específicas para cada bloque. En el Código 12.8, presentamos una arquitectura que combina convoluciones KAN, atención espacial y capas densas KAN.

Nótese cómo se configuran las capas de manera heterogénea: la primera capa convolucional utiliza activación de entrada polinomial (para alta expresividad) pero salida fija ReLU (para eficiencia), mientras que las capas profundas utilizan configuraciones KAN completas.

```

1 class CustomHybridKAN(nn.Module):
2     def __init__(self, input_channels=3, num_classes=10):
3         super(CustomHybridKAN, self).__init__()
4
5         # --- BLOQUE 1: Extracción y Atención ---
6         self.conv1 = DualFlexKANConv2d(
7             in_channels=input_channels, out_channels=32,
8             kernel_size=3, padding=1,
9             # Entrada adaptativa polinomial, salida eficiente ReLU
10            input_activation_type="polynomial",
11            output_activation_type="relu",
12            use_batch_norm=True
13        )
14        self.pool1 = nn.MaxPool2d(2)
15        # Inserción de atención tras la convolución
16        self.attn1 = KANConv2dAttention(channels=32)
17
18        # --- BLOQUE 2: Procesamiento Profundo ---
19        self.conv2 = DualFlexKANConv2d(
20            in_channels=32, out_channels=64,
21            kernel_size=3, padding=1,
22            # Configuración Full-KAN (entrada y salida adaptativas)
23            input_activation_type="polynomial",
24            output_activation_type="polynomial",
25            dropout_prob=0.1
26        )
27        self.pool2 = nn.MaxPool2d(2)
28
29        # --- BLOQUE 3: Clasificación Densa ---
30        self.flat_size = 64 * 8 * 8 # Suponiendo entrada 32x32
31
32        self.dense1 = DualFlexKANLinear(
33            in_features=self.flat_size, out_features=256,

```

```
34         input_function_strategy="per_input",
35         output_function_strategy="per_neuron"
36     )
37
38     # Atención sobre características latentes
39     self.attn_dense = KANFeatureAttention(256)
40
41     self.output = DualFlexKANLinear(
42         in_features=256, out_features=num_classes,
43         input_activation_type="silu", # Optimización final
44         output_activation_type="linear"
45     )
46
47     def forward(self, x):
48         # Flujo explícito de tensores
49         x = self.attn1(self.pool1(self.conv1(x)))
50         x = self.pool2(self.conv2(x))
51
52         x = x.view(x.size(0), -1) # Flatten
53
54         x = self.dense1(x)
55         x, _ = self.attn_dense(x) # Atención densa
56
57         return self.output(x)
```

Listing 12.8: Implementación de red híbrida KAN capa a capa

Capítulo 13 Aplicaciones Específicas

13.1. Regresión No Lineal

```
1 from sklearn.datasets import make_regression
2 from sklearn.preprocessing import StandardScaler
3
4 # Generar datos
5 X, y = make_regression(n_samples=1000, n_features=8, noise=0.1,
6                        random_state=42)
7
8 # Modelo para regresion
9 model_regression = DualFlexKAN(
10     layer_sizes=[8, 32, 16, 1],
11     input_function_strategies=["per_input", "per_input", "none"],
12     input_activation_types=["polynomial", "rbf", "polynomial"],
13     output_function_strategies=["per_neuron", "per_neuron", "fixed"],
14     output_activation_types=["polynomial", "rbf", "linear"],
15     dropout_probs=[0.05, 0.1, 0.0],
16     use_batch_norms=[True, True, False]
17 )
18
19 # Entrenamiento
20 criterion = torch.nn.MSELoss()
21 optimizer = torch.optim.AdamW(model_regression.parameters(), lr=1e-3)
22
23 X_tensor = torch.FloatTensor(X)
24 y_tensor = torch.FloatTensor(y)
25
26 model_regression.train()
27 for epoch in range(100):
28     optimizer.zero_grad()
29     output = model_regression(X_tensor).squeeze()
30     loss = criterion(output, y_tensor)
31     loss.backward()
32     optimizer.step()
33
34     if epoch % 20 == 0:
```

```
34 print(f'Epoch {epoch}, Loss: {loss.item():.6f}')
```

Listing 13.1: Ejemplo de regresión no lineal

13.2. Clasificación Multiclase

```
1 from sklearn.datasets import make_classification
2
3 # Generar datos de clasificacion
4 X_class, y_class = make_classification(
5     n_samples=2000, n_features=10, n_classes=4,
6     n_informative=8, random_state=42
7 )
8
9 # Modelo para clasificacion
10 model_classification = DualFlexKAN(
11     layer_sizes=[10, 64, 32, 4], # 4 clases
12     input_function_strategies=["per_input", "per_input", "none"],
13     input_activation_types=["polynomial", "legendre", "polynomial"],
14     output_function_strategies=["per_neuron", "per_neuron", "fixed"],
15     output_activation_types=["rbf", "polynomial", "softmax"],
16     dropout_probs=[0.15, 0.25, 0.0],
17     use_batch_norms=[True, True, False]
18 )
```

Listing 13.2: Ejemplo de clasificación multiclase

Apéndice A: Tabla de Referencia

Tabla 1: Referencia Rápida de Parámetros

Parámetro	Valores	Descripción
input_function_strategy	none, global, per_input, per_neuron_input	Estrategia funciones ent
output_function_strategy	none, global, per_neuron, fixed	Estrategia funciones sal
dropout_position	before_activation, after_activation, both, none	Posición dropout
regularization_order	dropout_first, batch_norm_first	Orden aplicación

Apéndice B: Tipos de Función

Este apéndice lista todos los identificadores de cadena válidos para los parámetros `input_activation_type` y `output_activation_type` en la clase `DualFlexKANLinear`.

C. Funciones Adaptativas (Entrenables)

Estas funciones contienen parámetros aprendibles y son el núcleo de la flexibilidad de las redes KAN. Requieren estrategias de tipo `global`, `per_input` o `per_neuron`.

Tabla 2: Funciones de Activación Adaptativas

Identificador	Base Matemática	Kwargs Típicos	Uso Recomendado
<code>polynomial</code>	Series de potencias $1, x, x^2 \dots$	<code>{degree: 3}</code>	Aproximación general simple, interpretable.
<code>legendre</code>	Polinomios ortogonales de Legendre	<code>{degree: 3}</code>	Estable en $[-1, 1]$, evita problemas numéricos de polinomios simples.
<code>chebyshev</code>	Polinomios de Chebyshev (1ª especie)	<code>{degree: 3}</code>	Minimiza el error máximo (minimax), aproximación uniforme.
<code>gegenbauer</code>	Polinomios Ultrasféricos	<code>{degree: 3, alpha: 0.5}</code>	Generalización de Legendre/Chebyshev con control de forma (α).
<code>jacobi</code>	Polinomios de Jacobi	<code>{degree: 3, alpha: 0., beta: 0.}</code>	Máxima flexibilidad ortogonal, controla peso en extremos del intervalo.
<code>bspline</code>	B-Splines (Curvas base)	<code>{num_knots: 10, degree: 3}</code>	Control local, funciones complejas y suaves. Estándar en KAN.
<code>fb spline</code> <code>fast_bspline</code>	B-Splines Optimizados	<code>{num_knots: 10, degree: 3}</code>	Versión acelerada computacionalmente para grandes volúmenes de datos.
<code>rbf</code>	Funciones de Base Radial (Gaussianas)	<code>{num_centers: 10}</code>	Interpolación basada en proximidad, clusters locales.
<code>sine</code>	Base Espectral (Fourier)	<code>{num_freqs: 5}</code>	Datos con periodicidad, estacionalidad o patrones ondulatorios.
<code>wavelet</code>	Wavelets (Sombrero Mexicano/Ricker)	<code>{num_wavelets: 5}</code>	Señales transitorias, localización tiempo-frecuencia.
<code>rational</code>	Funciones Racionales (Padé) $P(x)/Q(x)$	<code>{degrees: (3, 3)}</code>	Modelado de singularidades (polos), asíntotas y leyes físicas inversas.

D. Funciones Fijas (No Entrenables)

Estas funciones no añaden parámetros al modelo. Se utilizan principalmente con estrategias **fixed** para emular comportamientos de redes neuronales clásicas (MLP) o para preprocesamiento.

Tabla 3: Funciones de Activación Fijas (Estándar NN)

Identificador	Descripción	Uso Recomendado
linear	Identidad $f(x) = x$	Regresión sin límites, paso directo.
relu	$\max(0, x)$	Capas ocultas eficientes, sparsity.
leaky_relu	$\max(0, \alpha x)$	Evita neuronas muertas.
tanh	Tangente hiperbólica	Salidas acotadas en $[-1, 1]$.
sigmoid	Sigmoide $\frac{1}{1+e^{-x}}$	Probabilidades, puertas lógicas.
softmax	Exponencial normalizada	Clasificación multiclase (solo salida).
gelu	Gaussian Error Linear Unit	Modelos modernos (Transformers).
swish / silu	$x \cdot \sigma(x)$	Alternativa suave a ReLU.
mish	$x \cdot \tanh(\ln(1 + e^x))$	Alternativa suave a ReLU.

E. Transformaciones Matemáticas

Funciones fijas útiles para preprocesamiento de entrada o restricciones de salida específicas.

Tabla 4: Transformaciones Matemáticas

Identificador	Fórmula	Propósito
abs	$ x $	Magnitud, asegurar positividad.
square	x^2	Energía, penalización cuadrática.
sqrt	$\sqrt{ x + \epsilon}$	Estabilización de varianza.
exp	e^x	Inversión de logaritmos, tasas.
log	$\ln(x + \epsilon)$	Comprimir rangos dinámicos amplios.

Apéndice F: Tabla resumen de estrategias

Tabla 5: Resumen de estrategias de función de entrada y salida en DualFlexKANLinear. N_p indica el número de parámetros de la función base (ej. `degree+1`).

Sección	Estrategia	Descripción	Params. Extra	Compartición	Uso típico
10* ENTRADA	none	Sin transformación. Pasa la entrada original tal cual.	0	—	Capas lineales puras.
	fixed	Función fija no entrenable (ej. ReLU, Abs, Log).	0	Total	Preprocesamiento, normalización.
	global	Una única función paramétrica compartida para todas las entradas.	$1 \times N_p$	Total	Entradas homogéneas (misma escala/naturaleza).
	per_input	Una función aprendible distinta para cada dimensión de entrada.	$n_{in} \times N_p$	Por entrada	KAN Híbrida (Estándar). Variables heterogéneas.
	per_neuron_input	Una función distinta para cada conexión entre entrada y neurona.	$n_{in} \times n_{out} \times N_p$	Ninguna	KAN Pura. Máxima expresividad, alto coste.
8* SALIDA	none	Sin activación. Salida lineal directa ($Wx + b$).	0	—	Regresión, bloques residuales.
	fixed	Activación fija estándar (ReLU, Sigmoid, Tanh).	0	Total	Comportamiento MLP, estabilidad.
	global	Una función paramétrica compartida para todas las neuronas.	$1 \times N_p$	Total	Ajuste no-lineal adaptable común.
	per_neuron	Una función aprendible distinta para cada neurona de salida.	$n_{out} \times N_p$	Por neurona	Salidas complejas o multimodales.

Apéndice G: Dependencias

```
1 # Dependencias basicas
2 pip install torch torchvision
3 pip install numpy scipy matplotlib
4 pip install scikit-learn
5
6 # Dependencias opcionales
7 pip install jupyter tqdm
```

Listing 3: Instalación de dependencias

Apéndice H: Resumen de funciones

Este apéndice sirve como referencia rápida de la API de la librería DualFlexKAN. Se detallan las clases principales y funciones de utilidad divididas por módulo.

I. Núcleo del Modelo (KAN_shared_poly7.py)

Tabla 6: Componentes Principales de la Arquitectura KAN

Nombre	Tipo	Descripción	Sintaxis y Ejemplo
DualFlexKANLinear	Clase	Capa fundamental. Gestiona la transformación independiente de entrada (Ψ_{in}) y salida (Ψ_{out}), así como la regularización posicional.	Init: DualFlexKANLinear(in_ft, out_ft, input_function_strategy='per_input', ...) Call: $y = \text{layer}(x)$
DualFlexKAN	Clase	Contenedor de red multicapa. Construye una secuencia de capas DualFlexKANLinear según una lista de tamaños y configuraciones.	Init: $\text{model} = \text{DualFlexKAN}(\text{layer_sizes}=[10, 64, 1], \dots)$ Call: $\text{preds} = \text{model}(\text{inputs})$
polycoefs	Función	Parser de utilidades que extrae coeficientes numéricos de una representación en cadena de un polinomio.	$\text{coefs} = \text{polycoefs}("0.5x^2 + 2x")$
get_function_info	Método	Devuelve las ecuaciones matemáticas (string) de las funciones activas en una capa.	$\text{info} = \text{layer.get_function_info}()$
count_parameters	Método	Retorna un diccionario detallado con el recuento de parámetros (lineales vs. funcionales vs. batchnorm).	$\text{stats} = \text{model.count_parameters}()$
print_learned_functions	Método	Itera sobre toda la red e imprime en consola las ecuaciones aprendidas para cada neurona.	$\text{model.print_learned_functions}()$
print_architecture_info	Método	Imprime un resumen jerárquico de la arquitectura, estrategias y conteo de parámetros.	$\text{model.print_architecture_info}()$

J. Mecanismos de Atención (KAN_shared_poly7.py)

Tabla 7: Componentes de Atención y Selección de Características

Nombre	Tipo	Descripción	Sintaxis y Ejemplo
KANFeatureAttention	Clase	Mecanismo tipo "Squeeze-and-Excitation". Aprende un vector de pesos para amplificar o suprimir características de entrada dinámicamente.	Init: <code>attn = KANFeatureAttention(in_dim, reduction_ratio=4)</code> Call: <code>x_att, weights = attn(x)</code>
DualFlexKANAttentionLayer	Clase	Capa compuesta que integra el mecanismo de atención antes de una capa DualFlexKANLinear estándar.	Init: <code>layer = DualFlexKANAttentionLayer(in_ft, out_ft, attention_reduction=4, **kwargs)</code>
SelectiveAttentiveKANClase		Constructor de modelos multicapa que permite especificar qué capas exactas (por índice) incluyen mecanismos de atención.	Init: <code>model = SelectiveAttentiveKAN(sizes, attention_layers_indices=[0, 2], ...)</code>

K. Funciones de Activación (KAN_activation.py)

Tabla 8: Biblioteca de Funciones de Activación

Nombre	Tipo	Descripción	Kwargs Típicos
StandardActivationClase		Wrapper para activaciones fijas de PyTorch (ReLU, Tanh, SiLU, etc.).	<code>activation_name='elu'</code>
PolynomialActivationClase		Polinomio estándar de grado n : $c_0 + c_1x + \dots + c_nx^n$.	<code>{degree: 3}</code>
LegendreActivationClase		Polinomios ortogonales de Legendre. Más estables en $[-1, 1]$.	<code>{degree: 3}</code>

Continúa en la siguiente página

Cuadro 8 – continuación de la página anterior

Nombre	Tipo	Descripción	Kwargs Típicos
ChebyshevActivation	Clase	Polinomios de Chebyshev (1ª especie). Minimizan error máximo (minimax).	{degree: 4}
GegenbauerActivation	Clase	Polinomios ultrasféricos. Generalización con parámetro α .	{degree: 3, alpha: 0.5}
JacobiActivation	Clase	Polinomios ortogonales generales con pesos controlados por α, β .	{degree: 3, alpha: 0., beta: 0.}
BSplineActivation	Clase	Base de Splines cúbicos con soporte local. Estándar en KAN original.	{num_knots: 10, degree: 3}
FastBSplineActivation	Clase	Implementación optimizada de B-Splines para mayor velocidad de cómputo.	{num_knots: 10, degree: 3}
RBFActivation	Clase	Redes de Base Radial (Gaussianas). Interpolación local.	{num_centers: 8}
SineActivation	Clase	Base espectral (Fourier) para datos periódicos.	{num_freqs: 5}
WaveletActivation	Clase	Wavelets tipo "Mexican Hat" para localización tiempo-frecuencia.	{num_wavelets: 6}
RationalActivation	Clase	Funciones racionales (Padé) $P(x)/Q(x)$. Modela polos/singularidades.	{degrees: (3, 3)}

L. Inicialización (KAN_initialization.py)

Tabla 9: Sistema de Inicialización de Pesos

Nombre	Tipo	Descripción	Uso
smart_kan_initialization	Función	Punto de entrada principal. Aplica estrategias predefinidas (optimal, deep, conservative).	smart_kan_initialization(model, strategy='optimal')

Continúa...

Cuadro 9 – continuación

Nombre	Tipo	Descripción	Uso
KANInitializer	Clase	Motor de inicialización. Permite mezclar estrategias lineales (he, xavier) con polinomiales.	<code>init = KANInitializer(linear_strategy='he', ...)</code>
<code>initialize_model_layerwise</code>	Función	Aplica configuraciones distintas por capa (útil para encoder vs decoder).	<code>initialize_model_layerwise(model, configs_list)</code>
InitializationAnalyzer	Clase	Diagnóstico. Calcula estadísticas de pesos para detectar problemas de gradiente.	<code>InitializationAnalyzer.analyze_model(model)</code>

M. Persistencia y Utilidades (`save_load_utils.py`)

Tabla 10: Utilidades de Guardado y Carga

Nombre	Tipo	Descripción	Uso
<code>save_model_auto</code>	Función	Guarda modelo + código fuente + argumentos init. Permite cargar sin el archivo .py original.	<code>save_model_auto(model, 'net.pt')</code>
<code>load_model_auto</code>	Función	Carga un modelo guardado con <code>save_model_auto</code> .	<code>model = load_model_auto('net.pt')</code>
<code>extract_model_info</code>	Función	Introspección profunda del modelo para generar reportes.	<code>info = extract_model_info(model)</code>
<code>save_experiment</code>	Función	Empaqueta modelo, métricas y configuración en un solo archivo con timestamp.	<code>save_experiment('run1', model=net, acc=0.9)</code>

N. Módulos de Arquitecturas Específicas

Tabla 11: Arquitecturas Especializadas (Autoencoders, VAE, CNN)

Módulo/Clase	Archivo	Descripción
DualFlexKANAutoencoder	DFKAN_-autoencoder.py	Autoencoder asimétrico. Soporta $N_{in} \neq N_{out}$. Genera capas ocultas automáticamente si no se definen.
DualFlexKANBetaVAE	DFKAN_VAE.py	Variational Autoencoder con control β para disentanglement. Incluye reparametrización y pérdida KL.
DualFlexKANConv2d	DFKAN_conv.py	Capa convolucional donde el kernel lineal es precedido/sucedido por transformaciones KAN.
DualFlexKANCNN	DFKAN_conv.py	Constructor de CNNs completas (Conv blocks + Pooling + Dense KAN classifier).

Ñ. Módulos de Análisis e Interpretabilidad

Tabla 12: Herramientas de Análisis Post-Entrenamiento

Clase Principal	Archivo	Funcionalidad Clave
KANExplainability	KAN_-explainability_importance.py	- comprehensive_importance_analysis - Importancia por gradientes y varianza. - Análisis de sensibilidad y Knockout tests.
KANInteractionAnalysis	KAN_-explainability_interaction_analysis.py	- functional_decomposition (Waterfall plots). - Mapas de interacción input-output. - Detección de sinergias por pares.
KANFlowVisualization	KAN_flow_-visualization.py	- Heatmaps de activación por capa. - Rastreo de rutas críticas (Critical paths). - Detección de cuellos de botella de información.
KANPruningSimplification	KAN_pruning_simplification.py	- Poda de coeficientes y conexiones (Sparsity). - fit_symbolic_function: Regresión simbólica para obtener fórmulas simples.
KANAutoencoderAnalysis	KAN_autoencoder_analysis.py	- Simetría funcional Encoder-Decoder. - Interpolación y visualización de espacio latente. - Reconstrucción por componente.

O. Carga de Datos (dataset_loader.py)

Tabla 13: Gestión de Datasets

Nombre	Tipo	Descripción	Uso
<code>load_dataset</code>	Función	Fachada principal. Carga datasets estándar o sintéticos por nombre.	<code>tr, ts = load_dataset('mnist')</code>
<code>load_custom_dataset</code>	Función	Convierte arrays X, y arbitrarios en DataLoaders.	<code>tr, ts = load_custom_dataset(X, y)</code>
<code>FlexibleImageDataset</code>	Clase	Dataset PyTorch que maneja aplanado/redimensionado dinámico.	<code>ds = FlexibleImageDataset(X, y, as_vector=True)</code>

Apéndice P: *

Referencias

1. Liu et al. (2024). “KAN: Kolmogorov-Arnold Networks”
2. Kidger & Lyons (2020). “Universal Approximation with Deep Neural Networks”
3. PyTorch Documentation: <https://pytorch.org/docs/>

FAQ - Preguntas Frecuentes

P: ¿Cuál es la diferencia entre `per_input` y `per_neuron_input`?

R: `per_input` usa una función por cada característica de entrada, mientras que `per_neuron_input` usa una función específica para cada entrada de cada neurona, ofreciendo máxima flexibilidad pero con mayor costo computacional.

P: ¿Cuándo debo usar funciones fijas vs. adaptativas?

R: Usa funciones fijas (como ReLU, softmax) para capas de salida o cuando quieras mayor estabilidad. Usa funciones adaptativas (polynomial, legendre) para capas ocultas donde necesites mayor expresividad.

P: ¿Cómo elijo la posición del dropout y batch normalization?

R: Como regla general: dropout después de activación para estabilidad, batchnorm antes de activación para mejor convergencia.

P: ¿Por qué mi modelo KAN es más lento que un MLP tradicional?

R: Las funciones adaptativas requieren más cómputo que activaciones fijas. Considera usar estrategias menos complejas como `global` en lugar de `per_neuron`.

P: ¿Cómo reduzco el overfitting en modelos KAN?

R: Incrementa dropout, usa batch normalization, reduce la complejidad de las funciones, o usa regularización L2 en el optimizador.

Roadmap de Desarrollo

Versión 1.3

- Soporte para funciones wavelet
- Regularización adaptativa automática (en desarrollo)
- Funciones de activación personalizables (en desarrollo)
- Capas de atención
- Herramientas de visualización mejoradas (en desarrollo)

Índice de Funciones

Clases Principales

- `DualFlexKANLinear` - Capa individual con control dual
- `DualFlexKAN` - Red completa con múltiples capas
- `KANEnsemble` - Ensemble de múltiples modelos

Funciones de Utilidad

- `save_kan_model()` - Guardar modelo con configuración
- `load_kan_model()` - Cargar modelo desde archivo
- `validate_config()` - Validar configuración
- `create_kan_template()` - Crear configuraciones base
- `benchmark_kan_configs()` - Comparar configuraciones
- `transfer_learning_kan()` - Transfer learning

Apéndice Q: *

Agradecimientos

Este manual fue desarrollado gracias a las contribuciones de la comunidad de investigadores en redes neuronales y aprendizaje profundo. Agradecimientos especiales a:

- Los autores originales de Kolmogorov-Arnold Networks
- La comunidad de PyTorch por proporcionar las herramientas fundamentales
- Al grupo BioSiP por hacer posible todo esto

Historial de Versiones

Tabla 14: Historial de Versiones

Versión	Fecha	Cambios Principales
v0.1	Agosto 2025	Versión inicial completa Documentación preliminar de DualFlexKAN Sistema de regularización flexible Ejemplos prácticos y casos de uso
v1.0	Octubre 2025	Versión mejorada Documentación completa de DualFlexKAN Autoencoders KAN Autoencoders β VAE-KAN β -annealing en VAE-KAN Autoencoders KAN convolucionales 2D
v1.1	Noviembre 2025	Versión mejorada Inclusión de DualFlexKANConv Generalización a convoluciones 1D y 3D

Apéndice R: Licencia

MIT License

Copyright (c) 2025 Andrés Ortiz

Por la presente se concede permiso, libre de cargos, a cualquier persona que obtenga una copia de este software y de los archivos de documentación asociados (el "Software"), a utilizar el Software sin restricción, incluyendo sin limitación los derechos a usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del Software, y a permitir a las personas a las que se les proporcione el Software a hacer lo mismo, sujeto a las siguientes condiciones:

El aviso de copyright anterior y este aviso de permiso se incluirán en todas las copias o partes sustanciales del Software.

EL SOFTWARE SE PROPORCIONA "TAL CUAL", SIN GARANTÍA DE NINGÚN TIPO, EXPRESA O IMPLÍCITA, INCLUYENDO PERO NO LIMITADO A GARANTÍAS DE COMERCIALIZACIÓN, IDONEIDAD PARA UN PROPÓSITO PARTICULAR Y NO INFRACCIÓN. EN NINGÚN CASO LOS AUTORES O TITULARES DEL COPYRIGHT SERÁN RESPONSABLES DE NINGUNA RECLAMACIÓN, DAÑOS U OTRAS RESPONSABILIDADES, YA SEA EN UNA ACCIÓN DE CONTRATO, AGRAVIO O CUALQUIER OTRO MOTIVO, QUE SURJA DE O EN CONEXIÓN CON EL SOFTWARE O EL USO U OTRO TIPO DE ACCIONES EN EL SOFTWARE.

Manual DualFlexKAN v1.1

Documentación Técnica Completa

Última actualización: 27 de febrero de 2026

Andrés Ortiz García

Manual DualFlexKAN v1.1

Agosto 2025